Обратная разработка программного обеспечения x86, взлом и контрмеры

Русский перевод практического руководства по архитектуре и ассемблеру x86, анализу и отладке машинного кода, обратной разработке, взлому программного обеспечения, защитным механизмам и противодействию анализу.

Больше крутых материалов в моём телеграм канале @grogrigs

Глава 1. Декомпиляция и архитектура

Эффективный специалист по обратной разработке или взлому понимает системы, которые анализирует. Программное обеспечение проектируется для работы в определённой среде, и, если вы не понимаете, как устроена эта среда, вам будет трудно понять само программное обеспечение.

В этой главе рассматриваются шаги, необходимые для начала обратной разработки приложения. Декомпиляция играет важнейшую роль в преобразовании приложения из машинного кода в форму, которую человек может прочитать и понять. Чтобы действительно проанализировать полученный код, необходимо также понимать архитектуру компьютеров, для работы на которых он предназначен.

Декомпиляция

Большинство программистов пишут программы на языке высокого уровня, например C/C++ или Java, рассчитанном на чтение человеком. Однако компьютеры предназначены для выполнения машинного кода, который представляет инструкции в двоичном виде.

Компиляция - это процесс преобразования программы с языка программирования в машинный код. Следовательно, декомпиляцией называется обратный процесс, при котором машинный код преобразуют обратно в исходный язык программирования, восстанавливая первоначальный исходный код. Когда такая возможность существует, это самый простой подход к обратной разработке, поскольку исходный код предназначен для чтения и интерпретации человеком. Большая часть этой книги будет посвящена более типичному случаю, когда декомпиляция невозможна. Однако в учебных целях важно понимать: иногда исходный код можно восстановить путём декомпиляции, и, когда такая возможность есть, ею следует воспользоваться.

Когда декомпиляция полезна?

Для многих языков программирования полная декомпиляция невозможна. Программы на этих языках непосредственно преобразуются в машинный код, и в ходе этого процесса часть сведений, например имена переменных, утрачивается. Некоторые развитые декомпиляторы способны создавать для таких языков псевдокод, но этот процесс не даёт безупречного результата.

В то же время некоторые языки программирования используют так называемую компиляцию точно в срок (just-in-time, JIT). Когда программы на JIT-языках «собирают», исходный код преобразуется не в машинный, а в промежуточный язык (intermediate language, IL). JIT-компиляторы хранят копию кода на этом промежуточном языке до запуска программы, после чего преобразуют его в машинный код. К примерам JIT-языков относятся Java, Dalvik (Android) и .NET.

Например, Java хорошо известен своей практически полной независимостью от платформы, которая достигается благодаря применению промежуточного языка (байт-кода Java) и виртуальной машины Java (Java Virtual Machine, JVM). Программный код распространяется в виде байт-кода и компилируется только во время выполнения, а JVM преобразует промежуточный язык Java в машинный код, предназначенный именно для той машины, на которой работает программа. Такой подход может отрицательно влиять на размер файла и производительность, но окупается переносимостью.

JIT-компиляция также значительно упрощает обратную разработку таких приложений. Промежуточные языки достаточно похожи на первоначальный исходный код, поэтому их можно

декомпилировать, то есть преобразовать обратно в пригодный для использования исходный код. Поскольку исходный код рассчитан на чтение человеком, понять логику приложения и обнаружить средства защиты программы или другие встроенные секреты становится гораздо проще.

Декомпиляция JIT-языков программирования

Для JIT-языков наподобие .NET существует несколько бесплатных декомпиляторов. Один из широко используемых декомпиляторов .NET - JetBrains dotPeek, доступный по адресу www.jetbrains.com/decompiler. На рисунке 1.1 показан пример кода .NET, декомпилированного в dotPeek.

Как видно на рисунке, после декомпиляции код .NET легко читается, поскольку промежуточный язык кодирует в форме метаданных большой объём сведений, позволяя точнее восстановить исходный код. Любые конфиденциальные сведения или коммерческие тайны, содержащиеся в коде, оказываются легко доступны специалисту по обратной разработке.

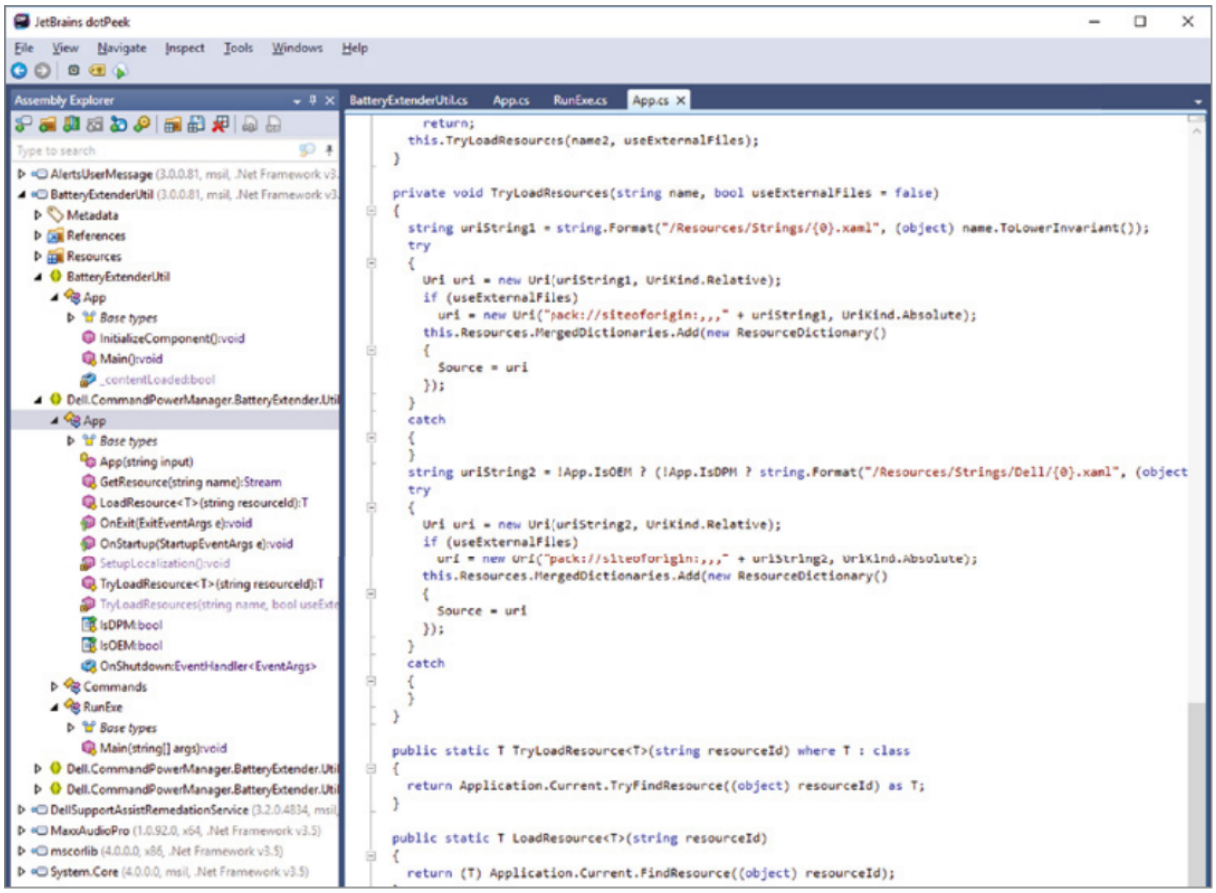


Рисунок 1.1. Декомпилятор .NET JetBrains dotPeek

Защита JIT-языков

В отличие от программ с настоящим машинным кодом, JIT-скомпилированные программы часто можно преобразовать в исходный код. Поскольку порог вхождения для обратной разработки такого кода ниже, многие способы защиты x86-кода от обратной разработки, рассматриваемые в последующих главах, оказываются излишними и чрезмерными.

Для декомпилируемых языков распространённым средством защиты от обратной разработки служит обфускация. На рисунке 1.2 показан пример приложения .NET до и после обфускации.

В верхней половине рисунка находится код до обфускации: имена функций и переменных, а также строки легко читаются. Сведения, содержащиеся в именах переменных, помогают специалисту по обратной разработке понять назначение каждой функции и работу приложения в целом.

В нижней половине рисунка показана обфусцированная версия того же кода. Теперь имена функций, имена переменных и строки искажены, поэтому понять назначение показанной функции, не говоря уже обо всём приложении, стало гораздо сложнее.

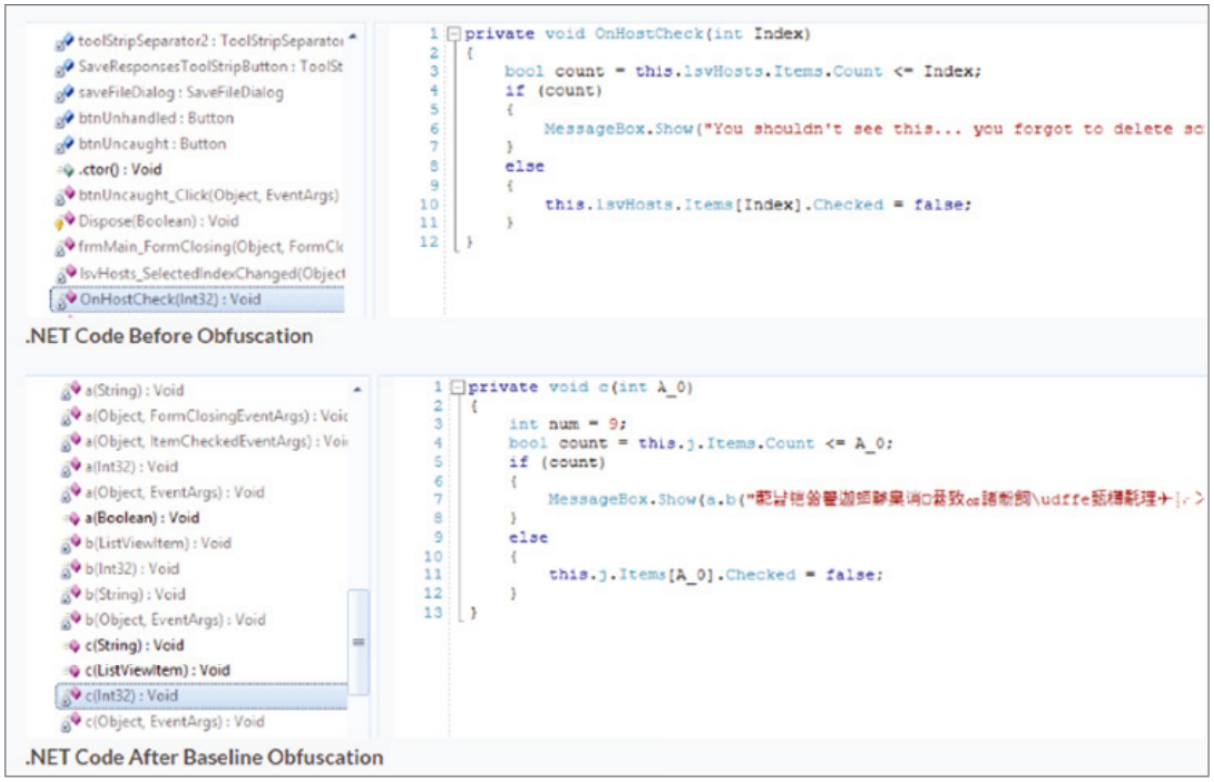


Рисунок 1.2. Обфускация в JetBrains dotPeek

Ещё одна важная рекомендация по обеспечению безопасности - не писать критически важный для безопасности или конфиденциальности код на JIT-языках, для которых обратная разработка не представляет труда. Вместо этого такой код следует писать на компилируемом языке, например C/C++, обратная разработка которого значительно сложнее. Этот код можно поместить в библиотеки DLL, связанные с исполняемым файлом, который содержит неконфиденциальный код на JIT-языке.

Лабораторная работа 1. Декомпиляция

Это первая практическая лабораторная работа в книге. Лабораторные работы и все относящиеся к ним инструкции находятся в соответствующей папке по следующему адресу:

github.com

Для выполнения этой лабораторной работы найдите папку `Lab Decompiling` и следуйте приведённым в ней инструкциям.

Отрабатываемые навыки

Каждая лабораторная работа в этой книге предназначена для обучения определённым навыкам и получения практического опыта их применения. В этой лабораторной работе отрабатываются следующие навыки:

• декомпиляция;
• выполнение вводной обратной разработки.

Для приобретения этих навыков вы будете использовать JetBrains dotPeek, чтобы провести обратную разработку и изменить приложение .NET.

Основные выводы

Декомпиляция - мощный и простой способ понять и изменить программу. Однако он применим не ко всем программам. Хотя программы, написанные на таких языках, как C/C++, можно декомпилировать с помощью таких инструментов, как Hex-Rays Decompiler из IDA или Ghidra, результат зачастую оказывается низкокачественным и неудобным в использовании.

При разработке приложений, которые содержат конфиденциальные сведения или которые вы не хотите позволить изменять, лучше использовать язык, плохо поддающийся декомпиляции. Например, для реализации конфиденциальных функций C/C++ подходит лучше, чем такой язык платформы .NET, как C#.

Архитектура

Декомпиляция - простой подход к обратной разработке, поскольку она возвращает нас к языкам высокого уровня и логическим конструкциям. Однако этот простой путь доступен нечасто. В случае языков, программы на которых преобразуются в машинный код, необходимо погрузиться глубже и понять устройство компьютерных архитектур, машинного кода и кода на языке ассемблера.

Архитектура компьютера

Принято считать, что обычному программисту не требуется глубокое понимание принципов работы компьютеров. Когда программа пишется на процедурном языке, всеми низкоуровневыми операциями занимается операционная система. Программа представляется процессом, который получает доступ к процессору, памяти и файловой системе всякий раз, когда они ему нужны. Создаётся впечатление, что процессы располагают собственными непрерывными адресными пространствами памяти, а файлы представляют собой всего лишь последовательности байтов, которые можно читать и записывать.

Однако на самом деле всё это не так: операционная система скрывает от вас истинное положение дел, чтобы упростить программирование. Специалисту по обратной разработке необходимо твёрдо понимать, как в действительности устроена архитектура компьютера. На рисунке 1.3 показаны основные компоненты компьютера, включая центральный процессор, мост, память и периферийные устройства.

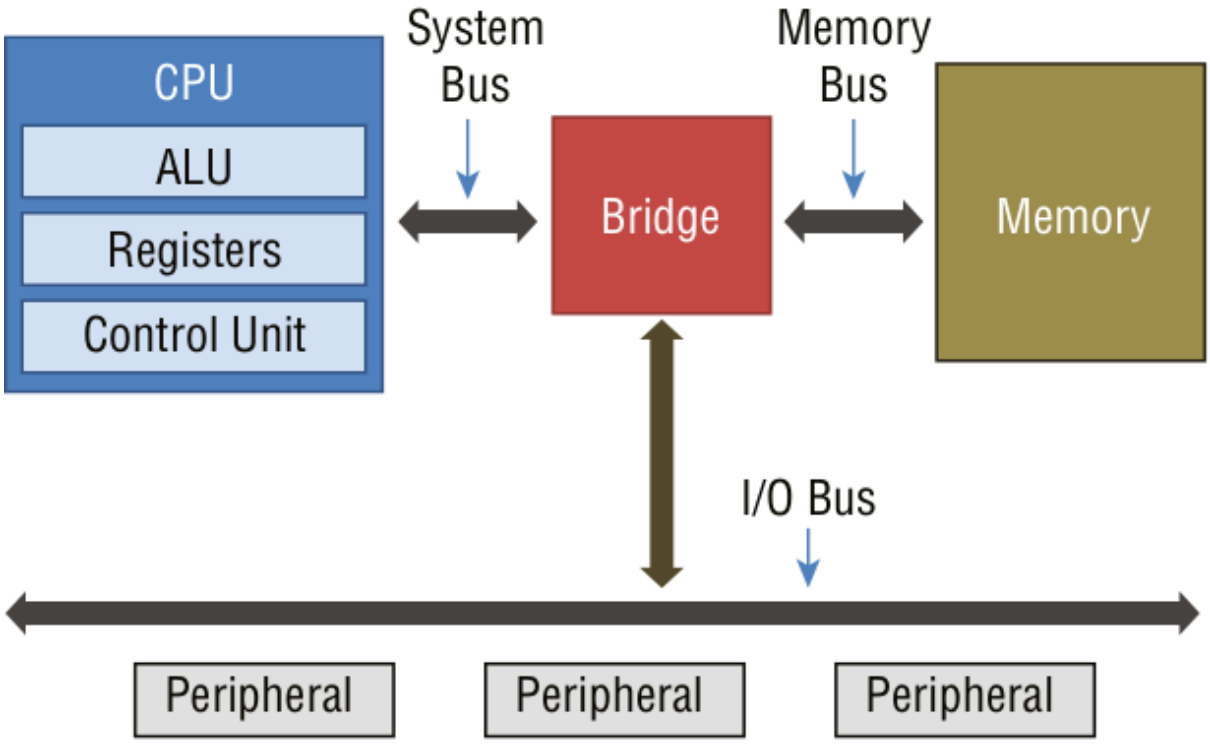


Рисунок 1.3. Архитектура компьютера

Центральный процессор

Центральный процессор (central processing unit, CPU) - это компонент компьютера, в котором происходит обработка. Внутри центрального процессора находятся следующие компоненты:

• **Арифметико-логическое устройство (АЛУ).** АЛУ выполняет в компьютере математические операции, например сложение и умножение.
• **Регистры.** Регистры служат для временного хранения данных и используются как основные источники входных данных и места назначения результатов инструкций x86. Они обеспечивают чрезвычайно быстрый доступ к одному машинному слову данных и обычно указываются по имени.
• **Устройства управления.** Устройства управления исполняют код. В частности, они считывают инструкции и координируют работу других элементов компьютера.

Мосты и периферийные устройства

Центральный процессор соединён шиной с мостом. Назначение моста состоит в том, чтобы связывать центральный процессор с другими компонентами системы, в том числе с памятью и шиной ввода-вывода, через которую к системе подключаются такие периферийные устройства, как клавиатура, мышь и динамики. Информация передаётся по шинам, а мост управляет этим потоком и следит, чтобы данные, поступившие по одной шине, направлялись в соответствующую выходную шину.

Периферийные устройства, подключённые через шину ввода-вывода, позволяют компьютеру взаимодействовать с внешним миром. В частности, через них компьютер отправляет данные графическому адаптеру, клавиатуре, мыши, динамикам и другим системам и получает данные от них.

Память и регистры

Как следует из названия, память - это место хранения данных в компьютере. Данные хранятся в виде линейной последовательности байтов, доступ к которым осуществляется по адресам. Такая организация обеспечивает умеренно быстрый доступ к данным, хранящимся в системе.

Когда программе требуется обратиться к данным в памяти, центральный процессор отправляет запрос по шине мосту, который пересылает его памяти; там выполняется обращение к данным по указанному адресу. Прежде чем программа сможет использовать запрошенные данные, им приходится пройти тот же путь в обратном направлении и вернуться в центральный процессор. Регистр, напротив, физически находится внутри центрального процессора, поэтому доступ к нему гораздо проще.

Регистры представляют собой хранилища внутри центрального процессора и, в отличие от памяти, не образуют линейной последовательности байтов. У регистров есть определённые имена и заданные размеры.

Регистры и память выполняют одну и ту же задачу: хранят данные. Однако они обладают разной специализацией, то есть предлагают качество в обмен на количество. Регистров мало, и они дороги, но обеспечивают чрезвычайно быстрый доступ к данным. Память дешева и многочисленна, однако скорость доступа к ней ниже.

Основная масса связанных с программой данных - сам код и обрабатываемые им данные - хранится в памяти. Во время работы программы небольшие порции данных копируются в регистры для обработки.

Язык ассемблера

Компьютеры работают на основе двоичной цифровой логики. Любой сигнал либо включён (1), либо выключен (0). Это относится и к программам, выполняющимся на компьютере. Все языки высокого уровня в конечном итоге преобразуются в последовательность битов, называемую машинным кодом. Машинный код определяет набор инструкций, которые выполняет компьютер для осуществления требуемой функции.

Введение в машинный код

Каждый программист начинает изучение языка с программы «hello world». В x86 машинный код программы «hello world» выглядит следующим образом:

```
55 89 e5 83 e4 f0 83 ec 10 b8 b0 84 04 08 89 04 24 e8 1a ff ff ff b8 00
00 00 00 c9 c3 90
```

Для удобства чтения и компактности этот машинный код записан в шестнадцатеричной форме, но его истинное представление - двоичная строка из единиц и нулей. В этой двоичной строке находятся инструкции, которые переключают транзисторы для выполнения вычислений, получают данные из памяти, отправляют сигналы по системным шинам, взаимодействуют с графическим адаптером и в конечном счёте выводят текст «hello world». Если эта строка символов кажется слишком короткой для выполнения всей такой работы, причина в том, что инструкции обращаются за помощью к операционной системе, которой в данном примере служит Linux.

Машинный код управляет процессором на максимально подробном уровне. В число выполняемых машинным кодом функций входят:

• перемещение данных в память и из памяти;
• перемещение данных в регистры и из регистров;
• управление системной шиной;
• управление АЛУ, устройством управления и другими компонентами.

Благодаря такому низкоуровневому управлению приложения, написанные в машинном коде, могут быть невероятно мощными и эффективными. Однако, хотя запоминать и вводить разнообразные последовательности битов для выполнения определённых задач весьма впечатляюще, это неэффективно и чревато ошибками.

От машинного кода к языку ассемблера

В машинном коде последовательность битов обозначает определённое действие. Например, `0x81`, или `10000001`, - это инструкция, которая складывает два значения и сохраняет результат в определённом месте.

Код на языке ассемблера задуман как понятная человеку версия машинного кода. Вместо того чтобы запоминать двоичную или шестнадцатеричную строку наподобие `0x81` или `10000001`, программист может использовать команду `add`. Мнемонике `add` соответствует код `0x81`, поэтому такая сокращённая запись упрощает программирование, не лишая его преимуществ непосредственной работы с машинным кодом.

Преобразование машинного кода в код на языке ассемблера значительно облегчает его понимание. Например, код приведённой выше программы «hello world» можно преобразовать в последовательность понятных инструкций.

Машинный код	**Язык ассемблера**
`55`	`push ebp`
`89 e5`	`mov ebp,esp`
`83 e4 f0`	`and esp, 0xfffffff0`
`83 ec 10`	`sub esp, 0x10`
`b8 b0 84 04 08`	`mov eax`
`89 04 24`	`mov [esp], eax`
`e8 1a ff ff ff`	`call 80482f4`
`b8 00 00 00 00`	`mov eax, 0x0`
`c9`	`leave`
`c3`	`ret`
`90`	`nop`

Если вы понимаете машинный код, писать непосредственно на нём может быть интересно, а в некоторых случаях и оправданно. Однако в подавляющем большинстве ситуаций это неэффективно и непрактично. Программирование на языке ассемблера даёт те же преимущества,

что и программирование в машинном коде, но гораздо практичнее.

После написания кода на языке ассемблера его можно преобразовать в машинный код с помощью ассемблера; этот процесс называется ассемблированием. Программу, уже представленную машинным кодом, можно преобразовать в код на языке ассемблера с помощью дизассемблера.

> **ОПРЕДЕЛЕНИЕ.** Ассемблеры преобразуют код на языке ассемблера в машинный код. Дизассемблеры преобразуют машинный код в код на языке ассемблера.

Многие программисты не пишут на машинном коде или языке ассемблера. Вместо этого они применяют языки более высокого уровня, которые скрывают больше подробностей. Например, приведённый ниже псевдокод похож на многие процедурные языки высокого уровня.

```
int x=1, y=2, z=x+y;
```

В процессе компиляции такой код на языке высокого уровня преобразуется в код на языке ассемблера, подобный следующему:

```
mov   [ebp-4], 0x1
mov   [ebp-8], 0x2
mov   eax, [ebp-8]
mov   edx, [ebp-4]
lea   eax, [edx+1*eax]
mov   [ebp-0xc], eax
```

Затем ассемблер может преобразовать код на языке ассемблера в следующий машинный код, пригодный для выполнения компьютером:

```
c7 45 fc 01 00 00 00 c7 45 f8 02 00 00 00 8b 45 f8 8b 55 fc 8d 04 02
89 45 f4
```

Архитектуры наборов инструкций и микроархитектуры

Слово «компьютер» охватывает широкий спектр систем. Умные часы и настольный компьютер работают по сходным принципам. Однако их внутренние компоненты могут значительно различаться.

Архитектура набора инструкций (instruction set architecture, ISA) описывает среду, в которой выполняются программы. В число характеристик, определяемых ISA, входят следующие:

• **Регистры.** ISA указывает, имеется ли у процессора один регистр или сотни регистров. Она также определяет размер этих регистров: содержат ли они 8 или 128 битов.
• **Адреса и форматы данных.** ISA задаёт формат адресов, используемых для обращения к данным в памяти. Она также определяет, сколько байтов система способна получить из памяти за один раз.
• **Машинные инструкции.** Различные ISA могут поддерживать разные наборы инструкций. ISA определяет наличие инструкций сложения, вычитания, проверки равенства, останова и других операций.

Определяя возможности физической системы, ISA косвенно определяет и язык ассемблера. Именно ISA задаёт доступные низкоуровневые инструкции и выполняемые ими действия.

Микроархитектура описывает способ реализации конкретной ISA в процессоре. На рисунке 1.4 приведён пример архитектуры Intel Core 2.

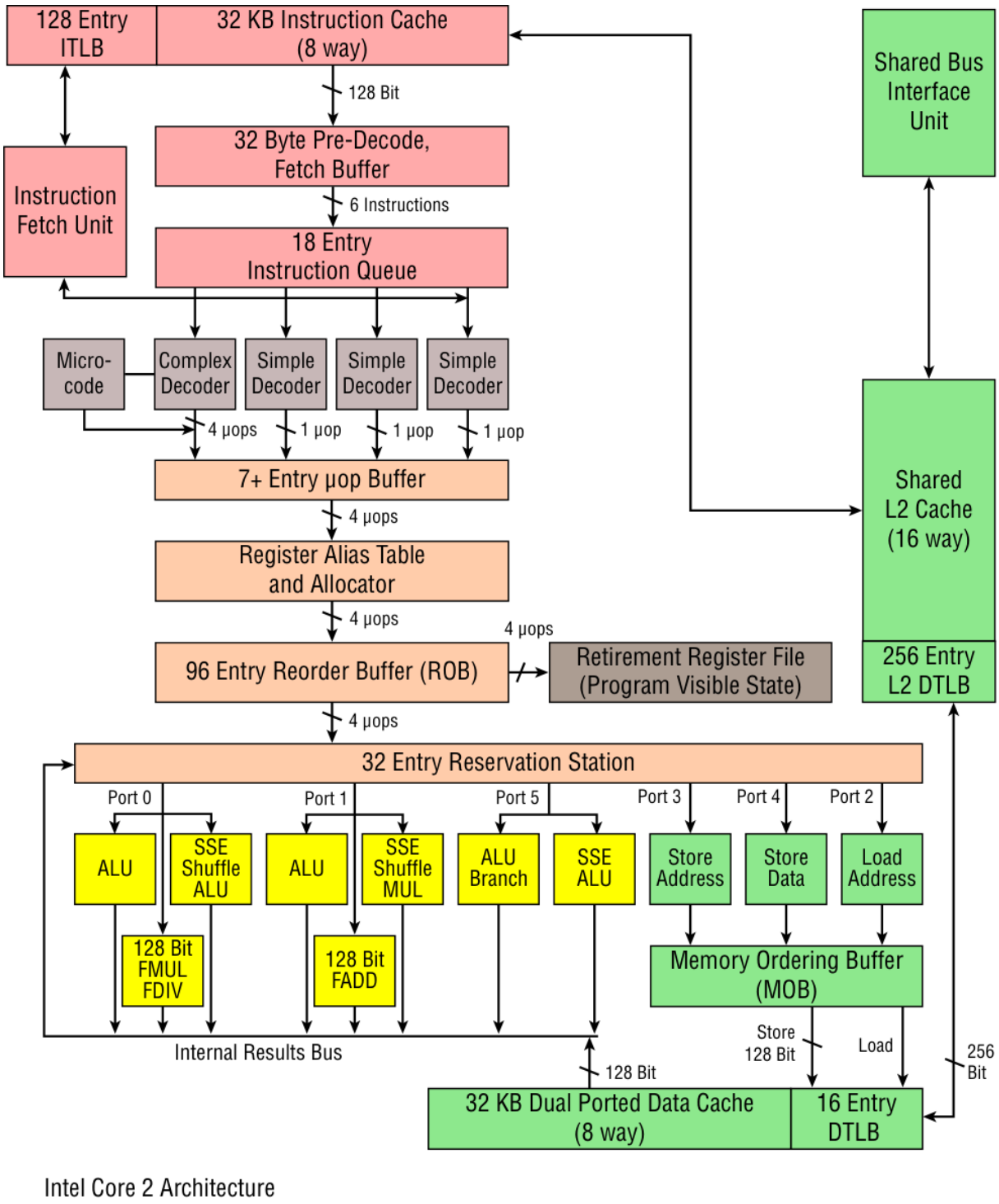


Рисунок 1.4. Архитектура Intel Core 2

ISA и микроархитектура вместе определяют архитектуру компьютера. Существование тысяч ISA и тысяч микроархитектур означает, что существуют и тысячи компьютерных архитектур.

> **ОПРЕДЕЛЕНИЕ.** Архитектура набора инструкций определяет работу регистров, адресов, форматов данных и машинных инструкций. Микроархитектуры реализуют ISA в процессоре. ISA и микроархитектура вместе определяют архитектуру компьютера.

Компьютерные архитектуры RISC и CISC

Хотя существуют тысячи компьютерных архитектур, в широком смысле их можно разделить на две основные категории. Архитектуры вычислений с сокращённым набором инструкций (reduced instruction set computing, RISC) определяют небольшое число более простых инструкций. Как правило, RISC-архитектуры дешевле и проще в создании, а аппаратные средства физически меньше и потребляют меньше энергии.

В противоположность им архитектура вычислений с полным набором инструкций (complex instruction set computing, CISC) определяет большее число более мощных инструкций. Процессоры CISC дороже и сложнее в создании, обычно они крупнее и потребляют больше энергии.

Хотя может показаться, что CISC-архитектуры объективно хуже RISC-архитектур, их главное преимущество заключается в простоте и эффективности программирования. Рассмотрим, например, гипотетический случай, в котором программе требуется умножить значение на 5 в системах RISC и CISC.

CISC	**RISC**
`mul [100], 5`	`load r0, 100`
	`mov r1, r0`
	`add r1, r0`
	`add r1, r0`
	`add r1, r0`
	`add r1, r0`
	`mov [100], r1`

В этом примере процессор CISC способен выполнить вычисление одной инструкцией, если располагает инструкцией умножения, которая может загрузить значение из памяти, умножить его и сохранить результат по тому же адресу памяти. В процессоре RISC инструкции умножения может не быть, поскольку это сложная операция. Вместо неё процессор RISC загружает значение из памяти, четыре раза складывает его с самим собой и сохраняет результат по тому же адресу памяти, выполняя в общей сложности семь шагов.

Архитектуры RISC и CISC обладают собственными преимуществами, недостатками и областями применения. Например, процессору RISC может потребоваться 100 инструкций для выполнения той же операции, которую процессор CISC способен выполнить одной инструкцией. Однако единственная операция CISC может выполняться в 100 раз дольше или потреблять в 100 раз больше энергии.

Сегодня широко используются оба набора инструкций, RISC и CISC. Ниже приведены некоторые примеры распространённых RISC-архитектур:

• ARM (используется в телефонах и планшетах);
• MIPS (используется во встраиваемых системах и сетевом оборудовании);
• PowerPC (используется в первоначальных компьютерах Mac и Xbox 360).

В этой книге основное внимание уделяется языку ассемблера x86, относящемуся к архитектуре CISC. Эта архитектура применяется во всех современных персональных компьютерах и серверах, поддерживается всеми основными операционными системами (Windows, Mac, Linux) и даже некоторыми игровыми системами, например Xbox One. Поэтому её изучение особенно полезно для

взлома программного обеспечения.

Итоги

Машинный код, который в действительности выполняется компьютерами, не предназначен для чтения и понимания человеком. Чтобы человек мог с ним работать, его необходимо преобразовать в другую форму.

Одним из вариантов такого преобразования служит декомпиляция, результат которой подобен первоначальному исходному коду или совпадает с ним. Однако декомпиляция возможна не всегда.

Для полностью компилируемых языков, таких как C/C++, и многих других языков необходимо дизассемблировать скомпилированный исполняемый файл и анализировать его на языке ассемблера. В сравнении с написанием и чтением кода на языке высокого уровня это требует гораздо более глубокого понимания архитектуры компьютера и принципов его фактической работы. Теперь, когда мы знаем, какую роль способна играть декомпиляция и зачем нужно дизассемблирование, в нескольких следующих главах мы рассмотрим работу компьютеров, чтобы научиться выполнять дизассемблирование на профессиональном уровне.

Глава 2. Ассемблер x86: данные, режимы, регистры и доступ к памяти

Для обратной разработки программного обеспечения чаще всего требуется дизассемблировать скомпилированный исполняемый файл и проанализировать результат. Результатом дизассемблирования становится код на языке ассемблера, а не на языке более высокого уровня.

Существует несколько языков ассемблера, и x86 относится к числу самых распространённых. В этой главе представлены некоторые ключевые понятия ассемблера x86, которые послужат основой для последующих глав.

Введение в x86

Существуют тысячи компьютерных архитектур. Хотя все они работают сходным образом - компьютер остаётся компьютером, - между ними имеются незначительные или существенные различия.

Для изучения обратной разработки необходимо выбрать архитектуру, на которой мы сосредоточимся. В этой книге будет использоваться x86, выбранная по нескольким причинам:

• **Повсеместное распространение.** x86 - самый распространённый язык ассемблера, поэтому он широко применим в обратной разработке.
• **Поддержка компьютерами.** Приложения x86 можно собирать, запускать и подвергать обратной разработке на любом настольном компьютере, ноутбуке или сервере.
• **Доля рынка.** x86 лежит в основе основных операционных систем (Windows, Linux и macOS), поэтому применяется в миллиардах систем.

Архитектура x86 существует уже несколько десятилетий и за это время значительно развилась. Впервые она была представлена Intel в 1974 году; ниже перечислены некоторые основные вехи в истории x86:

• **Intel 8080:** 8-разрядный микропроцессор, представленный в 1974 году;
• **Intel 8086:** 16-разрядный микропроцессор, представленный в 1978 году;
• **Intel 80386:** 32-разрядный микропроцессор, представленный в 1985 году;
• **Intel Prescott, AMD Opteron и Athlon 64:** 64-разрядные микропроцессоры, представленные в 2003/2004 годах.

На протяжении почти 50-летней истории в архитектуру x86 регулярно добавлялись новые возможности при сохранении обратной совместимости. Даже если выяснялось, что какая-либо возможность не используется, её никогда не удаляли из системы. В результате программы, написанные для процессора Intel 8086, выпущенного в 1978 году, по-прежнему могут без изменений работать на новейших микросхемах x86.

Такая ориентация на обратную совместимость породила огромную, сложную и интересную архитектуру. Последнее руководство Intel для разработчиков программного обеспечения, доступное по адресу www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html, насчитывает более 5000 страниц и лишь начинает раскрывать возможности этой архитектуры. В данной книге основное внимание уделяется пониманию основ x86, а этого достаточно, чтобы читать, писать и изменять большую часть x86-кода.

По мере развития архитектуры x86 термин x86 превратился в обобщающее название всех архитектур, произошедших от 16-разрядной архитектуры Intel 8086. К ним относится архитектура

Intel 80286, включающая как 16-разрядные, так и 32-разрядные архитектуры, а также архитектура Intel 80886, в которой добавлена 64-разрядная архитектура. Термин x64 обозначает именно 64-разрядную версию x86.

В примерах этой книги будет показана 32-разрядная архитектура x86. Все понятия из 32-разрядной x86 в точности переносятся на x64. Во время обучения работать с 32-разрядными примерами существенно проще, чем с 64-разрядными. Изучив по этой книге 32-разрядную x86, вы сразу сможете посмотреть на ассемблерный код x64 и понять его. При этом ваши глаза будут благодарны за то, что им не приходится постоянно разглядывать 64 бита, ведь даже на 32 бита смотреть несколько утомительно. Поэтому не беспокойтесь, что 32-разрядные примеры устарели или что с самого начала следует сосредоточиться на 64-разрядной архитектуре. Мы оба вначале изучали 32-разрядную архитектуру, много преподавали взлом программного обеспечения и можем уверенно сказать: если сначала заложить прочную 32-разрядную основу, 64-разрядная архитектура сведётся лишь к нескольким дополнительным именам регистров и более длинным значениям.

Синтаксис языка ассемблера

Выбрать x86 из тысяч возможных компьютерных архитектур важно, но этого недостаточно. Архитектура набора инструкций (instruction set architecture, ISA) определяет такие характеристики, как регистры, формат данных и машинные инструкции, но не задаёт синтаксис.

Если язык ассемблера соблюдает все правила работы с регистрами, адресацией и прочими элементами и определяет правильный набор инструкций, он является допустимым языком x86. Например, в языке x86 должна присутствовать операция умножения. Однако её мнемоникой в разных языках может быть `mul`, `MUL`, `multiply` или любой другой вариант.

Синтаксис языка ассемблера полностью определяется ассемблером. Не существует стандартного синтаксиса ни для языка ассемблера в целом, ни для ассемблера x86 в частности. В результате имеются сотни различных вариантов.

Тем не менее существует два распространённых варианта синтаксиса x86, которые используются большинством инструментов для работы с ассемблером x86: синтаксис AT&T и синтаксис Intel. В каждой из этих двух основных ветвей существуют сотни вариантов, зависящих от конкретного ассемблера.

Хотя ассемблеры Intel и AT&T относятся к x86, внешне они сильно различаются. Рассмотрим, например, команду, предназначенную для перемещения содержимого памяти по адресу `ebx+4*ecx+2020` в регистр `eax`.

В синтаксисах Intel и AT&T эта инструкция выглядит совершенно по-разному:

Синтаксис Intel	**Синтаксис AT&T**
`mov eax, [ebx+4*ecx+2020]`	`mov 0x7e4(%ebx,%ecx,4),%eax`

В синтаксисе Intel после инструкции `mov` указывается место, где будет сохранён результат. Доступ к памяти обозначается квадратными скобками, и вычисление адреса памяти `[ebx+4*ecx+2020]` выполняется внутри этих скобок.

Синтаксис AT&T отличается от синтаксиса Intel по нескольким признакам:

• **Порядок.** Аргументы меняются местами, поэтому место назначения указывается вторым.
• **Регистры.** В AT&T регистры обозначаются знаком процента (%), а в Intel этот знак не используется.

• **Доступ к памяти.** В AT&T доступ к памяти обозначается круглыми скобками, а в Intel - квадратными.
• **Вычисление.** Вычисление требуемого адреса памяти выглядит в синтаксисах AT&T и Intel совершенно по-разному.
• **Инструкции.** Хотя в данном примере это не показано, в AT&T часто применяются иные и более длинные мнемоники инструкций, чем в Intel.

Ради ясности и единообразия для примеров в этой книге выбран синтаксис Intel. Ниже приведены некоторые причины выбора Intel вместо AT&T:

• **Поддержка Intel.** Intel является ведущим разработчиком процессоров и использует синтаксис Intel.
• **Применение в инструментах.** Большинство основных средств обратной разработки, например IDA, используют синтаксис Intel.
• **Удобочитаемость.** Синтаксис Intel широко считается более ясным и простым для чтения и написания, чем синтаксис AT&T.

Представление данных

В отличие от людей, компьютеры работают с двоичными значениями, поэтому большинство средств обратной разработки не отображают числа в системе счисления с основанием 10. Чтобы понять действия приложения, необходимо понимать обрабатываемые им данные и способы их представления.

Основания систем счисления

Основание системы счисления определяет число символов, используемых для представления значения цифры. Большинство людей выполняют вычисления в системе с основанием 10, где применяются символы 0, 1, 2, 3, 4, 5, 6, 7, 8 и 9.

Однако это не единственный вариант. Можно использовать любое основание при наличии достаточного числа символов для представления значений. Например, в системе с основанием 5 используются символы 0-5, а в системе с основанием 8, также называемой восьмеричной, - символы 0-7.

> **СОВЕТ.** Основание системы, в которой записано число, может обозначаться нижним индексом. Например, число 10₁₀ записано в системе с основанием 10, а 10₂ является двоичным числом, то есть числом с основанием 2.

Для оснований больше 10 в качестве символов также используются буквы. Например, в системе с основанием 11 добавляется буква a, и применяются символы 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 и a. В системе с основанием 16, также называемой шестнадцатеричной, используются символы 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e и f.

> **СОВЕТ.** В системе с основанием 16, то есть шестнадцатеричной, регистр букв не имеет значения, поэтому и a, и A соответствуют значению 10 в десятичной системе.

В любой системе счисления необходимо уметь представлять значения, превышающие основание. Для этого используются несколько цифр. Например, счёт в системе с основанием 10 выглядит так: ... 8, 9, 10, 11, ..., 98, 99, 100, 101, ... В системе с основанием 16 он выглядит так: ... 8, 9, a, b, c, d, e, f, 10, 11, ... 19, 1a, ... 1f, 20, ...

Компьютеры являются двоичными системами и хранят и обрабатывают все данные с помощью единиц и нулей. Однако такая запись неэффективна и быстро становится громоздкой. Например, значение 2014₁₀ равно 11111011110₂.

Хотя компьютеры работают с двоичными данными, для удобства чтения инструменты часто отображают значения в шестнадцатеричной форме, или «hex». Шестнадцатеричные значения могут обозначаться нижним индексом (1d₁₆), префиксом `0x` (`0x1d`) или суффиксом `h` (`1dh`).

Одно из преимуществ шестнадцатеричной системы с основанием 16 состоит в том, что 16 является степенью числа 2. Поэтому значения можно легко преобразовывать между двоичной и шестнадцатеричной формами простой заменой символов. На рисунке 2.1 показано соответствие каждого шестнадцатеричного символа значениям в десятичной системе и системе с основанием 2, то есть двоичной.

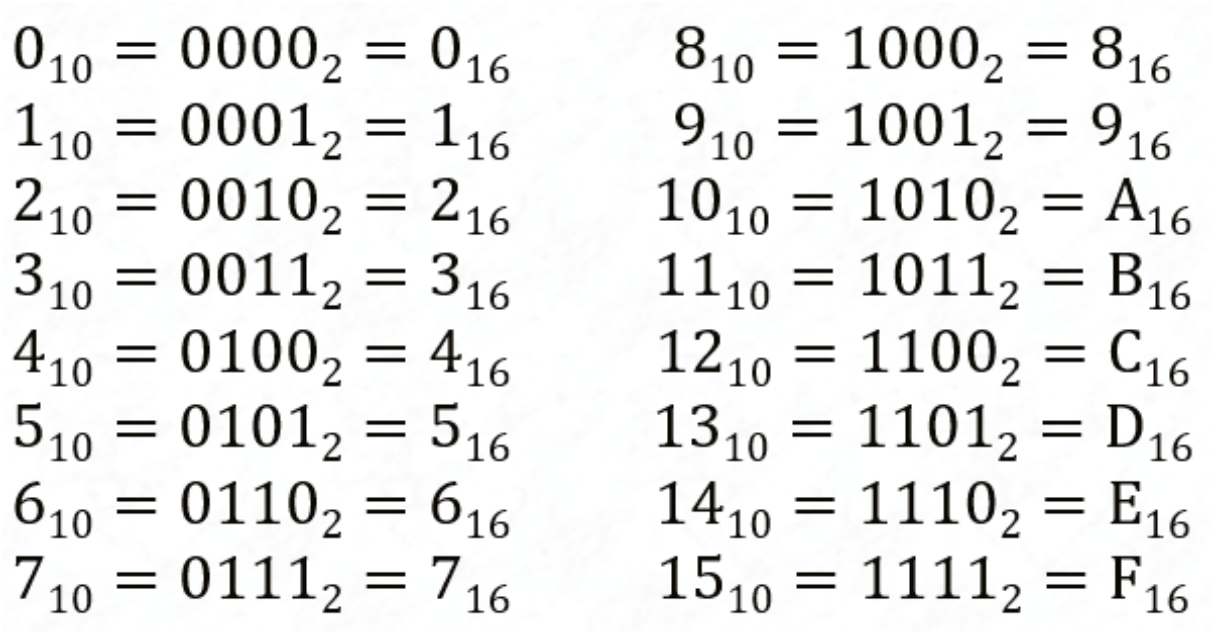


Рисунок 2.1. Шестнадцатеричная система

Рассмотрим, например, двоичное значение 11111011110₂. Каждая шестнадцатеричная цифра представляет четыре двоичные цифры, поэтому это значение можно разбить справа налево на три группы: 111, 1101 и 1110. Из рисунка видно, что эти группы соответствуют шестнадцатеричным цифрам 7, d и e, а всё значение можно представить как `0x7de`.

Значение	**Пояснение**
11111011110₂	Двоичное число
111 1101 1110₂	Разбиение справа налево на 4-битовые полубайты
7 d e	Преобразование каждого полубайта в шестнадцатеричную цифру
`0x7de`	Полученное шестнадцатеричное значение

Хотя такие преобразования можно выполнять вручную, зачастую быстрее и точнее воспользоваться инструментом. На рисунке 2.2 показан пример преобразования между системами

счисления с помощью калькулятора Windows.

Биты, байты и слова

Биты являются базовыми единицами, используемыми компьютерами. Однако они слишком малы, чтобы быть особенно полезными. Вместо обращения к отдельным битам и их обработки компьютеры используют в качестве минимальной единицы памяти байты. Во всех современных системах байт содержит 8 битов.

Хотя байты крупнее битов, для многих операций они тоже слишком малы. Компьютеры проектируются так, чтобы оптимально обращаться к определённому числу байтов за один раз. Это число байтов называется словом, обычно является степенью числа 2 и может различаться в разных компьютерах. Например, микроконтроллеры имеют небольшой размер слова, часто равный 1 или 2 байтам (8 или 16 битам). В компьютерах общего назначения размер слова обычно равен 4 или 8 байтам (32 или 64 битам).

Биты, байты и слова - самые важные термины при работе с памятью, но ими список не ограничивается. Ниже приведён полный перечень распространённых терминов:

• **Бит:** двоичная цифра, 0 или 1;
• **Байт:** 8 битов;
• **Полубайт:** 4 бита;
• **Двойной байт:** 16 битов;
• **Четверной байт:** 32 бита;
• **Слово:** зависящее от архитектуры количество байтов;
• **Полуслово:** половина слова;
• **Двойное слово (DWORD):** два слова;
• **Четверное слово (QWORD):** четыре слова;
• **Восьмерное слово, двойное четверное слово (DQWORD):** восемь слов.

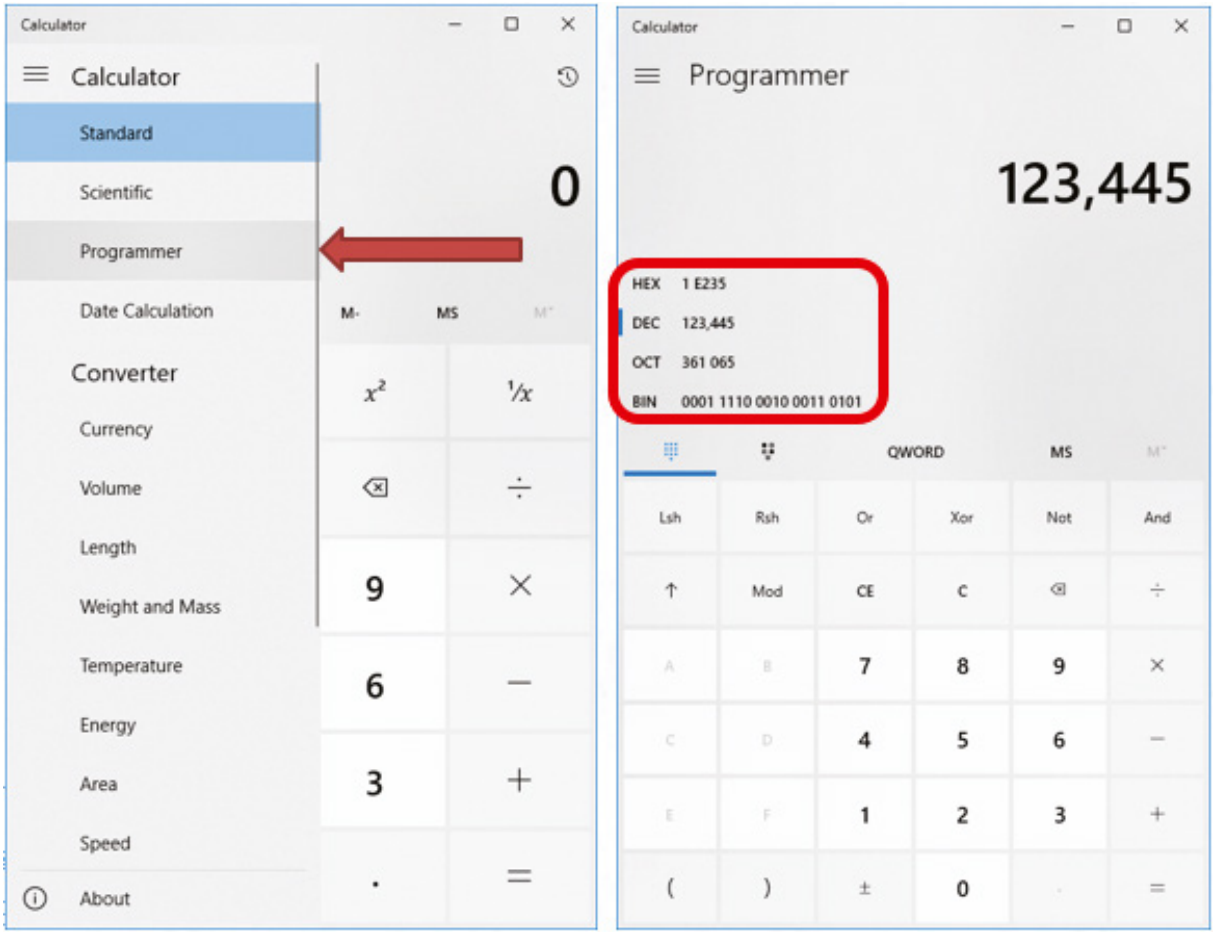


Рисунок 2.2. Преобразование систем счисления в калькуляторе Windows

В этой книге рассматривается 32-разрядная архитектура. В традиционной 32-разрядной архитектуре это означало бы, что размер слова равен 32 битам. Но здесь проявляется особенность архитектуры x86. Поскольку x86 сохранила обратную совместимость с первоначальной 16-разрядной архитектурой, слово в x86 содержит 16 битов, а двойное слово - 32 бита.

СОВЕТ. В 32-разрядной x86 байт содержит 8 битов, а двойное слово - 32 бита.

Работа с двоичными значениями

При обратной разработке часто приходится работать с большими двоичными числами, занимающими несколько разных байтов. Для правильной интерпретации числа, представленного двоичной строкой, необходимо понимать такие понятия, как расширение нулями, значимость битов и байтов и порядок байтов.

Расширение нулями и удобочитаемость

Двоичные значения часто дополняются или расширяются нулями до размера слова соответствующей архитектуры. В 32-разрядной архитектуре к значению слева добавляются нули, пока его длина не достигнет 32 битов. Например, значение 11001₂ будет дополнено нулями до 00000000 00000000 00000000 00011001₂.

Обратите внимание, что для повышения удобочитаемости биты также разделяются на группы по четыре или восемь. Это подобно тому, как в десятичной системе иногда через каждые три цифры добавляют запятые, например записывают 1,000 вместо 1000. Шестнадцатеричные значения также группируются по байтам, по два символа на байт. Например, значение 4D2₁₆, эквивалентное 1234₁₀, можно записать как 04₁₆ D2₁₆.

Значимость битов и байтов

Биты и байты двоичного числа могут обозначаться в соответствии с их относительным весом в числе. На рисунке 2.3 показаны некоторые распространённые обозначения.

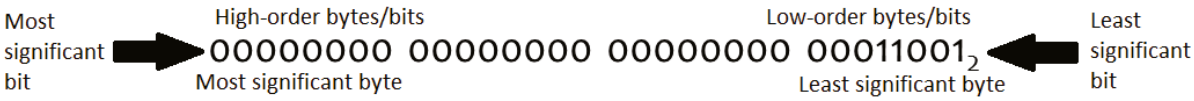


Рисунок 2.3. Обозначения значимости битов и байтов

В значении 00000000 00000000 00000000 00011001₂ младшим значащим битом (least significant bit, LSB) является крайний правый бит со значением 1. Старший значащий бит (most significant bit, MSB) находится слева и имеет значение 0. При преобразовании двоичного числа в десятичное MSB умножается на 2³¹, а LSB - на 2⁰.

Помимо MSB и LSB существуют также понятия младшего и старшего значащих байтов. В значении 00000000 00000000 00000000 00011001₂ младший значащий байт имеет значение 00011001₂, а старший значащий байт - 00000000₂.

Биты и байты могут также обозначаться по близости к концам значения. Например, биты и байты рядом с LSB называются младшими, а биты и байты рядом с MSB - старшими.

Порядок байтов

В памяти данные хранятся в байтах. Однако многие типы данных занимают несколько байтов. Например, значение типа `int` содержит 32 бита, или 4 байта.

Порядок байтов описывает последовательность размещения этих байтов в памяти. В системе с порядком байтов от младшего к старшему (little-endian) первым, то есть по наименьшему адресу, хранится младший значащий байт. В системе с порядком байтов от старшего к младшему (big-endian) первым, то есть по наименьшему адресу, хранится старший значащий байт.

Рассмотрим, например, значение 1337₁₀, которое в двоичной форме равно 0000 0000 0000 0000 0000 0101 0011 1001₂, а в шестнадцатеричной - `0x00000539`. На рисунке 2.4 показано, как эти значения хранились бы в памяти.

Address	1828	1829	1830	1831
Little Endian	0x39	0x05	0x00	0x00
Big Endian	0x00	0x00	0x05	0x39

Рисунок 2.4. Порядок байтов

> **ОПРЕДЕЛЕНИЕ.** В системе с порядком байтов от младшего к старшему младший значащий байт расположен по наименьшему адресу. В системе с порядком байтов от старшего к младшему по наименьшему адресу расположен старший значащий байт.

Независимо от порядка байтов системы адресом, связанным с переменной, является наименьший используемый, или базовый, адрес. В данном примере и при порядке от младшего к старшему, и при порядке от старшего к младшему это адрес 1828.

В этой книге рассматривается архитектура x86, использующая порядок байтов от младшего к старшему. Поэтому младший значащий байт порции данных будет находиться по смещению 0 от базового адреса. Вы заметите, что для человека такой порядок выглядит «обратным», поскольку мы читаем и пишем от старшего байта к младшему.

> **СОВЕТ.** x86 является архитектурой с порядком байтов от младшего к старшему, поэтому по наименьшему адресу содержится младший значащий бит.

Регистры

Регистры обеспечивают процессору высокоскоростной доступ к данным. Поскольку регистры физически находятся внутри центрального процессора, задержка обращения к ним гораздо меньше, чем к памяти, запросы к которой должны проходить по шинам и через мосты.

В 32-разрядной архитектуре регистр содержит 32 бита данных, и с ним можно обращаться как с переменной в программе. У каждого регистра есть уникальное имя, а находящиеся в нём данные можно изменять путём вычислений или загрузки новых значений из памяти.

Главное ограничение регистров состоит в том, что их число невелико и они должны совместно использоваться всей программой. Если программе не хватает регистров, ей приходится начинать сохранять сведения в памяти, что отрицательно сказывается на производительности. При ограниченном числе регистров обычный цикл выполнения выглядит следующим образом:

• получить данные из памяти и сохранить их в регистрах;
• обработать данные;
• сохранить данные обратно в память;
• повторить цикл.

Регистры в x86

Как уже упоминалось в обзоре архитектуры, регистры - это имеющие специальные имена места в центральном процессоре, которые позволяют очень быстро выполнять операции. Все регистры можно разделить на две отдельные категории:

• **Регистры общего назначения (general-purpose registers, GPR):** используются для обычного хранения данных, адресов и прочего, и ими можно непосредственно манипулировать;
• **Регистры специального назначения (special-purpose registers, SPR):** используются для хранения состояния программы.

Архитектура x86 определяет множество регистров, показанных на рисунке 2.5. Однако многие из них зарезервированы для использования самим центральным процессором, поэтому знать необходимо лишь их подмножество.

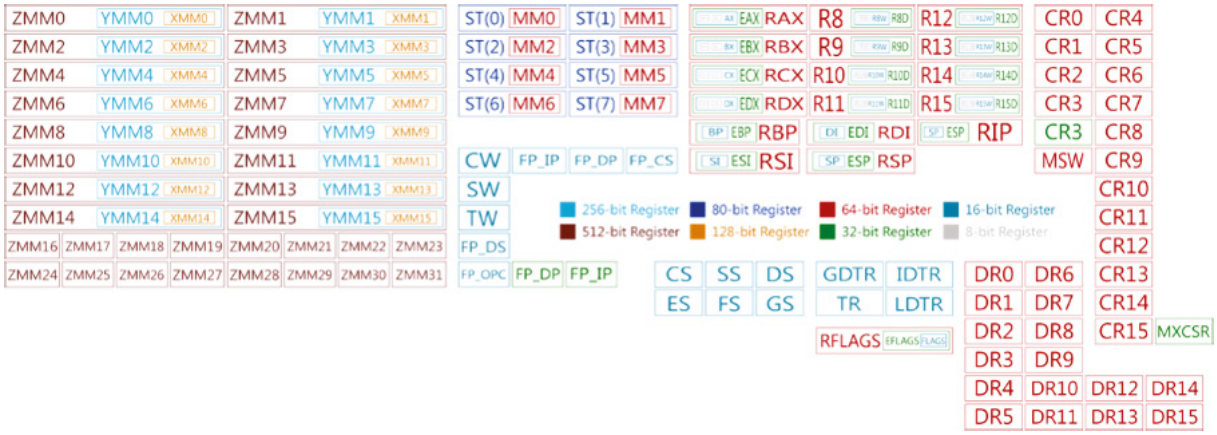


Рисунок 2.5. Регистры x86

Источник: Liam McSherry / Wikimedia Commons / CC BY-SA 3.0.

Регистры общего назначения x86

Регистры общего назначения выполняют в приложении основную работу: хранят значения, полученные из памяти, участвуют в операциях с данными и сохраняют результаты вычислений. Ниже перечислены наиболее важные регистры общего назначения x86, каждый из которых может хранить 32 бита данных. Каждый аккумулятор имеет традиционную роль, в честь которой он назван; однако регистры общего назначения можно использовать для любых целей, и счётчик можно поместить в любой регистр, а не только в `ecx`.

eax

`eax` - регистр-«аккумулятор». Его название связано с тем, что он часто используется для хранения результата арифметической операции. Например, программа может выполнить вычисление `eax += ebx`.

ebx

`ebx` - «базовый» регистр. Он часто используется для хранения базового адреса области памяти, в которой находится переменная. Например, выражение `[ebx + 5]` можно использовать для доступа к пятому элементу массива.

ecx

`ecx` - регистр-«счётчик», который традиционно используется для подсчёта. Например, `ecx` может отслеживать текущую итерацию цикла. В команде `for (i=0; i<10; i++)` переменная `i`, скорее всего, будет храниться в регистре `ecx`.

edx

`edx` - регистр «данных». Его название связано с тем, что он часто используется для хранения данных. Например, приложение может содержать инструкцию `sub edx, 7`.

esi

`esi` - регистр «индекса источника». Он традиционно используется для хранения индекса в исходном массиве. Например, в команде `array[i] = array[k]` значение `k`, скорее всего, будет храниться в `esi`.

edi

`edi` - регистр «индекса назначения». Он используется для хранения индекса в массиве назначения. Например, в команде `array[i] = array[k]` значение `i`, скорее всего, будет храниться в `edi`.

ebp

`ebp` - регистр «указателя базы». Его назначение состоит в хранении адреса основания текущего кадра стека. Понятия программного стека и кадров стека будут рассмотрены в последующих главах.

esp

`esp` - регистр «указателя стека». Он хранит адрес вершины текущего кадра стека.

Регистры специального назначения

Регистры специального назначения предназначены для конкретных задач, и непосредственно изменять их нельзя. Например, инструкция `mov eip, 1`, использующая регистр специального назначения, не будет ассемблирована, тогда как `mov eax, 1`, использующая регистр общего назначения, будет.

eip

`eip` - регистр «указателя инструкций». Он хранит адрес следующей инструкции, которая должна быть выполнена.

eflags

`eflags` - регистр «флагов». В нём хранятся «флаги», принимающие истинное или ложное значение и содержащие сведения о состоянии системы и результатах ранее выполненных инструкций.

> **СОВЕТ.** Регистры общего назначения доступны как для чтения, так и для записи, а регистры специального назначения - только для чтения.

Работа с регистрами

В языке ассемблера с регистрами общего назначения можно обращаться точно так же, как с переменными, используя их имена. Например, инструкция `mov eax, 1` сохраняет значение 1 в `eax`, а `add eax, ebx` прибавляет содержимое `eax` к `ebx`.

Обратите внимание, что все эти имена регистров начинаются с буквы `e`. Причина в том, что данные 32-разрядные регистры были «расширены» из первоначальных 16-разрядных регистров.

К младшей половине содержимого регистра можно обратиться, убрав из имени букву `e`. Например, регистр `ax` содержит младшие 16 битов регистра `eax`.

Если имя регистра заканчивается на `x` (`eax`, `ebx`, `ecx` и `edx`), этот 16-разрядный регистр можно дополнительно разделить на два 8-разрядных регистра, которые обозначаются буквами `l` и `h`. Регистр `al` содержит младшие 8 битов регистра `ax`, а `ah` - старшие 8 битов. Это показано на рисунке 2.6, где `eax=0x01234567`, `ax=0x4567`, `ah=0x45`, а `al=0x67`.

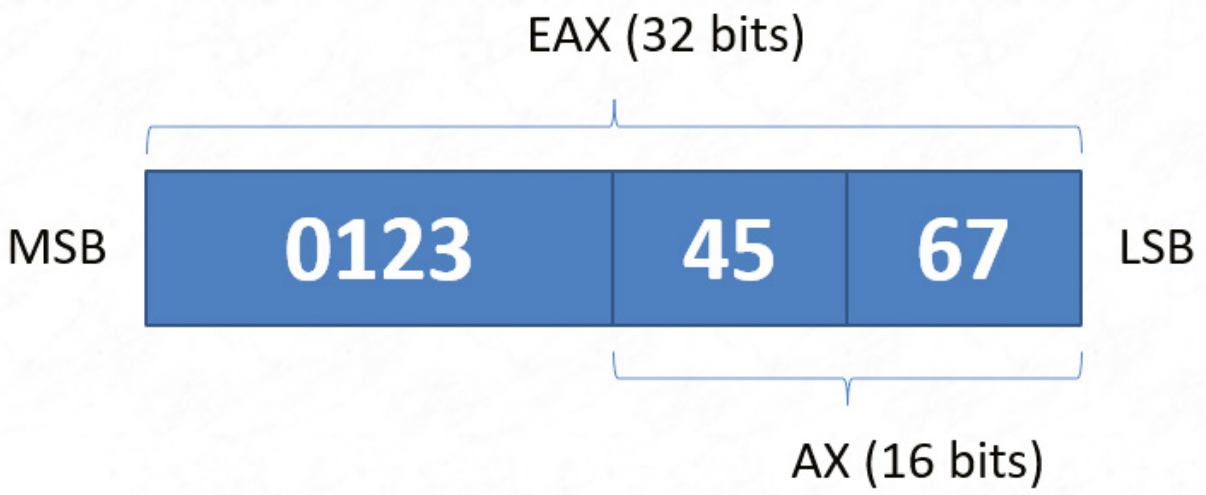


Рисунок 2.6. Части регистра eax

64-разрядные регистры

В 64-разрядной x86 все инструкции и поведение остаются такими же, как в 32-разрядной x86. Однако 64-разрядные архитектуры располагают большим числом регистров увеличенного размера.

На рисунке 2.7 показаны часто используемые регистры в 64-разрядной x86. В дополнение к иначе представленным 32-разрядным регистрам 64-разрядные архитектуры включают ещё восемь регистров с именами `r8`-`r15`.

Все 64-разрядные регистры также крупнее своих 32-разрядных аналогов. Для регистров, существующих в 32-разрядной x86, например `eax`, полное имя 64-разрядного аналога получается заменой `e` на `r`, и регистр называется `rax`. Доступ к младшим 32 битам регистра осуществляется по 32-разрядному имени, а использование таких имён, как `ax`, `al` и `ah`, остаётся неизменным.

Для новых регистров наподобие `r8` 64-разрядная x86 позволяет обращаться к младшим 32, 16 и 8 битам. Они обозначаются соответственно буквами `d` (`r8d`), `w` (`r8w`) и `b` (`r8b`), как показано на рисунке 2.8.

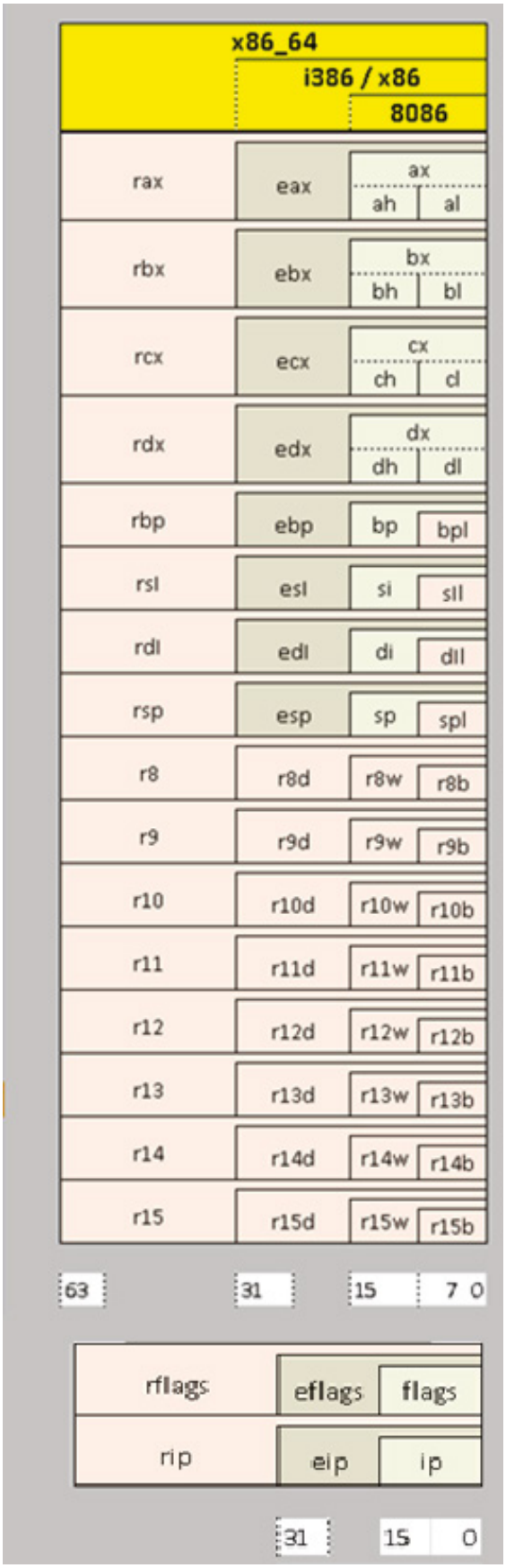


Рисунок 2.7. Распространённые регистры x64

Источник: Bobmon / Wikimedia Commons / CC BY-SA 4.0.

Доступ к памяти

В 32-разрядной или 64-разрядной системе доступно лишь ограниченное число регистров. Если не учитывать регистры специального назначения и регистры общего назначения, отслеживающие стек (`esp` и `ebp`), для вычислений общего назначения остаются только шесть регистров (`eax`, `ebx`, `ecx`, `edx`, `esi` и `edi`). Этого недостаточно для сколько-нибудь значительной работы, поэтому программа должна также уметь читать данные из памяти и записывать их туда.

В синтаксисе Intel для ассемблера x86 доступ к памяти обозначается квадратными скобками `[]`. Например, к данным по адресу `0x12345678` можно обратиться с помощью записи `[0x12345678]`. Адреса памяти также можно хранить в регистрах, как в записи `[eax]`.

Указание длины данных

При обращении к данным в памяти необходимо знать не только их адрес, но и размер области памяти, к которой следует обратиться. Например, запись `[0x12345678]` не указывает, требуется ли программе байт, слово, двойное слово или больший объём данных.

В некоторых случаях длину данных, к которым выполняется обращение, можно вывести из контекста. Например, в инструкции `mov eax, [0x12345678]` данные, полученные из памяти, будут сохранены в `eax`. Поскольку `eax` является 32-разрядным регистром, программа должна запрашивать 32 бита данных.

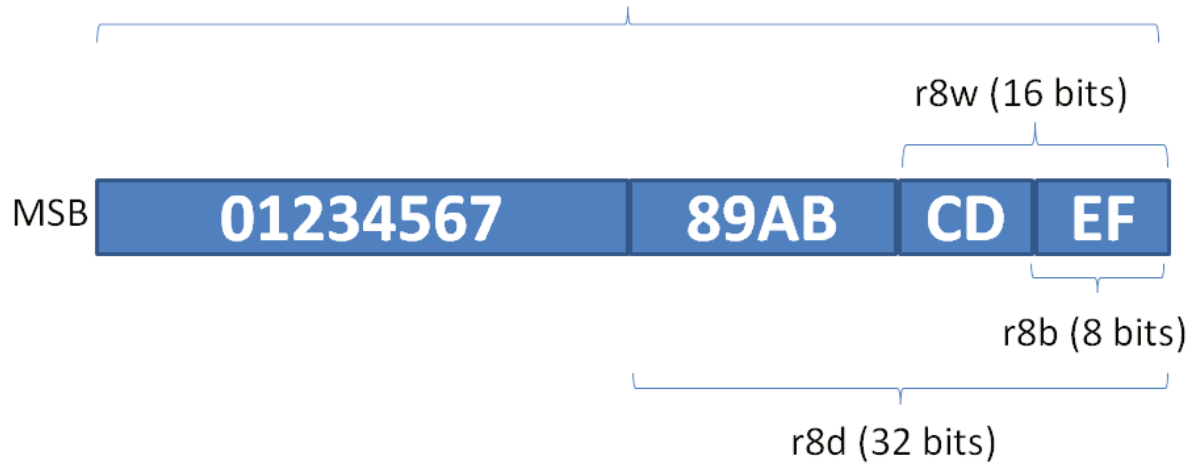


Рисунок 2.8. Части регистра r8

Так происходит не всегда. Рассмотрим, например, команду `mov [0x12345678], 1`, которая помещает значение 1 по определённому адресу памяти. Однако эта инструкция не указывает длину записываемого значения. Следует ли считать 1 байтом (`0000 0001`), словом (`0000 0000 0000 0001`) или двойным словом (`0000 0000 0000 0000 0000 0000 0000 0001`)? Для ясности и компактности ведущие нули часто удаляют из значений, поэтому при перемещении единицы потенциально допустимо любое из этих толкований.

> **СОВЕТ.** Традиционно в 32-разрядной x86 слова должны были бы иметь размер 32 бита. Однако из-за обратной совместимости с 16-разрядными архитектурами x86 слово имеет размер 16 битов, а двойное слово (`dword`) - 32 бита.

Когда размер обращения к памяти нельзя вывести из контекста, его необходимо явно указать в инструкции. Например, инструкция `byte [100]` обращается к байту по адресу 100, `word[ebx]` - к слову, на которое указывает `ebx`, а `dword[ax]` - к двойному слову, на которое указывает `ax`. На рисунке 2.9 показано различие между следующими тремя инструкциями:

```
mov byte[100], 1
mov word[100], 1
mov dword[100], 1
```

Address	Value
98	0x00
99	0xff
100	0x01
101	0xfd
102	0x92
103	0xe8
104	0x42
105	0x13

mov byte [100], 1

Address	Value
98	0x00
99	0xff
100	0x01
101	0x00
102	0x92
103	0xe8
104	0x42
105	0x13

mov word [100], 1

Address	Value
98	0x00
99	0xff
100	0x01
101	0x00
102	0x00
103	0x00
104	0x42
105	0x13

mov dword [100], 1

Рисунок 2.9. Сравнение инструкций mov с разными размерами операндов

Режимы адресации

В синтаксисе Intel для x86 адреса памяти обозначаются квадратными скобками. Например, запись `[0x1234]` указывает, что программа должна обратиться к памяти по адресу `0x1234`.

Однако адресация памяти не ограничивается указанием адресов непосредственными значениями наподобие `0x1234`. Язык x86 поддерживает несколько режимов адресации. Разные режимы адресации используются для доступа к различным типам переменных.

Абсолютная адресация

При абсолютной адресации адрес задаётся постоянным значением. Это постоянное значение можно указать в любой системе счисления, например `[1]` или `[0x1234]`. Оно также может быть результатом арифметической операции, как в `[0x1337 + 0777]`, либо обозначаться меткой `[label]`.

Пример: глобальные переменные

В C/C++ глобальные переменные предназначены для доступа из любой части программы. Для этого они хранятся в памяти по фиксированному адресу и не перемещаются, когда приложение переходит между различными кадрами стека.

Это означает, что в языке ассемблера точный адрес переменной всегда будет известен. Следовательно, доступ к глобальным переменным осуществляется с помощью абсолютной адресации, например `mov eax, [0x1000]`.

Косвенная адресация

При косвенной адресации адрес указывается с помощью регистров. Можно использовать как 16-разрядные регистры общего назначения, например `[ax]`, так и 32-разрядные, например `[eax]`. Однако 8-разрядные регистры общего назначения (`al`, `bh` и другие) и регистры специального назначения для адресации использовать нельзя.

Пример: указатели

Во многих языках программирования применяется понятие указателей: в одних языках непосредственно, а в других скрыто от программиста. Непосредственное использование указателей и манипуляции с ними - пример типа данных C/C++, для которого часто применяется косвенная адресация. Программа на C может содержать строку `int x = 1; int* p = &x;`, в которой указателю `p` задаётся ссылка на `x`. Если вы не знакомы с C, не беспокойтесь; достаточно знать, что `p` хранит адрес расположения `x` в памяти.

Однако значение `p` может измениться и указывать на другие объекты, поэтому адрес его цели не является фиксированным. Чтобы получить в языке ассемблера значение, на которое указывает `p`, сначала необходимо загрузить `p` в регистр, а затем использовать этот регистр для поиска требуемого значения. Это показано в следующих инструкциях x86:

```
mov ebx, [p]        ; Загрузить адрес, указанный p, в ebx
mov eax, [ebx]      ; Переместить значение, указанное p, в eax
```

Адресация «база + смещение»

Некоторые переменные, например массивы, хранятся в памяти с использованием базового адреса и смещений. Доступ к отдельным значениям массива можно получить с помощью базового адреса и смещения.

Адресация «база + смещение», или базовая адресация, использует для указания адреса сочетание значения регистра и смещения. Этот режим часто применяется для доступа к массивам. Поэтому, если в языке высокого уровня встречается запись `myList[8]`, выполняется обращение к элементу, отстоящему на восемь позиций от начала `myList`. Например, в языке ассемблера запись `[eax + 8]` обозначает восемь байтов от базового адреса массива, который хранится в `eax`.

Индексная адресация

Адресация «база + смещение» хорошо работает, если длина каждого элемента массива всегда равна одному байту. Для массивов с более крупными элементами смещение приходится вычислять вручную, что утомительно и чревато ошибками.

В таких случаях более подходящим вариантом может быть индексная адресация. Для указания адреса в ней используются индексный регистр, коэффициент масштабирования и смещение. Коэффициент масштабирования всегда должен быть равен 1, 2, 4 или 8.

Пример: массивы

Определим массив целых чисел `int x[100];`, объявляющий массив из 100 значений типа `int`. В памяти каждое значение массива хранится по определённому смещению от базового адреса. Это смещение определяется размером значений массива, например размером 32-разрядного, или 4-байтового, значения `int`.

Предположим, что массив `int` создан по адресу `0x1000`. Следующая инструкция переместит n-й элемент массива в `eax`, если n хранится в `ebx`:

```
mov eax, [ ebx * 4 + 0x1000]
```

Базово-индексная адресация

Базово-индексная адресация объединяет элементы индексной адресации и адресации «база + смещение». Для указания адреса в ней используются базовый регистр, индексный регистр, коэффициент масштабирования (1, 2, 4 или 8) и смещение.

Рассмотрим, например, адрес `[ ebx + edi * 4 + 0x1000 ]`. Его база хранится в `ebx`, индекс - в `edi`, а смещение равно `0x1000`.

Пример: структуры

Базово-индексная адресация идеально подходит для доступа к элементам вложенных типов данных. Рассмотрим, например, команду C `struct { int i; short a[4]; } s;`. Она создаёт структуру с несколькими полями, включая массив.

Каждый элемент этой структуры находится по определённому смещению, то есть массив `a` имеет определённое смещение, или базовый адрес. Однако элементы внутри `a` также находятся по различным смещениям от этого базового адреса.

Предположим, что базовый адрес структуры `s` хранится в `ebx`, а массив `a` расположен в 4 байтах от этого базового адреса. Следующая инструкция обращается к n-му элементу `a`, если n хранится в `ecx`:

```
mov eax, [ebx + 2 * ecx + 4]
```

Не слишком огорчайтесь, если более сложные способы адресации трудно понять. Операционная система скрывала от вас память, поэтому вполне естественно, что размышления о размещении массивов в памяти стали для вас новой областью. Сначала эти режимы адресации необходимо представить в теории, но их бывает трудно усвоить до тех пор, пока далее вы не увидите их применение в настоящем ассемблерном коде. И не сомневайтесь, вы его увидите.

Итоги

x86 - широко используемый язык ассемблера. Понимание его устройства необходимо, чтобы стать успешным специалистом по обратной разработке и взлому программного обеспечения.

В этой главе были рассмотрены некоторые ключевые понятия ассемблера x86. К ним относятся представление данных, синтаксис языка ассемблера, а также применение регистров и адресов памяти для доступа к данным и их хранения.

Глава 3. Ассемблер x86: инструкции

Взлом и обратная разработка подразумевают чтение, написание и изменение кода на языке ассемблера. В этой книге основное внимание уделяется языку ассемблера x86.

Чтобы заниматься обратной разработкой или даже писать программы на языке ассемблера, необязательно понимать каждую деталь ассемблера x86. В этой главе рассматриваются основы x86 и главные инструкции, из которых состоит более 90 процентов ассемблерного кода программного обеспечения.

Формат инструкций x86

Мнемоники используются в ассемблере x86 для создания ассемблерного кода, понятного человеку. Каждая мнемоническая инструкция ассемблируется в машинный код, управляющий процессором. Следовательно, процессор не имеет представления о мнемониках и знает только машинный код. Например, мнемоника `add` ассемблируется в значение машинного кода `0x04`.

В x86 инструкции записываются в определённом формате. Ниже приведён пример простой инструкции x86:

```
add eax, 1
```

В этой инструкции `add` - мнемоника, которая указывает процессору, что следует сделать. Инструкция также содержит пару операндов, обозначающих данные для этой операции. В данном случае операндами являются регистр `eax` и значение 1. В обычных условиях инструкция x86, если она вообще имеет операнды, может содержать до трёх операндов. Существуют специальные расширения языка, допускающие до четырёх операций (префикс VEX), но мы не станем углубляться в этот уголок ассемблера.

Операндами инструкций x86 могут быть регистры, непосредственные значения или адреса памяти. В качестве регистров обычно используются регистры общего назначения (GPR), а области памяти указываются адресами. Непосредственные значения - это числа или константы наподобие 12345.

Хотя инструкция x86 может включать любой из этих типов операндов, она способна содержать не более одного адреса памяти. Например, инструкции `add eax, ebx` и `add eax, [0x12345678]` допустимы, поскольку обращаются соответственно к двум регистрам и к регистру с одной областью памяти. Однако инструкция `add [0x12345678], [0x87654321]` недопустима, поскольку одновременно использует два адреса памяти. Причина заключается в тонком устройстве конвейера процессора, который способен выполнить лишь одно чтение из памяти за инструкцию.

Инструкции x86

Язык ассемблера x86 включает сотни различных инструкций. Ниже перечислены некоторые из наиболее часто используемых:

• **Арифметические операции**
• `add`
• `sub`
• `mul`
• `inc`

• `dec`
• **Манипуляции с битами**
• `and`
• `or`
• `xor`
• `not`
• **Стек**
• `call`
• `return`
• `push`
• `pop`
• **Перемещение данных**
• `mov`
• **Поток выполнения**
• `jmp`
• условные переходы
• **Сравнение**
• `test`
• `cmp`
• **Прочие инструкции**
• `lea`
• `nop`

Хотя этот список может показаться длинным, вспомните распространённые операторы языков программирования (`+`, `-`, `*`, `/`, `%`, `&&`, `||`, `&`, `|`, `^`, `!`, `~`, `<`, `>`, `>=`, `<=`, `==`, `.`, `->` и другие) и основные ключевые слова (`if`, `else`, `switch`, `while`, `do`, `case`, `break`, `continue`, `for` и другие). Чтобы реализовать всё это поведение в языке ассемблера, требуется немало возможностей.

По правде говоря, никто не знает всех инструкций x86, да это и не требуется, если только человек не стремится произвести впечатление на друзей. Полный список инструкций x86 можно найти по адресу ref.x86asm.net, а подробности любой инструкции при необходимости можно посмотреть в справочнике.

Однако для успеха в обратной разработке необходимо чётко понимать работу самых распространённых средств ассемблера x86. Если вы понимаете это важное подмножество инструкций x86, то сможете читать и понимать большинство программ x86.

mov

Как следует из названия, инструкция `mov` предназначена для перемещения данных из одного места в другое. Она позволяет копировать данные между регистрами и областями памяти либо помещать непосредственное значение в определённое место. Обратите внимание: несмотря на слово move в названии, инструкция копирует данные, а не перемещает их, то есть данные не удаляются из источника, а копируются из источника в место назначения.

Синтаксис этой инструкции имеет вид `mov назначение, источник`. Например, команда `mov eax, 5` помещает значение `5` в регистр `eax`. Аналогично инструкция `mov eax, [1]` перемещает в `eax` значение по адресу `0x1`.

При работе с `mov` и подобными инструкциями важно помнить, что имена используемых переменных влияют на длину перемещаемого значения. Например, инструкция `mov eax, [0x100]` перемещает в `eax` 32-разрядное значение, тогда как инструкция `mov dx, [0x100]` перемещает в `dx` 16-разрядное значение.

ПРИМЕЧАНИЕ. Операнд x86 может использовать значение регистра для указания адреса памяти. Например, инструкция `mov [eax], ebx` перемещает значение из `ebx` в область памяти, адрес которой хранится в `eax`. Следовательно, если значение `eax` равно `0x7777`, значение `ebx` будет сохранено по адресу памяти `0x7777`.

`mov` - чрезвычайно универсальная операция и прекрасный пример преимущества мнемоник перед машинным кодом. Как показано на рисунке 3.1, `mov` можно использовать множеством способов, и каждый из них преобразуется в разный машинный код в зависимости от двух используемых операндов. Однако на уровне мнемоник все эти варианты представлены как `mov`. Задача ассемблера - преобразовать мнемонику в правильный машинный код.

MOV—Move

Opcode	Instruction	Op/ En	64-Bit Mode	Compat/ Leg Mode	Description
88 /r	MOV r/m8, r8	MR	Valid	Valid	Move r8 to r/m8.
REX + 88 /r	MOV r/m8[1], r8[1]	MR	Valid	N.E.	Move r8 to r/m8.
89 /r	MOV r/m16, r16	MR	Valid	Valid	Move r16 to r/m16.
89 /r	MOV r/m32, r32	MR	Valid	Valid	Move r32 to r/m32.
REX.W + 89 /r	MOV r/m64, r64	MR	Valid	N.E.	Move r64 to r/m64.
8A /r	MOV r8, r/m8	RM	Valid	Valid	Move r/m8 to r8.
REX + 8A /r	MOV r8[1], r/m8[1]	RM	Valid	N.E.	Move r/m8 to r8.
8B /r	MOV r16, r/m16	RM	Valid	Valid	Move r/m16 to r16.
8B /r	MOV r32, r/m32	RM	Valid	Valid	Move r/m32 to r32.
REX.W + 8B /r	MOV r64, r/m64	RM	Valid	N.E.	Move r/m64 to r64.
8C /r	MOV r/m16, Sreg[2]	MR	Valid	Valid	Move segment register to r/m16.
8C /r	MOV r16/r32/m16, Sreg[2]	MR	Valid	Valid	Move zero extended 16-bit segment register to r16/r32/m16.
REX.W + 8C /r	MOV r64/m16, Sreg[2]	MR	Valid	Valid	Move zero extended 16-bit segment register to r64/m16.
8E /r	MOV Sreg, r/m16[2]	RM	Valid	Valid	Move r/m16 to segment register.
REX.W + 8E /r	MOV Sreg, r/m64[2]	RM	Valid	Valid	Move lower 16 bits of r/m64 to segment register.
A0	MOV AL, moffs8[3]	FD	Valid	Valid	Move byte at (seg:offset) to AL.
REX.W + A0	MOV AL, moffs8[3]	FD	Valid	N.E.	Move byte at (offset) to AL.
A1	MOV AX, moffs16[3]	FD	Valid	Valid	Move word at (seg:offset) to AX.
A1	MOV EAX, moffs32[3]	FD	Valid	Valid	Move doubleword at (seg:offset) to EAX.
REX.W + A1	MOV RAX, moffs64[3]	FD	Valid	N.E.	Move quadword at (offset) to RAX.
A2	MOV moffs8, AL	TD	Valid	Valid	Move AL to (seg:offset).
REX.W + A2	MOV moffs8[1], AL	TD	Valid	N.E.	Move AL to (offset).
A3	MOV moffs16[3], AX	TD	Valid	Valid	Move AX to (seg:offset).
A3	MOV moffs32[3], EAX	TD	Valid	Valid	Move EAX to (seg:offset).
REX.W + A3	MOV moffs64[3], RAX	TD	Valid	N.E.	Move RAX to (offset).
B0+ rb ib	MOV r8, imm8	OI	Valid	Valid	Move imm8 to r8.
REX + B0+ rb ib	MOV r8[1], imm8	OI	Valid	N.E.	Move imm8 to r8.
B8+ rw iw	MOV r16, imm16	OI	Valid	Valid	Move imm16 to r16.
B8+ rd id	MOV r32, imm32	OI	Valid	Valid	Move imm32 to r32.
REX.W + B8+ rd io	MOV r64, imm64	OI	Valid	N.E.	Move imm64 to r64.
C6 /0 ib	MOV r/m8, imm8	MI	Valid	Valid	Move imm8 to r/m8.
REX + C6 /0 ib	MOV r/m8[1], imm8	MI	Valid	N.E.	Move imm8 to r/m8.
C7 /0 iw	MOV r/m16, imm16	MI	Valid	Valid	Move imm16 to r/m16.
C7 /0 id	MOV r/m32, imm32	MI	Valid	Valid	Move imm32 to r/m32.
REX.W + C7 /0 id	MOV r/m64, imm32	MI	Valid	N.E.	Move imm32 sign extended to 64-bits to r/m64.

Рисунок 3.1. Инструкции mov

Ниже приведён полный перевод строк таблицы на рисунке 3.1. Обозначение `N.E.` сохранено в исходном виде и означает, что соответствующий вариант не кодируется в данном режиме.

Код операции	Инструкция	Op/En	64-разрядный режим	Режим совместимости/устаревший	Описание
88 /r	MOV r/m8, r8	MR	Допустима	Допустима	Переместить r8 в r/m8.
REX + 88 /r	MOV r/m8[1], r8[1]	MR	Допустима	N.E.	Переместить r8 в r/m8.
89 /r	MOV r/m16, r16	MR	Допустима	Допустима	Переместить r16 в r/m16.
89 /r	MOV r/m32, r32	MR	Допустима	Допустима	Переместить r32 в r/m32.
REX.W + 89 /r	MOV r/m64, r64	MR	Допустима	N.E.	Переместить r64 в r/m64.
8A /r	MOV r8, r/m8	RM	Допустима	Допустима	Переместить r/m8 в r8.
REX + 8A /r	MOV r8[1], r/m8[1]	RM	Допустима	N.E.	Переместить r/m8 в r8.
8B /r	MOV r16, r/m16	RM	Допустима	Допустима	Переместить r/m16 в r16.
8B /r	MOV r32, r/m32	RM	Допустима	Допустима	Переместить r/m32 в r32.
REX.W + 8B /r	MOV r64, r/m64	RM	Допустима	N.E.	Переместить r/m64 в r64.
8C /r	MOV r/m16, Sreg[2]	MR	Допустима	Допустима	Переместить сегментный регистр в r/m16.
8C /r	MOV r16/r32/m16, Sreg[2]	MR	Допустима	Допустима	Переместить расширенный нулями 16-разрядный сегментный регистр в r16/r32/m16.

Код операции	Инструкция	Op/En	64-разрядный режим	Режим совместимости/устаревший	Описание
REX.W + 8C /r	MOV r64/m16, Sreg²	MR	Допустима	Допустима	Переместить расширенный нулями 16-разрядный сегментный регистр в r64/m16.
8E /r	MOV Sreg, r/m16²	RM	Допустима	Допустима	Переместить r/m16 в сегментный регистр.
REX.W + 8E /r	MOV Sreg, r/m64²	RM	Допустима	Допустима	Переместить младшие 16 битов r/m64 в сегментный регистр.
A0	MOV AL, moffs8³	FD	Допустима	Допустима	Переместить байт по адресу (сегмент:смещение) в AL.
REX.W + A0	MOV AL, moffs8³	FD	Допустима	N.E.	Переместить байт по адресу (смещение) в AL.
A1	MOV AX, moffs16³	FD	Допустима	Допустима	Переместить слово по адресу (сегмент:смещение) в AX.
A1	MOV EAX, moffs32³	FD	Допустима	Допустима	Переместить двойное слово по адресу (сегмент:смещение) в EAX.
REX.W + A1	MOV RAX, moffs64³	FD	Допустима	N.E.	Переместить четверное слово по адресу (смещение) в RAX.
A2	MOV moffs8, AL	TD	Допустима	Допустима	Переместить AL по адресу (сегмент:смещение).

Код операции	Инструкция	Op/En	64-разрядный режим	Режим совместимости/устаревший	Описание
REX.W + A2	MOV moffs8[1], AL	TD	Допустима	N.E.	Переместить AL по адресу (смещение).
A3	MOV moffs16[3], AX	TD	Допустима	Допустима	Переместить AX по адресу (сегмент:смещение).
A3	MOV moffs32[3], EAX	TD	Допустима	Допустима	Переместить EAX по адресу (сегмент:смещение).
REX.W + A3	MOV moffs64[3], RAX	TD	Допустима	N.E.	Переместить RAX по адресу (смещение).
B0+ rb ib	MOV r8, imm8	OI	Допустима	Допустима	Переместить imm8 в r8.
REX + B0+ rb ib	MOV r8[1], imm8	OI	Допустима	N.E.	Переместить imm8 в r8.
B8+ rw iw	MOV r16, imm16	OI	Допустима	Допустима	Переместить imm16 в r16.
B8+ rd id	MOV r32, imm32	OI	Допустима	Допустима	Переместить imm32 в r32.
REX.W + B8+ rd io	MOV r64, imm64	OI	Допустима	N.E.	Переместить imm64 в r64.
C6 /0 ib	MOV r/m8, imm8	MI	Допустима	Допустима	Переместить imm8 в r/m8.
REX + C6 /0 ib	MOV r/m8[1], imm8	MI	Допустима	N.E.	Переместить imm8 в r/m8.
C7 /0 iw	MOV r/m16, imm16	MI	Допустима	Допустима	Переместить imm16 в r/m16.
C7 /0 id	MOV r/m32, imm32	MI	Допустима	Допустима	Переместить imm32 в r/m32.

Код операции	Инструкция	Op/En	64-разрядный режим	Режим совместимости/устаревший	Описание
REX.W + C7 /0 id	MOV r/m64, imm32	MI	Допустима	N.E.	Переместить imm32 с расширением знака до 64 битов в r/m64.

Практический пример

Как записать на языке ассемблера следующий псевдокод? Предположим, что переменная `i` находится по адресу 100, а `j` - по адресу 200.

```
int i = 42, j = i;
```

Эта единственная строка псевдокода может быть ассемблирована в три инструкции x86:

```
mov [100], 42
mov eax, [100]
mov [200], eax
```

Обратите внимание, что регистр `eax` используется для хранения значения, копируемого из памяти по адресу 100 в память по адресу 200. Причина в том, что одна инструкция не может выполнить два разных обращения к памяти. Для временного хранения необходимо использовать регистр, например `eax`.

При взгляде на код может показаться, что разумнее загрузить по адресу 200 непосредственное значение 42, чем выполнять две операции для его загрузки из памяти по адресу 100. Однако компилятор не станет и не должен так поступать.

Причина связана с возможностью многопоточного выполнения приложений. Если в системе выполняется другой поток, значение по адресу 100 могло измениться между шагом, на котором ему присваивается 42, и шагом присваивания этого значения области по адресу 200. Копирование значения из области 100 вместо использования непосредственного значения помогает гарантировать, что переменная `j` по адресу 200 получит самую свежую версию значения, сохранённого в `i`.

inc, dec

Инструкции x86 `inc` и `dec` соответственно увеличивают или уменьшают указанное значение на 1. В обычном коде им соответствуют инструкции `i++` и `i--`.

Эти инструкции принимают единственный операнд, который может быть регистром или адресом памяти. Например, инструкция `inc eax` увеличивает на 1 значение в `eax`, а `dec [0x12345678]` уменьшает на 1 значение, хранящееся в памяти по адресу `0x12345678`.

add, sub

Инструкции `add` и `sub` соответственно прибавляют значение к определённому значению или вычитают его. Эти инструкции принимают два операнда. Например, инструкция `add` записывается как `add назначение, значение`.

Местом назначения в инструкции add может быть регистр или область памяти, а значением - регистр, область памяти или непосредственное значение. Операция вычисляет назначение + значение и сохраняет результат в месте назначения. Это означает, что входное содержимое места назначения участвует в математическом выражении, но затем перезаписывается результатом. Обратите внимание, что размеры двух операндов должны совпадать. Например, add eax, ebx - допустимая инструкция (32 бита плюс 32 бита), а sub eax, bx - недопустимая (32 бита минус 16 битов).

При использовании add и sub важно учитывать размеры значений. Например, инструкция sub ecx, [100] подразумевает 32-разрядное значение, поскольку местом назначения служит ecx. Однако в инструкции add dword [edx], 100 требуется спецификатор размера dword: 32-разрядное значение edx указывает, что адрес памяти имеет длину 32 бита, но не задаёт размер данных, изменяемых по этому адресу.

mul

Операция mul выполняет умножение целых чисел без знака. Однако она несколько необычна: принимает только один операнд, но неявно использует два дополнительных регистра. Синтаксис операции имеет вид mul операнд, где операнд может быть регистром или адресом памяти. Операция умножает значение в eax на значение, указанное операндом.

Результат операции mul сохраняется в edx:eax, причём edx содержит старшие 32 бита результата. Значения edx и eax всегда изменяются инструкцией mul, даже если длина результата меньше 32 битов и edx не требуется. Инструкция mul интересна тем, что при 32-разрядных вычислениях позволяет получить 64-разрядный результат (edx:eax).

Пример операции - mul eax, возводящая в квадрат 32-разрядное значение в eax. Если операнд содержит адрес памяти, длина значения может различаться. Например, mul dword [0x555] умножает eax на 32-разрядное значение по адресу 0x555, а mul byte [0x123] использует при умножении 8-разрядное значение по адресу 0x123.

div

Операция div выполняет деление беззнаковых значений. Как и mul, она принимает один операнд и неявно изменяет регистры eax и edx. В данном случае частное сохраняется в eax, а остаток - в edx. Если кому-то требуется быстро напомнить математические термины, не стесняйтесь: у меня самого от математических словечек болит голова; при делении 5 на 2 частное равно 2, а остаток - 1.

В качестве входных данных операция div использует и eax, и edx, объединяя их в том же формате, в котором mul выдаёт результат: старшие 32 бита содержатся в edx. Как и в случае mul, результат всегда изменяет eax и edx, даже если edx не требуется, например когда остаток равен нулю.

Пример операции деления - div eax. Она эквивалентна вычислению eax, edx = edx:eax / eax. В данном случае операндом служит 32-разрядный регистр, но адреса памяти позволяют указывать и использовать делители другой длины.

Практический пример

Предположим, требуется вычислить остаток от деления 123 на 4. Это можно сделать четырьмя инструкциями на языке ассемблера:

```
mov eax, 123  ; Загрузить младшие 32 бита делимого в eax
mov edx, 0    ; Очистить регистр edx, содержащий старшие 32 бита
              ; делимого
mov ecx, 4    ; Загрузить делитель в ecx, поскольку div не принимает
              ; непосредственный операнд
div ecx       ; Выполнить деление
```

По завершении этого процесса частное хранится в `eax`, а остаток - в `edx`.

and, or, xor

Стандарт x86 поддерживает несколько логических операций. Операции `and`, `or` и `xor` принимают по два операнда. Ниже приведены таблицы истинности этих трёх операций. Возможные входные значения показаны в верхней строке и левом столбце таблицы. Например, чтобы найти результат `1 AND 1`, нужно найти пересечение столбца 1 и строки 1; результат равен 1. Следовательно, `1 AND 1` равно 1.

AND	**1**	**0**
1	1	0
0	0	0

OR	**1**	**0**
1	1	1
0	1	0

XOR	**1**	**0**
1	0	1
0	1	0

Все три операции используют одинаковый синтаксис: `мнемоника назначение, источник`. Например, синтаксис операции `and` имеет вид `and назначение, источник`. Как и в операции `add`, место назначения должно быть регистром или адресом памяти, тогда как источником может быть регистр, адрес памяти или непосредственное значение. И снова, как в `add`, место назначения используется в вычислении, но затем перезаписывается для сохранения результата.

Логические операции можно применять для множества целей. Например, операция `or eax, 0xffffffff` позволяет быстро установить все биты `eax` в 1. Операция `and dword [0xdeadbeef], 0x1` маскирует все биты, кроме младшего бита 32-разрядного значения по адресу `0xdeadbeef`. Операция `xor eax, eax` является распространённым способом обнуления `eax`.

not

Операция `not` - логическая операция, вычисляющая обратный код значения. Если термин «обратный код» вам незнаком, его можно представить как замену всех нулей единицами и всех единиц нулями. Операция инвертирует число. Она принимает единственный операнд и имеет

синтаксис `not операнд`.

Операция `not` может работать со значениями различной длины. Например, `not ch` вычисляет обратный код 8-разрядного регистра `ch`. Команда `not dword [2020]` вычисляет обратный код 32-разрядного значения по адресу 2020.

shr, shl

`shr` и `shl` - две доступные в x86 операции сдвига: `shr` означает сдвиг вправо, а `shl` - сдвиг влево. Они принимают два операнда: место расположения сдвигаемого значения и величину сдвига. Пример операции сдвига имеет вид `shr регистр, непосредственное_значение`.

`shr` и `shl` являются операциями логического сдвига. Это означает, что при сдвиге значения на указанную непосредственным значением величину они будут расширять значение нулями слева или справа. Следовательно, любые новые разряды, появившиеся в результате сдвига, автоматически будут равны 0.

Например, операция `shr al, 3` сдвигает значение в `al` вправо на три бита. Если `al` содержит значение `00010000`, результат будет равен `00000010`.

> **СОВЕТ.** При расширении сдвинутого вправо значения нулями освободившиеся биты заполняются нулями; это называется логическим сдвигом. При расширении сдвинутого вправо значения с сохранением знака освободившиеся биты заполняются значением старшего значащего бита; это называется арифметическим сдвигом.

sar, sal

`sar` и `sal` являются операциями арифметического сдвига. Их синтаксис совпадает с синтаксисом логических сдвигов, но реализация различается. `sar` выполняет арифметический сдвиг вправо, а `sal` - арифметический сдвиг влево.

При сдвиге влево `sal` работает так же, как `shl`, расширяя значение нулями. Например, инструкции `shl al, 3` и `sal al, 3`, если в `al` хранится значение `00000100`, дадут значение `00100000`. Все новые разряды числа будут заполнены нулями.

Однако операция `sar` расширяет значение с сохранением знака, а `shl` - нулями. Расширение с сохранением знака означает копирование значения старшего значащего бита. Например, если `al` содержит `10000000`, команда `shr al, 3` выдаст `00010000`, как показано ниже:

```
10000000     Исходное значение
01000000     Сдвиг на 1 бит
00100000     Сдвиг на 2 бита
00010000     Сдвиг на 3 бита
```

Однако результатом инструкции `sar al, 3` будет `11110000`. Поскольку старший значащий бит равен 1, единица копируется во все новые разряды.

nop

Название операции `nop` означает «нет операции». Это однобайтовая операция (`0x90`), которая ничего не делает.

Хотя технически nop ничего не делает, эта инструкция применяется для нескольких вполне законных целей:

• управление временными характеристиками;
• выравнивание памяти;
• предотвращение конфликтов;
• слот задержки перехода в архитектурах RISC;
• заполнитель, который позднее будет заменён будущей заплатой.

А в области безопасности она используется для следующего:

• взлома (nop-цепочки);
• взлома защиты программ (замена инструкций на nop).

lea

Название операции lea означает «загрузить эффективный адрес». Она принимает два операнда: место назначения, которым может быть регистр или адрес памяти, и источник, который должен быть адресом памяти. Инструкция lea вычисляет адрес указанного операнда-источника и помещает его в место назначения. Для знакомых с указателями: она подобна оператору &.

Хотя lea предназначена для работы с адресами, её также часто используют для простых математических операций. Например, операция lea eax, [ebx + ecx + 5] запрашивает адрес, на который указывает ebx+ecx+5, и сохраняет его в eax. По существу, она вычисляет ebx + ecx + 5 и сохраняет результат в eax. Более стандартный вариант использования lea, lea eax, [100], поместит значение 100 в eax.

На первый взгляд это может показаться несколько нелепым или бессмысленным, однако lea полезна тем, что повышает эффективность работы с массивами в языке ассемблера. В массивах значения хранятся по определённым смещениям от базового адреса. Помните режим адресации «база + смещение»? С помощью lea можно эффективно вычислить адрес конкретного элемента массива. Например, предположим, что eax содержит базовый адрес массива символов. В таком случае инструкция lea ebx, [eax + 2] поместит в ebx адрес второго элемента массива. Эта единственная инструкция эффективнее последовательности mov ebx, eax и add ebx, 2, дающей тот же результат.

Практический пример

Как записать на языке ассемблера следующий псевдокод? Предположим, что i находится по адресу 100, j - по адресу 200, а k - по адресу 300.

```
int i = 7;
char j = 5;
int k = i + j;
```

Этот псевдокод будет ассемблирован в следующие инструкции x86:

```
mov dword [ 100 ], 7  ; задать i
mov byte [ 200 ], 5   ; задать j

mov eax, [ 100 ]      ; загрузить i в eax
xor ebx, ebx          ; обнулить ebx
mov bl, [ 200 ]       ; загрузить j в ebx

add eax, ebx          ; прибавить ebx к eax, сохранить в eax

mov [ 300 ], eax      ; сохранить результат в k
```

Обратите внимание на использование в этом примере и ebx, и bl. Сохраняемое в регистре значение помещается в bl. Однако при выполнении сложения используется весь регистр ebx. Причина заключается в продвижении типов: если прибавить 1-байтовое значение к 4-байтовому, 1-байтовое значение продвигается до 4 байтов, а дополнительные байты должны быть равны 0. Поэтому в данном случае значение 0x05 в bl продвигается до 0x00000005 в ebx. Операция XOR для очистки ebx была необходима, чтобы прежнее значение регистра ebx было полностью удалено и не повлияло на результат сложения.

Собираем всё вместе

До сих пор многие примеры представляли собой простые действия с использованием лишь пары операций x86. Теперь попробуйте написать ассемблерный код для следующего псевдокода, предполагая, что i находится по адресу 100, j - по адресу 200, а k - по адресу 300.

```
int i = 7;
char j = 5;
int k = i * i + j * j;
```

Этот псевдокод ассемблируется в следующие инструкции x86:

```
mov dword [ 100 ], 7  ; задать i
mov byte [ 200 ], 5   ; задать j

mov ecx, [ 100 ]      ; загрузить i в ecx
xor ebx, ebx          ; обнулить ebx
mov bl, [ 200 ]       ; загрузить j в ebx

mov eax, ecx          ; скопировать ecx в eax (eax = ecx = i)
mul ecx               ; умножить ecx на eax, сохранить результат в eax
mov ecx, eax          ; сохранить результат обратно в ecx, освободив eax

mov eax, ebx          ; скопировать ebx в eax (eax = ebx = j)
mul ebx               ; умножить ebx на eax, сохранить результат в eax

add eax, ecx          ; прибавить ecx к eax, сохранить результат в eax
mov [ 300 ], eax      ; сохранить окончательное значение в k
```

Распространённые ошибки в инструкциях x86

x86 - мощный язык ассемблера, и большинство его инструкций подчиняется согласованному набору правил. Однако в нём имеются несогласованности, способные сбить с толку.

Ниже приведены примеры распространённых ошибок, которые допускают при попытке написать собственный код x86 и из-за которых код не удаётся ассемблировать:

• mov [bl], 0xf: x86 поддерживает косвенную адресацию с использованием 16- и 32-разрядных регистров общего назначения. Поскольку длина bl составляет лишь 8 битов, его нельзя использовать для адресации.
• mov [0xabcd], 1337: эта инструкция не указывает размер перемещаемого значения, поскольку 1337 можно записать как 0x0539 или 0x00000539.
• mov word [0xabcd], eax: в инструкции указан неправильный размер памяти, поскольку слово содержит 16 битов, а eax - 32 бита.
• mov byte [1], byte [2]: в одной инструкции нельзя использовать две области памяти.
• mov sl, al: хотя у eax есть регистр al, регистра sl не существует.

• `mov 0x1234, eax`: значение `0x1234` является непосредственным значением, а не адресом памяти, и не может быть местом назначения команды.
• `mov eax, dx`: в этой инструкции не совпадают размеры 16-разрядного источника `dx` и 32-разрядного места назначения `eax`.

Если сомневаетесь, ищите в справочнике

Помните: никто, даже я, как бы трудно мне ни было это признать, не знает наизусть всех инструкций x86. Пишете ли вы код x86 или читаете его, если вам встретилось нечто непонятное, главное - уметь быстро найти справочные сведения. Для быстрого поиска у нас всегда открыта вкладка ref.x86asm.net.

Итоги

x86 - сложный, мощный язык ассемблера. Однако для эффективного взлома и обратной разработки программного обеспечения необязательно понимать в нём каждый отдельный бит.

В этой главе были рассмотрены инструкции x86, составляющие подавляющую часть ассемблерного кода. Изучение этих инструкций необходимо и закладывает прочную основу для обратной разработки.

Глава 4. Сборка и запуск программ на языке ассемблера

Обратная разработка программного обеспечения заключается в преобразовании скомпилированного исполняемого файла в код, понятный человеку. Однако умение выполнять обратный процесс, то есть собирать и запускать программу на языке ассемблера, может оказаться неоценимым для понимания этого преобразования.

В этой главе рассматриваются некоторые ключевые понятия, необходимые для понимания сборки и запуска ассемблерных программ. В частности, мы разберём, как такие программы взаимодействуют с внешним миром, как их действительно собирать и запускать и как они работают со строками.

Вывод

ИСТОРИЯ. В колледже я обнаружил, что довольно весело сходить в комиссионный магазин, купить кучу сломанной электроники, разобрать её и собрать из деталей нечто другое. На рисунке 4.1 показана одна из первых вещей, которые я когда-либо построил. Мне очень нравится работать над непрактичными вещами, потому что размышлять, проектировать, строить и учиться весело, но стоит начать беспокоиться о реальном применении и удобстве использования, как удовольствие исчезает. Поэтому я намеренно стараюсь работать над непрактичными вещами.

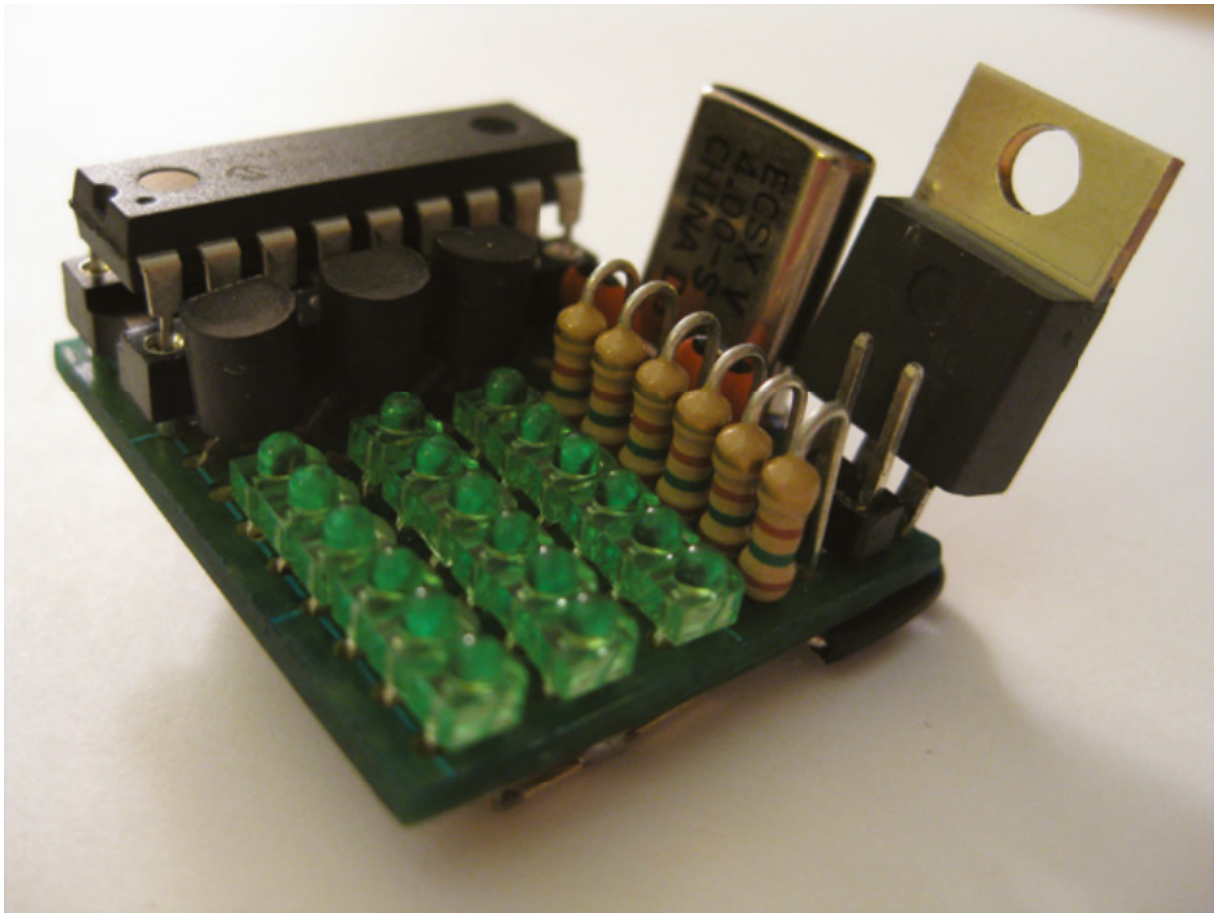


Рисунок 4.1. Двоичные наручные часы

> Я пытался придумать самую непрактичную вещь, которую мог бы здесь сделать. Это двоичные наручные часы, которые необходимо подключать к электрической розетке. Ремешка для них я так и не нашёл, но хотел придумать максимально гиковскую вещь, и эти часы показались достаточно гиковскими.

Язык ассемблера и машинный код замечательны, но в какой-то момент коду понадобится взаимодействовать с внешним миром. Для этого необходим способ выводить информацию.

Если вы когда-нибудь рассматривали процессор, то видели, что он покрыт маленькими выводами. Выводы позволяют процессору взаимодействовать с внешним миром. С помощью языка ассемблера можно управлять этими выводами, включая и выключая их, и тем самым, например, включать и выключать светодиоды. Современный процессор x86 имеет от 400 до 1000 выводов, что позволяет управлять множеством устройств.

Выводы объединены в группы, называемые портами. Благодаря портам вместо утомительного и длительного управления отдельными выводами можно одновременно управлять несколькими. Установка значения на выводах эквивалентна записи в порт, а получение значения с выводов - чтению из порта. Для x86 определено множество различных портов. В таблице 4.1 приведено несколько примеров из небольшого подмножества.

Таблица 4.1. Порты x86

Диапазон портов	**Назначение**
`0x0000-0x001F`	Первый устаревший контроллер DMA, часто используемый для передачи данных на гибкие диски
`0x0020-0x0021`	Первый программируемый контроллер прерываний
`0x0022-0x0023`	Доступ к регистрам, зависящим от модели процессоров Cyrix
`0x0040-0x0047`	Программируемый интервальный таймер (PIT)
`0x0060-0x0064`	Контроллер PS/2 «8042» или его предшественники
`0x0070-0x0071`	Регистры CMOS и RTC
`0x0080-0x008F`	DMA, страничные регистры
`0x0092`	Расположение регистра быстрого вентиля A20
`0x00A0-0x00A1`	Второй PIC
`0x00C0-0x00DF`	Второй контроллер DMA, часто используемый звуковыми платами Sound Blaster
`0x00E9`	Место применения приёма с портом E9

Диапазон портов	Назначение
`0x0170-0x0177`	Вторичный контроллер жёсткого диска ATA
`0x01F0-0x01F7`	Основной контроллер жёсткого диска ATA
`0x0278-0x027A`	Параллельный порт
`0x02F8-0x02FF`	Второй последовательный порт
`0x03B0-0x03DF`	Диапазон, используемый IBM VGA и её непосредственными предшественниками
`0x03F0-0x03F7`	Контроллер гибких дисков
`0x03F8-0x03FF`	Первый последовательный порт

Управление выводами

В x86 выводами можно управлять с помощью инструкций `in` и `out`, которые принимают в качестве параметров регистр и порт.

Синтаксис инструкции `in` имеет вид `in регистр, порт`. Например, `in al, 0x64` получает состояние клавиатуры.

В инструкции `out` порядок параметров обратный, а синтаксис имеет вид `out порт, регистр`. Например, `out 0x3c0, eax` задаёт значение пикселя.

В действительности непосредственного жёсткого соединения с портами назначения или источника зачастую нет и всё устроено несколько сложнее. Выводы подключены к общей шине, а задача направления операций чтения и записи нужному получателю возложена на отдельную плату или мост. Инструкции `in` и `out` обращаются к заранее определённым адресам шины, которые преобразует мост. Однако общая идея остаётся прежней.

Утомительная работа

Вернёмся к идее вывода: с помощью инструкций `in` и `out` можно включать и выключать отдельные пиксели. Однако один экран может содержать тысячи или миллионы пикселей. Устанавливать каждый из них по отдельности было бы утомительно и неэффективно.

Поэтому эти подробности скрываются за абстракцией. При отображении изображений графический адаптер занимается установкой каждого отдельного пикселя. Однако выяснять точный способ взаимодействия с графическим адаптером может быть утомительно.

Именно здесь в дело вступает операционная система. Она способна взять на себя сложность взаимодействия с графическим адаптером, который устанавливает значения пикселей и отображает изображение. Для взаимодействия с операционной системой требуется системный вызов. Поэтому, если вы хотите играть с x86 на эпическом уровне сложности, можно напрямую взаимодействовать с графическим адаптером, но для целей этой книги мы предпочитаем режим «сложно, но с подсказками» и поручим эту тяжёлую работу операционной системе.

Системные вызовы

В x86 системные вызовы предоставляют ограниченные возможности ввода-вывода, вызывая соответствующее поведение через операционную систему (ОС). Наборы доступных системных вызовов различаются в зависимости от операционной системы.

Поскольку системные вызовы являются понятием операционной системы, они зависят от ОС; мы рассмотрим некоторые наиболее полезные системные вызовы Linux. Системные вызовы выполняются путём загрузки номера функции в регистр `eax`. Затем в Linux системный вызов производится посредством прерывания, вызываемого инструкцией `int 0x80`.

sys_write

В языке программирования высокого уровня функция `sys_write` имела бы сигнатуру `ssize_t sys_write(unsigned int fd, const char * buf, size_t count)`. Эта функция возвращает размер, обозначающий объём записанных данных.

Функция `sys_write` принимает три аргумента. Первый, `fd`, является дескриптором файла, который указывает, куда следует записать данные. Значение 1 означает запись данных в консоль Linux. Аргумент `buf` содержит данные, выводимые при записи, а `count` сообщает функции число печатаемых символов.

В ассемблере x86 функции нельзя вызывать с помощью такого описания. Вместо этого аргументы загружаются в регистры, как показано в таблице 4.2. После загрузки этих регистров системный вызов можно выполнить инструкцией `int 0x80`.

Таблица 4.2. sys_write

Регистр	Значение	Описание
`eax`	`4`	Идентификатор `sys_write`
`ebx`	1 (вывод в консоль)	Дескриптор файла
`ecx`	`const char* buf`	Записываемая строка
`edx`	`size_t count`	Длина строки

Регистры, используемые функцией `sys_write`, необходимо загрузить последовательностью инструкций на языке ассемблера. В следующем примере показано возможное применение `sys_write`:

```
mov   edx,len       ; длина сообщения
mov   ecx,buff      ; записываемое сообщение
mov   ebx,1         ; дескриптор файла (stdout)
mov   eax,4         ; системная функция (sys_write)
int   0x80          ; вызвать ядро
```

sys_exit

Системный вызов `sys_exit` эквивалентен возврату состояния из `main` в языках программирования высокого уровня. Он завершает программу. Вызов принимает единственный аргумент, код состояния, который хранится в `ebx`, как показано в таблице 4.3.

Таблица 4.3. sys_exit

Регистр	Значение	Описание
eax	1	Идентификатор sys_exit
ebx	int	Код состояния

Вызов sys_exit начинается с загрузки значений в регистры eax и ebx, как показано в следующем примере:

```
mov eax, 1          ; системная функция (sys_exit)
mov ebx, 0          ; вернуть 0;
int 0x80            ; вызвать ядро
```

Вывод строки

Для вывода строки необходимо включать и выключать определённые выводы процессора. Выполнив системный вызов, ассемблерная программа может поручить операционной системе определение выводов, которые нужно включить и выключить. ОС сообщает об этом графическому адаптеру, который выбирает включаемые и выключаемые биты. Затем сведения передаются микроконтроллеру монитора, заставляя его включать и выключать свои выводы, что создаёт изображение на экране. По пути в этом процессе могут участвовать и десятки других контроллеров.

Ассемблерная программа, которая выводит строку, а затем завершается, использует оба системных вызова: sys_write и sys_exit. Общий синтаксис файла будет рассмотрен в следующем разделе. А пока, чтобы вы заинтересовались и читали следующий раздел на краешке стула, взглянем на него одним глазком. Следующий пример выводит сообщение «Hello, world!» в консоль:

```
global _start

section .text
_start:
    mov eax, 4 ; записать
    mov ebx, 1 ; stdout
    mov ecx, msg
    mov edx, msg.len
    int 0x80

    mov eax, 1 ; завершить
    mov ebx, 0
    int 0x80

section .data
msg:    db  "Hello, world!", 10
.len:   equ $ -msg
```

Сборка и компоновка

Обратная разработка и взлом защиты заключаются в понимании чужого существующего ассемблерного кода. Однако при исправлении или взломе защиты любого рода вы обнаружите, что писать собственный код и выполнять обратную разработку гораздо проще, если понимать противоположный процесс: написание, сборку и ассемблирование собственного ассемблерного кода. Сборка и компоновка - важнейший этап преобразования ассемблерного кода в работающее приложение.

Сборка и компоновка в Linux

Процесс сборки и компоновки ассемблерного кода зависит от операционной системы, поэтому данный раздел посвящён Linux. В среде Linux файлам ассемблерного кода традиционно присваивается расширение `.asm`, например `program.asm`. Затем программу можно ассемблировать, скомпоновать и выполнить с помощью следующих трёх команд:

```
nasm  -f elf program.asm
ld -melf_i386 program.o  -o program.out
./program.out
```

Первая команда использует Netwide Assembler, `nasm`, чтобы ассемблировать код в объектный файл. Флаг `-f` задаёт формат ELF, то есть формат исполняемого файла Linux. Результатом будет объектный файл с именем `program.o`.

На следующем этапе выполняется компоновка с помощью `ld`, компоновщика GNU. Параметр `-melf_i386` задаёт архитектуру для компоновки и указывает, что должен быть создан двоичный файл ELF для i386 (x86). Флаг `-o` задаёт имя выходного файла: `program.out`.

После завершения компоновки файл `program.out` представляет собой полностью работоспособный исполняемый файл Linux. Его можно запустить командой `./program.out`.

Написание ассемблерной программы

Предыдущий пример показывает, как собрать и скомпоновать ассемблерную программу в Linux. Однако прежде для этого требуется написать программу на языке ассемблера! В данном разделе рассматриваются основные понятия, необходимые для её написания.

Разделы и Stat

> **ПРИМЕЧАНИЕ ПЕРЕВОДЧИКА.** В исходнике заголовок напечатан как `Sections and Stat`, хотя последующий текст описывает разделы и метку `_start`.

В следующем примере показана общая структура ассемблерного файла:

```
section .text ; раздел для кода
global _start ; экспортировать начальный метод
_start:       ; выполнение начинается здесь

; здесь находится код

section .data ; раздел для данных

; здесь находятся переменные
```

Исходный ассемблерный файл разделяется на несколько основных разделов. Раздел `.text` содержит собственно ассемблерный код. Он начинается с команды `global _start`, которая экспортирует метку `_start`, сообщая внешним программам, откуда начинать выполнение кода. Затем следует метка `_start`, обозначающая адрес памяти первой инструкции программы. Остальной код располагается после этой первой инструкции.

После раздела `.text` находится раздел `.data`, содержащий данные, необходимые ассемблерной программе для работы. Распространённым примером данных внутри ассемблерной программы служат определённые в ней переменные.

Метки

Метка `_start` жизненно необходима для работы ассемблерной программы, но можно определять и другие метки. Текст `label:` в коде создаёт метку с именем `label`, представляющую постоянное значение, эквивалентное соответствующему адресу памяти.

После определения метку можно использовать вместо обычного адреса памяти. Метки допустимы везде, где может использоваться константа или непосредственное значение. В следующих примерах показаны эквивалентные инструкции с метками и без них:

```
mov eax, [ label ]         ; обратиться к dword, сохранённому по метке
mov eax, [ 0x1000 ]        ; если метка стояла у данных по адресу 0x1000,
                           ; эта инструкция эквивалентна предыдущей

jmp label2                 ; перейти к коду по метке label2
jmp 1337h                  ; если метка label2 стояла у кода по адресу 1337h,
                           ; эта инструкция эквивалентна предыдущей
```

Метки используются только для упрощения чтения и написания ассемблерного кода. После сборки кода слово `label` будет заменено ассемблером и компоновщиком соответствующим адресом памяти.

Константы

Константы упрощают работу с данными. Например, вместо запоминания максимального размера буфера, равного 1000, гораздо удобнее определить константу `MAX_SIZE` со значением 1000.

Константы в ассемблере x86 можно определять директивой `EQU`. Например, константа `MAX_SIZE` со значением 1000 определяется командой `MAX_SIZE EQU 1000`.

Глобальные данные

`nasm` позволяет объявлять место для глобальных данных различных размеров. Ниже приведены некоторые команды:

- `db`: зарезервировать место для одного байта;
- `dw`: зарезервировать место для одного слова, то есть двух байтов;
- `dd`: зарезервировать место для одного двойного слова, то есть четырёх байтов;
- `dq`: зарезервировать место для одного четверного слова, то есть восьми байтов.

Следующий пример содержит инструкции, которые используют эти команды для выделения места под данные различных типов:

```
db 0x01       ; сохранить значение "1" в одном байте
db 1, 2, 3    ; сохранить массив 1, 2, 3 как однобайтовые элементы
db 'a'        ; сохранить ASCII-значение 'a' в одном байте
db "hello", 0 ; сохранить строку "hello" с завершающим нулём
dw 0x1234     ; сохранить 0x1234 как двухбайтовое значение
dd 0xdeadbeef ; сохранить 0xdeadbeef как четырёхбайтовое значение
dq 1          ; сохранить 1 как восьмибайтовое значение
```

Хранение данных в памяти не приносит пользы, если позднее к ним нельзя обратиться. Обычно при определении глобальных данных им также назначается метка, позволяющая ссылаться на них в коде.

В следующем примере показана простая ассемблерная программа, которая выделяет место для двойного слова со значением 0, назначает ему метку `i` и помещает в него значение 1:

```
section .text
mov dword [ i ], 1
section .data
i: dd 0
```

В этом примере важно отметить, что `i` - не переменная, а символ, созданный с помощью метки. Использование `i` в коде равнозначно использованию адреса памяти выделенных данных.

Строки

В языке ассемблера строка определяется как последовательность байтов, где каждый символ занимает один байт. Например, слово «hello» можно сохранить в памяти командой `label: db "hello"`, а в коде ссылаться на него как на `label`.

При работе со строками в языке ассемблера важно помнить, что по умолчанию они не завершаются нулём. Чтобы явно завершить строку нулём, добавьте в конец нулевой байт (`0x0`), как в `label: db "hello", 0`. Завершение нулём используется почти во всех языках программирования для хранения строк, но до сих пор эту работу за вас выполнял компилятор. Теперь, когда вы овладели силой ассемблера, это придётся делать вручную, чтобы любые функции, использующие строки, выполнялись правильно. Если в конце строки нет нулевого терминатора, строковые функции продолжат получать память после предполагаемого конца строки и пытаться использовать или печатать её как символы. Результат непредсказуем: от безобидного вывода нескольких непечатаемых символов до аварийного завершения программы при попытке получить данные из памяти, к которой у неё нет разрешения на доступ.

times

Префикс `times` позволяет указать, что определённую инструкцию или префикс следует повторить заданное число раз. Это полезно для создания буферов фиксированной длины и в других ситуациях, как показано в следующих примерах:

```
times 100 db 0   ; создать 100 байтов, инициализированных нулями

times 64 db 0x55 ; создать 64 байта, каждый инициализирован значением 55h

; дополнить "hello world" до длины 64
buffer: db      'hello, world' times 64-$+buffer db ' '
```

$

`$` - сокращённое обозначение адреса текущей строки. Его можно использовать подобно метке, как показано в следующих примерах:

```
jmp $                ; бесконечный цикл

string: db "hello"
length EQU $-string  ; вычислить длину строки в предыдущей строке
```

В этом примере `length` - имя переменной. Значение `length` задаётся как текущий адрес (`$`) минус адрес, обозначенный меткой `string`. Поскольку `length` находится сразу после строки, фактически берётся адрес после «hello» и из него вычитается адрес начала, что даёт длину строки «hello».

ПРИМЕЧАНИЕ. Префиксы наподобие `times` и `$` относятся к особенностям `nasm` и не появляются в собранном коде. В других ассемблерах могут быть другие сокращения.

objdump

Такие инструменты, как `nasm` и `ld`, используются для создания исполняемого файла из ассемблерного кода. Object Dump (`objdump`) - инструмент Linux, который обращает этот процесс, принимая исполняемый файл и выводя его ассемблерный код. По мере продвижения по книге мы будем знакомиться со всё более мощными инструментами, но начинаем с `objdump`, поскольку он имеется в каждой системе на основе Linux и обеспечивает хорошую основу.

`objdump` способен выводить ассемблерный код любого приложения, выполняемого в Linux. Поэтому у него имеется множество возможных параметров настройки. Однако для начала обратной разработки наиболее важны следующие два:

• `-d`: указать `objdump` дизассемблировать содержимое всех разделов;
• `-Mintel`: указать выводить ассемблерный код в синтаксисе Intel; к сожалению, по умолчанию используется AT&T.

С учётом этого синтаксис дизассемблирования программы с именем `appname` имеет вид `objdump -d -Mintel appname`. Ниже приведён пример вывода:

```
804a030          <test_key>:

804a030          55               push            ebp

804a031          89 5e            mov             ebp, esp

804a033          53               push            ebx

804a034          83 ec 14         sub             esp, 0x14
```

Вывод `objdump` состоит из трёх столбцов. Первый содержит адреса памяти, то есть виртуальные адреса, по которым инструкции будут расположены при выполнении кода. Второй содержит машинный код x86 по соответствующему адресу, а третий - эквивалент этого машинного кода на языке ассемблера x86.

Главным исключением из этой структуры являются метки, подобные показанной в верхней строке предыдущей таблицы. У такой метки есть имя. Обратите внимание, что связанный с меткой адрес совпадает с адресом первой инструкции кода; сама метка не занимает места в памяти.

Лабораторная работа: Hello World

Настало время второй лабораторной работы. Перейдите в репозиторий книги на GitHub по адресу github.com и найдите лабораторную работу «hello world».

Навыки

Эта лабораторная работа позволяет научиться писать и собирать приложения с кодом x86. В число проверяемых ключевых навыков входят:

• регистры;

• память;
• инструкции;
• системные вызовы;
• сборка и компоновка ассемблерного кода x86.

Лабораторная работа также даёт практический опыт работы с несколькими инструментами:

• `nasm`;
• `ld`;
• Make-файлы;
• `objdump`.

Основные выводы

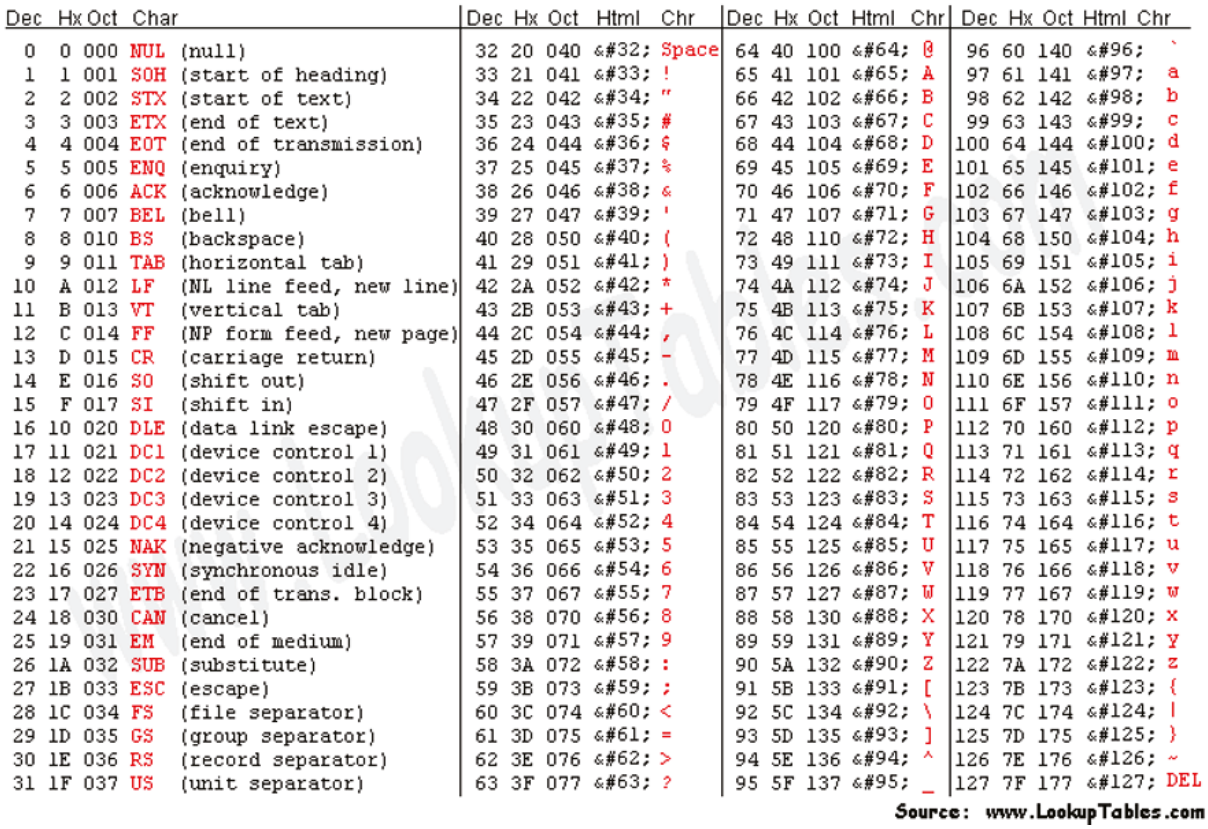
В основе приложений лежат те или иные ассемблерные инструкции. На персональном компьютере это обычно x86, но иногда может использоваться JIT-язык или промежуточный язык (IL).

Понимание создания программ на таком низкоуровневом языке помогает понять и их разбор. При взломе программ умение писать код x86 может оказаться неоценимым для разработки заплат, обходящих средства защиты программного обеспечения.

ASCII

Американский стандартный код обмена информацией (American Standard Code for Information Interchange, ASCII) и формат преобразования Unicode (Unicode Transformation Format, UTF) являются стандартами, определяющими представление текста компьютерами. В действительности ASCII является подмножеством UTF-8.

ASCII был разработан в 1960 году и использует семь битов для представления каждого символа. На рисунке 4.2 приведена полная таблица ASCII.



Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

Рисунок 4.2. Таблица ASCII

Стандарт ASCII поддерживает следующие типы символов:

- цифры (0-9);
- строчные буквы (a-z);
- прописные буквы (A-Z);
- распространённые знаки препинания.

Понимание ASCII полезно для обратной разработки, поскольку именно в таком виде строки, скорее всего, представлены в ассемблерном коде и памяти. Например, строка «Hello, world» хранится в памяти как `0x48`, `0x65`, `0x6C`, `0x6C`, `0x6F`, `0x2C`, `0x20`, `0x77`, `0x6F`, `0x72`, `0x6C`, `0x64`.

Распознавание строк ASCII

Одна из трудностей обратной разработки состоит в определении того, является ли последовательность байтов строкой ASCII, числом или чем-то ещё. Например, последовательность байтов `0x48`, `0x65`, `0x6C`, `0x6C`, `0x6F`, `0x2C`, `0x20`, `0x77`, `0x6F`, `0x72`, `0x6C` и `0x64` может быть строкой «Hello, world», значениями 1,819,043,144 или одним из множества других вариантов.

> **СОВЕТ.** Трудность распознавания строк ASCII объясняет, почему такие инструменты, как `strings`, часто возвращают много мусора. Они лишь ищут последовательности байтов, которые можно истолковать как строки печатаемых символов.

Единственный способ точно установить, что последовательность байтов является строкой, - посмотреть, как программа её использует. Если байты передаются функции, которая интерпретирует их как строку, то, вероятно, это строка.

Во многих языках полезным признаком строки служит последовательность печатаемых байтов, завершённая символом NULL. В действительности процессору ничто не сообщает, где строка

начинается или заканчивается, поэтому, если у строки отсутствует нулевой терминатор, процесс просто продолжит читать символы, пока не столкнётся с проблемой!

Например, следующая программа выведет «hello», затем 16 букв B, после чего продолжит читать и печатать память, пока не встретит байт NULL:

```
#include <stdio.h>
int main()
{
	char mybuffer[16];
	for (int i = 0; i < 16; i++)
	{
		mybuffer[i] = 'B';
	}
	Printf("hello %s\n", mybuffer);
}
```

На рисунке 4.3 показан вывод этого кода.



Рисунок 4.3. Вывод программы

Совет по работе с ASCII

Стандарт ASCII устроен так, что прописные и строчные буквы всегда разделены значением `0x20`, как показано на рисунке 4.4. В языках программирования высокого уровня функция `toUpper` просто прибавляет `0x20` к значению строчной буквы, а функция `toLower` просто вычитает `0x20`.

Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
64	40	100	@	@	96	60	140	`	`
65	41	101	A	A	97	61	141	a	a
66	42	102	B	B	98	62	142	b	b
67	43	103	C	C	99	63	143	c	c
68	44	104	D	D	100	64	144	d	d
69	45	105	E	E	101	65	145	e	e
70	46	106	F	F	102	66	146	f	f
71	47	107	G	G	103	67	147	g	g
72	48	110	H	H	104	68	150	h	h
73	49	111	I	I	105	69	151	i	i

Рисунок 4.4. Значения прописных и строчных букв ASCII

Итоги

В этой главе было рассмотрено прямое построение ассемблерных программ. Сюда относятся их взаимодействие с внешним миром, процесс сборки и компоновки и работа со строками.

Понимание этих прямых процессов может оказаться неоценимым для понимания их обратного хода. Если вы знаете, как ассемблерный код превращается из исходного текста в исполняемый файл, то лучше понимаете, как разобрать его и снова собрать воедино.

Глава 5. Понимание кодов условий

Ассемблерные инструкции обычно содержат регистры назначения, в которых сохраняется результат операции. Однако некоторые инструкции могут оказывать влияние, выходящее за рамки того, что записывается в регистр назначения.

x86 использует коды условий для отслеживания таких последствий. В этой главе рассматриваются коды условий и описываются основные из них, которые необходимо понимать для эффективной обратной разработки приложений x86.

Коды условий

Большинству архитектур, включая x86, требуется средство отслеживания основных свойств предыдущих операций. Например, при вычислении инструкции `if` программа должна проверить условие, а затем отреагировать на результат. Способность сохранять сведения о состоянии между инструкциями необходима для выполнения этой и подобных операций.

Для хранения сведений о состоянии компьютер имеет регистр специального назначения (SPR), называемый `flags`. В 32-разрядной системе он называется `eflags`, а его 16- и 64-разрядные версии - соответственно `flags` и `rflags`.

eflags

Регистр `eflags` состоит из набора флагов, каждый из которых представлен одним битом. Каждый бит может иметь истинное (1) или ложное (0) значение.

Флаги регистра `eflags` делятся на три типа:

• **Флаги состояния.** Представляют состояние некоторой операции, например указывают, равен ли нулю результат предыдущей операции.
• **Флаги управления.** Влияют на работу процессора, например разрешают и запрещают прерывания.
• **Системные флаги.** Отражают состояние процессора, например указывают, виртуализирована ли система.

32 бита регистра `eflags` позволяют хранить значительный объём сведений о состоянии. Для обратной разработки важны лишь некоторые флаги состояния.

Из флагов состояния для обратной разработки существенны четыре: флаг переноса, флаг нуля, флаг знака и флаг переполнения.

Флаг переноса

Флаг переноса (carry flag, CF) является битом 0 регистра `eflags`. Он указывает, привела ли последняя арифметическая операция к переносу.

Перенос означает, что при сложении единица вышла за старший бит или при вычитании единица была заимствована в старший бит. Рассмотрим, например, следующее вычисление, в результате которого будет установлен флаг переноса:

```
                    без знака      со знаком
```

```
  0011 0000              48            48
+ 1110 0000           + 224         + -32
1 0001 0000              16            16
```

Помните, что в самом двоичном представлении ничто не указывает, каким является значение; всё зависит от способа его использования или интерпретации. Поэтому двоичное представление в этом примере можно истолковать как беззнаковое или знаковое значение. Как видно, знаковый вариант допускает отрицательные и положительные значения, тогда как беззнаковый - только положительные.

Если проследить сложение в этом примере, видно, что из крайнего левого столбца переносится единица. Сравнив знаковые и беззнаковые значения, можно увидеть, что для беззнакового варианта флаг переноса представляет переполнение: результат оказался слишком большим для хранения в имеющемся размере, которым в данном случае является один байт. Именно так этот флаг традиционно применяется для обнаружения переполнения или антипереполнения в беззнаковой арифметике. Если возникает условие переноса, CF устанавливается в 1.

Флаг нуля

Флаг нуля (zero flag, ZF) является битом 6 регистра `eflags` и указывает, закончилась ли последняя арифметическая операция нулём. Например, следующее вычисление установит флаг нуля:

```
  0100 0000          64
- 0100 0000        - 64
  0000 0000          00
```

Флаг нуля понять проще, поскольку результат просто состоит из одних нулей. Различий в интерпретации знакового и беззнакового варианта нет. Если результат равен 0, флаг нуля устанавливается в 1.

Флаг знака

Седьмой бит регистра `eflags`, флаг знака (sign flag, SF), указывает, был ли установлен знаковый бит в результате предыдущей арифметической операции. В знаковых числах знаковым является старший бит используемого регистра.

Например, в инструкции `add ax, bx` знаковым является бит 15 регистра `ax`. В инструкции `sub bl, dl` знаковым является бит 7 регистра `bl`. Если бит установлен, значение считается отрицательным, а если не установлен - положительным. Когда знаковый бит установлен, SF также устанавливается в 1. Если старший бит результата установлен, мы знаем, что число отрицательное, но нельзя просто применить обычное преобразование остальных битов в десятичное число и получить значение. Отрицательное число хранится в формате «дополнительного кода», который требует дополнительных преобразований для восстановления истинного значения.

Флаг переполнения

Флаг переполнения (overflow flag, OF) является одиннадцатым битом регистра `eflags` и указывает, привела ли предыдущая арифметическая операция к переполнению. Переполнение возникает, когда перенос в старший бит не совпадает с переносом из него. Подобно тому как флаг переноса полезен для беззнаковой арифметики, флаг переполнения используется в знаковой арифметике для обнаружения неправильного результата.

Часто это указывает на один из двух случаев:

• положительное + положительное = отрицательное;
• отрицательное + отрицательное = положительное.

Для первого случая рассмотрим следующее вычисление:

```
  0101 0000          80
+ 0101 0000        + 80
0 1010 0000         -96
```

В этом вычислении складываются два положительных значения. Однако результат устанавливает знаковый бит, указывая на отрицательное число. Если проследить двоичное сложение по столбцам, видно, что единица переносится в крайний левый столбец, но из него ничего не переносится, то есть выходит 0. Следовательно, входящий перенос не совпал с исходящим, а в десятичной форме видно, что получено неправильное отрицательное значение.

В качестве примера второго случая рассмотрим следующее:

```
  1000 0000         -128
+ 1011 0000        + -80
1 0011 0000           48
```

В данном случае складываются два отрицательных числа. Однако из-за переполнения результат становится положительным. Снова проследив операцию на двоичном уровне, можно увидеть, что переноса в крайний левый столбец нет, но из него выполняется перенос, поэтому входящий перенос не совпадает с исходящим. В десятичной форме видно, что получено неправильное положительное значение.

Другие флаги состояния

Хотя эти четыре флага состояния наиболее важны, существуют и другие. Ниже перечислены некоторые менее важные флаги, которые всё же полезно знать:

• **Корректировка (AF).** Указывает, что последняя арифметическая операция привела к переносу из младших 4 битов; используется для арифметики BCD.
• **Ловушка (TF).** Разрешает режим пошагового выполнения центрального процессора, используемый при отладке.
• **Разрешение прерываний (IF).** Разрешает центральному процессору обрабатывать системные прерывания.
• **Направление (DF).** Задаёт направление обработки строк справа налево.
• **Чётность (PF).** Указывает, что результат последней арифметической или логической операции имеет чётную чётность, то есть содержит чётное число единиц в младшем байте.

Операции, влияющие на флаги состояния

Различные операции могут влиять на флаги состояния. В качестве четырёх примеров рассмотрим `add`, `sub`, `cmp` и `test`.

add

Инструкция `add` потенциально может изменять флаги переноса, нуля, знака и переполнения. Например, инструкция `add al, bl` способна вызывать разные сочетания флагов в зависимости от значений в `al` и `bl`. На рисунке 5.1 показаны результаты пяти разных операций `add`.

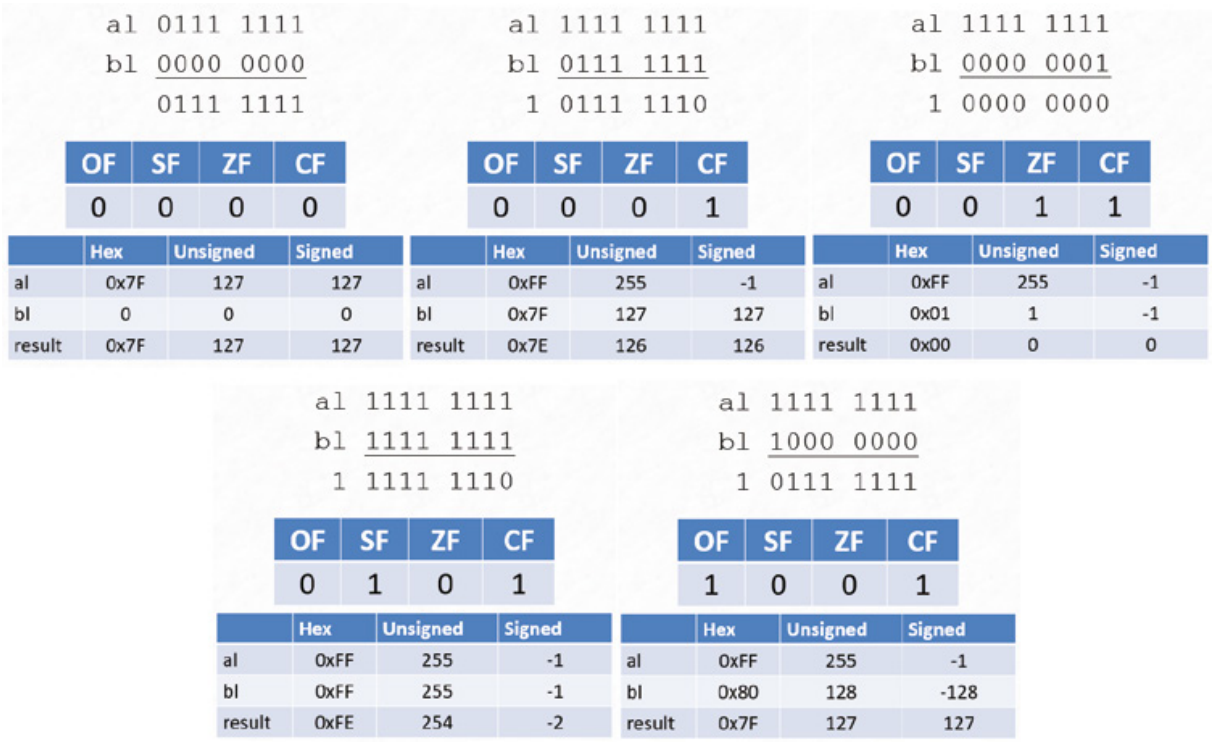


Рисунок 5.1. Воздействие add al, bl при разных входных значениях

Данные пяти примеров рисунка 5.1 полностью воспроизведены ниже. В столбцах «без знака / со знаком» указаны интерпретации `al`, `bl` и результата в соответствующем порядке.

Пример	`al` **(hex; без знака / со знаком)**	`bl` **(hex; без знака / со знаком)**	**Результат (hex; без знака / со знаком)**	**OF**	**SF**	**ZF**	**CF**
1	`0x7F`; 127 / 127	`0`; 0 / 0	`0x7F`; 127 / 127	0	0	0	0
2	`0xFF`; 255 / -1	`0x7F`; 127 / 127	`0x7E`; 126 / 126	0	0	0	1
3	`0xFF`; 255 / -1	`0x01`; 1 / -1	`0x00`; 0 / 0	0	0	1	1
4	`0xFF`; 255 / -1	`0xFF`; 255 / -1	`0xFE`; 254 / -2	0	1	0	1
5	`0xFF`; 255 / -1	`0x80`; 128 / -128	`0x7F`; 127 / 127	1	0	0	1

Обратите внимание, как использование знаковых или беззнаковых целых чисел влияет на интерпретацию значений и правильность результата. Например, результат второй операции `add` правилен для знаковых значений, но неправилен для беззнаковых.

sub

Инструкция `sub` также потенциально может изменять те же флаги, что и `add`, то есть все четыре важных флага состояния. На рисунке 5.2 показаны различные результаты `sub al, bl` при разных значениях `al` и `bl`.

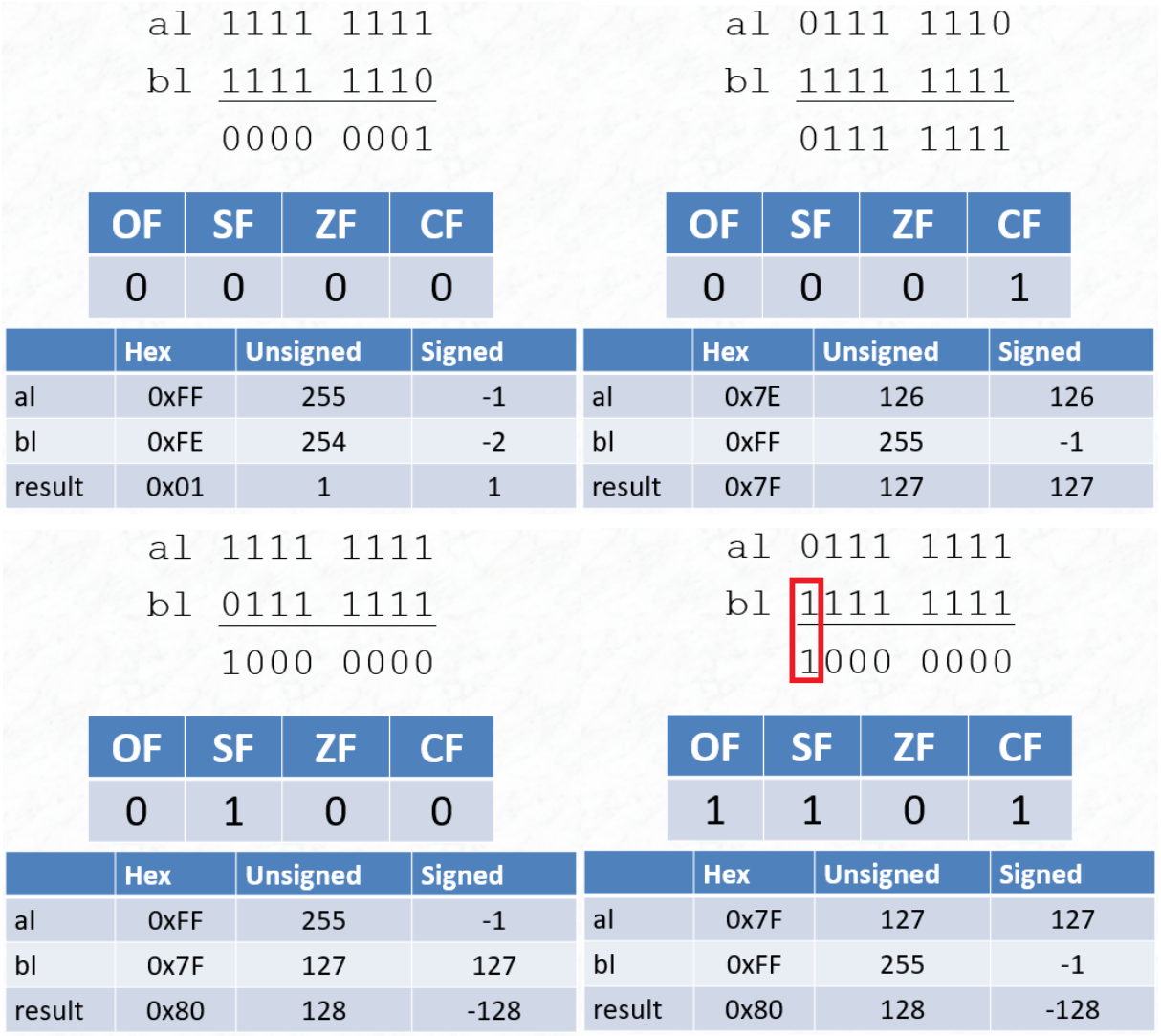


Рисунок 5.2. Воздействие sub al, bl при разных входных значениях

Пример	`al` (hex; без знака / со знаком)	`bl` (hex; без знака / со знаком)	Результат (hex; без знака / со знаком)	OF	SF	ZF	CF
1	`0xFF`; 255 / -1	`0xFE`; 254 / -2	`0x01`; 1 / 1	0	0	0	0
2	`0x7E`; 126 / 126	`0xFF`; 255 / -1	`0x7F`; 127 / 127	0	0	0	1
3	`0xFF`; 255 / -1	`0x7F`; 127 / 127	`0x80`; 128 / -128	0	1	0	0

Пример	`al` (hex; без знака / со знаком)	`bl` (hex; без знака / со знаком)	Результат (hex; без знака / со знаком)	OF	SF	ZF	CF
4	`0x7F`; 127 / 127	`0xFF`; 255 / -1	`0x80`; 128 / -128	1	1	0	1

Как и в случае `add`, правильность результата `sub` зависит от значений в `al` и `bl`. Например, обе интерпретации первого примера правильны, но во втором правильна только знаковая версия.

cmp

Инструкция `cmp` предназначена для сравнения двух значений, которыми могут быть память, константы или регистры. Она вычитает второй операнд из первого. Однако результат вычитания отбрасывается, а флаги изменяются.

Цель `cmp` - определить, больше ли одно значение другого, меньше него или равно ему. Рассмотрим следующий пример, где `eax < ebx`:

```
mov eax, 0x100
mov ebx, 0x200
cmp eax, ebx  ; вычисляет eax-ebx
```

Последняя инструкция выполняет вычитание, результат которого будет отрицательным, поскольку `ebx` больше `eax`. Поэтому флаг знака установится в 1, указывая на отрицательный результат, а флаг нуля - в 0, поскольку результат не равен нулю.

В другом примере значение первого операнда `eax` может быть больше значения второго операнда `ebx`:

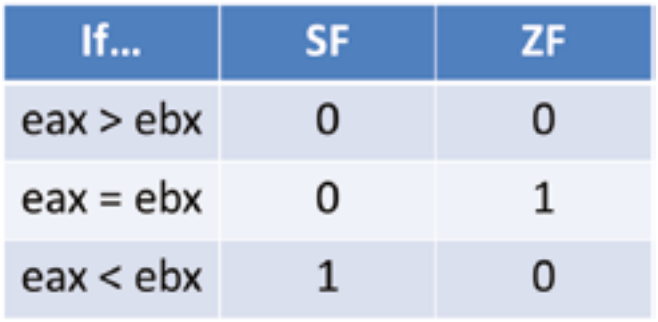
```
mov eax, 0x300
mov ebx, 0x200
cmp eax, ebx  ; вычисляет eax-ebx
```

В данном случае результат вычитания будет положительным. Поэтому и флаг знака, и флаг нуля будут равны нулю.

Последний возможный случай `cmp` возникает, если два операнда равны:

```
mov eax, 0x500
mov ebx, 0x500
cmp eax, ebx  ; вычисляет eax-ebx
```

Если операнды равны, результат вычитания равен нулю, поэтому флаг нуля устанавливается, а флаг знака - нет. На рисунке 5.3 приведена таблица истинности, обобщающая влияние операций `cmp` на флаги знака и нуля.



If...	SF	ZF
eax > ebx	0	0
eax = ebx	0	1
eax < ebx	1	0

Рисунок 5.3. Таблица истинности cmp

Условие	SF	ZF
`eax > ebx`	0	0
`eax = ebx`	0	1
`eax < ebx`	1	0

test

Инструкция `test` выполняет побитовую операцию and над двумя операндами, которыми могут быть память, константы или регистры. Как и в случае `cmp`, результат операции отбрасывается, но значения флагов изменяются.

Инструкция `test` часто используется для проверки того, установлены ли в значении один или несколько определённых битов, посредством проверки флага нуля. Например, следующие инструкции проверяют, установлен ли бит 0 или 2:

```
mov ax, 0x1450
test ax, 0x05 ; проверить, установлен ли бит 0 или 2 (0x5 в двоичной форме равно 0000 0101)
```

Эти инструкции эквивалентны следующей математической операции:

```
  0001 0100 0101 0000      (0x1450)
& 0000 0000 0000 0101      (0x0005)
  0000 0000 0000 0000
```

Результат этой операции and равен нулю, поэтому флаг нуля будет установлен. Это указывает, что ни бит 0, ни бит 2 не были установлены.

Следующие инструкции выполняют ту же проверку, когда в ax помещено значение `0x1451`:

```
mov ax, 0x1451
test ax, 0x05 ; проверить, установлен ли бит 0 или 2
```

Эти инструкции эквивалентны следующему вычислению:

```
  0001 0100 0101 0001      (0x1451)
& 0000 0000 0000 0101      (0x0005)
  0000 0000 0000 0001
```

В данном случае результат операции and ненулевой, поэтому флаг нуля не устанавливается. Это указывает, что был установлен как минимум один из двух битов.

> **СОВЕТ.** Инструкцию `test` можно использовать для определения того, является число чётным или нечётным.

Итоги

Коды условий используются для записи некоторых последствий операции, которые могут не отображаться в регистре назначения. Например, код условия может указывать, равен ли результат операции нулю или вызвала ли операция переполнение. Отслеживание этих кодов условий необходимо для понимания текущего состояния приложения при его обратной разработке.

Глава 6. Анализ и отладка ассемблерного кода

В предыдущих главах основное внимание уделялось теории и основам обратной разработки. Для успеха необходимо знать устройство x86 и распространённые форматы инструкций.

В этой главе применяется практический подход к обратной разработке и взлому программного обеспечения. Здесь представлен `gdb`, мощный отладчик, а также рассматриваются важные приёмы и рекомендации по обратной разработке и взлому программ.

Анализ двоичных файлов

Анализ существующих исполняемых файлов составляет значительную часть обратной разработки. Двоичный анализ можно выполнять несколькими способами, включая статический и динамический анализ и отладку.

Статический и динамический анализ

Функциональность программы можно анализировать несколькими способами. Два основных метода - статический и динамический анализ.

Статический анализ подразумевает изучение исходного кода без его запуска. У статического анализа имеется несколько преимуществ:

• хорошая отправная точка для дальнейшего анализа;
• безопасный метод анализа потенциально вредоносной программы;
• отсутствие необходимости в доступе к специализированным архитектурам.

У статического анализа есть свои преимущества, и одно из главных состоит в том, что он доступен всегда. Однако он может занимать много времени и не позволяет обнаружить всё. Всегда существуют части кода, обретающие смысл только во время выполнения. При анализе сложного кода без наблюдения за его работой бывает трудно или невозможно предвидеть, куда приведёт, например, переход. Кроме того, многие пути выполнения определяются входными данными программы, поэтому статического анализа недостаточно для рассуждений о дальнейшем ходе выполнения, что усложняет анализ.

Динамический анализ - дополняющий метод, при котором программу запускают и анализируют её поведение во время работы. К преимуществам динамического анализа относятся:

• более быстрый анализ;
• более широкое обнаружение потенциальных проблем.

Динамический анализ может принимать множество форм. В обратной разработке одной из наиболее распространённых является отладка. Наблюдение за работающим приложением позволяет уточнить многие неизвестные при статическом анализе обстоятельства, например наиболее вероятное место перехода. Однако динамический анализ требует запускать исследуемый код, а в зависимости от кода это не всегда возможно.

Код может быть фрагментом более крупного приложения, требовать уникальной среды выполнения, к которой у вас нет доступа, либо, в случае вредоносной программы, представлять потенциальную опасность при запуске.

Отладка

Помните, что цель обратной разработки и взлома программного обеспечения заключается в понимании и изменении существующей программы. Отладка - один из самых быстрых и эффективных способов достижения этой цели. Динамически анализируя функции программы и на ходу изменяя её поведение, можно собрать сведения, необходимые для взлома защиты, и испытать возможные варианты взлома.

Отладка обычно представляет собой многоэтапный процесс. Типичный порядок отладки выглядит следующим образом:

1. Установить точки останова в интересующих местах.
2. Запустить код.
3. Выполнение приостанавливается, то есть «останавливается», в точке останова.
4. Изучить состояние программы.
5. При необходимости внести изменения.
6. Повторить.

Точки останова

Точки останова приказывают процессору остановить выполнение программы в определённом месте. Они бывают двух видов:

• **Программные.** Программные точки останова устанавливаются на ассемблерных инструкциях, и их число не ограничено.
• **Аппаратные.** Ограниченное число аппаратных точек останова, четыре в x86, можно устанавливать на ассемблерных инструкциях или обращениях к памяти.

В первых лабораторных работах этой книги используются программные точки останова, а в последующих появляются аппаратные. В книге будут продемонстрированы разные отладчики, и точки останова в каждом из них настраиваются по-разному.

Программные точки останова

Для большинства отладчиков программные точки останова являются вариантом по умолчанию. При установке программной точки останова отладчик в действительности изменяет инструкцию за кулисами, заменяя её инструкцией точки останова. В x86 такой инструкцией является `int3` (`0xcc`). Программная точка останова ограничена выполнением: для её срабатывания должна быть исполнена инструкция `int3`.

Когда процессор достигает инструкции точки останова, он прекращает выполнение и возвращает управление отладчику. Это позволяет специалисту по обратной разработке изучить состояние программы и при необходимости внести изменения.

Главное ограничение программных точек останова состоит в том, что программа, читающая собственную память, легко может их обнаружить. Применяя средства противодействия отладке, программа способна удалить точку останова или предпринять в ответ другие защитные действия.

Аппаратные точки останова

Большинство отладчиков поддерживают аппаратные точки останова. Однако обычно их необходимо выбирать и настраивать вручную.

Аппаратная точка останова не изменяет код программы, как это делает программная. Вместо этого адреса точек останова хранятся в аппаратных регистрах.

В x86 для аппаратных точек останова используются отладочные регистры `DR0-7`. Регистры `DR0-4` содержат адреса точек останова, а `DR6,7` - сведения о конфигурации.

Аппаратные точки останова можно настроить на срабатывание при выполнении, чтении или записи определённого адреса. Когда процессор обнаруживает условие, совпадающее с регистрами точки останова, он передаёт управление отладчику.

Аппаратные точки останова полезны тем, что способны обнаруживать обращения к памяти. Например, с их помощью можно определить, в каком месте кода задаётся определённый байт или используется строка.

Аппаратные точки останова также помогают обойти защиту программы от программных точек. Если программа сканирует собственный код в поисках инструкций `int3`, она не заметит аппаратные точки останова, которые код не изменяют. Это не даёт полной защиты от средств противодействия отладчику: обладая глубокими знаниями системы, приложение может добраться достаточно глубоко, чтобы наблюдать и за аппаратными точками останова, но по сравнению с программными точками это значительно повышает сложность обнаружения.

gdb

GNU Debugger (`gdb`) фактически является стандартом отладки в Linux. Он установлен во многих дистрибутивах Linux и может быть установлен в любом из них. К основным возможностям `gdb` относятся:

• отладчик командной строки без графического интерфейса;
• поддержка сценариев;
• поддержка удалённой отладки.

`gdb` настолько распространён, что многие системы и процессоры включают заглушку `gdb` для поддержки отладки с его помощью. Многие отладчики ограничены несколькими архитектурами и платформами, тогда как `gdb` работает на сотнях. В этой книге рассматриваются разные отладчики, а позднее будут представлены отладчики с графическим пользовательским интерфейсом (GUI). Однако важно заложить базовое понимание работы с GDB в командной строке. Многие более «красивые» отладчики используют GDB и его протокол внутри, просто оборачивая их привлекательным интерфейсом.

Отладка с помощью gdb

Как программа командной строки, `gdb` управляется командами, вводимыми в приглашении `(gdb)`. Интерфейс `gdb` может казаться архаичным, но это чрезвычайно мощный и очень популярный отладчик.

Одна из полезных возможностей `gdb` состоит в том, что команды можно вводить в кратчайшей однозначной форме. Например, `run` сокращается до `r`, `info registers` - до `info reg`, а `disassemble` - до `disas`. По мере работы вы привыкнете к некоторым из этих сокращений.

Запуск gdb

gdb запускается командой gdb. Например, исполняемый файл `printreg-shift.out` можно запустить командой gdb `printreg-shift.out`, как показано на рисунке 6.1.

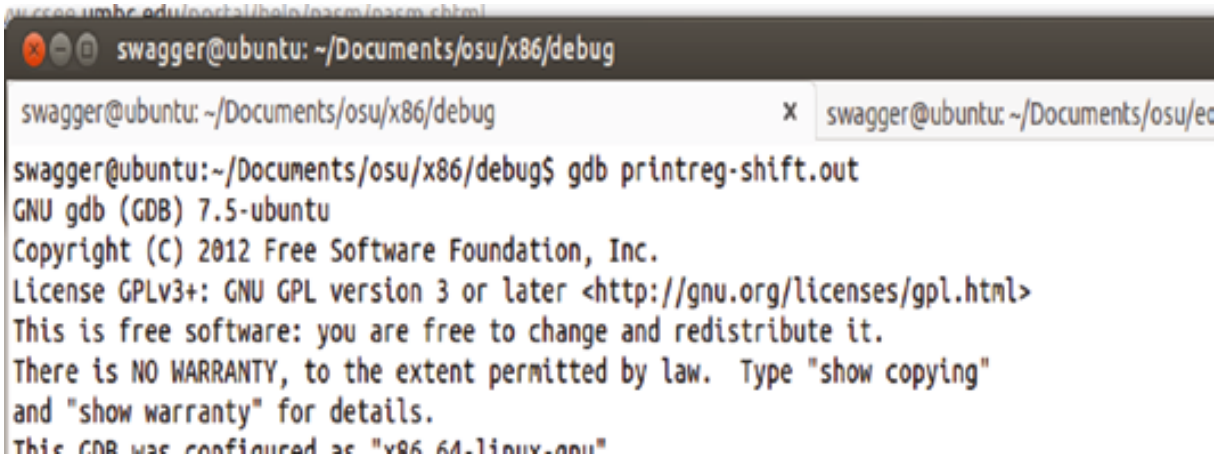


Рисунок 6.1. Команда gdb

```
swagger@ubuntu:~/Documents/osu/x86/debug$ gdb printreg-shift.out
GNU gdb (GDB) 7.5-ubuntu
Copyright (C) 2012 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
```

Дизассемблирование в gdb

Помните, что x86 имеет несколько вариантов синтаксиса, включая Intel и AT&T. Для выбора синтаксиса Intel в gdb используется инструкция `(gdb) set disassembly-flavor intel`.

После выбора варианта синтаксиса существует несколько способов начать отладку:

- `disassemble` начинает дизассемблирование с текущего указателя инструкций;
- `disassemble адрес` начинает дизассемблирование с указанного адреса;
- `disassemble метка` начинает дизассемблирование с указанной метки, например `loop` или `main`.

На рисунке 6.2 приведён пример дизассемблирования с использованием метки, где требуемый фрагмент кода начинается с метки `loop`. Слева показан исходный ассемблерный код, а справа - его эквивалентное дизассемблирование в gdb.

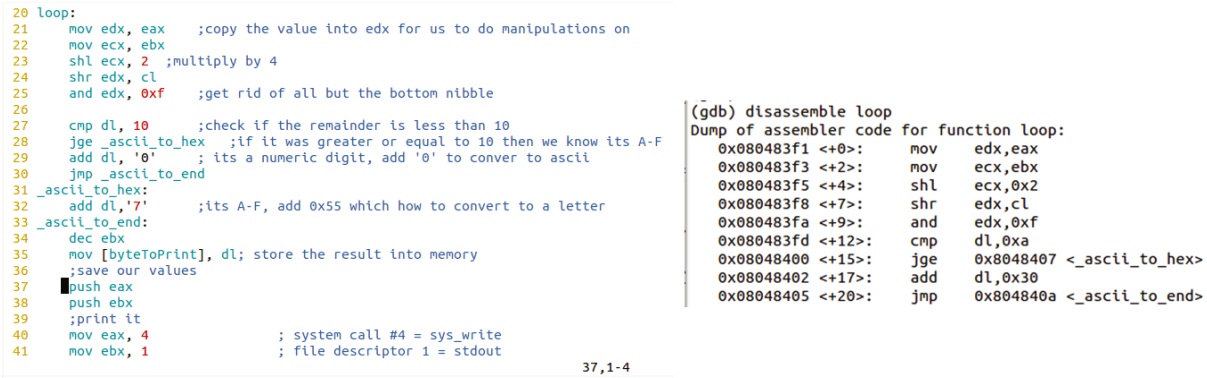


Рисунок 6.2. Дизассемблирование в gdb

```
(gdb) disassemble loop
Dump of assembler code for function loop:
   0x080483f1 <+0>:     mov    edx,eax
   0x080483f3 <+2>:     mov    ecx,ebx
   0x080483f5 <+4>:     shl    ecx,0x2
   0x080483f8 <+7>:     shr    edx,cl
   0x080483fa <+9>:     and    edx,0xf
   0x080483fd <+12>:    cmp    dl,0xa
   0x08048400 <+15>:    jge    0x08048407 <_ascii_to_hex>
   0x08048402 <+17>:    add    dl,0x30
   0x08048405 <+20>:    jmp    0x0804840a <_ascii_to_end>
```

После указания начальной точки можно задать число инструкций для дизассемблирования. Например, команда `disassemble main +50` начинает дизассемблирование с метки `main` и печатает 50 инструкций.

Запуск и остановка кода в gdb

Команда `run` запускает программу с самого начала. При этом все сведения о состоянии, накопленные при выполнении программы до этого момента, отбрасываются.

Команда `continue` возобновляет выполнение после приостановки. Например, точка останова может остановить выполнение для просмотра стека программы, после чего команда `continue` возобновит его.

Выполнение программы можно прекратить командами `quit` и `kill`. `kill` завершает работающий процесс, а `quit` завершает его и выходит из `gdb`.

Точки останова gdb

Точки останова прекращают выполнение кода, позволяя анализировать состояние программы. В `gdb` команда `break адрес` задаёт точку останова по определённому адресу, а `break метка` использует метку для обозначения требуемого места точки останова, как показано на рисунке 6.3.

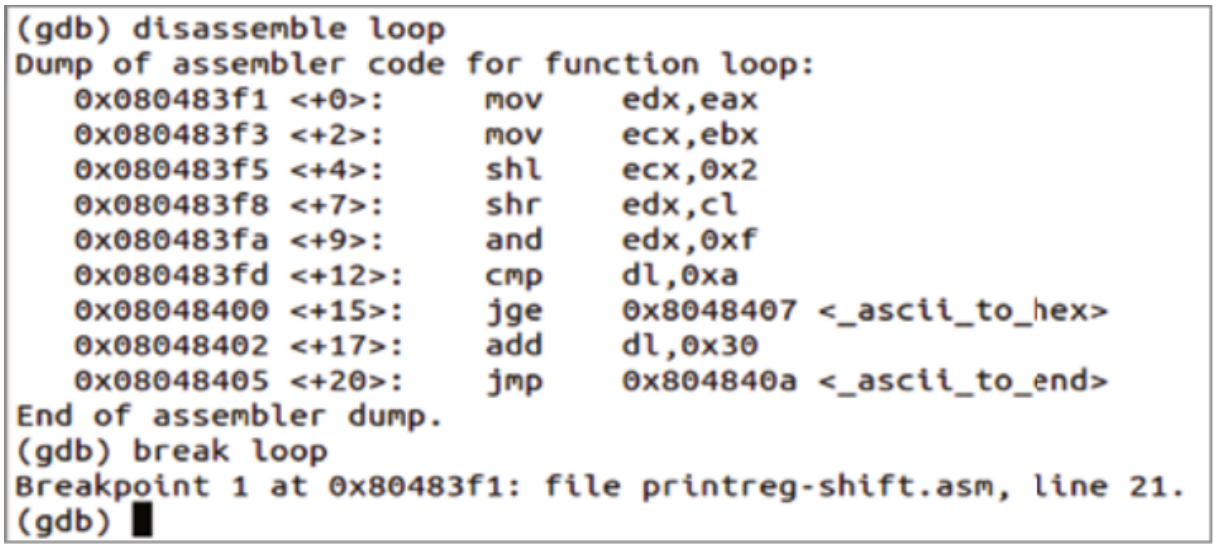


Рисунок 6.3. Установка точки останова в gdb

```
(gdb) disassemble loop
Dump of assembler code for function loop:
   0x080483f1 <+0>:     mov    edx,eax
   0x080483f3 <+2>:     mov    ecx,ebx
   0x080483f5 <+4>:     shl    ecx,0x2
```

```
   0x080483f8 <+7>:     shr    edx,cl
   0x080483fa <+9>:     and    edx,0xf
   0x080483fd <+12>:    cmp    dl,0xa
   0x08048400 <+15>:    jge    0x08048407 <_ascii_to_hex>
   0x08048402 <+17>:    add    dl,0x30
   0x08048405 <+20>:    jmp    0x0804840a <_ascii_to_end>
End of assembler dump.
(gdb) break loop
Breakpoint 1 at 0x080483f1: file printreg-shift.asm, line 21.
(gdb)
```

Команды gdb info

Команда `info` в gdb позволяет получать сведения разных типов. К распространённым командам `info` относятся:

• `info files`: показывает различные части дизассемблированного файла. На рисунке 6.4 приведён пример для простого исполняемого файла `a.out`.



Рисунок 6.4. Команда gdb info files

```
(gdb) info files
Symbols from "/home/swagger/Documents/osu/x86/a.out".
Unix child process:
        Using the running image of child process 61165.
        While running this, GDB does not access memory from...
Local exec file:
        `/home/swagger/Documents/osu/x86/a.out', file type elf32-i386.
        Entry point: 0x80480d1
        0x08048080 - 0x080480dd is .text
        0x080490e0 - 0x080490e5 is .data
(gdb)
```

• `info breakpoints`: перечисляет точки останова, определённые в данный момент для дизассемблированной программы;
• `info register`: отображает текущие значения регистров x86, как показано на рисунке 6.5;
• `info variables`: показывает все определённые в приложении переменные, как показано на рисунке 6.6.

Помимо команды `info register`, значения отдельных регистров можно выводить командой `print $reg`:

```
(gdb) print $esp
$1 = (void *) 0xffffd260
(gdb)
```

```
Starting program: /home/swagger/Documents/osu/x86/debug/printreg-shift.out

Breakpoint 1, loop () at printreg-shift.asm:21
21              mov edx, eax    ;copy the value into edx for us to do manipulations on
(gdb) info register
eax            0xabcdef12       -1412567278
ecx            0xffffd304       -11516
edx            0xffffd294       -11628
ebx            0x7      7
esp            0xffffd268       0xffffd268
ebp            0x0      0x0
esi            0x0      0
edi            0x0      0
eip            0x80483f1        0x80483f1 <loop>
eflags         0x246    [ PF ZF IF ]
cs             0x23     35
ss             0x2b     43
ds             0x2b     43
es             0x2b     43
fs             0x0      0
gs             0x63     99
(gdb)
```


Рисунок 6.5. Команда gdb info register

```
(gdb) info register
eax            0xabcdef12       -1412567278
ecx            0xffffd304       -11516
edx            0xffffd294       -11628
ebx            0x7              7
esp            0xffffd268       0xffffd268
ebp            0x0              0x0
esi            0x0              0
edi            0x0              0
eip            0x80483f1        0x80483f1 <loop>
eflags         0x246            [ PF ZF IF ]
cs             0x23             35
ss             0x2b             43
ds             0x2b             43
es             0x2b             43
fs             0x0              0
gs             0x63             99
(gdb)
```

```
(gdb) info variables
All defined variables:

Non-debugging symbols:
0x080490e0  loop_index
0x080490e4  byteToPrint
0x080490e5  __bss_start
0x080490e5  _edata
0x080490e8  _end
(gdb)
```

Рисунок 6.6. Команда gdb info variable

```
(gdb) info variables
All defined variables:

Non-debugging symbols:
0x080490e0  loop_index
0x080490e4  byteToPrint
0x080490e5  __bss_start
0x080490e5  _edata
```

```
0x080490e8  _end
(gdb)
```

Пошаговое выполнение инструкций

Команды `run` и `continue` просто снова запускают программу до тех пор, пока что-либо, например точка останова, не заставит выполнение прекратиться. Из-за этого труднее наблюдать за действиями программы или изменением переменных со временем.

Команда `stepi` выполняет по одной инструкции, как показано на рисунке 6.7, что позволяет проводить более глубокий анализ. Обратите внимание, что комментарии отображаются потому, что приложение было собрано в режиме отладки.

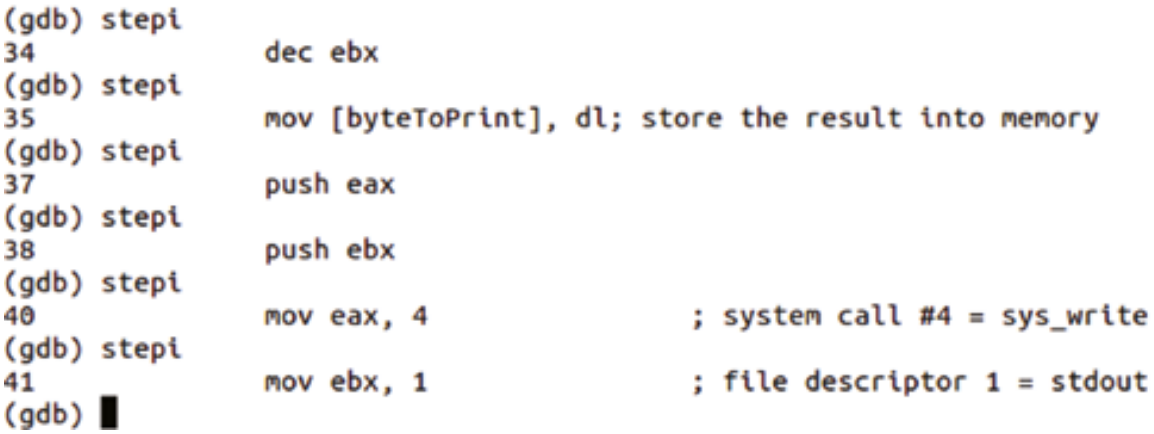


Рисунок 6.7. Команда gdb stepi

```
(gdb) stepi
34            dec ebx
(gdb) stepi
35            mov [byteToPrint], dl; сохранить результат в памяти
(gdb) stepi
37            push eax
(gdb) stepi
38            push ebx
(gdb) stepi
40            mov eax, 4          ; системный вызов #4 = sys_write
(gdb) stepi
41            mov ebx, 1          ; файловый дескриптор 1 = stdout
(gdb)
```

> **ПРИМЕЧАНИЕ.** Отладочные сведения, например имена функций и переменных, комментарии и прочее, можно включить при компиляции с помощью флага отладки, поддерживаемого большинством компиляторов. Например, в `gcc`/`g++` это флаг `-g`. Однако при обратной разработке коммерческого исполняемого файла такие сведения встречаются редко.

Если следующая выполняемая инструкция является вызовом функции, `stepi` войдёт внутрь вызываемой функции. Во многих случаях это нежелательно, особенно если функция хорошо понятна. Например, достаточно знать, что инструкция выводит строку, и нет необходимости исследовать каждую инструкцию внутри функции `printf`.

Команда `nexti` позволяет выполнить вызов функции, не входя в неё. Она выполняет вызов и переходит к следующей видимой инструкции.

Исследование памяти

В gdb для исследования памяти применяется команда x, сокращение от «eXamine memory». Её синтаксис имеет вид x/nfu addr.

В этой команде n, f и u являются необязательными параметрами со следующими значениями:

• n: число отображаемых единиц (u) памяти;
• f: формат отображения:
• s: строка с нулевым терминатором;
• i: машинная инструкция;
• x: шестнадцатеричный формат, используемый по умолчанию;
• u: размер единицы:
• b: байты;
• h: полуслова, то есть два байта;
• w: слова, используемые по умолчанию;
• g: гигантские слова, то есть восемь байтов.

На рисунке 6.6 была показана переменная byteToPrint, расположенная по адресу 0x080490e4. Чтобы вывести 16 байтов памяти по этому адресу, используется команда x/16x 0x80490e4, как показано на рисунке 6.8. Вывод команды показывает, что интересующий байт имеет значение 0x41, соответствующее символу A в ASCII.

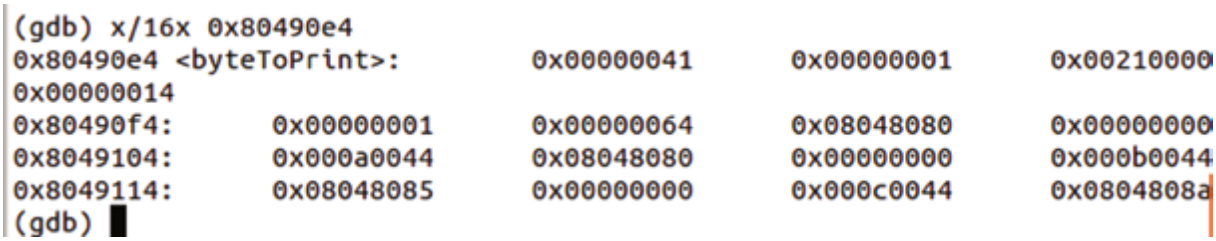


Рисунок 6.8. Команда gdb x

```
(gdb) x/16x 0x80490e4
0x80490e4 <byteToPrint>:  0x00000041  0x00000001  0x00210000
0x00000014:               0x00000001  0x00000064  0x08048080  0x00000000
0x80490f4:                0x00000001  0x08048080  0x00000000  0x0000b044
0x8049104:                0x000a0044  0x08048080  0x00000000  0x0804808a
(gdb)
```

Помимо адресов памяти команда x может использовать регистры для указания места дампа. Например, на рисунке 6.9 показан вывод 10 байтов в шестнадцатеричном формате по адресу, указанному esp.

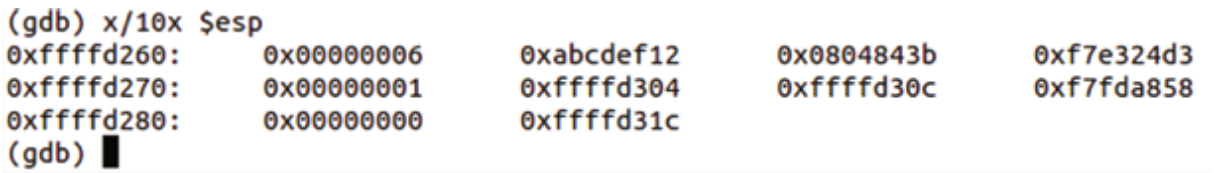


Рисунок 6.9. Вывод 10 байтов командой gdb x

```
(gdb) x/10x $esp
0xffffd260:  0x00000006  0xabcdef12  0x0804843b  0xf7e324d3
0xffffd270:  0x00000001  0xffffd304  0xffffd30c  0xf7fda858
0xffffd280:  0x00000000  0xffffd31c
(gdb)
```

Ошибки сегментации

Ошибки сегментации, или segfault, возникают, когда центральный процессор пытается прочитать или записать недоступную область памяти. Это может произойти потому, что указанной области не существует либо процессору не хватает разрешений для доступа к ней или её изменения.

Например, команда `mov eax, [0x00000000]` всегда приводит к ошибке сегментации. Причина в том, что адрес `0x0` обычно не отображён в память приложения или недоступен ему, поэтому чтение из памяти по адресу `0x0` в `gdb` вызывает ошибку сегментации.

Ошибки сегментации могут возникать по самым разным причинам. При эксплуатации программного обеспечения такую ошибку способно вызвать переполнение буфера. При взломе защиты она может возникнуть из-за неправильной заплаты или ошибки при изменении выполнения во время отладки. Начав писать и изменять ассемблерный код, вы близко познакомитесь с ошибкой сегментации. При взломе защиты может оказаться полезным держать рядом антистрессовый мячик и сжимать его при каждом появлении segfault. У этой ситуации есть и светлая сторона: если программа подключена к GDB, при ошибке сегментации `gdb` покажет вызвавшую её строку, что помогает найти место, где код пошёл наперекосяк.

Лабораторная работа: Shark Sim 3000

Эта лабораторная работа служит разминочной задачей по обратной разработке. Приложение намеренно спроектировано так, чтобы его поведение было неочевидным. Опыт расшифровки и анализа подобных программ является основой взлома программного обеспечения.

Перейдите на страницу книги в GitHub и найдите лабораторную работу Shark Sim 3000.

Навыки

Эта лабораторная работа предназначена для проверки основных навыков обратной разработки, включая способность разобрать и понять неизвестную программу. Проверяются следующие навыки:

• ASCII;
• коды условий;
• отладка скомпилированных программ;
• расшифровка неизвестных ассемблерных инструкций и программ.

Основные выводы

Отладка - неоценимый инструмент обратной разработки приложения. Основополагающая часть обратной разработки состоит в расшифровке новых инструкций: времени на понимание всего хватает не всегда!

Один из секретов успеха в обратной разработке - не застревать на неизвестных ассемблерных инструкциях. Постарайтесь быстро понять основы их работы, чтобы двигаться дальше. В последующих главах вы продолжите работать с `gdb` и другими, более мощными отладчиками. Теперь, когда вы начали изменять ассемблерный код, можно перейти к работе вообще без исходного текста.

Отсечение шума

Эффективная обратная разработка и взлом программного обеспечения требуют владения несколькими разными наборами навыков. Однако один из самых важных - способность отсекать шум и сосредоточиваться на существенном.

Даже небольшие программы содержат слишком много кода, чтобы анализировать всё. Подавляющее большинство инструкций не связано с основными функциями приложения, и тратить время на их обратную разработку бессмысленно. Часто знание того, на чём не следует сосредоточиваться, важнее знания того, что следует изучать.

При выборе того, чему уделять и не уделять внимание, необходимо понимать основы. К чрезвычайно распространённому коду, который следует узнавать сразу, относятся:

• конструкции потока управления;
• устройство стека: локальные переменные, входные и выходные параметры;
• шаблонный код компилятора: прологи и эпилоги, канарейки, выделение стека и управление регистрами.

Всё это будет подробно рассмотрено в следующих главах, чтобы вы научились быстро распознавать подобный код.

При расстановке приоритетов обратной разработки следует соблюдать следующий порядок операций: вызовы функций -> поток управления -> инструкции -> шаблонный код.

• **Вызовы функций.** Сосредоточьтесь на определении вызываемых функций. Зачастую знания о том, что функция вызывает `CreateDialog`, достаточно для понимания её назначения.
• **Поток управления.** При необходимости исследуйте поток управления, например установите, что `CreateDialog` вызывается в цикле внутри этой функции.
• **Отдельные инструкции.** Если этого недостаточно, исследуйте отдельные инструкции.
• **Шаблонный код компилятора.** Исследование шаблонного кода почти никогда не приносит пользы для обратной разработки или взлома программ. Однако важно понимать типичный шаблонный код, чтобы быстро распознавать и игнорировать его.

Итоги

В этой главе был представлен `gdb`, мощный и широко используемый отладчик для систем Linux. Будущему специалисту по обратной разработке или взлому защиты важно познакомиться с `gdb` и получить практический опыт работы с ним, поскольку этот инструмент позволяет анализировать программное обеспечение для множества различных систем.

Глава 7. Функции и поток управления

Программа представляет собой последовательность инструкций, однако приложение не всегда переходит линейно от одной инструкции к следующей. При обратной разработке и взломе приложения жизненно важно понимать потоки управления и различные факторы, способные на них влиять, например операторы `if` и циклы в языках высокого уровня.

При обратной разработке кода x86 или программы на языке высокого уровня вы, скорее всего, также столкнётесь с функциями. В этой главе рассматривается, как работают функции в x86 и как они влияют на стек программы.

Поток управления

До сих пор рассматривавшийся в этой книге ассемблерный код следовал последовательному потоку инструкций. Выполнение просто продолжалось сверху вниз. Однако большинство приложений не являются полностью последовательными. Рассмотрим следующий блок кода:

```
if (x) {
    // Сделать что-либо
}
```

При выполнении этого кода процессор вычисляет условие `x` и определяет, истинно ли оно. Если да, процессор переходит к инструкциям внутри блока `if`.

Однако если условие `x` не истинно, инструкции внутри блока `if` пропускаются. Для этого должна существовать возможность указать процессору, какие инструкции выполнять, а какие нет, тем самым изменив поток выполнения.

Указатель инструкций

Регистр `eip` известен как указатель инструкций и содержит адрес следующей инструкции, подлежащей выполнению. После выполнения инструкции процессор автоматически увеличивает значение, хранящееся в `eip`.

Увеличение `eip` после каждой инструкции позволяет выполнять последовательную серию инструкций. Однако в некоторых случаях требуется выполнять код условно. Для этого `eip` необходимо обновлять иначе. Напрямую изменять `eip` нельзя (вспомните, что это регистр специального назначения, SPR). Вместо этого для изменения `eip` применяются инструкции управления потоком.

Инструкции управления потоком

Наиболее распространённые отклонения от обычного потока выполнения, вынуждающие изменять `eip`, называются переходами или ветвлениями. Например, в следующем блоке кода ветвление находится в операторе `if`:

```
int x = 1;
int y = 2*x;
if (!y) { // ветвление!
    x = 2;
}
```

При ассемблировании такого кода высокого уровня инструкции перехода указывают, какое значение следует записать в регистр eip. Инструкция перехода имеет синтаксис jmp op, где op может быть адресом памяти или меткой.

Инструкция jmp является безусловным переходом, который выполняется всегда (условные переходы рассматриваются далее в этой главе).

jmp

Инструкция jmp имеет синтаксис jmp op. Её назначение - передать поток управления программы в ячейку памяти op, установив eip равным значению, хранящемуся в op. Ниже приведены несколько примеров инструкций jmp:

```
jmp eax       ; Копирует eax в eip (переходит к eax)
jmp label     ; Переходит к инструкции по метке label
jmp $         ; Бесконечный цикл в nasm (ценный
              ; инструмент отладки в ассемблере)
```

Инструкции jmp можно использовать для реализации различных функций. Например, следующие инструкции в бесконечном цикле ведут счёт вверх от нуля:

```
      mov eax, 0
loop: add eax, 1
      jmp loop
```

Условные переходы

Условные переходы позволяют связать выполнение или невыполнение перехода с истинностью либо ложностью условия. Нужно ли выполнять переход, определяется по значениям, хранящимся во флагах состояния. Например, рассмотрим следующие инструкции:

```
cmp eax, ebx
jle done
```

Инструкция jle (переход, если меньше или равно) перейдёт по указанному адресу или к указанной метке, если регистр флагов показывает, что результат предыдущего сравнения был меньше или равен. В данном случае непосредственно предшествующая ей инструкция cmp сравнивает eax и ebx и устанавливает флаги. Напомним, что cmp вычитает операнд 2 из операнда 1 и отбрасывает результат. Поэтому для выполнения условия «меньше или равно» значение eax должно быть меньше или равно ebx. В таком случае процессор перейдёт к метке done. В противном случае переход будет пропущен, а выполнение продолжится с инструкции, следующей за jle.

В языке x86 существует множество инструкций условного перехода. В таблице 7.1 перечислены эти инструкции и условия, определяющие выполнение перехода.

Таблица 7.1. Инструкции условного перехода x86

Инструкция	Значение	Условия
je	Перейти, если равно.	ZF = 1
jz	Перейти, если последний результат равен нулю.	ZF = 1
jne	Перейти, если не равно.	ZF = 0

Инструкция	Значение	Условия
`jge`	Перейти, если больше или равно.	`SF = OF`
`jl`	Перейти, если меньше.	`SF != OF`
`jle`	Перейти, если меньше или равно.	`ZF = 1 OR SF != OF`
`jg`	Перейти, если больше.	`ZF = 0 AND SF == OF`

Из таблицы можно заметить, что у некоторых инструкций условия совпадают. Например, и `jz`, и `je` выполнят переход, если флаг нуля (`ZF`) установлен в единицу. Это означает, что переход выполняется, если два указанных значения равны. Но логически эти инструкции трактуются по-разному. «Перейти, если предыдущий результат равен нулю» может использоваться после вычитания двух чисел, тогда как «перейти, если равно», скорее всего, будет использоваться после сравнения.

Например, рассмотрим случай, когда `eax = ebx = 0x10`. Инструкция `cmp eax, ebx` выполняет вычитание и при исполнении установит флаг нуля. Если вслед за ней идёт `jz` или `je`, любая из этих инструкций выполнит переход.

Эти инструкции взаимозаменяемы, однако обычно их выбирают в зависимости от инструкции, установившей определяющие переход флаги. Например, если для проверки условия применяется инструкция `sub eax, ebx`, то, вероятно, будет использована `jz`, поскольку проверяется нулевой результат математической операции. Если применяется инструкция `cmp eax, ebx`, будет использована `je`, поскольку сравнение проверяет эквивалентность.

Помните, что `cmp` за кулисами выполняет вычитание, поэтому она воздействует на флаги так же, как операция `sub`. `jz` и `je` - синонимичные инструкции, предназначенные исключительно для повышения читаемости ассемблерного кода.

Ловушки условных переходов

Чтобы определить, следует ли выполнять переход, условные переходы используют флаги состояния. Но каждая инструкция действует изолированно, и условный переход не знает, к какому сравнению или математическому выражению вы хотите его применить. Если между условной инструкцией и переходом флаги состояния изменятся, это может привести к проблемам. Компилятор такой путаницы не допустит, но при самостоятельном написании ассемблерного кода рассуждать об этом придётся вам.

Например, рассмотрим следующий набор инструкций:

```
cmp eax, ebx
cmp edx, ecx
jle done
```

Возможно, предполагалось, что выполнение перехода будет определять инструкция `cmp eax, ebx`. Однако непосредственно перед переходом флаги последней устанавливает `cmp edx, ecx`, перезаписывая прежние значения. Следовательно, переход выполняется по результату второго сравнения, а не первого.

Когда подряд идут несколько инструкций `cmp`, может быть очевидно, что флаги для перехода устанавливает последняя из них. Но с другими инструкциями это бывает менее очевидно, как в

следующем примере:

```
cmp eax, ebx
add ecx, 1
je done
```

В этой последовательности, возможно, предполагалось использовать для установки флагов перехода инструкцию `cmp`. Однако инструкция `add` также обновляет флаги состояния и перезаписывает прежние значения. Вместо перехода при `eax = ebx` переход выполняется, если инструкция `add` устанавливает флаг нуля (то есть если `ecx + 1 = 0`).

Пример

Инструкции перехода часто применяются для реализации операторов `if` и циклов. Следующий ассемблерный код с помощью цикла суммирует числа от 0 до 4:

```
        mov   eax, 0    ; инициализировать eax (аккумулятор) значением 0
        mov   ecx, 0    ; инициализировать ecx (счётчик) значением 0

loop:
        add   eax, ecx  ; прибавить значение текущей итерации
        add   ecx, 1    ; увеличить счётчик
        cmp   ecx, 5    ; уже выполнено 5 итераций?
        jne   loop      ; продолжить цикл, если ещё не 5

done:
```

Итератор этого цикла хранится в регистре `ecx` и инициализируется до начала цикла. Регистр `eax` является регистром-аккумулятором и содержит текущую сумму.

Цикл начинается с прибавления текущего значения счётчика к аккумулятору, после чего счётчик увеличивается. Так при выполнении итераций реализуется требуемая логика суммирования значений от 0 до 4.

Ветвление происходит на инструкции `jne` (переход, если не равно) в предпоследней строке. Предыдущая инструкция `cmp` проверяет, равен ли счётчик цикла 5, и соответствующим образом устанавливает флаги состояния. Если счётчик не равен 5, переход срабатывает и регистр `eip` получает адрес, указанный меткой `loop`, начиная новую итерацию. Если счётчик равен 5, переход не выполняется и процессор продолжает с метки `done`.

Логические конструкции в x86

В C/C++ и похожих языках высокого уровня имеется несколько логических конструкций, вызывающих непоследовательное выполнение кода. Ниже приведены некоторые примеры:

```
if (...) { ... }
if (...) { ... } else { ... }
if (...) { ... } else if (...) { ... } else { ... }
while (...) { ... }
do { ... } while (...);
for (...; ...; ...) { ... }
switch (...) { ... }
```

В ассемблере эти логические конструкции записываются с помощью сочетания инструкций сравнения (`cmp`) и перехода (`jmp`, `je`, `jne`, `jl`, `jle`, `jg` и `jge`). При компиляции кода компилятор автоматически выполняет преобразование в ассемблерный код.

При написании ассемблерного кода преобразовывать высокоуровневые понятия в ассемблер приходится вручную. А при анализе чужого кода необходимо понимать, как эти структуры выглядят в ассемблере. Чтобы научиться их распознавать, сосредоточьтесь на том, как преобразовать понятия языка высокого уровня в ассемблер. Это двухэтапный процесс:

1. **Удалить блоки кода.** Переписать код, заменив логические конструкции операторами `goto`.
2. **Ассемблировать.** Переписать программу на ассемблере.

if (...) { ... }

Оператор `if` - одна из простейших высокоуровневых логических конструкций. С блоками кода он выглядит так:

```
if (condition)
{
    code_if_true;
}
```

Первый шаг - удалить блоки кода. Блоки кода - это код, заключённый в фигурные скобки `{}`. Удаляя эти блоки, используйте операторы `goto`, указывающие, куда должно перейти выполнение. Понятие `goto` есть не во всех языках высокого уровня, но рассматривайте это как псевдокод и используйте `goto`. Ниже тот же оператор `if` записан без блоков кода:

```
if (!condition)
    goto skip_block;

code_if_true;

skip_block:
```

Обратите внимание, что в этой версии условие инвертировано. Причина в том, что переход за блок `if` происходит только при ложном условии, тогда как блок оператора `if` задаёт действия для истинного условия. При удалении блоков кода условие всегда приходится инвертировать.

Для преобразования этого псевдокода в работающую программу нужно заменить `condition` и `code_if_true` настоящим кодом.

С блоками	**Без блоков**

<pre>if (x==5)
{
 x++;
 y=x;
}</pre>
	<pre>if (x!=5)
	 goto skip_block;
	
	x++;
	y=x;
	
	skip_block:</pre>

После удаления блоков преобразовать код в ассемблер гораздо проще. Теперь его можно непосредственно сопоставить с эквивалентами x86.

Код	**Ассемблер x86**

<pre>if (x!=5)
 goto skip_block;
 	
x++;
y=x;
 	
skip_block:</pre>
	<pre>cmp dword [x], 5
	jne skip_block
	
	inc dword [x]
	mov eax, [x]
	mov [y], eax
	
	skip_block:</pre>

if (...) { ... } else { ... }

Добавление ветви `else` к конструкции `if` увеличивает сложность и необходимое число переходов. Помимо пропуска блока `if`, если условие ложно, конструкция `if (...) { ... } else { ... }` после выполнения кода в блоке `if` переходит через блок `else`.

Следующие примеры показывают, как эта логическая конструкция выглядит с блоками и без них:

С блоками	**Без блоков**

<pre>if (condition)
{
 code_if_true;
}
else
{
 code_if_false;
}</pre>
	<pre>if (!condition)
	 goto false_block;
	
	code_if_true;
	goto skip_block;
	
	false_block:
	code_if_false;
	
	skip_block:</pre>

Обратите внимание: в операторах `goto` используются две разные метки. Метка `false_block` служит для пропуска блока `if`, когда условие ложно, а метка `skip_block` - для перехода за блок `else` после выполнения блока `if`. Как и прежде, при переписывании кода без блоков инвертируйте условный оператор.

После замены псевдокода настоящим кодом получаются следующие варианты с блоками и без них:

С блоками	**Без блоков**

<pre>if (x)
{
 x++;
}
else
{
 x--;
}</pre>
	<pre>if (!x)
	 goto false_block;
	
	x++;
	goto skip_block;
	
	false_block:
	x--;
	
	skip_block:</pre>

Как и раньше, удаление блоков упрощает преобразование высокоуровневого кода в ассемблерный.

Код	**Ассемблер x86**
<pre>if (!x) goto false_block; x++; goto skip_block; false_block: x--; skip_block:</pre>	<pre>cmp dword [x], 0 je false_block inc dword [x] jmp skip_block false_block: dec dword [x] skip_block:</pre>

if (...) { ... } else if { ... } else { ... }

Операторы `if` можно усложнять, проверяя несколько различных условий. Ниже показан оператор `if` с `else if` и `else`, записанный с блоками и без них. Но процесс остаётся прежним: инвертировать условие и добавить `goto`.

С блоками	**Без блоков**

<pre>if (condition_1)
{
 code_if_1;
}
else if (condition_2)
{
 code_if_2;
}
else
{
 code_if_false;
}</pre>
	<pre>if (!condition_1)
	 goto test_2;
	code_if_1;
	goto skip_block;
	
	test_2:
	if (!condition_2)
	 goto false_block;
	code_if_2;
	goto skip_block;
	
	false_block:
	code_if_false;
	
	skip_block:</pre>

В этой версии кода для преобразования в вариант без блоков необходимо несколько меток и переходов. Рассмотрим практический пример, реализующий систему оценок с чрезвычайно

щедрым округлением.

С блоками	Без блоков

<pre>if (score>70)
{
 grade='a';
}
else if (score>50)
{
 grade='b';
}
else
{
 grade='c';
}</pre>
	<pre>if (score<=70)
	 goto test_2;
	grade='a';
	goto skip_block;
	
	test_2:
	if (score<=50)
	 goto false_block;
	grade='b';
	goto skip_block;
	
	false_block:
	grade='c';
	
	skip_block:</pre>

Обратите внимание: для преобразования в вариант без блоков условия вновь приходится обращать. Строгие операторы «меньше» превращаются в «больше или равно 1». В следующем примере показано, как затем этот код легко преобразуется в ассемблерный:

Код	Ассемблер x86
<pre>if (score<=70) goto test_2; grade='a'; goto skip_block; test_2: if (score<=50) goto false_block; grade='b'; goto skip_block; false_block: grade='c'; skip_block:</pre>	<pre>cmp dword [score], 70 jle test_2 mov byte [grade], 'a' jmp skip_block test_2: cmp dword [score], 50 jle false_block mov byte [grade], 'b' jmp skip_block false_block: mov byte [grade], 'c' skip_block:</pre>

do { ... } while (...);

В языках программирования высокого уровня существует несколько разных циклических структур, каждая из которых работает немного иначе. Цикл `do...while` гарантированно выполняет по меньшей мере одну итерацию, прежде чем вычислит условие, которое может завершить цикл. Ниже приведён пример цикла `do...while` с блоками:

```
do
{
    code;
}
while (condition);
```

В отличие от операторов `if`, цикл `do...while` вычисляет своё условие в конце, поэтому для следующих итераций требуется обратный переход. Как и прежде, код нужно переписать без блоков. Ниже тот же цикл `do...while` использует вместо блоков операторы `goto`:

```
loop:

code;

if (condition)
    goto loop;
```

В отличие от оператора `if`, цикл `do...while` не инвертирует проверяемое условие. Причина в том, что обратный переход выполняется только при истинном условии, когда требуется ещё одна итерация цикла.

Теперь рассмотрим вариант кода с настоящими условиями и логикой.

С блоками	Без блоков

<pre>do
{
 y*=x;
 x--;
}
while (x);</pre>
	<pre>loop:
	y*=x;
	x--;
	
	if (x)
	 goto loop;</pre>

Преобразовать этот код в вариант без блоков относительно просто, поскольку поток инструкций в основном остаётся тем же. Главное отличие состоит в том, что оператор `while` заменяется оператором `if` и переходом `goto`.

В отличие от предыдущих примеров, большая часть сложности преобразования в ассемблер здесь вызвана сложностью примера кода, а не ветвлениями.

Код	Ассемблер x86
<pre>loop: y*=x; x--; if (x) goto loop;</pre>	<pre>loop: mov eax, [y] mul dword [x] mov [y], eax sub dword [x], 1 cmp dword [x], 0 jne loop</pre>

while (...) { ... }

Цикл `do...while` гарантирует выполнение одной итерации до проверки условия. Цикл `while` вычисляет условие сразу, поэтому находящийся внутри него код может не выполниться ни разу. Следующий код демонстрирует цикл `while` с блоками и без них. Цикл `while` можно разложить на оператор `if`, поэтому при его преобразовании действует обычная схема для оператора `if`.

С блоками	Без блоков
<pre>while (condition) { code; }</pre>	<pre>loop: if (!condition) goto done; code; goto loop; done:</pre>

Обратите внимание: поскольку условие вычисляется в начале, оно инвертируется, как в операторах `if`. Ниже показано, как это может выглядеть после перехода от псевдокода к настоящему коду:

С блоками	Без блоков

<pre>while (tired)
{
 sleep();
}</pre>
	<pre>loop:
	if (!tired)
	 goto done;
	
	sleep();
	goto loop;
	
	done:</pre>

После удаления блоков код можно преобразовать в ассемблер x86, как показано ниже:

Код	Ассемблер x86
<pre>loop: if (!tired) goto done; sleep(); goto loop; done:</pre>	<pre>loop: cmp dword [tired], 0 je done call sleep jmp loop done:</pre>

for (...; ...; ...) { ... }

Цикл `for` работает иначе, чем `while` или `do...while`. Вместо переменного числа повторений, определяемого условием, цикл `for` наряду с условием цикла инициализирует значение и обновляет его.

Оператор `for` содержит три выражения. Первое инициализирует счётчик цикла. Второе задаёт условие завершения выполнения цикла, а третье определяет, как будет изменяться счётчик между итерациями. Ниже это показано в псевдокоде с блоками и без них:

С блоками	**Без блоков**
<pre>for (expr_1; expr_2; expr_3) { code; }</pre>	<pre>expr_1; loop: if (!expr_2) goto done; code; expr_3; goto loop; done:</pre>

При преобразовании цикла `for` в вариант без блоков три выражения оператора `for` распределяются по коду. Первое выражение является предварительным условием и выполняется единственный раз до цикла, а второе начинает цикл. Последнее условие, изменяющее значение счётчика, выполняется в конце каждой итерации.

Эти три выражения оператора `for` легче понять на настоящем коде. Например, следующий код определяет счётчик цикла `i` и инициализирует его нулём. На каждой итерации этот счётчик увеличивается на 1 (`i++`), а цикл прекращает работу, когда `i` достигает 100.

С блоками	**Без блоков**

<pre>for (i=0; i<100; i++)
{
 sum+=i;
}</pre>
	<pre>i=0;
	
	loop:
	if (i>=100)
	 goto done;
	
	sum+=i;
	i++;
	goto loop;
	
	done:</pre>

Обратите внимание: подобно циклу `while` или оператору `if`, цикл `for` инвертирует условие. И снова причина в том, что условие вычисляется в начале цикла, а не в конце, как у `do...while`.

Ниже показано, как цикл `for` выглядит на ассемблере x86:

Код	**Ассемблер x86**

<pre>i=0;
 	
loop:
if (i>=100)
 goto done;
 	
sum+=i;
i++;
goto loop;
 	
done:</pre>
	<pre>mov dword [i], 0
	
	loop:
	cmp dword [i], 100
	jge done
	
	mov eax,[i]
	add [sum],eax
	inc dword [i]
	jmp loop
	
	done:</pre>

switch (...) { ... }

Оператор `switch` - существующая в некоторых языках программирования логическая структура, упрощающая условную логику. Назначение оператора `switch` состоит в том, чтобы выполнить одну из нескольких разных операций в зависимости от значения определённой переменной. Следующий оператор `switch` проверяет значение, хранящееся в `op`, и выводит символ, представляющий эту операцию:

```
typedef enum {ADD, SUB, MUL, DIV, MOD} op_t;

switch (op) {
    case ADD:
```

```
        c='+'; break;
    case SUB:
        c='-'; break;
    case MUL:
        c='*'; break;
    case DIV:
        c='/'; break;
    case MOD:
        c='%'; break;
    default:
        c='?'; break;
}
```

Любой оператор `switch` можно записать как последовательность операторов `if` и `else if`. Однако такой код быстро становится и сложным для написания, и неэффективным при выполнении. Если совпадение обнаруживается только в самом последнем варианте, приходится выполнить каждое предыдущее сравнение. Ниже приведён эквивалент предыдущего оператора `switch`, записанный с помощью `if` и `else if`:

```
if (op==ADD)
    c='+';
else if (op==SUB)
    c='-';
else if (op==MUL)
    c='*';
else if (op==DIV)
    c='/';
else if (op==MOD)
    c='%';
else
    c='?';
```

Построение таблицы переходов

При проверке этого списка операторов `if` и `else if` процессор должен выполнить пять проверок, чтобы понять, что делать с `MOD`, а это крайне неэффективно. Представьте сценарий с сотнями вариантов... или тысячами. Выполнение было бы невероятно неэффективным. Для оптимизации этого процесса компилятор может вместо этого построить таблицу переходов.

Таблица переходов - это структура данных ассемблера, содержащая список целевых адресов для оператора `switch`, как показано на рис. 7.1. Определив правильный вариант оператора `switch`, программа может использовать номер этого варианта как индекс массива адресов и перейти непосредственно к нужному блоку кода.

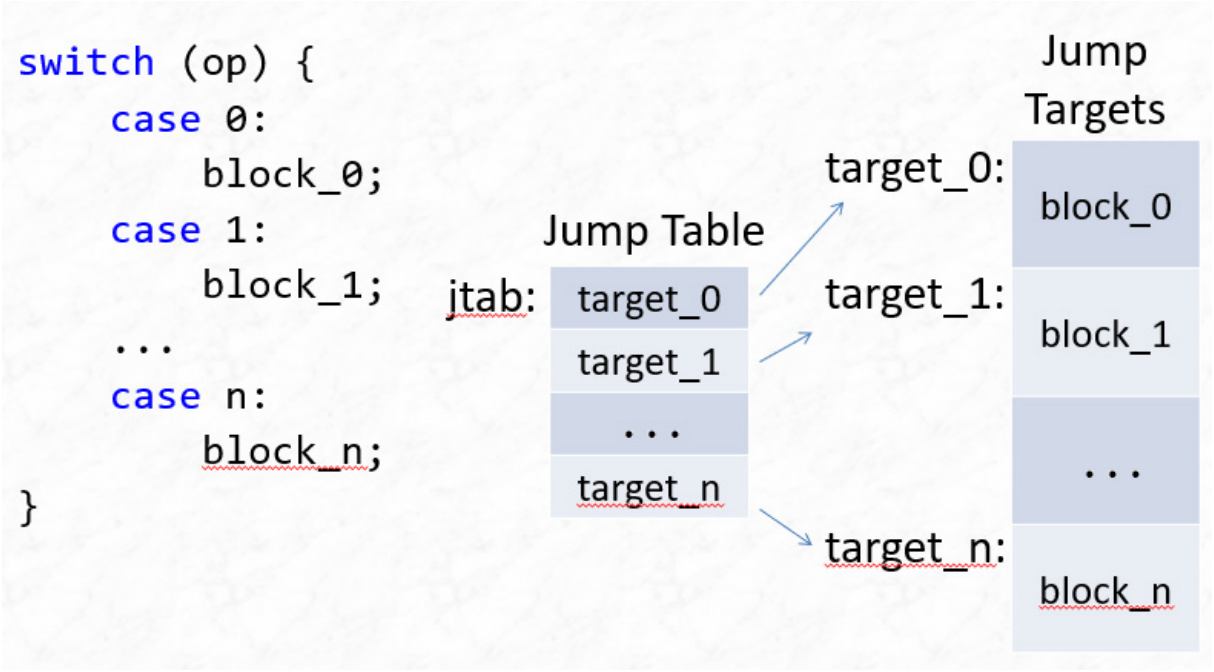


Рис. 7.1. Пример таблицы переходов

Следующий код показывает, как может выглядеть на ассемблере программа, использующая таблицу переходов:

```
.section data
table:
dd target_0
dd target_1
dd target_2
dd target_3
dd target_4

.section text
mov eax, [op]
cmp eax, 5
jge default
jmp [table+eax*4]

target_0:
mov byte [c], '+'
jmp done

target_1:
mov byte [c], '-'
jmp done

target_2:
mov byte [c], '*'
jmp done

target_3:
mov byte [c], '/'
jmp done

target_4:
mov byte [c], '%'
jmp done

default:
mov byte [c], '?'
jmp done

done:
```

Код начинается с таблицы переходов, содержащей адреса различных блоков кода в памяти. Каждый из этих блоков, помеченный как `target_x`, помещает соответствующий символ в `byte [c]`, а затем переходит к метке `done`.

Между таблицей и целевыми блоками кода находится код, который фактически реализует оператор `switch` под заголовком `.section text`. Этот код начинается с перемещения `op` в `eax`. Затем он проверяет, больше или равно 5 значение `op`. Если да, выполняется переход к варианту `default`.

Если значение `op` меньше 5, ему соответствует одна из целей. Используя его как индекс в таблице переходов, процессор может получить адрес соответствующего блока кода и перейти туда для выполнения кода.

В этом случае благодаря таблице переходов время выполнения одинаково для любого варианта. Последний вариант не требует больше инструкций или сравнений, чем первый.

Пропущенные варианты

Таблица переходов предполагает, что варианты покрывают непрерывный диапазон значений. Например, в следующем фрагменте кода оператор `switch` имеет варианты 1, 2, 4 и 5. В таком случае отсутствие 3 может стать проблемой. В третью позицию таблицы переходов необходимо что-либо поместить.

```
switch (x) {
    case 0:
        ...
    case 1:
        ...
    case 2:
        ...
    case 4:
        ...
    default:
        ...
}
```

Пропущенные значения таблицы переходов можно заполнить адресом `default` или `done`, если варианта `default` нет, как показано ниже. Тогда, если `op` равен 3 и процессор попытается перейти по соответствующей позиции таблицы переходов, он попадёт в правильное место.

Таблица переходов
`target_0`
`target_1`
`target_2`
`default`
`target_4`

Ненулевые основания

Таблица переходов рассчитана на набор вариантов, начинающийся с 0. Однако оператор `switch` может иметь ненулевые значения вариантов, как в следующем фрагменте кода:

```
switch (x) {
    case 'a':
        ...
    case 'b':
        ...
    case 'c':
        ...
    case 'd':
        ...
}
```

В этом случае необходимо найти способ привести варианты к нулевой базе. Для ASCII нужно найти наименьший вариант таблицы переходов, которым в данном примере является a, или 97. Реализуя таблицу переходов для этого оператора `switch`, код может использовать смещение, обращаясь к значениям как к `table[x-97]`. В следующей таблице переходов `target_a` будет указывать на вариант a оператора `switch`.

Таблица переходов
`target_a`
`target_b`
`target_c`
`target_d`

Непрактичные таблицы переходов

Компилятор использует такие приёмы для повышения эффективности кода, и вы тоже можете применять их при написании кода вручную. Однако иногда таблица переходов просто не подходит. Например, рассмотрим следующий код:

```
switch (x) {
    case 1:
        printf("this is the beginning."); break;
    case 1000:
        printf("this is the end."); break;
}
```

Для этого оператора `switch` потребовалась бы таблица переходов из 1000 элементов, причём 998 из них указывали бы на метку `done`. В таком случае оператор `if/else` будет более эффективным вариантом.

Это не единственный непрактичный случай: большой размер таблицы с сотнями вариантов, индекс, который трудно привести к нулевой базе, или слишком большое число пробелов в таблице также могут сделать таблицу переходов непрактичной. Но как при написании, так и при разборе ассемблерного кода важно понимать эти структуры и способы их применения.

continue

В некоторых языках высокого уровня имеется ключевое слово `continue`. Оно используется внутри цикла и приказывает процессору перейти к следующей итерации, пропустив все последующие инструкции.

Например, в следующем фрагменте кода второй участок с меткой `code` будет недостижим. При каждом вычислении оператора `continue` процессор переходит непосредственно к оператору `while`.

```
do
{
    code;
    continue;
    code;
}
while (condition);
```

Цикл с оператором `continue`, записанный без блоков кода, похож на обычный цикл соответствующего типа. Как показано ниже, `continue` можно реализовать с помощью `goto`, переходящего к метке непосредственно перед условием цикла.

```
loop:

code;
goto check_condition;
code;

check_condition:
if (condition)
    goto loop;
```

Следующие примеры показывают цикл с оператором `continue`, использующий настоящий код:

С блоками	**Без блоков**

<pre>do
{
 x--;
 continue;
 x++;
}
while (x);</pre>
	<pre>loop:
	
	x--;
	goto check_condition;
	x++;
	
	check_condition:
	if (x)
	 goto loop;</pre>

В этом коде инструкция `x++;` никогда не выполнится, поскольку `continue` всегда заставляет её пропустить. Обычно `continue` находится внутри оператора `if`, иначе следующий за ним код лишён смысла. Этот искусственный пример призван показать работу `continue` без сложности, возникающей при вложении операторов `if` в цикл.

После удаления блоков код можно преобразовать в ассемблерный. Как и прежде, `goto` реализуется безусловным переходом.

Код	**Ассемблер x86**
<pre>loop: x--; goto check_condition; x++; check_condition: if (x) goto loop;</pre>	<pre>loop: sub dword [x], 1 jmp check_condition add dword [x], 1 check_condition: cmp dword [x], 0 jne loop</pre>

break

Ключевое слово `break` также существует в некоторых языках программирования и приказывает процессору выйти из текущего цикла. Как и с `continue`, второй участок кода в следующем примере никогда не выполнится, поскольку обычно `break` находится внутри условного оператора, однако данный пример предназначен лишь для демонстрации механизма `break`.

```
do
{
    code;
```

```
    break;
    code;
}
while (condition);
```

Следующий пример показывает, как этот код реализуется без блоков. Ключевое слово `break` тоже заменяется на `goto`, однако на этот раз переход ведёт не к точке перед условием, а за пределы цикла.

```
loop:

code;
goto break;
code;

if (condition)
    goto loop;

break:
```

В следующем примере оператор `break` завершит цикл после однократного вычисления операции `x--`. В таком случае оператор `x++` и условие цикла никогда не будут выполнены, но, повторим, это лишь образец кода. Он показывает, как легко поместить `break` внутрь условного оператора.

С блоками	**Без блоков**

<pre>do
{
 x--;
 break;
 x++;
}
while (x);</pre>
	<pre>loop:
	
	x--;
	goto break;
	x++;
	
	if (x)
	 goto loop;
	
	break:</pre>

Следующий пример показывает преобразование этого кода в ассемблерный:

Код	**Ассемблер x86**
<pre>loop: x--; goto break; x++; if (x) goto loop; break:</pre>	<pre>loop: sub dword [x], 1 jmp break add dword [x], 1 cmp dword [x], 0 jne loop break:</pre>

&&

В языках высокого уровня условие оператора `if` или цикла может вычислять несколько разных условий, например логическое И (&&). В следующем примере блок `if` будет выполнен, только если истинны и `condition_1`, и `condition_2`.

```
if ( condition_1 && condition_2 ) {
    code;
}
```

Чтобы удалить из этого кода блоки, составной оператор `if` необходимо разбить на два отдельных оператора `if`. Каждый из них отрицает одно из условий, входивших в исходный оператор `if`.

```
if (!condition_1) goto skip_block;
if (!condition_2) goto skip_block;
true:
code;
skip_block:
```

После такой перезаписи без блоков преобразуйте код в ассемблерный по той же формуле.

||

Ещё один вариант в операторах `if` - объединить несколько условий логическим ИЛИ (`||`). Пример показан в следующем псевдокоде:

```
if ( condition_1 || condition_2 ) {
    code;
}
```

При удалении блоков логическое ИЛИ также разбивается на два оператора `if`. Однако эти операторы `if` выглядят иначе, чем при логическом И.

```
if (condition_1) goto true;
if (!condition_2) goto skip_block;
true:
code;
skip_block:
```

Оператор логического ИЛИ истинен, если истинно любое из двух условий. В предыдущем примере первый оператор `if` использует исходное `condition_1` и переходит к метке `true`, поскольку при истинности этого условия проверять второе уже не требуется.

Однако если это условие ложно, код продолжает вычислением `condition_2`. Это условие инвертировано, а переход ведёт за конец оператора `if`.

Если `condition_2` истинно, выполнение проваливается в блок `true`. В противном случае оно переходит за оператор `if`.

Стек

В ассемблере стек используется для хранения нескольких типов данных, включая:

• локальные переменные;
• рабочее пространство;
• параметры и вызов функций.

Стек получил своё название потому, что представляет собой структуру «последним пришёл - первым ушёл» (last-in-first-out, LIFO), подобную стопке бумаги. Концептуально это соответствует потоку управления программы, и стеки чрезвычайно распространены в самых разных архитектурах.

У стека имеется указатель стека, обозначающий его вершину. В этом разделе подробно рассматриваются несколько инструкций, специально предназначенных для работы со стеком. Операция `push` сохраняет новое значение на текущей вершине стека и обновляет указатель стека, чтобы он обозначал новую вершину (представьте, что добавляете новый лист бумаги на стопку). Операция `pop` перемещает значение с вершины стека в регистр или по адресу памяти и обновляет указатель стека, чтобы он обозначал находящееся ниже значение, которое становится новой вершиной стека (как если бы вы сняли со стопки верхний лист бумаги).

Как работает стек

При вызове новых функций стек программы растёт вверх от базового адреса, а при возврате из функций уменьшается. Когда стек становится выше, адреса уменьшаются, а когда он становится ниже, адреса увеличиваются.

Стек x86

Стек не является принципиально отдельным объектом или пространством памяти процессора. Это выделенная и назначенная для работы стеком область памяти. Она существует в памяти рядом с остальной программой, а данные представляют собой просто пространство, выделенное приложением. Например, следующий код выделяет 128 байт, которые можно использовать как пространство стека:

```
section .data
times 128 db 0
stack equ $-4
```

Для управления стеком в x86 используются два регистра:

• `esp`: указатель стека содержит адрес вершины стека.
• `ebp`: указатель базы содержит адрес основания кадра стека.

Логически стек должен расти вверх, а в x86 это означает уменьшение адресов, как показано на рис. 7.2. Представьте перевёрнутый термометр, у которого 0 находится сверху, а наибольшее число - снизу.



Рис. 7.2. Рост адресов стека

Рассмотрим следующий фрагмент ассемблерного файла:

```
mov esp, stack
...
section .data
times 128 db 0
stack equ $-4
```

Инструкция `times 128 db 0` выделяет пространство под стек размером 128 байт. Затем инструкция `stack equ $-4` определяет константу `stack` с начальным значением, равным текущему расположению (`$`) минус 4 (то есть на 4 байта внутрь от конца стека); она содержит адрес последнего двойного слова (4-байтового фрагмента) этого стека.

Первая инструкция примера инициализирует указатель стека `esp` этой константой. После этого стек может расти вверх по мере добавления новых данных инструкциями `push`.

Push и pop

Инструкции push и pop служат для добавления данных в стек и их удаления из стека.

Push

Инструкция push добавляет на вершину стека 4 байта, или 1 двойное слово. Она принимает один аргумент, которым может быть имя регистра, адрес памяти или константа.

```
push <register>
push <memory>
push <constant>
```

При выполнении push указатель стека esp автоматически уменьшается на 4, обозначая новую вершину стека. Затем помещаемое в стек значение записывается в это место.

Таблица 7.2 показывает работу инструкции push. В данном случае указатель стека esp сначала имеет значение 0x120, как показано слева в таблице 7.2. Затем выполняются следующие инструкции:

```
; esp = 0x120
mov eax, 0xFEDA8712
push eax
; esp = 0x11C
```

Эти инструкции помещают в стек 1 двойное слово, или 4 байта. Для этого указатель стека уменьшается и начинает указывать на адрес 0x11C. Затем в стек сохраняется значение 0xFEDA8712, как показано справа в таблице 7.2.

Таблица 7.2. Помещение переменной в стек

Адрес до push	**Значение до** push	**Адрес после** push	**Значение после** push
0x11B		0x11B	
0x11C		0x11C (esp)	0x12
0x11D		0x11D	0x87
0x11E		0x11E	0xDA
0x11F		0x11F	0xFE
0x120 (esp)	0x11	0x120	0x11
0x121	0x22	0x121	0x22
0x122	0x33	0x122	0x33

Одна инструкция push объединяет в себе несколько шагов. Следующий фрагмент кода эквивалентен предыдущему:

```
;esp = 0x120
mov eax, 0xFEDA8712
sub esp, 4
mov [esp], eax
```

```
; esp = 0x11C
```

В этом примере значение указателя стека явно уменьшается, чтобы он указывал на новую вершину. Затем в это место перемещается значение, хранящееся в `eax`. Два подхода взаимозаменяемы, но при использовании `push` требуется меньше инструкций.

Pop

В x86 инструкция `pop` противоположна инструкции `push` и удаляет из стека одно двойное слово. Она имеет синтаксис `pop dst`, где `dst` может быть регистром или адресом памяти.

Инструкция `pop` обращает операции, выполняемые `push`. Сначала она перемещает значение, хранящееся по адресу `[esp]`, в указанный регистр или ячейку памяти. Затем автоматически увеличивает `esp` на 4, чтобы он указывал на новую вершину стека.

Таблица 7.3 показывает, как с помощью `pop` отменить действие `push` из предыдущего примера. В конце того примера стек выглядел как столбцы слева в этой таблице.

```
;esp = 0x11C
pop eax
;esp = 0x120
;eax = 0xfeda8712
```

Таблица 7.3. Извлечение переменной из стека

Адрес до `pop`	**Значение до** `pop`	**Адрес после** `pop`	**Значение после** `pop`
`0x11B`		`0x11B`	
`0x11C (esp)`	`0x12`	`0x11C`	
`0x11D`	`0x87`	`0x11D`	
`0x11E`	`0xDA`	`0x11E`	
`0x11F`	`0xFE`	`0x11F`	
`0x120`	`0x11`	`0x120 (esp)`	`0x11`
`0x121`	`0x22`	`0x121`	`0x22`
`0x122`	`0x33`	`0x122`	`0x33`

Выполнение инструкции `pop eax` изменит стек так, как показано справа. В ходе этого процесса регистр `eax` получит значение `0xFEDA8712`. Затем указатель стека `esp` увеличится на 4 и станет равен `0x120`.

Как и `push`, инструкция `pop` объединяет двухэтапный процесс в одну инструкцию. Следующий код эквивалентен ей, но явно выполняет каждый шаг:

```
;esp = 0x11C
mov eax, [esp]
add esp, 4
;esp = 0x120
;eax = 0xfeda8712
```

Стек как рабочая область

В x86 имеется ограниченное число регистров, и их легко исчерпать. Нередко требуется некая область обмена или рабочая область для временного хранения ещё нужных данных. Стек предоставляет удобное место для временного хранения значений.

Например, рассмотрим следующий код:

```
mov eax, 0xcafed00d
mov ebx, 0x00c0ffee
add eax, ebx
push eax     ; сохранить, чтобы освободить eax
...          ; выполнить другие действия
pop eax      ; получить сохранённый eax
```

В этом примере инструкции `push` и `pop` служат для временного хранения содержимого `eax` в стеке. Благодаря этому регистр освобождается для других вычислений. Когда сохранённое значение снова понадобится, с помощью `pop` его можно вернуть в `eax`.

Осторожное использование pop

Данные на компьютере удаляются редко. Например, при удалении файла в файловой системе или освобождении переменной программой связанная с ними память просто помечается как доступная для других целей. Хранившиеся данные по-прежнему остаются на диске.

То же самое справедливо для стека x86. Когда значение извлекается из стека, оно копируется в регистр или ячейку памяти, но само значение остаётся в стеке. После изменения указателя стека извлечённое значение оказывается вне допустимого диапазона стека.

После извлечения значение следует считать освобождённым и более непригодным для безопасного использования. Любая попытка обратиться к данным вне допустимого диапазона стека опасна. Например, ни одна правомерная ассемблерная инструкция не должна содержать `[esp-...]`.

Рассмотрим следующий пример, показывающий трассировки стека в разных местах; соответствующая трассировка указана в комментариях (например, `;(1)`). Каждое место с комментарием показывает стек после выполнения этой строки, как представлено в таблице 7.4.

```
;(1) esp = 0x10C
push 0xbadc0de ; (2)
pop eax  ;(3)  eax = 0xbadc0de
push 0xc0ffee ;(4)
```

Таблица 7.4. Примеры трассировки стека

Адрес	(1), значение	(2), значение	(3), значение	(4), значение
0x1000	??	??	??	??
0x1004	??	??	??	??
0x1008	??	0xbadc0de (esp)	0xbadc0de	0xc0ffee (esp)
0x100c	0x11223344 (esp)	0x11223344	0x11223344 (esp)	0x11223344
0x1010	0x55667788	0x55667788	0x55667788	0x55667788

Обратите внимание: в трассировке 3 значение `0xbadc0de` извлечено из стека, но всё ещё остаётся там, пока его не перезапишет что-либо другое, как в состоянии стека 4. Повторим: в правомерных условиях нельзя полагаться на данные выше текущего выделенного стека (`[esp-..]`) или использовать их, но это знание действительно может оказаться полезным для иных, менее правомерных целей. В последующих примерах содержимое «невыделенной» части стека либо больше не приводится, либо значение ~~зачёркивается~~, чтобы показать, что оно освобождено.

Вызовы функций и кадры стека

В языках программирования высокого уровня существует понятие функций - фрагментов кода, которые можно вызывать из других функций. Понятие функций имеется и в x86.

При вызове функции состояние стека изменяется. Понимание этих изменений необходимо для понимания работы приложения.

Функции в x86

Инструкции x86 `call` и `ret` позволяют создавать функции, подобные функциям языков высокого уровня.

call

Инструкция `call` имеет синтаксис `call op`, где `op` обозначает адрес вызываемой функции. Аргументом `op` может быть регистр, метка или адрес памяти.

```
call eax    ; перейти к eax
call label  ; перейти к label
call 0x1000 ; перейти к 0x1000
```

Подобно `push` и `pop`, `call` фактически объединяет несколько шагов в одну операцию. Сначала она создаёт адрес возврата, помещая в стек адрес следующей инструкции. Затем выполняет безусловный переход к участку кода, указанному операндом `op`.

ret

Инструкция `ret` не принимает аргументов. Её назначение - вернуть выполнение в вызывающую функцию.

Это выполняется в два этапа. Сначала `ret` извлекает из стека адрес возврата, сохранённый инструкцией `call`. Затем выполняет безусловный переход по этому адресу.

Как работают функции x86

С помощью `call` и `ret` можно создавать функции непосредственно на x86 или преобразовывать их из других языков Рассмотрим следующий код:

```
void a() {
}
void b() {
```

```
    a();
}
```

Этот код определяет две функции, a и b, причём b вызывает a. На x86 этому эквивалентен следующий код:

```
b:
    call a
    ret

a:
    ret
```

Таблица 7.5 показывает, как выполнение этого кода повлияет на стек. Предположим, что инструкции хранятся в следующих ячейках памяти, а соответствующие трассировки стека отмечены в комментариях. Напомним, что стек показывает состояние после выполнения указанной инструкции.

```
b:
       ;(1)
0x10000  call a  ;(2)
0x10003  ret

a:
0x20012  ret  ;(3)
```

Крайнее левое представление в таблице 7.5 показывает начальное состояние стека. В этот момент esp указывает на адрес 0x9010, а eip имеет значение 0x1000.

Среднее представление показывает происходящее при выполнении вызова a. В этот момент значение eip, теперь равное 0x10003, помещается в стек. Теперь eip указывает на первую строку кода в a, имеющую адрес 0x20012.

После возврата из a исходное значение eip извлекается из стека, из-за чего он начинает указывать на инструкцию ret в b по адресу 0x10003. Указатель стека также обновляется и указывает на адрес 0x9010. Обратите внимание: значение 0x10003 остаётся в памяти, но теперь находится вне стека, поэтому его больше нельзя использовать или считать надёжным.

Таблица 7.5. Вызовы функций и стек

Адрес/регистр	(1)	(2)	(3)
0x9000	??	??	??
0x9004	??	??	??
0x9008	??	??	??
0x900c	??	0x10003	~~0x10003~~
0x9010	??	??	??
esp	0x9010	0x900c	0x9010
eip	0x10000	0x20012	0x10003

Объединив представления о стеке и функциях, рассмотрим пример. Пусть заданы следующие функции:

```
void a() { int x; b(); }
void b() { int x; c(); }
void c() { int x; }
```

Всё начинается с функции a(), имеющей одну локальную переменную x и вызывающей функцию b. Таблица 7.6 показывает структуру стека после вызова a. Обратите внимание: для локальных данных a выделяется место, и они добавляются в стек.

Таблица 7.6. Стек программы после вызова a

Стек
Локальные данные a

При выполнении a объявляет x, a затем вызывает b. Это означает, что поток выполнения переключается на код функции b, а затем возвращается в a.

Чтобы обеспечить возврат в правильное место a, процессор сохраняет в стеке адрес возврата. Этот адрес возврата является адресом инструкции в a, следующей за вызовом b.

После помещения в стек адреса возврата a процессор сохраняет там локальные данные b. Таблица 7.7 показывает состояние стека до выполнения процессором первой инструкции b.

Таблица 7.7. Стек программы после вызова b

Стек
Локальные данные b
Адрес возврата a
Локальные данные a

Подобно a, функция b объявит свои локальные переменные, а затем вызовет другую функцию, c. При выполнении этого вызова в стек будут помещены адрес возврата для b, а также локальные переменные вызываемой функции. Когда подготовка c завершится, стек будет выглядеть как в таблице 7.8.

Таблица 7.8. Стек программы после вызова c

Стек
Локальные данные c
Адрес возврата b
Локальные данные b
Адрес возврата a
Локальные данные a

В c объявляется локальная переменная x, после чего функция завершается. Когда процессор закончит работу в c, поток кода должен вернуться в вызывающую функцию b.

В этот момент локальные данные c находятся на вершине стека, но больше не нужны. Процессор может извлечь эти данные из стека, изменив указатель стека так, чтобы тот указывал на адрес возврата b.

Затем процессор может извлечь из стека этот адрес возврата, сохранив его в `eip`, и обновить указатель стека с помощью инструкции `ret`. Благодаря этому программа возвращается в `b` и продолжает выполнять код, следующий за вызовом `c`. Теперь стек вновь находится в состоянии, показанном в таблице 7.7.

Вызов `c` является последней инструкцией в `b`, поэтому `b` также немедленно выполнит возврат. Подобно возврату из `c`, это включает извлечение локальных переменных из стека и обновление `eip` путём извлечения адреса возврата `a` из стека в `eip` (с помощью `ret`). После завершения стек будет выглядеть как в таблице 7.6.

После этого возврата из `b` функция `a` также выполнит возврат. Локальные данные `a` будут извлечены из стека. Затем регистр `eip` обновится на основании адреса возврата вызывающей функции, вызов которой произошёл до начала нашего анализа и не показан в таблице 7.6, а выполнение возобновится внутри этой функции.

Анализ стека

Вызовы функций и возвраты из них воздействуют на стек программы. Например, рассмотрим следующий код:

```
void a() { }
void b() { }
void c() { a(); b(); }
```

Этот код определяет три функции, `a`, `b` и `c`, причём `c` вызывает две остальные.

Когда функция выполняется или находится в стеке вызовов выполняющейся функции, её адрес возврата присутствует в стеке. Например, во время выполнения `a` в стеке находятся адреса возврата для `a` и `c`. Аналогично, во время выполнения `b` в стеке находятся адреса возврата для `b` и `c`.

Изучение сохранённых в стеке адресов возврата позволяет увидеть, каким образом программа достигла определённой точки. Для каждой функции в стеке вызовов в стеке будут видны её адрес возврата, локальные переменные и рабочие данные.

Такая процедура называется раскруткой стека. В `gdb` текущее состояние стека показывает команда `info stack`.

Соглашения о вызовах

Инструкции `call` и `ret` позволяют создавать функции на ассемблере. Однако при наличии только `call` и `ret` эти функции должны быть самодостаточными, без возможности передавать данные между функциями.

В языках программирования высокого уровня функции обычно имеют параметры или аргументы - переменные, передаваемые функции вызывающей функцией. Однако в машинном коде понятия параметров нет: существуют только регистры и память.

В x86 имеются все средства, необходимые для создания параметров. Языки программирования высокого уровня, использующие параметры, преобразуются в ассемблер. Выбрать способ применения этих средств должен программист или компилятор.

Зачем нужны соглашения

Благодаря регистрам, стеку и даже ячейкам памяти x86 позволяет передавать значения из одной функции в другую. Параметры можно хранить в регистрах либо помещать в стек и извлекать из него.

Однако если функции собираются единообразно использовать параметры, регистры или стек, им необходимы взаимодействие или договорённости. Если вызывающая функция использует определённые места для передачи параметров, вызываемая функция должна знать, какие места предназначены для каких значений. То же справедливо, если вызываемая функция возвращает данные вызывающей.

```
void caller() {   ... callee()} // определение терминов
```

Кроме того, если вызывающая функция хранит свои внутренние значения в регистре, вызываемая функция должна знать, что эти значения нельзя перезаписывать. Это особенно важно, если вызываемая функция использует операции вроде `mul`, которые изменяют регистры, но воздействие которых легко упустить.

В небольшой программе разработчик может заложить это знание в код. Если функция a принимает три параметра, разработчик может создать схему их передачи через регистры или стек. Аналогично, структуру, необходимую функции `b`, можно передать, выделив определённое место в памяти.

Однако такой подход способен работать в малом масштабе, но не масштабируется и подвержен ошибкам. Из-за упущения операция `mul` может случайно затереть жизненно важные данные. Кроме того, подобная специализированная схема усложняет работу коллективов разработчиков.

Введение в соглашения о вызовах

Соглашения о вызовах предназначены для упрощения передачи данных между функциями: они определяют правила взаимодействия функций. Они являются частью двоичного интерфейса приложений (application binary interface, ABI), то есть самого низкоуровневого определения взаимодействия частей кода.

Соглашение о вызовах должно определять несколько правил, в том числе следующие:

• **Расположение параметров:** где параметры передаются от вызывающей функции к вызываемой - в стеке или регистрах?
• **Порядок параметров:** как параметры организуются в стеке или регистрах?
• **Очистка стека:** если используется стек, какая функция отвечает за удаление из него значений - вызывающая или вызываемая?
• **Доступ к регистрам:** какие регистры вызываемая функция может использовать без сохранения их прежних значений и восстановления перед возвратом?
• **Возвращаемые значения:** где и как значения возвращаются из вызываемой функции в вызывающую?

Соглашения о вызовах могут различаться в зависимости от нескольких факторов, включая:

• архитектуру (x86 или ARM);
• операционную систему (UN*X или Windows);
• язык программирования (C или Java);
• даже компилятор (GCC или Microsoft).

На заре программирования стандартов фактически не существовало. Поэтому программы не могли работать вместе, если их разработчики не договаривались о соглашении о вызовах.

Первоначально существовало множество разных компаний, каждая со своими соглашениями. Со временем их число сократилось до нескольких популярных стандартов, среди которых:

• `cdecl`;
• `syscall`;
• `optlink`;
• `pascall`;
• `register`;
• `stdcall`;
• `fastcall`;
• `safecall`;
• `thiscall`.

cdecl

`cdecl` (произносится «си декл») - сокращение от C declaration («объявление C») и одно из самых распространённых соглашений о вызовах в архитектуре x86. Хотя оно возникло вместе с C, `cdecl` применяется для множества разных языков программирования и архитектур. Это также полезный стандарт при написании ассемблерного кода вручную.

`cdecl` определяет следующие правила:

• **Параметры в стеке:** аргументы помещаются в стек справа налево для передачи вызываемой функции.
• **Очистка вызывающей функцией:** после возврата из вызываемой функции вызывающая функция отвечает за удаление аргументов из стека.
• **Возвращаемое значение:** для возвращаемого функцией значения используется регистр `eax`.
• **Доступные регистры:** вызываемая функция может свободно изменять `eax`, `ecx` и `edx`. Перед вызовом вызывающая функция должна сохранить все нужные ей значения в этих регистрах. Перед использованием других регистров вызываемая функция должна сохранить их значения, а перед возвратом - восстановить.

Рассмотрим инструкцию `int s = add(1,2)`. При использовании стандарта `cdecl` она преобразуется в следующий ассемблерный код x86:

```
; Сохранить регистры, которые по cdecl нам нужно сохранить.
; Необязательно, если мы не собираемся изменять эти регистры.
push    eax
push    ecx
push    edx
; Поместить параметры в стек справа налево. Исходным
; кодом был add(1,2), поэтому справа налево идут 2, затем 1
push    2
push    1

; Вызвать add.
call    add

; Удалить параметры из стека. Мы поместили два 4-байтовых значения,
; поэтому можем либо выполнить два pop, либо прибавить 8 к указателю стека
add esp, 8

; Сохранить возвращённое значение из eax (куда cdecl помещает результаты)
mov [s], eax

; Восстановить сохранённые регистры; помните: последним пришёл - первым ушёл,
; поэтому edx был помещён последним и должен быть извлечён первым
pop edx
pop ecx
pop eax
```

Сохранение регистров

В `cdecl` функции могут свободно изменять `eax`, `ecx` и `edx`, не сохраняя их значений. Поэтому следующая функция `f` допустима по этому стандарту.

```
f:  mov ecx, 0xd15ea5e
    mov edx, 0xfee1dead
    lea eax, [ecx + edx]
    ret
```

Однако значения любых других регистров, используемых вызываемой функцией, необходимо сохранить до изменения и восстановить перед возвратом.

```
f:  push   ebx
    push   ebp
    push   esi
    mov    ebp, 0xd15ea5e
    mov    ebx, 0xfee1dead
    lea    esi, [ebp + ebx]
    pop    esi
    pop    ebp
    pop    ebx
    ret
```

Эта функция использует `ebx`, `ebp` и `esi`, поэтому перед использованием регистров помещает их прежние значения в стек, а перед возвратом извлекает эти значения обратно в регистры.

Благодаря `cdecl` вызывающая функция знает, значениям каких регистров можно доверять после вызова другой функции. Вызываемая функция может произвольно изменять значения `eax`, `ecx` и `edx`, поэтому вызывающей функции следует сохранять значения этих регистров, если они понадобятся ей позднее. Однако значения всех остальных регистров должна сохранять вызываемая функция, поэтому до вызова сохранять их не нужно.

Например, рассмотрим следующий блок кода:

```
g:
    mov   ebx, 0xd15ea5e
    mov   ecx, 0xfee1dead
    call  f
```

После вызова `f` функция `g` может рассчитывать, что `ebx` сохранит значение `0xd15ea5e`. Однако она не может предполагать, что `ecx` по-прежнему имеет значение `0xfee1dead`.

Возвращаемые значения

В языках программирования высокого уровня функции часто используют возвращаемые значения для передачи информации вызывающим функциям. Например, функция может возвращать 0 при успешном завершении или код ошибки, если что-либо пошло не так. Следующая функция возвращает при завершении значение 1:

```
int f()
{
    return 1;
}
```

При использовании соглашения о вызовах `cdecl` это возвращаемое значение хранится в регистре `eax`. Предыдущей функции `f` эквивалентен следующий код x86:

```
f:
    mov  eax, 1
```

```
    ret
```

Функции могут иметь разные типы и соответствующие этим типам возвращаемые значения. Например, следующая функция предназначена для возврата указателя `char*` и по умолчанию возвращает нулевой указатель:

```
char* f()
{
    return NULL;
}
```

В x86 регистр можно использовать как указатель. Следующий код x86 использует значение 0 как эквивалент нулевого указателя; `eax` также можно было бы использовать для указания местоположения массива `char` в памяти.

```
f:
    mov  eax, 0
    ret
```

Доступ к параметрам

Стандарт `cdecl` использует стек для передачи параметров функции. При обращении к параметрам следует помнить следующее:

• вершина стека (последнее помещённое значение) находится по адресу `[esp]`;
• стек растёт вниз (в сторону меньших адресов);
• инструкция `call` помещает адрес возврата в стек;
• вызывающая функция помещает аргументы в стек справа налево;
• возвращаемое значение вызываемой функции должно храниться в `eax`.

Учитывая эти факторы, рассмотрим, как в x86 реализуется вызов следующей функции:

```
int add (int x, int y)
{
    return x+y;
}
```

Эквивалентом на x86 будет следующий код:

```
f:
    push  1   ; y
    push  2   ; x
    call  add
    mov   [s], eax  ; сохранить возвращаемое значение в памяти
    pop   eax
    pop   eax
    ret

; int add(int x, int y) { return x+y; }
add:
    mov eax, [esp+4]    ; получить x из стека
    mov edx, [esp+8]    ; получить y из стека
    add eax, edx
    ret
```

Вызов функции влияет на текущий стек программы. Таблица 7.9 показывает состояние стека внутри функции `add`.

Таблица 7.9. Стек в функции add

Адрес	Значение
`0xeff0`	

Адрес	Значение
0xeff4	
0xeff8 (esp)	Адрес возврата
0xeffc	2
0xf000	1

Хотя обращаться к параметрам через [esp] возможно, такой подход способен привести к проблемам. Рассмотрим, как следующие инструкции внутри add влияют на значение esp:

```
; int f(int x);
f:
    mov  eax, [esp+4]    ; x находится по адресу [esp+4]
    push ebx             ; сохранить ebx
    mov  ebx, [esp+8]    ; теперь x находится по адресу [esp+8]
    ...
```

Когда вызываемая функция извлекает параметры из стека, положение вершины стека меняется. В результате изменяются и положения параметров относительно esp.

Кадры стека

Значение esp меняется слишком часто, чтобы служить полезной точкой отсчёта для расположения переменных в стеке. При каждом помещении или извлечении значения изменяются значение esp и относительные положения других переменных стека.

Здесь в дело вступает другой стековый регистр ebp, также известный как указатель базы или указатель кадра. Регистр ebp указывает на нижнюю границу текущего кадра стека - нижнюю границу участка памяти в стеке, используемого определённой функцией.

Прологи и эпилоги

Функции x86 обычно начинаются и заканчиваются фрагментами шаблонного кода. Назначение этого кода - создать и удалить кадр стека функции.

Создание кадра стека

Пролог, или преамбула, функции создаёт кадр стека и находится в самом начале функции. Пролог выполняет две задачи:

• сохраняет предыдущий кадр стека с помощью push ebp;
• устанавливает новый кадр стека с помощью mov ebp, esp.

Эти инструкции находятся в самом начале функции. Таблица 7.10 показывает воздействие каждой из них на стек.

```
;(1)
push ebp ;(2)
move ebp, esp ; (3)
push 0x11223344 ;(4)
```

Примечание переводчика. В исходнике напечатано `move ebp, esp`; корректная мнемоника x86 - `mov ebp, esp`. Ошибка сохранена в листинге.

Таблица 7.10. Воздействие пролога функции на стек

Адрес	(1), значение	(2), значение	(3), значение	(4), значение
0xefe8	??	??	??	??
0xeff0	??	??	??	0x11223344 (esp)
0xeff4	??	Старое ebp (esp)	Старое ebp (esp, ebp)	Старое ebp (ebp)
0xeff8	Адрес возврата (esp)	Адрес возврата	Адрес возврата	Адрес возврата
0xeffc	2	2	2	2
0xf000	1	1	1	1

Первая часть таблицы показывает стек до начала функции. В этот момент в стек уже помещены параметры функции (справа налево), а также адрес возврата вызывающей функции.

При выполнении инструкции `push ebp` указатель базы предыдущей функции сохраняется в стеке. Получившийся стек показан во второй части таблицы.

Третья часть показывает стек после выполнения инструкции `mov ebp, esp`. Хотя сам стек не обновляется, новый ebp указывает туда же, куда esp, - на адрес возврата вызывающей функции.

После выполнения этих инструкций вызываемая функция может помещать в стек локальные переменные. Это изменяет значение esp, но значение ebp остаётся постоянным (четвёртая часть таблицы). Благодаря этому к параметрам и локальным переменным можно обращаться относительно фиксированной точки ebp, а не более изменчивого esp.

Удаление кадра стека

Создание нового кадра стека для текущей функции означает, что кадр стека вызывающей функции утрачен. Перед возвратом функция должна отменить внесённые изменения и восстановить кадр стека вызывающей функции.

Эту процедуру выполняет эпилог функции, находящийся в её конце. Он состоит из трёх инструкций:

```
mov esp, ebp
pop ebp
ret
```

Первый этап - удалить все добавленные в стек данные. Поскольку данные фактически не стираются из памяти, для этого достаточно изменить указатель стека инструкцией `mov esp, ebp`. Эта операция восстанавливает состояние стека, показанное в таблице 7.11.

```
; тело функции (1)
mov esp, ebp ;(2)
pop ebp ; (3)
```

Затем указатель базы следует восстановить до значения вызывающей функции. Напомним, что в преамбуле оно было помещено в стек, поэтому это выполняется инструкцией `pop ebp`,

возвращающей стек в исходное состояние. Теперь стек находится в состоянии, подходящем для возврата в вызывающую функцию.

Хотя удалить кадр можно этими двумя инструкциями, x86 предлагает другой вариант. Инструкция `leave` эквивалентна следующим двум инструкциям:

```
mov esp, ebp
pop ebp
```

Таблица 7.11. Воздействие эпилога функции на стек

Адрес	(1), значение	(2), значение	(3), значение
0xefe8	0x3325d321 (esp)	0x3325d321	0x3325d321
0xeff0	0x11223344	0x11223344	0x11223344
0xeff4	Старое ebp (ebp)	Старое ebp (ebp, esp)	Старое ebp (esp)
0xeff8	Адрес возврата	Адрес возврата	Адрес возврата
0xeffc	2	2	2
0xf000	1	1	1

Доступ к параметрам

Кадры стека предназначены для упрощения доступа к параметрам и другим значениям в стеке изнутри функции. Использование неизменного `ebp` как точки отсчёта упрощает определение местонахождения конкретного значения в стеке. Например, таблица 7.12 показывает расположение некоторых значений в стеке любой функции, использующей соглашение `cdecl`.

Таблица 7.12. Расположение распространённых значений в стеке

Расположение	Значение
[ebp + 0]	Указатель предыдущего кадра
[ebp + 4]	Адрес возврата функции
[ebp + 8]	Первый параметр
[ebp + 12]	Второй параметр
[ebp + 16]	Третий параметр
...	...

Поскольку `ebp` не перемещается во время выполнения функции, эти соотношения и смещения всегда остаются одинаковыми. Это означает, что если вызывающая функция передала переменные, первая всегда будет находиться по адресу `ebp+8`, вторая - по адресу `ebp+12` и так далее.

Следующий пример показывает создание и использование параметров функции. Как и во всех примерах со стеком, помните, что каждое состояние стека показано после выполнения инструкции.

```
f:
    push  1   ; y
    push  2   ; x   ;(1)
    call  add
    mov   [s], eax
    pop   eax
    pop   eax
    ret

; int add(int x, int y) { return x+y; }
add:
       ; (2)
    push ebp
    mov  ebp, esp   ;(3)
    mov  eax, [ebp+8]     ; получить x из стека
    mov  edx, [ebp+12]    ; получить y из стека
    add  eax, edx
    mov  esp, ebp
    pop  ebp
    ret
```

Таблица 7.13 показывает содержимое стека в точках (1), (2) и (3) предыдущего кода.

Таблица 7.13. Содержимое стека в точках 1, 2 и 3 программы

Адрес	(1), значение	(2), значение	(3), значение
0xeff4	??		Старое ebp (esp, ebp)
0xeff8	??	Адрес возврата (esp)	Адрес возврата
0xeffc	2 (esp)	2	2
0xf000	1	1	1

После достижения точки 3 благодаря знанию cdecl и кадра стека можно с уверенностью сказать, что первый параметр x находится по адресу ebp+8 и имеет значение 2. Второй параметр y находится по адресу ebp+12 и имеет значение 1.

Локальные переменные

Локальные переменные функции хранятся в стеке выше указателя предыдущего кадра (по меньшим адресам). После создания кадра стека место для локальных переменных можно выделить, просто вычтя из esp необходимый объём. Это выделение автоматически отменяется в эпилоге функции, когда указатель стека восстанавливается по указателю базы.

Например, рассмотрим следующую функцию:

```
void one_up(int x)
{
    int y = x + 1;
}
```

Помимо входного аргумента x, она также определяет локальную переменную y, которая будет храниться в стеке. Следующий код показывает эту функцию после преобразования в x86:

```
one_up:
    push ebp
    mov  ebp, esp
```

```
sub  esp, 4        ; выделить место для локальной y (4 байта)
mov  eax, [ebp+8]  ; загрузить параметр x
inc  eax           ; x + 1
mov  [ebp-4], eax  ; сохранить локальную y
  ; здесь показан стек
mov  esp, ebp
pop  ebp
ret
```

Кадр стека этой программы показан в таблице 7.14. Обратите внимание: хотя теперь `esp` указывает на `0xeff4`, после выделения в стеке локальных переменных значение `ebp` остаётся прежним и указывает на сохранённый `ebp` вызывающей функции. И к параметрам, и к локальным переменным можно легко обращаться относительно `ebp`.

Таблица 7.14. Кадр стека программы one_up

Адрес	Значение
`0xeff4 (esp)`	y
`0xeff8 (ebp)`	Старое `ebp`
`0xeffc`	Адрес возврата
`0xf000`	x

Подобно параметрам, `cdecl` обеспечивает хранение локальных переменных в одинаковых местах разных функций. Таблица 7.15 показывает расположение локальных переменных относительно `ebp`. В обратной разработке знание `ebp` становится невероятно мощным средством.

> **СОВЕТ.** Понимание того, что `ebp` минус что-либо обращается к локальной переменной, то есть к чему-либо выделенному внутри функции, тогда как `ebp` плюс что-либо обращается к информации, предоставленной функции вызывающей стороной, помогает быстро замечать интересные фрагменты кода или находить критически важные функции, которыми можно манипулировать, например через ввод программы.

Таблица 7.15. Расположение локальных переменных в стеке

Расположение	Значение
`[ebp - 4]`	Первая локальная переменная
`[ebp - 8]`	Вторая локальная переменная
`[ebp - 12]`	Третья локальная переменная
`...`	`...`

Сокращённые приёмы

Каждый параметр или локальную переменную можно помещать в стек отдельно; каждая инструкция `push` обновляет указатель стека и перемещает значение на место.

Вместо этого компиляторы часто сразу выделяют всё пространство, прежде чем размещать значения. Например, следующие инструкции реже встречаются в скомпилированном коде:

```
push 1
push 2
```

Гораздо вероятнее, что они будут ассемблированы в следующие:

```
sub esp, 8             ; выделить 8 байт в стеке
mov dword [esp+4], 1   ; поместить 1 в стек
mov dword [esp], 2     ; поместить 2 в стек
```

Выравнивание стека

Некоторые компиляторы при входе в функцию обеспечивают 32-байтовое выравнивание стека. Это означает стремление сделать адрес памяти в указателе стека без остатка делящимся на 32. Исторически системам было эффективнее извлекать память по границам, выровненным на 32 байта. Возможно, такого выигрыша в эффективности уже нет, но иногда всё ещё можно увидеть, как компиляторы выполняют действия для сохранения выравнивания стека.

Одно из проявлений состоит в том, что при выделении пространства под локальные переменные может выделяться избыточное пространство для сохранения этого выравнивания.

Поэтому внутри кадра стека функции часто имеется неиспользуемое пространство. При обратной разработке не зацикливайтесь на избыточном пространстве: оно совершенно нормально. При написании собственного кода это не требуется делать вручную, однако цель этой книги - научить вас распознавать такое в коде и понимать, что в основном это можно игнорировать.

Общая картина

Вызов функции вносит несколько изменений в стек программы. Чтобы увидеть все изменения вместе, рассмотрим следующую программу:

```
void hack(...)
{
    ...
}

void drink(...)
{
    ...
    hack(...);
    ...
}
```

Каждая из двух функций этой программы может иметь ноль или больше параметров и локальных переменных. На рис. 7.3 показана структура кадров стека обеих функций.

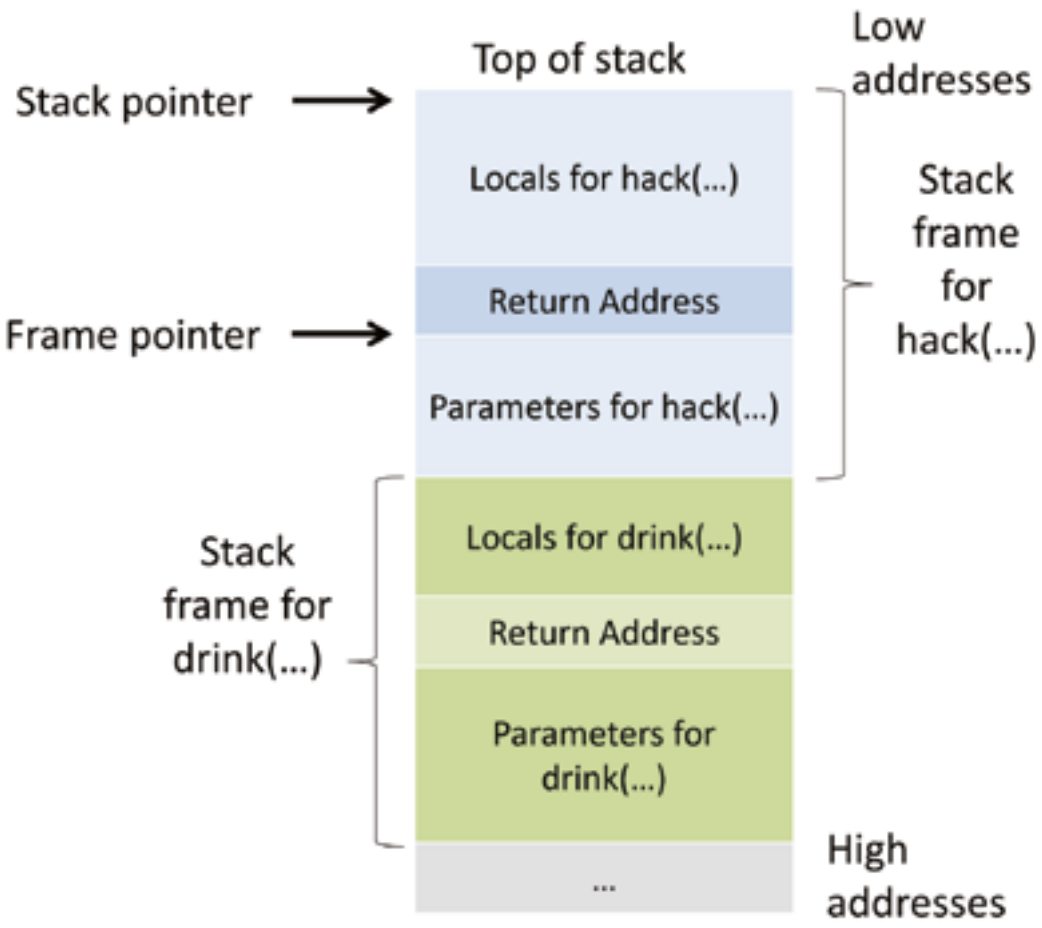


Рис. 7.3. Кадры стека функций hack и drink

Что нужно запомнить

Ассемблерные программы x86 могут быть сложными. Для эффективной обратной разработки x86 жизненно важно запомнить некоторые вещи.

Первая из них - структура кадра стека функции. В таблице 7.16 показан полный кадр стека, включая параметры, адреса возврата, локальные переменные и рабочее пространство.

Таблица 7.16. Полный кадр стека функции

Расположение в стеке	Значение
`...`	`...`
`[ebp-12]` или `[ebp-0xC]`	Третья локальная переменная
`[ebp-8]`	Вторая локальная переменная
`[ebp-4]`	Первая локальная переменная
`[ebp]`	Указатель предыдущего кадра
`[ebp+4]`	Адрес возврата функции

Расположение в стеке	Значение
`[ebp+8]`	Первый параметр
`[ebp+12]` или `[ebp+0xC]`	Второй параметр
`[ebp+16]` или `[ebp+0xF]`	Третий параметр
`...`	`...`

> **Примечание переводчика.** В исходной таблице для третьего параметра напечатано `[ebp+0xF]`, хотя десятичному смещению 16 соответствует `0x10`. Исходное значение сохранено.

Также важно запомнить различие между шаблонными и полными прологами и эпилогами функций. Таблица 7.17 показывает, чем шаблонный пролог отличается от пролога, включающего выделение в стеке пространства под локальные переменные.

Таблица 7.17. Два типа прологов

Шаблонный пролог	Полный пролог
<pre>push ebp ; сохранить кадр стека mov ebp, esp ; начать новый кадр</pre>	<pre>push ebp ; сохранить кадр стека mov ebp, esp ; начать новый кадр sub esp, 20 ; выделить пять 4-байтовых локальных переменных push ebx ; сохранить изменяемые регистры push esi (и так далее)</pre>

Эпилог функции обращает воздействие пролога функции. В таблице 7.18 показаны соответствующие эпилоги для каждого из этих прологов.

Таблица 7.18. Два типа эпилогов

Шаблонный эпилог	Полный эпилог
<pre>mov esp, ebp ; отбросить локальные переменные pop ebp ; восстановить кадр ret ; возврат</pre>	<pre>(и так далее) pop esi ; восстановить изменённые регистры pop ebx mov esp, ebp ; отбросить локальные переменные pop ebp ; восстановить кадр ret ; возврат</pre>

Итоги

В этой главе были рассмотрены жизненно важные понятия обратной разработки и взлома приложений. Прежде чем двигаться дальше, убедитесь, что твёрдо понимаете, как может работать поток управления внутри приложений, а также все подробности функций и их кадров стека.

Глава 8. Компиляторы и оптимизаторы

Для многих языков программирования высокого уровня компиляция является жизненно важной частью преобразования приложения из исходного кода в машиночитаемый двоичный код. В ходе этого процесса компилятор может вносить в код небольшие изменения, чтобы сделать его как можно более быстрым и эффективным.

Компиляция и оптимизация приложения способны усложнить его обратную разработку. В этой главе рассказывается, как найти отправную точку для обратной разработки приложения, а также о некоторых распространённых действиях компиляторов, затрудняющих обратную разработку.

Поиск начального кода

При компиляции кода компилятор добавляет большой объём шаблонного кода, выполняемого ещё до вызова настоящего кода приложения. Одно из искусств, которым необходимо овладеть специалисту по обратной разработке, - научиться пропускать его и сосредотачиваться на целевом коде, а не на шаблонной подготовке. Однако определить точку входа в целевой код бывает непросто.

При обратной разработке чужого кода маловероятно, что он будет скомпилирован с отладочными символами. Это означает, что имена функций и переменных, а также прочая информация, способная подсказать настоящую точку входа в код, удалены из приложения. На рис. 8.1 показано, как в gdb выглядит открытие файла без отладочных символов.

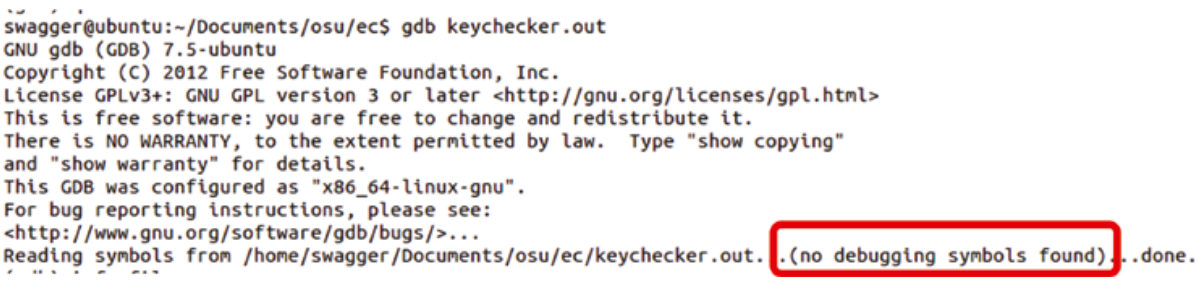


Рис. 8.1. Приложение без отладочных символов в gdb

Ключевая строка вывода:

```
Reading symbols from /home/swagger/Documents/osu/ec/keychecker.out...
(no debugging symbols found)...done.
```

Отсутствие отладочных символов создаёт серьёзную проблему, поскольку приложения, написанные на языках высокого уровня, а не на чистом ассемблере, содержат гораздо больше служебного кода и созданных компилятором символов. Ниже приведён пример вывода команды `info files` в gdb, показывающий множество разделов простого исполняемого файла:

```
Entry point: 0x80483a0
0x08048154 -0x08048167 is .interp
0x08048168 -0x08048188 is .note.ABI-tag
0x08048188 -0x080481ac is .note.gnu.build-id
0x080481ac -0x080481cc is .gnu.hash
0x080481cc -0x0804823c is .dynsym
0x0804823c -0x080482a6 is .dynstr
0x080482a6 -0x080482b4 is .gnu.version
0x080482b4 -0x080482e4 is .gnu.version_r
0x080482e4 -0x080482ec is .rel.dyn
0x080482ec -0x08048314 is .rel.plt
0x08048314 -0x08048338 is .init
```

```
0x08048340 -0x080483a0 is .plt
0x080483a0 -0x08048648 is .text
0x08048648 -0x0804865d is .fini
0x08048660 -0x080486a9 is .rodata
0x080486ac -0x080486f0 is .eh_frame_hdr
0x080486f0 -0x080487f4 is .eh_frame
0x08049f08 -0x08049f0c is .init_array
0x08049f0c -0x08049f10 is .fini_array
0x08049f10 -0x08049f14 is .jcr
0x08049f14 -0x08049ffc is .dynamic
0x08049ffc -0x0804a000 is .got
0x0804a000 -0x0804a020 is .got.plt
0x0804a020 -0x0804a028 is .data
0x0804a028 -0x0804a02c is .bss
0xf7fdc114 -0xf7fdc138 is .note.gnu.build-id in /lib/ld-linux.so.2
0xf7fdc138 -0xf7fdc1f4 is .hash in /lib/ld-linux.so.2
0xf7fdc1f4 -0xf7fdc2d4 is .gnu.hash in /lib/ld-linux.so.2
0xf7fdc2d4 -0xf7fdc494 is .dynsym in /lib/ld-linux.so.2
0xf7fdc494 -0xf7fdc612 is .dynstr in /lib/ld-linux.so.2
0xf7fdc612 -0xf7fdc64a is .gnu.version in /lib/ld-linux.so.2
0xf7fdc64c -0xf7fdc714 is .gnu.version_d in /lib/ld-linux.so.2
0xf7fdc714 -0xf7fdc77c is .rel.dyn in /lib/ld-linux.so.2
0xf7fdc77c -0xf7fdc7ac is .rel.plt in /lib/ld-linux.so.2
0xf7fdc7b0 -0xf7fdc820 is .plt in /lib/ld-linux.so.2
0xf7fdc820 -0xf7ff4baf is .text in /lib/ld-linux.so.2
0xf7ff4bc0 -0xf7ff8a60 is .rodata in /lib/ld-linux.so.2
0xf7ff8a60 -0xf7ff90ec is .eh_frame_hdr in /lib/ld-linux.so.2
0xf7ff90ec -0xf7ffb654 is .eh_frame in /lib/ld-linux.so.2
0xf7ffccc0 -0xf7ffcf3c is .data.rel.ro in /lib/ld-linux.so.2
0xf7ffcf3c -0xf7ffcff4 is .dynamic in /lib/ld-linux.so.2
```

В более сложных двоичных файлах со множеством зависимостей и библиотек этот список может стать ещё длиннее. По данному выводу известно, что раздел `.text` исполняемого файла находится по адресу `0x080483a0`. Дизассемблирование кода по этому адресу может подсказать точку входа в целевой код. На рис. 8.2 показан результат дизассемблирования кода по этому адресу в gdb.

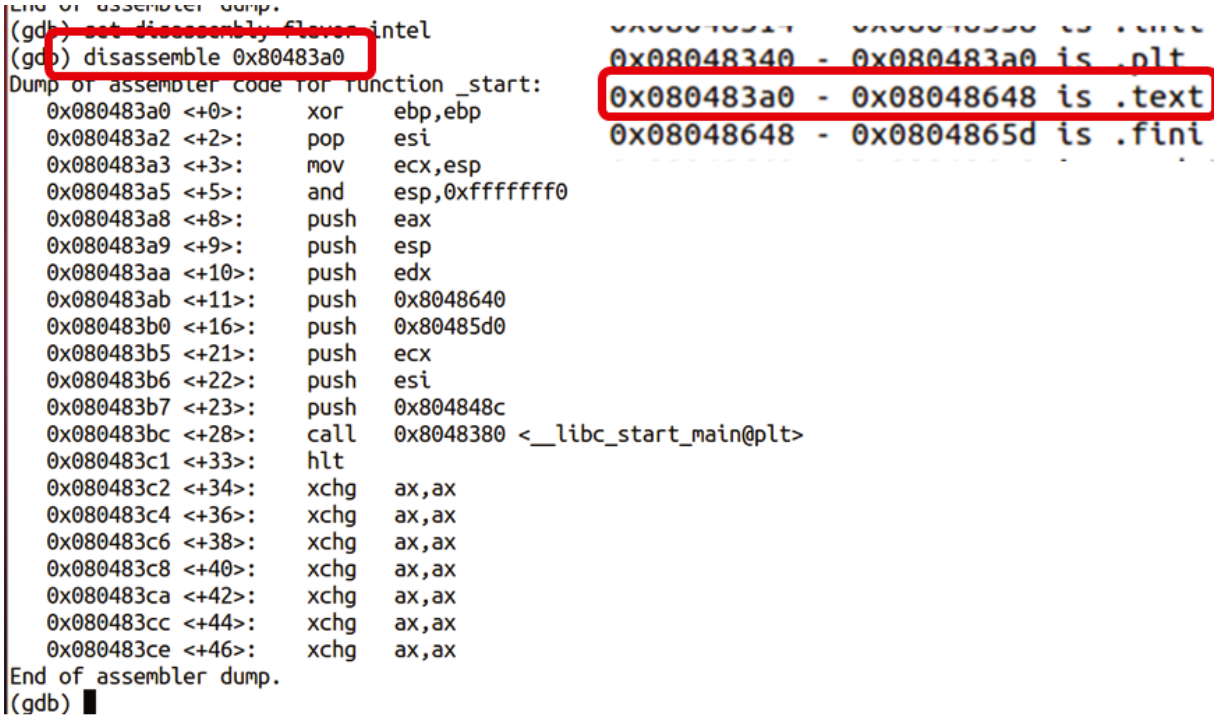


Рис. 8.2. Дизассемблирование раздела .text в gdb

Основной вывод снимка:

```
(gdb) disassemble 0x80483a0
Dump of assembler code for function _start:
0x080483a0 <+0>:  xor  ebp,ebp
```

```
0x080483a2 <+2>:  pop  esi
0x080483a3 <+3>:  mov  ecx,esp
0x080483a5 <+5>:  and  esp,0xfffffff0
0x080483a8 <+8>:  push eax
0x080483a9 <+9>:  push esp
0x080483aa <+10>: push edx
0x080483ab <+11>: push 0x8048640
0x080483b0 <+16>: push 0x80485d0
0x080483b5 <+21>: push ecx
0x080483b6 <+22>: push esi
0x080483b7 <+23>: push 0x804848c
0x080483bc <+28>: call 0x8048380 <__libc_start_main@plt>
0x080483c1 <+33>: hlt
```

Способ поиска точки входа в целевой код зависит от конкретного компилятора и языка сборки. Здесь будет показан поиск начального кода в приложении C/C++, поскольку этот язык и сегодня остаётся одним из самых распространённых. Для начала найдите вызов `__libc_start_main`. Адрес целевого кода будет передан этой функции как параметр, а из знаний о соглашениях вызовов известно, что искать нужно значение, помещённое в стек перед вызовом.

На рис. 8.2 адрес `0x804848c` помещается в стек непосредственно перед вызовом `__libc_start_main`, становясь параметром функции. Следовательно, целевой код начинается по этому адресу. На рис. 8.3 показано дизассемблирование функции `main`, включая вызовы `libc`.

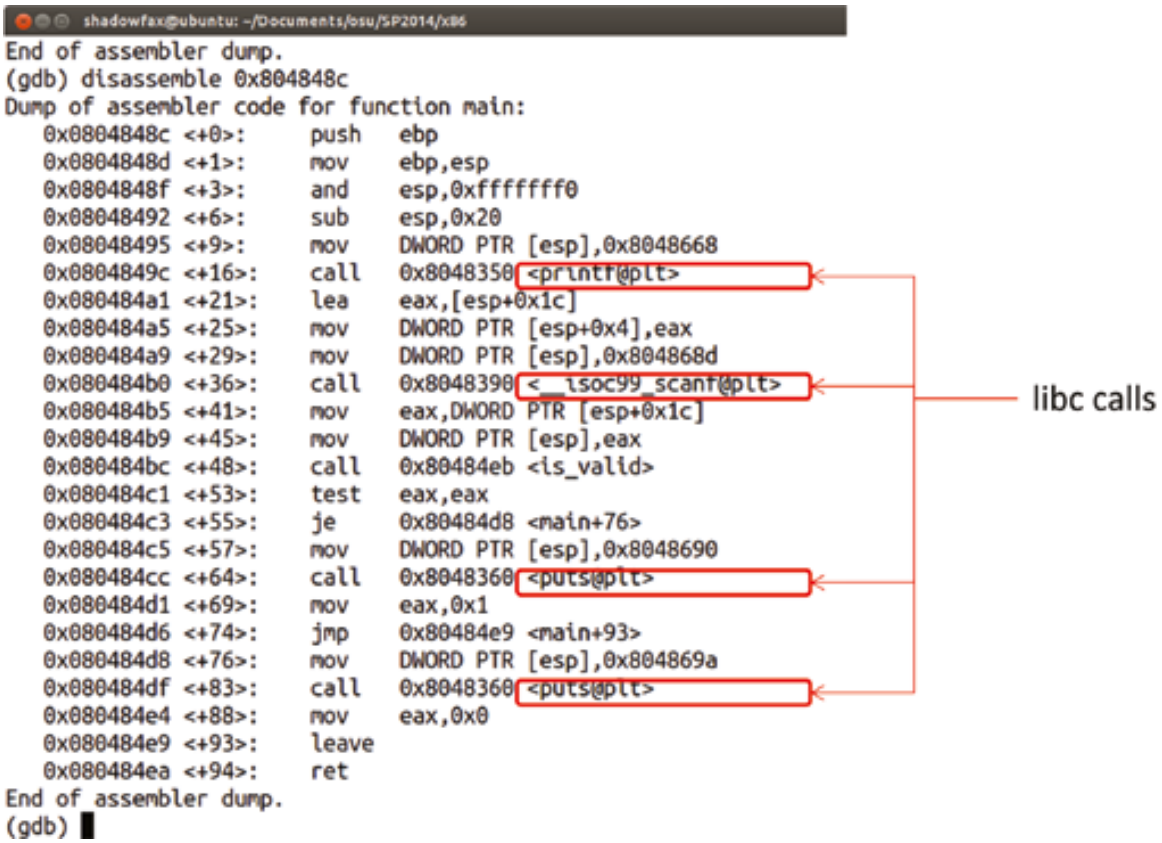


Рис. 8.3. Дизассемблирование функции main в gdb

На снимке отмечены вызовы функций `libc`: `printf@plt`, `__isoc99_scanf@plt`, `puts@plt` и ещё один `puts@plt`.

Компиляторы

Компиляторы принимают код и переводят его в машинный код, читаемый процессором. Компиляторы могут по-разному влиять на обратную разработку, как намеренно, так и ненамеренно. В этом разделе основное внимание уделяется ненамеренным изменениям; намеренные методы, например обфускация, будут рассмотрены в главе 12 «Защита».

Оптимизация

Компиляторы можно настраивать на оптимизацию кода по различным показателям, включая скорость и размер на диске, либо вообще отключать оптимизацию. В зависимости от применения оптимизаций код может выглядеть совершенно по-разному.

Рассмотрим следующий пример. Этот код реализует простой оператор `if` с двумя условиями.

```
int main(int argc, char* argv[])
{
    if (argc >= 3 && argc <= 8)
    {
        printf("valid number of args\n");
    }
}
```

На рис. 8.4 показано, как код выглядит в дизассемблере (подробнее о нём будет рассказано в главе 11 «Исправление и расширенные инструменты», не беспокойтесь), если он скомпилирован без оптимизаций. Обратите внимание: обе проверки условий, сравнивающие значения с 2 и 8, ясно видны в коде.

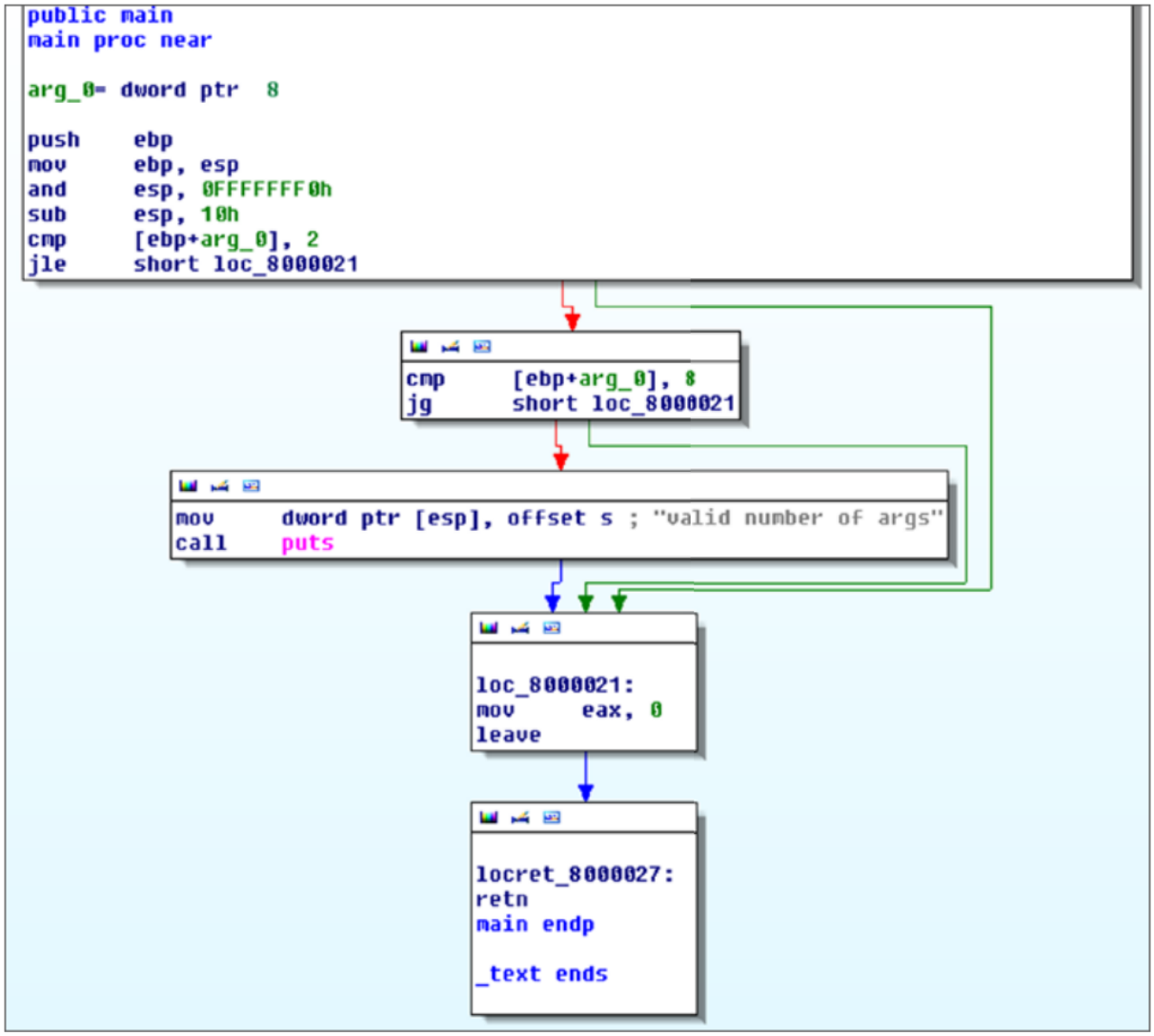


Рис. 8.4. Неоптимизированный код в дизассемблере

Ключевые инструкции двух отдельных сравнений:

```
cmp [ebp+arg_0], 2
jle short loc_8000021
cmp [ebp+arg_0], 8
jg  short loc_8000021
```

На рис. 8.5 тот же код показан после оптимизации по скорости и размеру. Сравнения со значениями 2 и 8 в коде больше не видны, и код уже не похож на оператор `if` с двумя условиями.

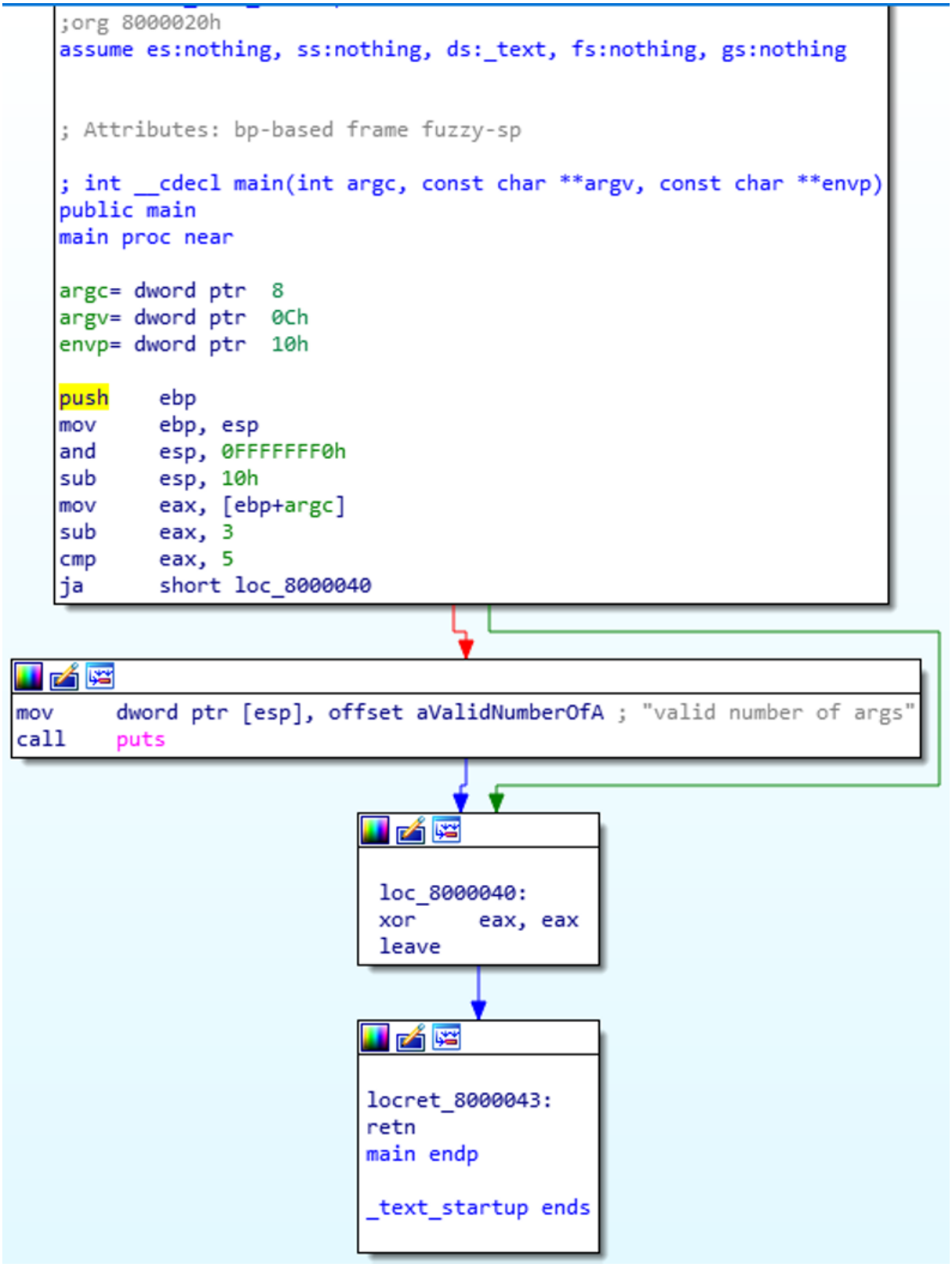


Рис. 8.5. Код в дизассемблере после оптимизации по скорости и размеру

Ключевой оптимизированный фрагмент:

```
mov eax, [ebp+argc]
sub eax, 3
cmp eax, 5
ja  short loc_8000040
```

На рис. 8.6 показан код, оптимизированный исключительно по размеру на диске. Два сравнения снова отсутствуют.

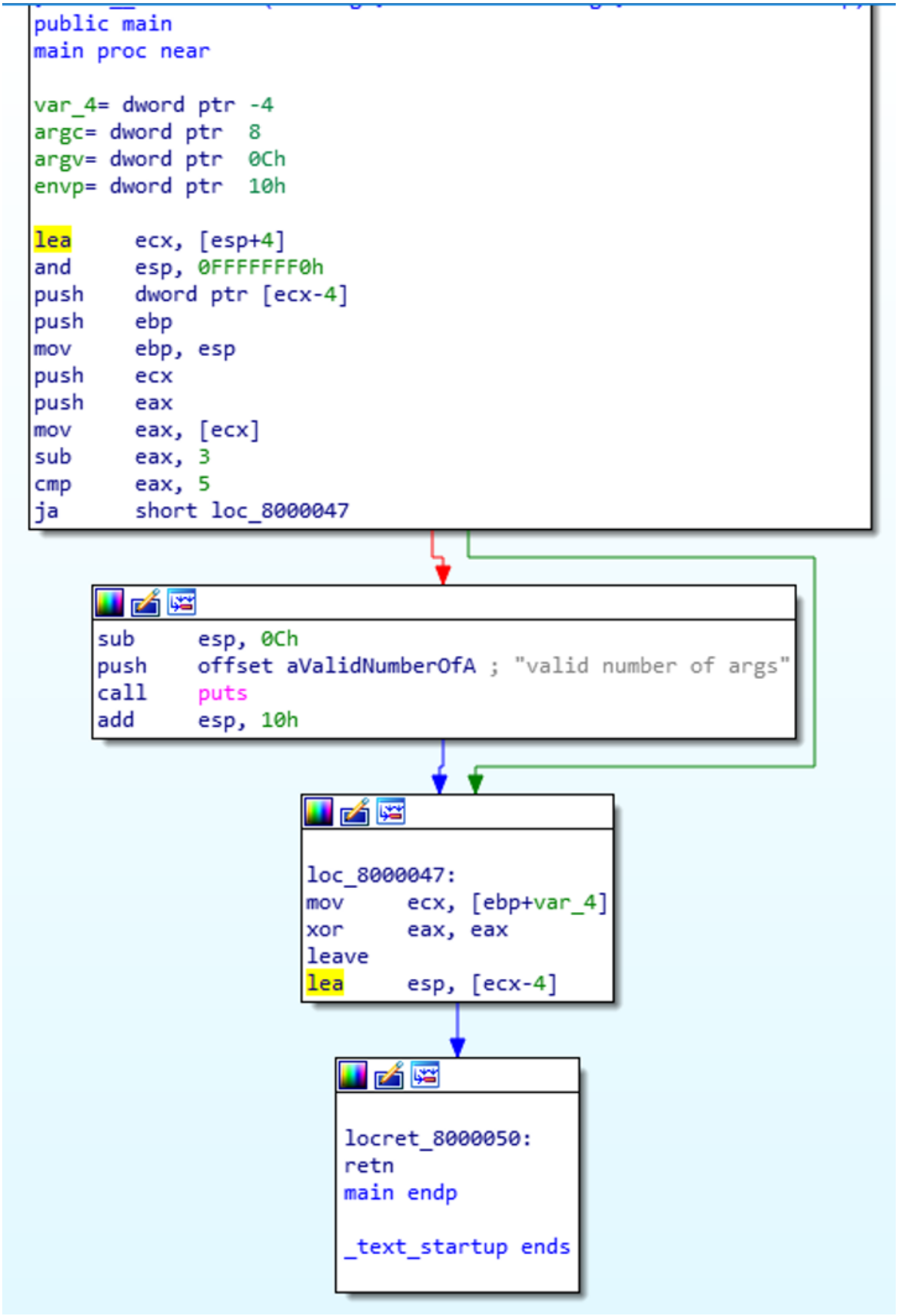


Рис. 8.6. Код в дизассемблере после оптимизации по размеру

На этом снимке та же проверка представлена следующими ключевыми инструкциями:

```
mov eax, [ecx]
sub eax, 3
cmp eax, 5
ja  short loc_8000047
```

Если изучить код, видно, что он проверяет условие `(argc-3) > 5`. Если `argc < 3`, то вычитание 3 вызовет потерю значимости снизу, и значение в `eax` станет большим положительным числом. Если `argc > 8`, то `argc-3 > 5`. В обоих случаях результат будет больше 5, поэтому оптимизированный оператор эквивалентен исходной проверке. Оптимизации компилятора дают эквивалентную логику, но способны значительно затруднить чтение кода и рассуждение о нём.

У большинства компиляторов есть параметры, задающие уровень оптимизации. Во время обучения, если вам трудно выполнять обратную разработку написанного вами приложения, попробуйте при компиляции отключить оптимизации. И наоборот, если вы хотите затруднить обратную разработку своего кода, оптимизации компилятора являются простым и полезным способом сделать это.

Удаление символов

Удаление символов из двоичного файла означает удаление всей информации, не требующейся для выполнения кода, включая таблицу символов. Необработанный таким образом двоичный файл сохраняет таблицу символов, а обработанный - нет.

Символы могут быть чрезвычайно полезны при отладке приложения. Например, рассмотрим следующий код:

```
// Объявить внешнюю функцию
extern double bar(double x);

// Определить общедоступную функцию
double foo(int count)
{
    double sum = 0.0;

    // Суммировать все значения от bar(1) до bar(count)
    for (int i = 1; i <= count; i++)
        sum += bar((double) i);
    return sum;
}
```

Если этот код обработать компилятором, он будет содержать как минимум записи таблицы символов, показанные на рис. 8.7. Символы настолько полезны при отладке, что Microsoft позволяет загружать символы для своих приложений на случай необходимости устранения неполадок! Эта дополнительная информация может оказаться бесценной для понимания назначения приложения.

Symbol name	Type	Scope
bar	function, double	extern
x	double	function parameter
foo	function, double	global
count	int	function parameter
sum	double	block local
i	int	for-loop statement

Рис. 8.7. Отладочные символы приложения

Имя символа	Тип	Область видимости
bar	функция, double	внешний
x	double	параметр функции
foo	функция, double	глобальный
count	int	параметр функции
sum	double	локальный в блоке
i	int	оператор цикла for

Если символы удалены из файла, при открытии в gdb будет сообщено, что отладочные символы не найдены, как показано на рис. 8.1. Обратная разработка таких файлов значительно сложнее.

Символы можно удалить из приложения несколькими способами. Один вариант - использовать флаги компилятора, например `gcc -fno-rtti -s`. Другой - применить после сборки инструменты удаления символов, например `strip` в Linux.

Символы облегчают злоумышленнику обратную разработку приложения, поскольку помогают находить интересующие участки и понимать назначение определённых переменных. Однако существуют законные причины оставить символы в приложении. Например, символы помогают создавать отчёты о сбоях и журналы ошибок, а также обеспечивают правомерную отладку для исправления ошибок клиентов. Во время обучения, если вы пишете и компилируете собственный код для практики, сначала убедитесь, что выполняете сборку с сохранением символов. По мере совершенствования навыков удаляйте символы. При обратной разработке чужого кода крайне маловероятно, что в нём оставлены символы, но такое всё же случается!

Компоновка

В наши дни приложения редко пишутся изолированно. Гораздо чаще в них включают библиотеки, предоставляющие основные возможности, например связь, журналирование, рисование и так далее. При компиляции приложения, использующего библиотеки, существует два способа их сборки. Библиотеки могут быть скомпонованы с приложением статически или динамически. С точки зрения взлома программного обеспечения у каждого варианта есть преимущества и недостатки.

Статическая компоновка

При статической компоновке библиотеки встраиваются в само приложение. Это повышает скорость выполнения, поскольку целевые адреса всех вызовов библиотеки встраиваются в приложение во время компиляции. Кроме того, статически скомпонованные приложения более переносимы, поскольку меньше зависят от окружения.

Однако у статической компоновки есть и недостатки. Статически скомпонованные приложения имеют больший размер, поскольку в исполняемый файл встраивается вся библиотека, даже если из большой библиотеки используется лишь одна функция. Кроме того, любое обновление библиотеки требует повторной компиляции использующих её приложений.

Раздувание файлов из-за статической компоновки может быть значительным. Например, как показано на рис. 8.8, даже простая однострочная программа «hello world» подключает десятки библиотек.

Динамическая компоновка

Динамическая компоновка - другой вариант и выбор по умолчанию для многих компиляторов. При динамической компоновке необходимые библиотеки находятся в системе во время выполнения. Если библиотека ещё не загружена в системную память, её необходимо найти в системе и загрузить в память совместно используемых библиотек; однако распространённые библиотеки, вероятно, уже загружены и доступны для использования.

```
0xf7fdc114 - 0xf7fdc138 is .note.gnu.build-id in /lib/ld-linux.so.2
0xf7fdc138 - 0xf7fdc1f4 is .hash in /lib/ld-linux.so.2
0xf7fdc1f4 - 0xf7fdc2d4 is .gnu.hash in /lib/ld-linux.so.2
0xf7fdc2d4 - 0xf7fdc494 is .dynsym in /lib/ld-linux.so.2
0xf7fdc494 - 0xf7fdc612 is .dynstr in /lib/ld-linux.so.2
0xf7fdc612 - 0xf7fdc64a is .gnu.version in /lib/ld-linux.so.2
0xf7fdc64c - 0xf7fdc714 is .gnu.version_d in /lib/ld-linux.so.2
0xf7fdc714 - 0xf7fdc77c is .rel.dyn in /lib/ld-linux.so.2
0xf7fdc77c - 0xf7fdc7ac is .rel.plt in /lib/ld-linux.so.2
0xf7fdc7b0 - 0xf7fdc820 is .plt in /lib/ld-linux.so.2
0xf7fdc820 - 0xf7ff4baf is .text in /lib/ld-linux.so.2  96KB!
0xf7ff4bc0 - 0xf7ff8a60 is .rodata in /lib/ld-linux.so.2
0xf7ff8a60 - 0xf7ff90ec is .eh_frame_hdr in /lib/ld-linux.so.2
0xf7ff90ec - 0xf7ffb654 is .eh_frame in /lib/ld-linux.so.2
0xf7ffccc0 - 0xf7ffcf3c is .data.rel.ro in /lib/ld-linux.so.2
0xf7ffcf3c - 0xf7ffcff4 is .dynamic in /lib/ld-linux.so.2

..(many more)...
```

Рис. 8.8. Подключённые библиотеки программы «hello world»

Снимок показывает множество разделов `/lib/ld-linux.so.2`; один только раздел `.text` отмечен размером 96 КБ, после чего следует пометка о множестве других элементов.

Динамическая компоновка уменьшает размер приложения и, если обновление обратно совместимо, устраняет необходимость повторно компилировать приложение после обновления библиотеки. Кроме того, динамически скомпонованные приложения могут загружаться быстрее, если используемые ими библиотеки уже находятся в памяти.

Однако динамически скомпонованные приложения зависят от наличия необходимых библиотек в системе и могут быть медленнее статически скомпонованных, если зависимости ещё не загружены и их требуется найти и загрузить. Помимо необходимости загружать отсутствующие в памяти библиотеки, динамически скомпонованные приложения во время выполнения должны находить адрес вызываемых функций. Для этого приходится искать библиотеку в пространстве совместно используемой памяти, что может потребовать значительного числа подкачек страниц памяти.

Влияние компоновки на безопасность

Выбор статической или динамической компоновки зависит от разработчика или компилятора. Но если надеть шляпу взломщика программ, оба варианта имеют свои последствия для безопасности.

Специалисты по обратной разработке обычно предпочитают, чтобы приложение было скомпоновано статически. Статическая компоновка упрощает определение точного адреса загрузки функций совместно используемых библиотек, что полезно при создании эксплойтов. Это означает, что для эксплуатации можно задействовать код из совместно используемых библиотек, и во время выполнения этот код будет находиться в предсказуемом месте внутри двоичного файла. Использовать динамически подключённую библиотеку возможно, но намного сложнее: всякий раз приходится искать нужную библиотеку в памяти совместно используемых библиотек и определять её адрес, поскольку она перемещается и непредсказуема.

Взломщики защит, напротив, склонны предпочитать динамически подключённые библиотеки. При динамической компоновке приходится просматривать гораздо меньше кода, а взломщиков интересует исключительно собственный код приложения, но не код совместно используемых библиотек.

Итоги

Компиляция и оптимизация приложения способны значительно усложнить его обратную разработку, даже если компилятор не выполняет намеренную обфускацию. Однако, как и любая защита от обратной разработки, они могут лишь замедлить процесс, поскольку невзламываемого программного обеспечения не существует.

Глава 9. Обратная разработка: инструменты и стратегии

До сих пор основное внимание в этой книге уделялось пониманию внутреннего устройства компьютеров. Это необходимо, чтобы эффективно взламывать программное обеспечение.

Теперь, когда фундамент заложен, внимание переключается на искусство взлома программ. Для экспериментов и практики взлома будут использоваться различные цели:

• **Настоящее программное обеспечение:** программы из реального мира. При анализе настоящего программного обеспечения необходимо учитывать законодательство об авторском праве, чтобы не допустить его нарушения.
• **Искусственно созданные примеры:** приложения, написанные для этой книги с целью продемонстрировать определённые понятия.
• **Crackme:** небольшие допускающие взлом программы, написанные другими взломщиками программного обеспечения, чтобы продемонстрировать идею и бросить вызов коллегам.

Подобные используемым в этом курсе `crackme` являются искусственно созданными примерами и дают начинающему взломщику несколько преимуществ. Как правило, они спроектированы так, чтобы иметь решение, быть законными для взлома и безопасными для запуска в отладчике.

Кроме того, `crackme` часто снабжаются метками, отражающими их направленность, требуемый уровень подготовки и так далее. Благодаря этому можно целенаправленно искать задачи, соответствующие вашим интересам и уровню навыков, например для продвинутого взломщика C или начинающего взломщика Java.

Лабораторная работа: RE Bingo

Эта лабораторная работа даёт практический опыт обратной разработки кода, собранного (и обфусцированного) компилятором.

Лабораторные работы и все связанные с ними инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой лабораторной работы найдите **Lab RE Bingo** и следуйте предоставленным инструкциям.

Навыки

В лабораторной работе используется `objdump`, чтобы потренироваться распознавать конструкции потока управления и параметры компилятора при обратной разработке. Проверяются, в частности, следующие ключевые навыки:

• обратная разработка x86;
• конструкции потока управления;
• влияние параметров компилятора.

Основные выводы

Быстрое распознавание конструкций потока управления способно значительно ускорить обратную разработку. Эти конструкции дают представление о логике приложения, делая его более читаемым и понятным.

Однако конфигурация компилятора существенно влияет на скорость обратной разработки. Например, удаление символов и оптимизация в целом замедляют работу.

В больших и сложных программах нередко приходится автоматизировать часть обратной разработки. Для конкретной цели часто пишут специальные инструменты. Распаковка, деобфускация и обход проверок против отладки - распространённые задачи автоматизации.

Базовая разведка при обратной разработке

Взломщик программного обеспечения чаще всего сталкивается со следующей ситуацией:

• вы хотите взломать программу;
• у вас нет исходного кода;
• у вас есть исполняемый файл.

В такой ситуации необходимо быстро оценить целевой исполняемый файл и найти отправную точку для анализа. Одними из самых распространённых начальных инструментов специалистов по обратной разработке являются `objdump`, `strace`, `ltrace` и `strings`. По мере продвижения по книге появятся более совершенные инструменты, но эти относятся к основополагающим и являются хорошей отправной точкой.

objdump

Object Dump (`objdump`) - инструмент Linux для вывода дизассемблированного кода любой программы. Как показано на рис. 9.1, у него имеется множество параметров. Для быстрой обратной разработки наиболее важны следующие:

• `-d`: приказывает `objdump` дизассемблировать содержимое всех разделов;
• `-Mintel`: приказывает `objdump` отображать ассемблерный код в синтаксисе Intel, а не AT&T.

Например, чтобы дизассемблировать приложение с именем `appname`, используйте команду:

```
objdump -d -Mintel appname
```

```
shadowfax@ubuntu:~/Desktop/blackhat$ objdump
Usage: objdump <option(s)> <file(s)>
 Display information from object <file(s)>.
 At least one of the following switches must be given:
  -a, --archive-headers    Display archive header information
  -f, --file-headers       Display the contents of the overall file header
  -p, --private-headers    Display object format specific file header contents
  -P, --private=OPT,OPT... Display object format specific contents
  -h, --[section-]headers  Display the contents of the section headers
  -x, --all-headers        Display the contents of all headers
  -d, --disassemble        Display assembler contents of executable sections
```

Рис. 9.1. Параметры objdump

На рис. 9.2 показан результат запуска `objdump` для примера приложения. Обратите внимание: `objdump` отображает адреса памяти, имена функций, машинный код x86 и ассемблерный код x86.

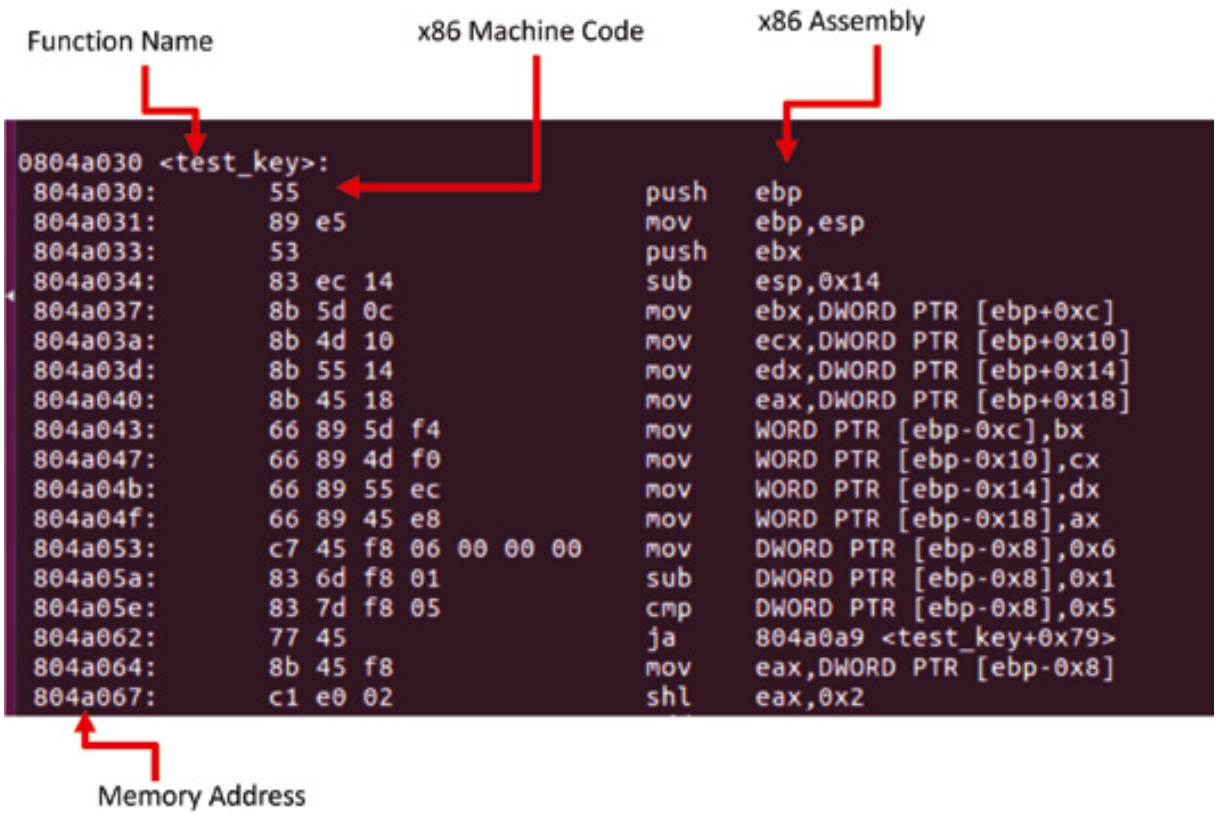


Рис. 9.2. Пример вывода objdump

Подписи на снимке обозначают:

- **Function Name** - имя функции;
- **x86 Machine Code** - машинный код x86;
- **x86 Assembly** - ассемблерный код x86;
- **Memory Address** - адрес памяти.

strace и ltrace

`strace` и `ltrace` позволяют отслеживать библиотечные (`ltrace`) и системные (`strace`) вызовы. С их помощью можно пройти по выполнению программы и получить представление о том, что делают другие программы.

Чтобы выполнить сколько-нибудь полезное действие, любая программа на любом языке должна обращаться к системным вызовам. Определение используемых ею библиотек и внешних функций может быть чрезвычайно полезно при разведке системы.

Обратите внимание: эти инструменты показывают не только то, что используется, но и кто это использует, то есть какой адрес приложения выполнил вызов. Поэтому они также помогают сосредоточиться на полезных функциях. Например, можно увидеть, какой фрагмент кода часто вызывает криптографические библиотеки; с точки зрения взлома он, вероятно, представляет интерес.

ltrace

`ltrace` (library trace, трассировка библиотек) - утилита командной строки Linux, отслеживающая библиотечные вызовы. Библиотечные вызовы - это вызовы из приложения в динамически подключённые библиотеки. Синтаксис команды: `ltrace <command>`.

Например, если в код включён `#include <stdio.h>`, эта библиотека динамически подключается при загрузке программы. Вызов `printf` или `fopen` является вызовом стандартной библиотеки C. Такая конструкция справедлива для всех языков программирования: во всех них существует понятие подключения внешних библиотек.

strace

`strace` (system trace, системная трассировка) - утилита командной строки Linux, отслеживающая системные вызовы. Синтаксис команды: `strace <command>`.

Системные вызовы - это обращения приложения к операционной системе, которая управляет такими объектами, как файлы и окно консоли. Внутри функций вроде `fopen` и `printf` в конечном счёте обязательно выполняются обращения к операционной системе. Как и с `ltrace`, это справедливо для всех языков программирования; приложения, не использующие функций уровня ОС, встречаются редко.

Пример strace: echo

Наблюдение за системными вызовами даёт грубый способ трассировки программы. Предположим, вы написали утилиту `echo` и хотите проследить за её работой.

`echo` - команда Linux, повторяющая ввод в выводе. Например, команда `echo hello!` выводит в терминал строку `hello!`.

Но что она делает в действительности? Выполнение `strace echo hello!` выдаст результат, похожий на рис. 9.3.

```
steph@steph-XPS-9320:~$ strace echo hello!
execve("/usr/bin/echo", ["echo", "hello!"], 0x7ffec41908e8 /* 44 vars */) = 0
brk(NULL)                               = 0x55976ad13000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffc703b2180) = -1 EINVAL (Invalid argument)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f814463f000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=78555, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 78555, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f814462b000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
pread64(3, "\4\0\0\0 \0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48, 848) = 48
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\244;\374\204(\337f#\315I\214\234\f\256\271\32"..., 68, 896) = 68
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2216304, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2260560, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f8144400000
mmap(0x7f8144428000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f8144428000
mmap(0x7f81445bd000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) = 0x7f81445bd000
mmap(0x7f8144615000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x214000) = 0x7f8144615000
mmap(0x7f814461b000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f814461b000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f8144628000
arch_prctl(ARCH_SET_FS, 0x7f8144628740) = 0
set_tid_address(0x7f8144628a10)         = 142481
set_robust_list(0x7f8144628a20, 24)     = 0
rseq(0x7f81446290e0, 0x20, 0, 0x53053053) = 0
mprotect(0x7f8144615000, 16384, PROT_READ) = 0
mprotect(0x559768f6d000, 4096, PROT_READ) = 0
mprotect(0x7f8144679000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f814462b000, 78555)           = 0
getrandom("\xa1\x77\x9f\xec\xe6\x33\xaa\xf1", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x55976ad13000
brk(0x55976ad34000)                     = 0x55976ad34000
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=5712208, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 5712208, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f8143e00000
close(3)                                = 0
newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}, AT_EMPTY_PATH) = 0
write(1, "hello!\n", 7hello!
)                 = 7
close(1)                                = 0
close(2)                                = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

Рис. 9.3. Вывод strace для echo hello!

Это сложный снимок, который непросто расшифровать. В начале результата видны стандартные системные вызовы, используемые для запуска программы `echo`.

Наиболее интересны следующие строки в самом конце:

```
write(1, "hello!\n", 7hello!
)                 = 7
close(1)          = 0
```

Они означают, что `echo` записала строку в поток 1, который, как вы помните, является `stdout`. Команда `write` вернула значение 7, поскольку было записано семь символов. Наконец, `echo` закрыла поток 1, получив код успешного выполнения 0. Это кажется простым, но представьте, что таким способом отслеживается место, куда приложение записало фрагмент конфигурационных данных. Допустим, вы изменили настройку и хотите увидеть, как она сохраняется в системе.

Пример strace: вредоносные котята

Comet Cursor был одним из ранних примеров шпионского программного обеспечения для ОС Windows. Он позволял пользователям изменять вид указателя мыши, а веб-сайтам - применять специальные указатели. Приложение устанавливалось без разрешения пользователя и тайно следило за ним.

Как показано на рис. 9.4, в интернете существует множество приложений с указателями-котятами. В этом примере используется условное приложение для указателя, которое тайно обращается к российскому IP-адресу.

Запуск кода не обнаруживает признаков вредоносной функциональности:

```
deltaop@deltaleph-ubuntu:~$ ./kittens
Registering kitten cursor!
Done!  Enjoy the kitties!
deltaop@deltaleph-ubuntu:~$
```

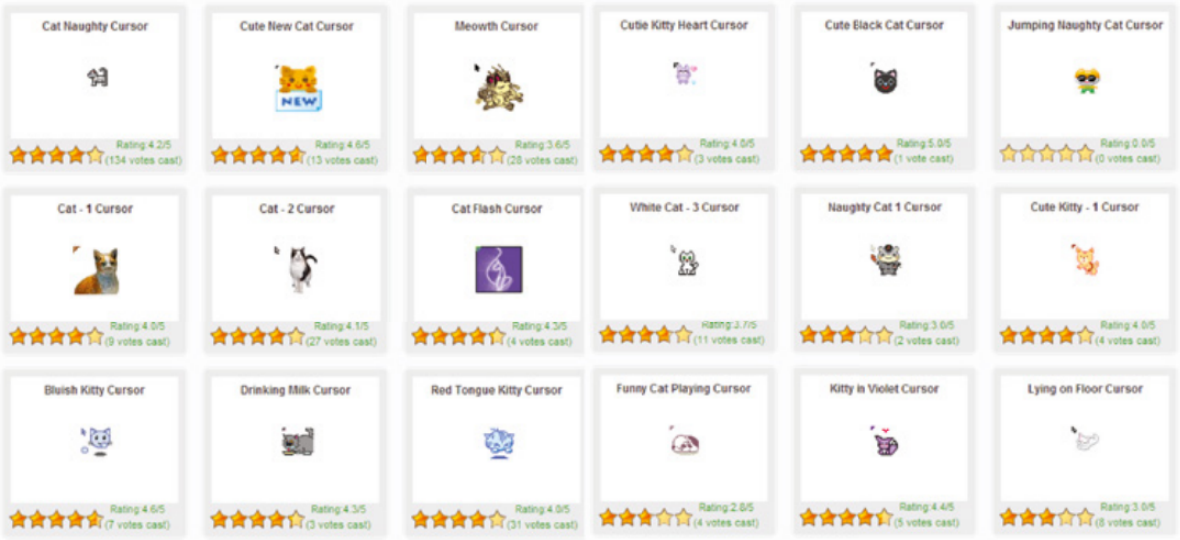


Рис. 9.4. Приложения с указателями-котятами

Однако анализ кода в `strace` рассказывает другую историю:

```
deltaop@deltaleph-ubuntu:~$ strace ./kittens
...
poll([{fd=3, events=POLLOUT}], 1, 0)    = 1 ([{fd=3, revents=POLLOUT}])
send(3, "!$\1\0\0\1\0\0\0\0\0\0\7kremlin\2ru\0\0\34\0\1",
      28, MSG_NOSIGNAL) = 28
poll([{fd=3, events=POLLIN}], 1, 5000)  = 1 ([{fd=3, revents=POLLIN}])
ioctl(3, FIONREAD, [28])                = 0
recvfrom(3, "!$\201\200\0\1\0\0\0\0\0\0\7kremlin\2ru\0\0\34\0\1", 1024,
         0, {sa_family=AF_INET, sin_port=htons(53),sin_addr=inet_
            addr("192.168.1.1")}, [16]) = 28
close(3)                                = 0
socket(PF_INET, SOCK_DGRAM|SOCK_NONBLOCK, IPPROTO_IP) = 3
connect(3, {sa_family=AF_INET, sin_port=htons(53),
       sin_addr=inet_addr("192.168.1.1")}, 16) = 0
...
```

Этот пример вывода `strace` показывает несколько событий. Чтобы сосредоточиться на интересующих событиях, используйте `grep`, который ограничивает результат строками, соответствующими искомой строке, в данном случае `connect`.

```
deltaop@deltaleph-ubuntu:~$ strace -f ./kittens 2>&1 | grep connect

connect(3, {sa_family=AF_FILE, path="/var/run/nscd/socket"},
       110) =  -1 ENOENT
connect(3, {sa_family=AF_FILE, path="/var/run/nscd/socket"},
       110) =  -1 ENOENT
connect(3, {sa_family=AF_INET, sin_port=htons(53),
       sin_addr=inet_addr("192.168.1.1")}, 16) = 0
connect(3, {sa_family=AF_INET, sin_port=htons(53),
       sin_addr=inet_addr("192.168.1.1")}, 16) = 0
connect(3, {sa_family=AF_INET, sin_port=htons(53),
       sin_addr=inet_addr("192.168.1.1")}, 16) = 0
connect(3, {sa_family=AF_INET, sin_port=htons(80),
       sin_addr=inet_addr("195.208.24.91")}, 16) = 0
write(2, "connected.\n", 11) = 11
```

Предыдущий результат отбирает события, содержащие слово `connect`. В него входит несколько интернет-соединений, в том числе с `195.208.24.91`. Это подозрительно, поскольку адрес является внешним, да и зачем указателю мыши вообще к нему обращаться?

strings

`strings` - утилита Linux для извлечения используемых приложением печатаемых строк. Она ищет последовательности печатаемых символов ASCII заданной (настраиваемой) минимальной длины и выводит все найденные.

`strings` может быть очень полезна при обратной разработке, поскольку даёт высокоуровневое представление о возможных действиях программы. Кроме того, далее вы увидите, как по найденным интересным строкам легко определить связанный с ними фрагмент кода. Например, строка `incorrect password` позволяет быстро отыскать код обработки пароля. Следующие строки дают ценные подсказки о приложении:

• `Enter password:`;
• `open_socket`;
• `YOUR FILES HAVE BEEN ENCRYPTED!`.

Синтаксис команды: `strings program`. Хотя обычно она используется без параметров, при обратной разработке иногда полезны следующие флаги:

• `-a`: показать все строки файла, а не только строки в загруженных разделах объектных файлов. Часто полезно для обфусцированных, вложенных или иных необычных двоичных файлов;
• `-n`: задать минимальное число последовательных печатаемых символов, при котором последовательность байтов считается строкой. Значение по умолчанию равно 4. Этот параметр часто полезен для увеличения или уменьшения числа найденных строк.

Dependency Walker

Обход зависимостей - метод быстрого изучения импорта и экспорта приложения. Dependency Walker является одним из инструментов такого рода. Ссылки приведены в разделе **Tools** репозитория.

Обход зависимостей даёт ценное высокоуровневое представление о действиях программы и часто служит полезным первым шагом при взломе. Большинство приложений не реализуют самостоятельно все свои функции: они используют функции операционной системы или внешних библиотек. Каждый выход приложения за пределы собственного кода отображается как импортированная функция. Кроме того, приложения часто предоставляют функции другим приложениям; всякая функция, «доступная для совместного использования», отображается как экспорт приложения.

Загрузка программы в Dependency Walker показывает используемые ею DLL и ожидаемые вызовы API. На рис. 9.5 видно, что программа создаст несколько ключей реестра.

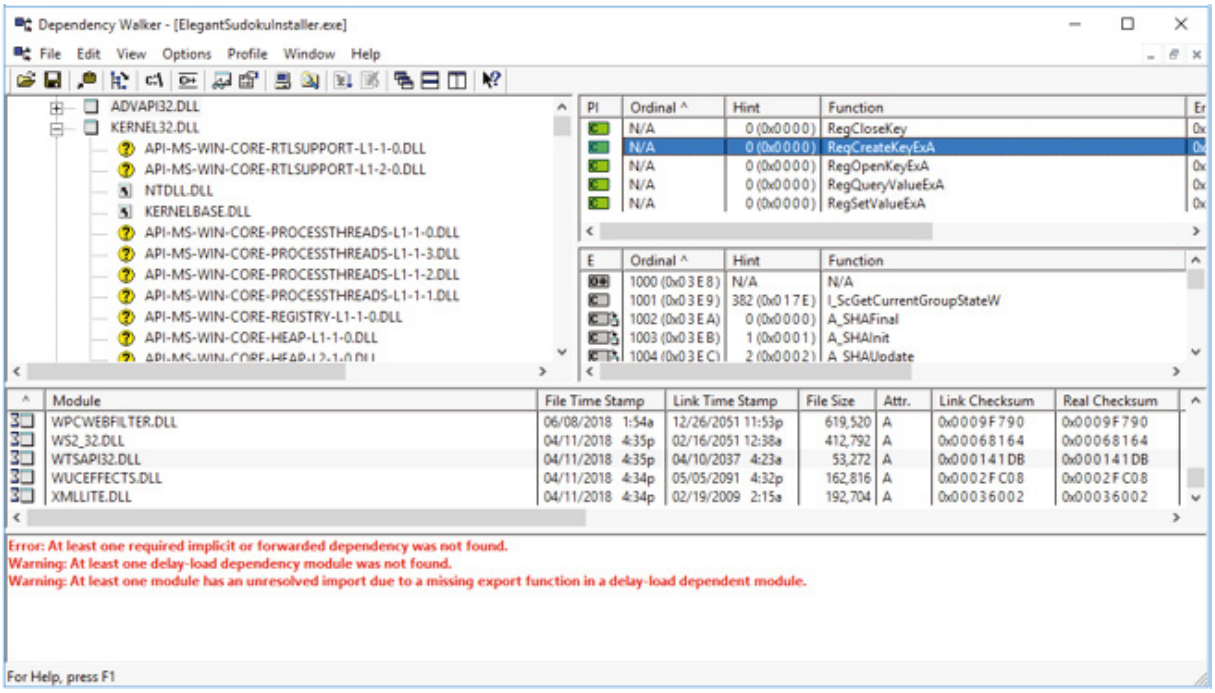


Рис. 9.5. Изучение изменений реестра в Dependency Walker

На снимке видны вызовы реестра `RegCloseKey`, `RegCreateKeyExA`, `RegOpenKeyExA`, `RegQueryValueExA` и `RegSetValueExA`.

Стратегия обратной разработки

Обратная разработка по-прежнему больше искусство, чем наука. Хотя существует множество прекрасных вспомогательных инструментов и методов, эффективная обратная разработка в конечном счёте в значительной мере зависит от интуиции и опыта.

Поэтому дать предписывающее решение невозможно. Однако существуют общие подходы и передовые практики, способные помочь.

Находите интересующие области

Приложения содержат огромные объёмы кода, большая часть которого не относится к задаче или не требует обратной разработки. Важный первый шаг при обратной разработке приложения - найти нужную область программы.

Далее вы узнаете множество интересных способов сузить область поиска, но уже сейчас на основании знаний о предыдущих инструментах доступны следующие:

• **Интересные строки:** найдите интересующие строки программы, например `Incorrect Key`, и места их использования, например вызовы `printf` с этими строками.
• **Пользовательский ввод:** найдите места получения пользовательского ввода, например `scanf`, диалоговые окна и так далее, а затем места его обработки.
• **Системный ввод:** найдите места чтения системного ввода, например файлов конфигурации и параметров реестра.
• **Код аутентификации:** если возможно, с помощью отладчика приостановите программу после ввода имени пользователя или ключа. Затем найдите в памяти введённые значения, установите на

эти ячейки аппаратные точки останова и снова запустите приложение, чтобы определить, где значения читаются или записываются.

Итеративно аннотируйте код

Даже после обнаружения интересующего кода понять его бывает трудно. Один из подходов к сложному коду - выполнить несколько проходов, при каждом добавляя информацию, например комментарии.

Для этого аннотируйте целевую область до тех пор, пока не поймёте её работу, следуя такому процессу:

• **Определите и отметьте локальные переменные:** по правилам соглашения о вызовах найдите локальные переменные, например `[ebp-4]` в `cdecl`. Сначала им можно присвоить неопределённые имена, например `local1`.
• **Определите и отметьте параметры функций:** по правилам соглашения о вызовах найдите параметры, например `[ebp+12]` в `cdecl`. Сначала им также можно присвоить неопределённые имена, например `arg1`.
• **Определите вызовы API, например** `atoi`**:** используйте знания о параметрах API для дальнейшего уточнения локальных переменных. Например, документация API указывает, что `atoi` получает строку для преобразования в целое число, поэтому параметр можно переименовать в `integer_string`.
• **Добавьте комментарии к сложным потокам управления:** например, «этот код раскладывает число на множители».
• **Уточняйте описания по наблюдаемым потокам данных:** например, `local1` может стать `loop_counter`, если используется как счётчик цикла `for`.

Важная часть эффективной обратной разработки - высокая скорость. Даже в небольших программах слишком много кода, чтобы анализировать всё.

Подавляющее большинство кода приложения не будет иметь отношения к вашей цели. Знание того, на чём следует сосредоточиться, часто менее важно, чем знание того, на чём сосредотачиваться не следует. Умение понимать, какие участки можно пропустить, приходит со временем.

Итоги

В этой главе представлены некоторые основные инструменты и методы, которые вы будете использовать как специалист по обратной разработке и взлому программ. Прежде чем двигаться дальше, уделите время практике и получите опыт работы с этими инструментами. Он окажется бесценным позднее, когда вы перейдёте к более сложному программному обеспечению и более продвинутым методам обратной разработки и взлома.

Глава 10. Взлом: инструменты и стратегии

Взлом - это искусство обратной разработки программного обеспечения с целью обойти защиту или иную нежелательную функциональность. В этой главе рассматриваются некоторые основные инструменты и стратегии взлома программ, включая применение генераторов ключей и исправлений для нейтрализации средств проверки ключей.

Средства проверки ключей

Один из наиболее распространённых способов лицензирования программного обеспечения основан на лицензионных ключах. В попытке противодействовать пиратству для завершения каждой установки программы требуется уникальный ключ. В многоуровневом программном обеспечении некоторые функции могут быть доступны всегда и бесплатно, тогда как остальные находятся за лицензионным барьером; возможен и вариант, когда без лицензионного ключа программа вообще не работает.

Лицензионные ключи - распространённое средство борьбы с пиратством, имеющее свои преимущества. Два наиболее существенных:

• лицензионные ключи легко создавать и проверять;
• отношение числа действительных ключей к недействительным настолько мало, что случайный подбор вряд ли даст действительный ключ, если ключ имеет разумную длину.

Однако, как и всякая защита, при плохой реализации они могут быть крайне уязвимы для взлома и не являются совершенно непогрешимыми. Достаточно знающий и мотивированный взломщик в конечном счёте сможет их нейтрализовать или обойти. Тем не менее это по-прежнему одна из более сильных форм защиты; просто ещё одно напоминание о том, что стопроцентно защищённого программного обеспечения не существует.

Раньше, когда автономные системы были распространены сильнее, проверка и подтверждение лицензии часто выполнялись полностью без сети, то есть вся логика проверки ключа находилась в системе. Теперь, при повсеместном подключении, часто встречаются проверки лицензионных ключей, сочетающие автономную и сетевую части и обращающиеся к серверу лицензий для дополнительного подтверждения. Существует несколько способов реализации проверок ключей с разной эффективностью.

Плохой способ

В прошлом очень популярные компьютерные игры StarCraft и Half-Life использовали в качестве лицензионного ключа контрольную сумму. Напомним, что контрольные суммы часто являются очень простыми математическими выражениями, вычисляемыми над двоичным блоком; некоторые всего лишь складывают все числа. В контрольной сумме этих игр 13-я цифра проверяла первые 12.

Это означало, что пользователь мог ввести любые первые 12 цифр, а затем вычислить 13-ю, получив действительную контрольную сумму. Этот изъян защиты породил печально известный ключ `1234-56789-1234`, действительный для этих игр и широко использовавшийся для их пиратского копирования.

Одна из главных проблем в этих случаях состояла в чрезмерной простоте алгоритма вычисления контрольной суммы.

```
x = 3;
for(int i = 0; i < 12; i++)
{
    x += (2 * x) ^ digit[i];
}
lastDigit = x % 10;
```

К взлому можно подойти двумя способами. Первый - выполнить алгоритм и вычислить действительное значение последней цифры, как показано выше.

Второй - атака полным перебором. Поскольку определить требовалось всего одну цифру, для последней цифры имеется только 10 вариантов `[0-9]`. Можно случайно выбрать набор из 12 цифр, а затем поочерёдно проверить 10 вариантов последней до успеха. Печально известный ключ `1234-56789-1234` прославился потому, что легко запоминался, однако любой из двух подходов - вычисление или полный перебор - позволял создать сколько угодно новых ключей.

Разумный способ

Атака полным перебором на лицензионный ключ гарантированно сработает... когда-нибудь. Самое большее, что способен сделать лицензионный ключ, - отнять у взломщика столько времени, чтобы проведение атаки полным перебором стало неосуществимым или невозможным.

Как же защищаться от полного перебора? Распространённый в других областях вариант - криптографический хеш. Например, лицензионный ключ можно реализовать одним из следующих способов:

• **Имя пользователя:** `SHA(username)`;
• **Случайное значение:** `WXYZ-SHA(WXYZ)`.

Хеш-функция значительно усложняет атаку полным перебором. Однако, взглянув на код, взломщик без труда определит работу алгоритма. Если вы злоумышленник, то используете изученные навыки обратной разработки, чтобы найти и раскрыть алгоритм; если защитник - это важнейший фрагмент кода, который необходимо защищать.

Вместо стандартного хеша можно использовать собственный сложный. Обычно в безопасности это ужасная идея, но для данного применения такой выбор не является неслыханным. Цель состоит не в абсолютной защите, а лишь в замедлении обратной разработки. У всех специалистов по безопасности, которых передёрнуло от предложения создать собственный хеш, отметим оговорку: вы должны быть способны сделать его достаточно хорошим. Защитнику следует помнить, что существует множество инструментов для распространённых методов хеширования, поэтому злоумышленник прежде всего попытается раскрыть ключ именно ими.

Кроме того, находите способы добавить уникальную сложность, чтобы ключ можно было применять только в определённых условиях и нельзя было широко распространять. Такие схемы, как объединение в хешируемом значении имени и версии продукта с именем компьютера, добавляют солидный уровень сложности. Тогда взломанный действительный ключ одной установки не открывает другие выпуски.

Лучший способ

Хеши лучше, и при правильной реализации они могут быть вполне приемлемы. Но существуют ещё более хорошие варианты. Прекрасный пример - подход Microsoft к созданию лицензионных ключей для своего программного обеспечения.

Вместо алгоритмов хеширования Windows использует криптографию с открытым ключом. С её помощью цифровую подпись можно создать закрытым ключом и проверить открытым. Это означает, что приложение способно проверить лицензионный ключ с цифровой подписью, не раскрывая секретных ключей.

При создании лицензионных ключей Windows использует множество сведений о программном обеспечении, включая, помимо прочего:

• разрядность (32, 64);
• тип (домашняя, профессиональная, корпоративная версия);
• идентификатор продукта;
• особенности оборудования.

Включение всех этих сведений помогает привязать ключ продукта к конкретной установке программного обеспечения. Если вас интересуют подробности протокола, в интернете имеется множество материалов, разбирающих создание ключей Microsoft.

Ключи с цифровой подписью

Цифровые подписи лицензионных ключей, подобные применяемым Windows, значительно усложняют создание поддельных действительных ключей. Действительную подпись необходимо создавать закрытым ключом, но проверять её можно несекретным открытым ключом.

Цифровые подписи предотвращают прямолинейное создание лицензионных ключей и оставляют злоумышленнику два варианта. Первый - опубликовать законный ключ, который можно отследить до конкретного пользователя. Другой - изменить программу, удалив код проверки ключа, что увеличивает время и сложность пиратского копирования программы.

Наилучший способ

До сих пор примеры были сосредоточены преимущественно на автономной проверке лицензионных ключей, когда весь код проверки и разблокировки программы находится в системе. Однако в условиях нынешней повсеместной подключённости усилить защиту можно добавлением сетевой части.

Это может принимать разные формы, но один из современных вариантов состоит в том, что каждый экземпляр программного обеспечения распространяется с лицензионным ключом в виде большого случайного числа. При установке и регистрации продукта это значение отправляется серверу лицензий, который проверяет его действительность и отсутствие предыдущего использования. При нынешнем цифровом распространении программ полученный ключ не является действительным даже до покупки: если бы вы угадали его за 10 минут до покупки программы, он бы не сработал.

Можно применять и гибридные подходы, при которых значительная часть алгоритма проверки посредством хеширования или криптографии с открытым ключом находится в системе, но также имеется этап обращения к серверу лицензий, проверяющему прежнее использование или отзыв ключа.

Другие предложения

Представленные методы соответствуют распространённым отраслевым практикам и наиболее часто применяемым способам. Но в безопасности нет универсального решения, и некоторые из следующих методов могут встретиться при взломе либо оказаться полезными для защиты при уникальных ограничениях.

Предпочитайте автономную активацию

Хотя добавление сетевых серверов ключей выглядит мощным средством защиты и действительно им является, следует признать, что этот метод создаёт огромные трудности управления и инфраструктуры. Управление серверами ключей - задача не из лёгких, а сами они становятся маяком для кибератак. Поэтому многие компании всё ещё не способны или не готовы брать на себя такой уровень хаоса и предпочитают усиленную автономную проверку. Поддержка автономной проверки ключей устраняет сложности управления сервером и учитывает пользователей без доступа к интернету.

Выполняйте частичную проверку ключа

В автономном режиме отсутствует механизм отзыва, позволяющий сделать некоторые ключи недействительными. Чтобы единственный опубликованный ключ не работал во всех будущих версиях программы, проверяйте только часть ключа. Простой пример - проверять лишь первый символ каждой группы лицензионного ключа, например `X`, `9`, `B` и `B` в `X4Z-951-B41-BR0`.

Если кто-либо выпустит генератор ключей для вашего приложения, выпустите новую версию, проверяющую другую часть ключа. Например, перейдите к проверке второго символа каждой группы (`4`, `5`, `4` и `R`). Это ограничит возможный ущерб от одного генератора ключей.

Кодируйте в ключе полезные данные

Кодирование полезных данных в ключе помогает ограничить область его применения. Например, ключ может задавать максимальную версию приложения, к которой применим, тем самым ограничивая последствия компрометации ключа.

Генераторы ключей

Если программное обеспечение использует ключ для активации, взломщики захотят создать для него генератор ключей. Это справедливо независимо от выбранного типа активации. Затем генераторы распространяются, чтобы пользователи могли создавать «бесплатные» ключи к программе.

Позднее вы увидите, как исправить программу, просто удалив проверку ключа, но пока сосредоточимся на создании генератора и предположим, что обойти проверку напрямую нельзя. Генераторы ключей обычно требуют более глубокого анализа программы и лучшего понимания алгоритма ключа.

Зачем создавать генераторы ключей?

Если создавать генераторы ключей сложнее, зачем это делать? Есть несколько причин.

В программном обеспечении могут присутствовать различные защитные меры, из-за которых исправление становится более трудным путём, например:

• защита от модификации;
• динамические проверки;
• защита от отладки;
• программные охранные механизмы.

Кроме того, исправление может потребовать распространения изменённой копии целевой программы, на которую может быть нанесена водяная метка. Водяная маркировка - метод отслеживания программы до её первоначального покупателя. Такие метки позволяют связать взломанную копию со взломщиком, чего он, очевидно, не желает.

Программа может выполнять сетевые проверки, выявляющие исправленные или изменённые версии. Возможен и вариант, когда программа расшифровывает себя на основании введённого ключа (распаковка, рассматриваемая в главе 13 «Передовые методы защиты»), а полное удаление проверки не позволит ей расшифроваться.

Генераторы ключей также более устойчивы к будущим версиям, чем исправления. Разработчику приложения нелегко отозвать действительные ключи.

Наконец, взломщики могут выбирать генераторы именно потому, что они сложнее. В некоторых случаях исправить программу просто, тогда как успешный генератор представляет собой задачу, приносящую определённый престиж.

Философия создания ключей

При взломе средств проверки полезно представлять проверку в виде `f(u) == g(k)`, где:

• `u` - имя пользователя, введённое пользователем;
• `f` - функция преобразования имени пользователя;
• `k` - ключ, введённый пользователем;
• `g` - функция преобразования ключа.

В этой модели проверяется равенство `f(u) == g(k)`. Если говорить не математическим языком, имя пользователя некоторым образом преобразуется или изменяется, а затем результат сравнивается с некоторым преобразованием ключа. В этом и последующих примерах входом служит имя пользователя, но им может быть любое сочетание данных: номер версии, имя компьютера и так далее. Суть в том, что некоторые данные поступают в преобразование, дающее результат. Этот результат сравнивается с введённым ключом, который тоже прошёл какое-либо преобразование. Преобразование может отсутствовать, и тогда результатом является сам ключ, либо может включать дополнительное хеширование или изменение. С учётом этой модели возможны несколько вариантов проверок.

Возвращаясь к начальному примеру StarCraft/Half-Life, `u` фактически представляет первые 12 цифр ключа, а `k` - последнюю. Здесь имя пользователя не вводится: одна часть ключа проверяет другую.

Другой вариант - `u`, а значит и `f(u)`, является константой, то есть используются жёстко заданные ключи. Здесь имя пользователя также не вводится: ключ преобразуется и сравнивается с фиксированным значением. Например: «сумма всех цифр ключа равна 1337».

Взлом различных типов проверок ключей

Рассматривая средства проверки через формулу `f(u) == g(k)`, можно начать строить методы взлома различных вариантов.

Проверка ключа типа I: преобразуется только имя пользователя

В этом случае имя пользователя преобразуется некоторой функцией, после чего сравнивается с введённым ключом. Можно считать, что `g()` не изменяет ключ. Это упрощает проверку до `f(u) == k`. Программа преобразует имя пользователя и проверяет совпадение преобразованного значения с введённым пользователем ключом.

Для взлома такого типа найдите и извлеките функцию преобразования `f` в приложение-генератор ключей. Например, перемножьте порядковые номера символов имени пользователя и сопоставьте результат ключу. Генератор запросит желаемое имя пользователя, вычислит `f(u)` и выведет действительный ключ.

Проверка ключа типа II: преобразуются оба значения

В типе II имя пользователя по-прежнему преобразуется, но дополнительно `g` изменяет ключ. В строгой математической формулировке у `g` имеется обратная функция: существует `g-1`, и `g-1(g(k)) == k`). Проще говоря, `g` выполняет изменение, а всякое изменение можно обратить, то есть выполнить строго противоположное действие.

В такой схеме программа преобразует имя пользователя и введённый ключ, после чего проверяет совпадение результатов. Однако функцию `g` можно обратить, то есть «развернуть» или «отменить».

> **Примечание переводчика.** Лишняя закрывающая скобка в формуле `g-1(g(k)) == k)` присутствует в исходнике.

Чтобы взломать такую проверку, выполните обратную разработку `g` и получите `g-1`.. Часто для этого достаточно «отменить» каждое преобразование `g` в обратном порядке. Затем создайте ключ как `g-1(f(u))`.

> **Примечание переводчика.** Две точки после `g-1` в предыдущем предложении сохранены из исходника.

Например, предположим, что `g(k) = k * 2 + 1000`. Тогда `g-1(h) = (h - 1000) / 2`.

В таком случае генератор запросит желаемое имя пользователя, как в типе I, и вычислит `f(u)`, но теперь результат является изменённым ключом, поэтому его нужно развернуть с помощью `g-1(h)`. Конечный результат и будет действительным ключом.

Проверка ключа типа III: допускается полный перебор

В типе III коллизию с `f(u)` можно найти полным перебором значений `g(k)`. Такой подход жизнеспособен, если пространство ключей очень мало или доступна большая вычислительная мощность.

В этой схеме программа преобразует имя пользователя и введённый ключ, после чего проверяет совпадение результатов, как в типе II. Но решение равенства `f(u) == g(k)` ищется многократной проверкой случайных или псевдослучайных значений `k`.

Чтобы взломать этот тип, определите формат `k`. Затем извлеките `g` в автономный генератор ключей. Наконец, создавайте случайные значения `k`, пока не найдётся решение `f(u) == g(k)`.

Например, рассмотрим случай `g(k) = CRC32(k)`. Если для изменения ключа применяется столь малый алгоритм, как CRC32, полный перебор становится вполне простым на обычном компьютере. Поскольку диапазон возможных значений CRC32 настолько мал, выполнить полный перебор возможно.

Защита от генераторов ключей

Проверки могут сочетать эти типы. Например, преобразование ключа `g` может одновременно допускать полный перебор и быть обратимым.

Обычно проверка ключа должна относиться к одной из этих категорий. Иначе изначально не существовало бы способа создавать ключи.

Проверка типа I самая слабая. Взломщику достаточно извлечь алгоритм из средства проверки, даже не выполняя обратную разработку самого алгоритма.

Проверка типа III лучше. Она требует от злоумышленника извлечь оба алгоритма и найти способ полного перебора преобразования ключа, который не всегда очевиден.

Проверка типа II, вероятно, лучшая, но её труднее всего хорошо спроектировать. Для взлома злоумышленнику необходимо вывести обратную функцию преобразования ключа. Это может потребовать глубокого анализа алгоритма и замедлить атаку.

Как всегда, универсального решения нет. В конечном счёте любую проверку ключа можно взломать, и всё, что способен сделать защитник, - замедлить злоумышленника.

Лабораторная работа: вводный генератор ключей

Эта лабораторная работа даёт опыт создания генератора ключей для простой программы.

Лабораторные работы и все связанные инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой работы найдите **Introductory Keygen** и следуйте предоставленным инструкциям.

Навыки

В лабораторной работе практикуется применение `objdump` и утилиты `strings` для создания генератора ключей. Проверяются, в частности, следующие навыки:

- начальная разведка;
- обратная разработка x86;
- создание ключей.

Основные выводы

Помимо изменения программы, нередко её можно взломать, лишь наблюдая за её работой. Правильный подход часто определяется ограничениями программы, и выбор подходящего варианта является важным навыком.

Procmon

В обратной разработке требуется узнать как можно больше о работе программы. Прежде чем переходить к сверхсложной отладке, начните с простого наблюдения за поведением программы.

Procmon - инструмент из набора Sysinternals, доступного по адресу http://technet.microsoft.com/en-us/sysinternals/bb842062. Этот репозиторий содержит около 60 утилит Windows, созданных и бесплатно распространяемых Microsoft. Обратите внимание: эти инструменты работают только в ОС Windows.

Пример: Notepad.exe

Рассмотрим, что делает `notepad.exe`, когда вы создаёте новый файл, изменяете шрифт, а затем сохраняете содержимое. Для этого выполните следующие действия:

1. Откройте `Procmon.exe`.
2. Запустите Notepad.
3. Введите какой-либо текст в документ Notepad.
4. Откройте меню **Format**, а затем выберите пункт **Font**.
5. В окне **Font** измените шрифт на Webdings.
6. В окне **Font** установите размер 20.
7. Нажмите кнопку **OK**.
8. Сохраните документ Notepad под именем `Example1.txt`.
9. Закройте Notepad.

Остановите сбор данных Process Monitor, нажав кнопку **Capture**, как показано на рис. 10.1. Теперь на значке должен появиться крест поверх увеличительного стекла. К этому моменту Process Monitor записал все события файлов, реестра и процессов/потоков.

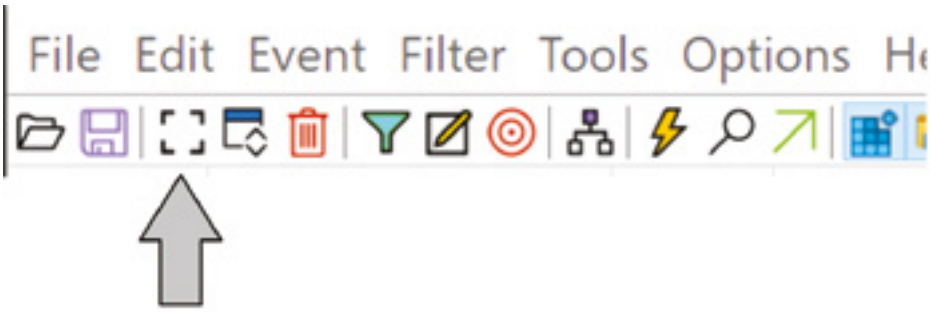


Рис. 10.1. Остановка Process Monitor

Process Monitor записывает тысячи событий в секунду, из-за чего возникает слишком много записей для ручного просмотра. Результаты необходимо отфильтровать до интересующих событий. Для этого откройте меню **Filter**, нажав значок воронки, как показано на рис. 10.2.



Рис. 10.2. Фильтрация событий в Procmon

Чтобы видеть только события, относящиеся к процессу `Notepad.exe`, задайте фильтр, согласно которому **Process Name** равен `Notepad.exe`, как показано на рис. 10.3. Для этого выполните следующие действия:

1. Выберите **Process Name** в списке **Column**.
2. Выберите **is** в списке **Relation**.
3. Введите `Notepad.exe` в текстовом поле **Value**.
4. Выберите **Include** в списке **Action**.
5. Нажмите кнопку **Add**.
6. Нажмите **Apply**, затем **OK**.

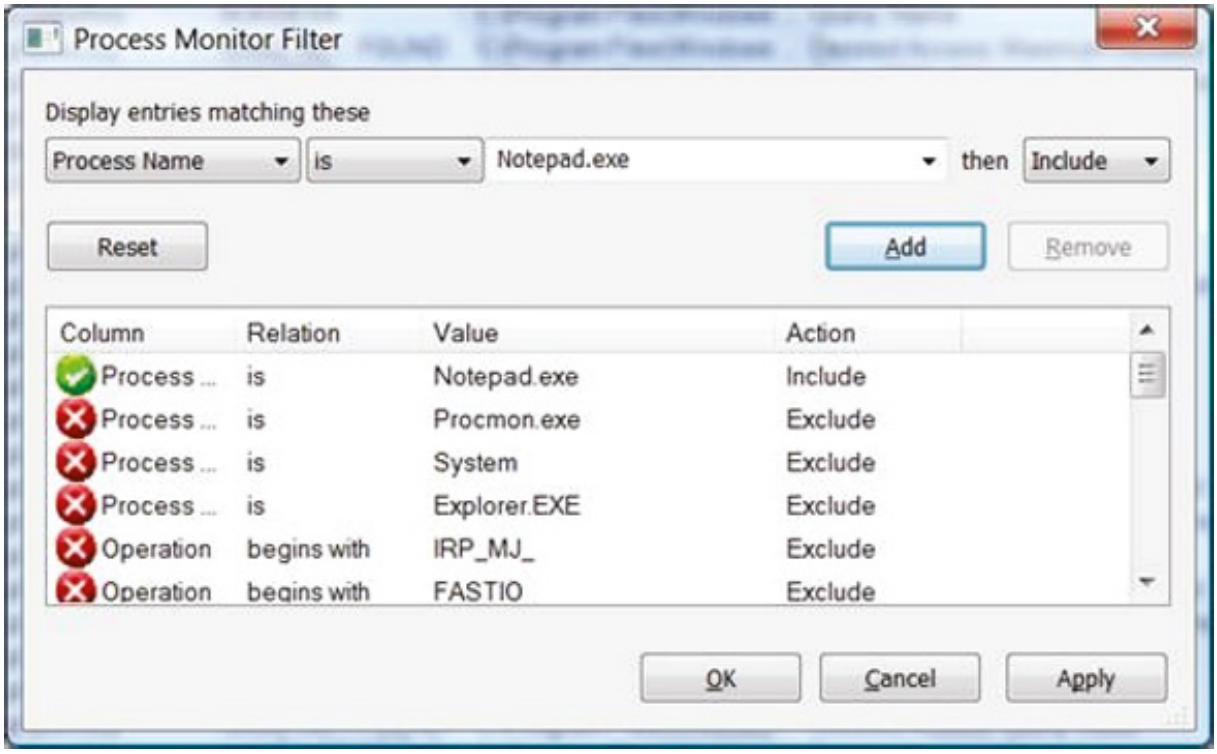


Рис. 10.3. Задание фильтра в Procmon

Фильтрация по имени процесса резко уменьшает число событий. Однако этого всё ещё недостаточно.

Для поиска интересующих событий необходимо задать дополнительные фильтры. В Procmon имеется несколько категорий событий, по которым можно фильтровать:

- реестр;
- файлы;
- сеть;
- потоки процессов.

Для начала сосредоточьтесь на значениях реестра, изменяемых Notepad. В Process Monitor для этого имеется удобная кнопка, показанная на рис. 10.4.

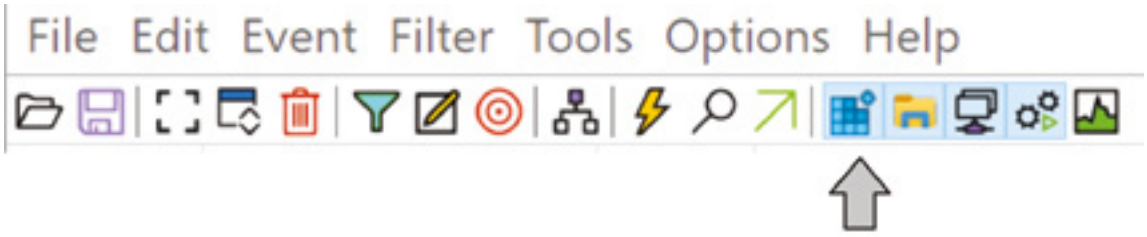


Рис. 10.4. Фильтрация событий реестра в Procmon

Если Notepad сохранил значения в реестре, он создаст запись события с типом **Operation** `RegSetValue`. Щёлкнув записи журнала Procmon правой кнопкой мыши, можно включать или исключать определённые типы событий, как показано на рис. 10.5. Это позволяет дополнительно уточнять результаты и сосредотачиваться на интересующих событиях.

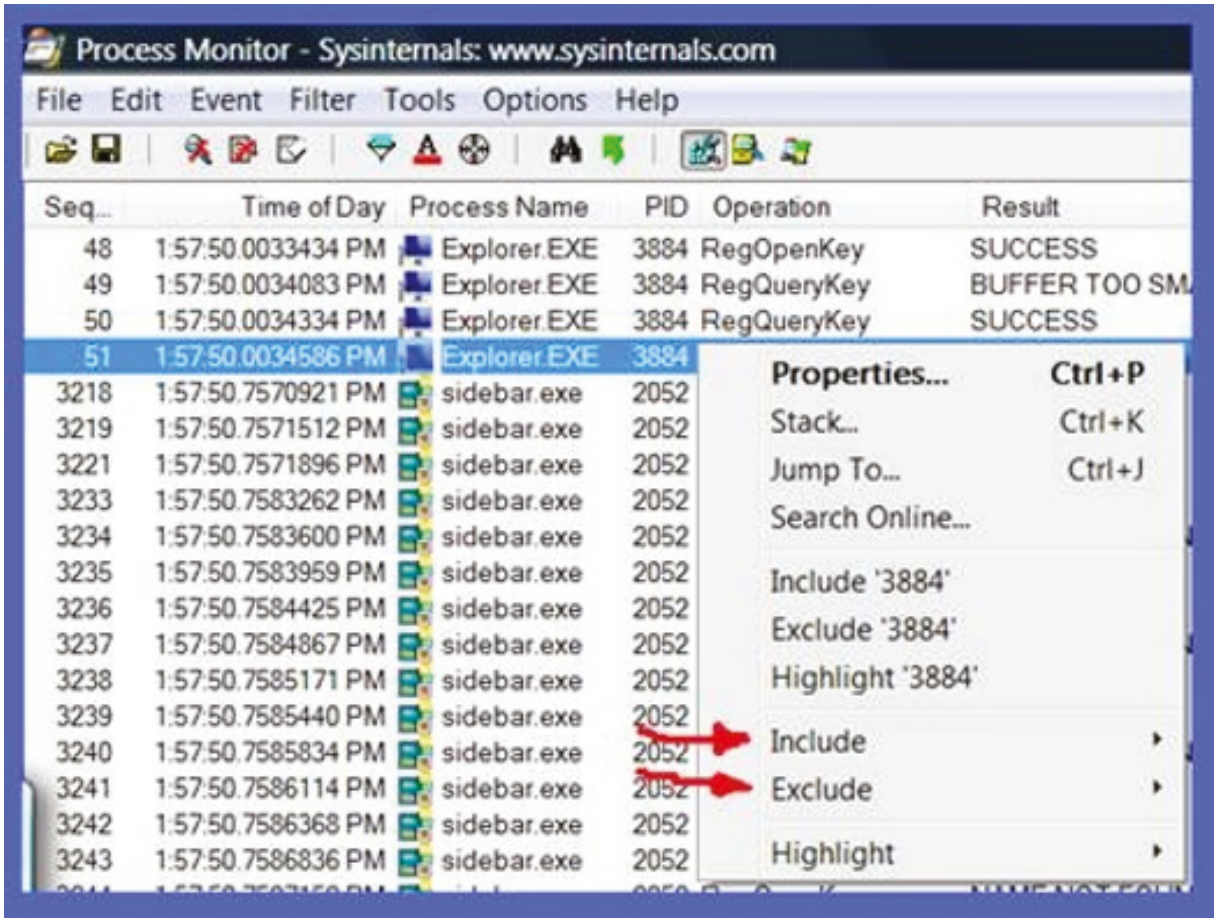


Рис. 10.5. Включение и исключение категорий событий в Procmon

На рис. 10.6 показана запись Procmon, которая, по-видимому, связана с изменением шрифта Notepad. Чтобы увидеть дополнительные сведения, щёлкните запись правой кнопкой и выберите **Properties**.

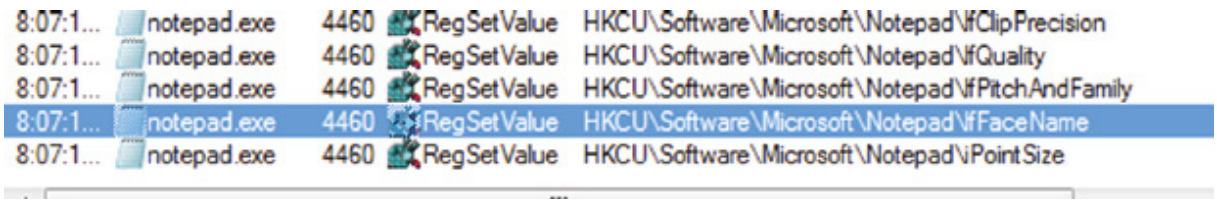


Рис. 10.6. Событие реестра при изменении шрифта Notepad

На рис. 10.7 показаны свойства события. В поле **Data** виден текст `Webdings`, означающий, что событие было вызвано сменой шрифта Notepad на Webdings.

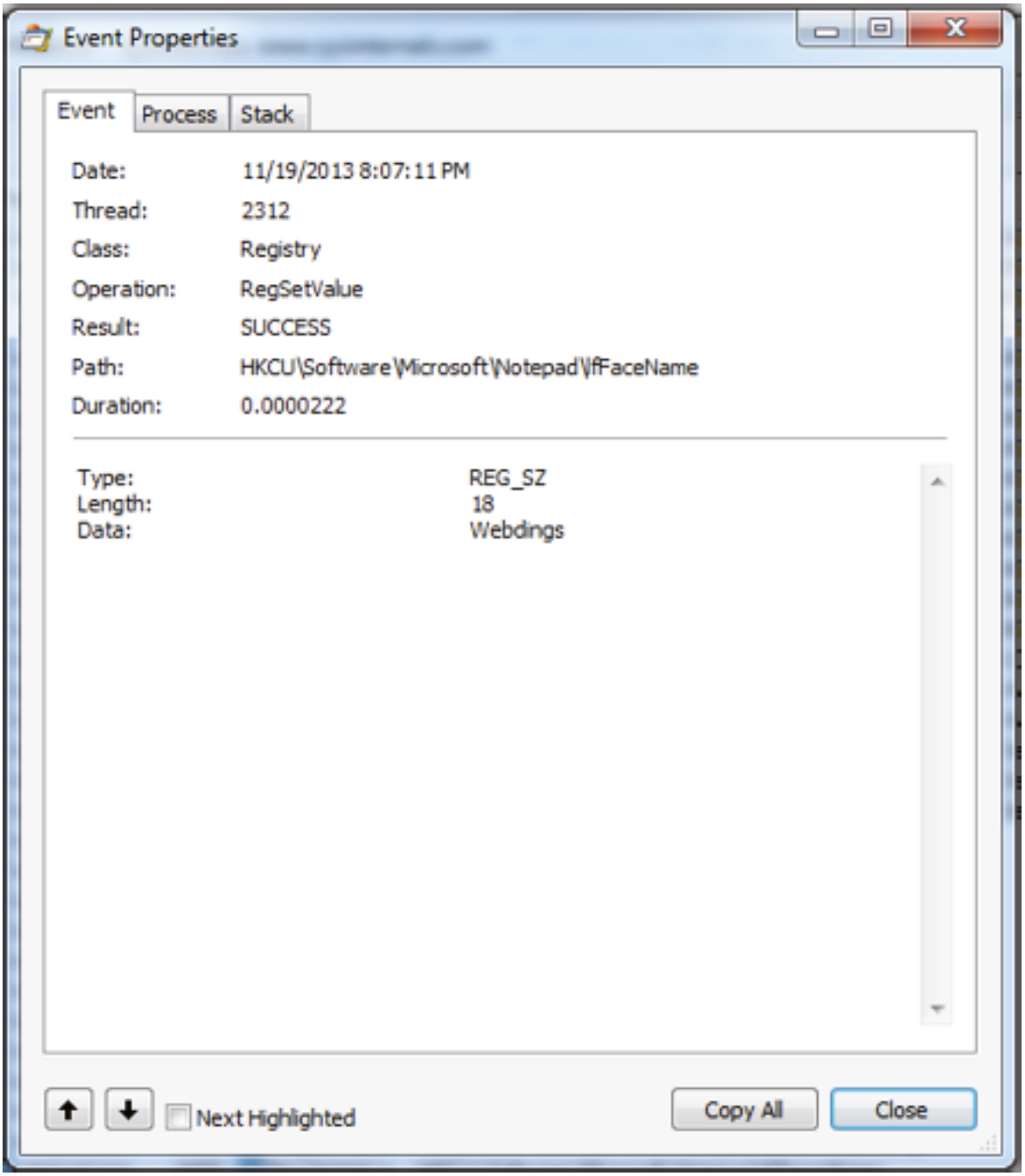


Рис. 10.7. Свойства события в Procmon

В окне указаны, в частности, операция `RegSetValue`, результат `SUCCESS`, путь `HKCU\Software\Microsoft\Notepad\lfFaceName`, тип `REG_SZ` и данные `Webdings`.

Как Procmon помогает обратной разработке и взлому

Procmon позволил увидеть изменения реестра, внесённые Notepad. Однако его возможности этим не ограничиваются. Дальнейшее изучение инструмента открывает множество полезных сведений.

Стеки вызовов

В окне **Properties** события имеется несколько вкладок. Переход на вкладку **Stack** показывает последовательность вызовов, приведших к этой точке, как на рис. 10.8.

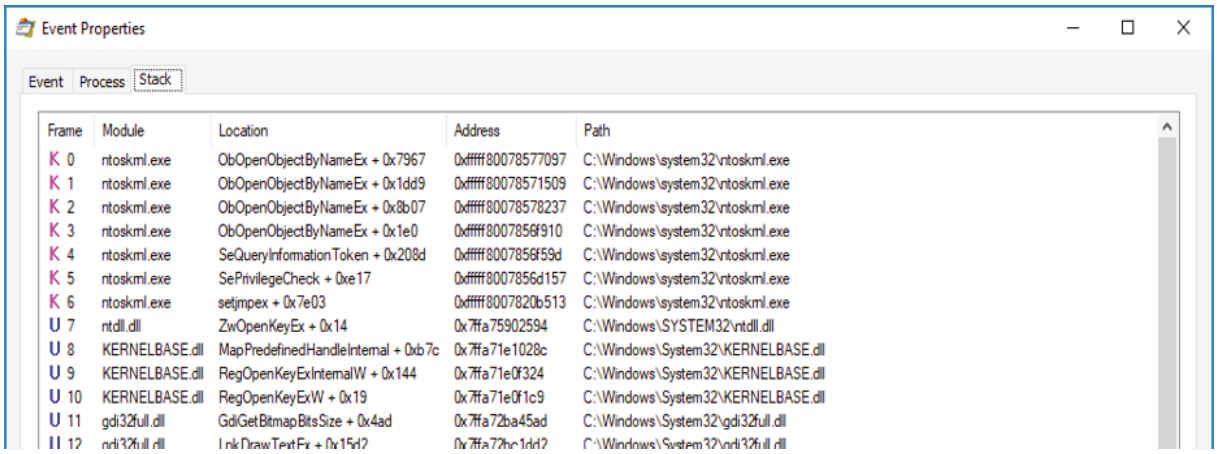


Рис. 10.8. Представление стека в окне Properties Procmon

Ниже в трассировке стека можно увидеть точку, в которой программа покинула `notepad.exe`, как показано на рис. 10.9. Такая точка перехода из приложения в библиотеки может быть хорошим местом для начала обратной разработки.

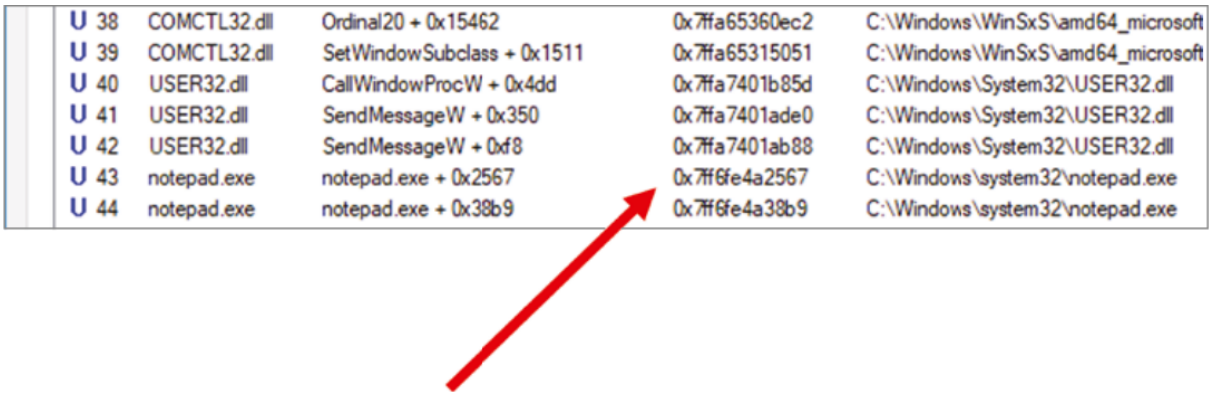


Рис. 10.9. Трассировка стека notepad.exe

На границе видны кадры `notepad.exe`, за которыми следуют вызовы `USER32.dll`, включая `SendMessageW` и `CallWindowProcW`.

Файловые операции

Procmon также записывает события файловых операций, например открытия, закрытия и изменения файлов. Пример показан на рис. 10.10.

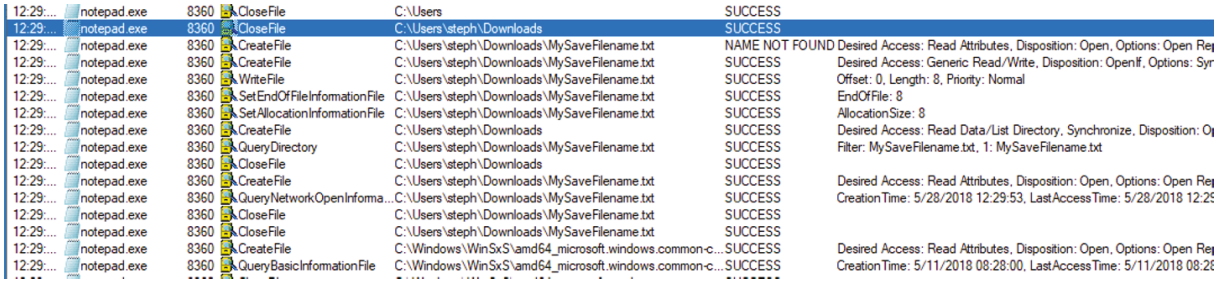


Рис. 10.10. Файловые операции в Procmon

Такие файловые события могут дать полезные сведения для обратной разработки. Например, они помогают находить и анализировать файлы конфигурации, функции экспорта и проприетарные форматы файлов.

Запросы реестра

Пример с `Notepad.exe` показал, как найти операцию реестра при изменении шрифта в Notepad. Однако это не единственный способ применения запросов реестра.

Например, на рис. 10.11 видно, что Notepad искал два ключа со словом `Security`, но не нашёл их. Можно добавить эти ключи в реестр и записать в них собственные значения, изменив работу Notepad.



Рис. 10.11. Запросы реестра Security в Procmon

Обе операции `RegQueryValue` завершились результатом `NAME NOT FOUND`.

Resource Hacker

Resource Hacker, также известный как ResHacker или ResHack, - бесплатная утилита извлечения или компилятор ресурсов для Windows. С помощью Resource Hacker можно добавлять, изменять или заменять большинство ресурсов внутри двоичных файлов Windows, включая строки, изображения, диалоговые окна, меню, а также ресурсы VersionInfo и Manifest. Ссылки на инструменты приведены в разделе **Tools** репозитория X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES.

Resource Hacker может быть полезен для изучения структуры двоичного файла до начала взлома. Он позволяет находить и понимать структуру назойливых окон, экранов ввода ключа, меню справки и других элементов.

Resource Hacker также позволяет добавлять программе функциональность до или после взлома. Например, в существующее приложение можно добавить новые значки, меню и темы оформления.

Для начала откройте файл `.exe` в ResHack и изучите его строки, изображения, диалоговые окна, меню и так далее, как показано на рис. 10.12. Затем выберите элемент в ResHack слева, чтобы справа увидеть, как он выглядит в приложении.

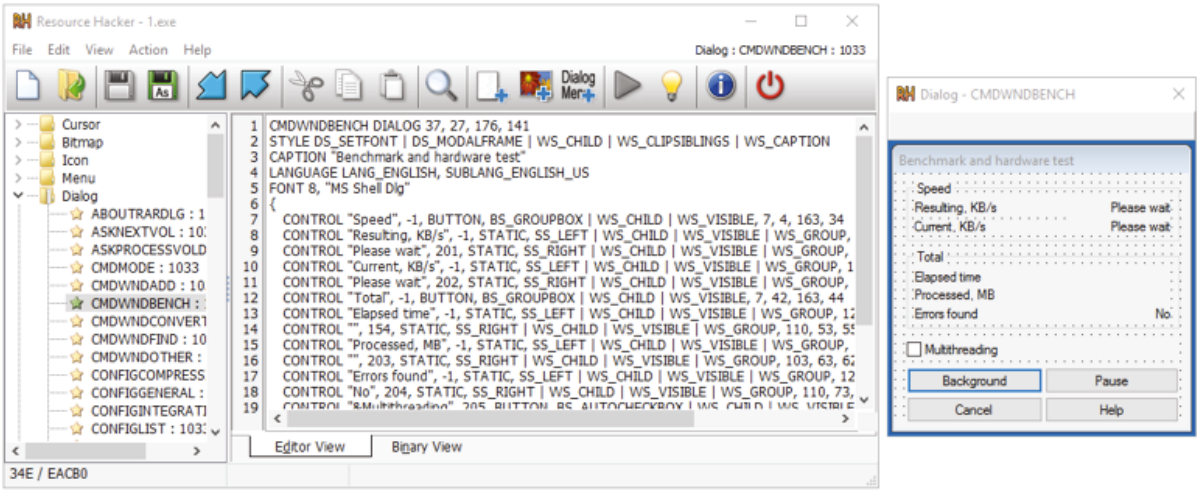


Рис. 10.12. Пример приложения в Resource Hacker

Пример

Предположим, в программе появилось окно, показанное на рис. 10.13. Как взломщику вам требуется понять, как программа использует это окно.

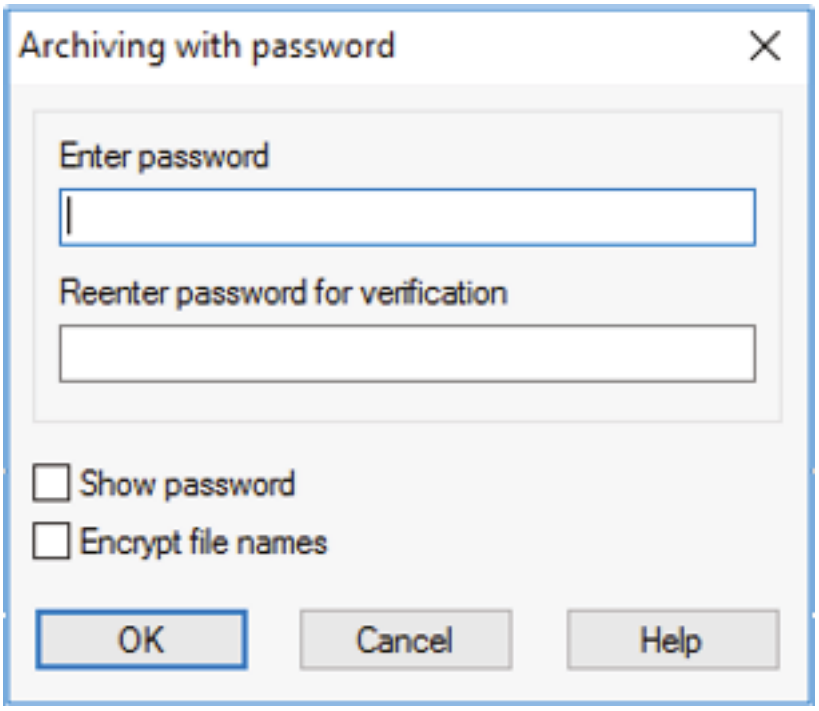


Рис. 10.13. Окно пароля

В окне присутствуют поля «Введите пароль» и «Повторно введите пароль для проверки», флажки «Показать пароль» и «Шифровать имена файлов», а также кнопки **OK**, **Cancel** и **Help**.

Чтобы выяснить назначение окна, откройте программу в ResHack. Затем нажмите `Ctrl+F` и найдите одну из строк диалогового окна, как показано на рис. 10.14.

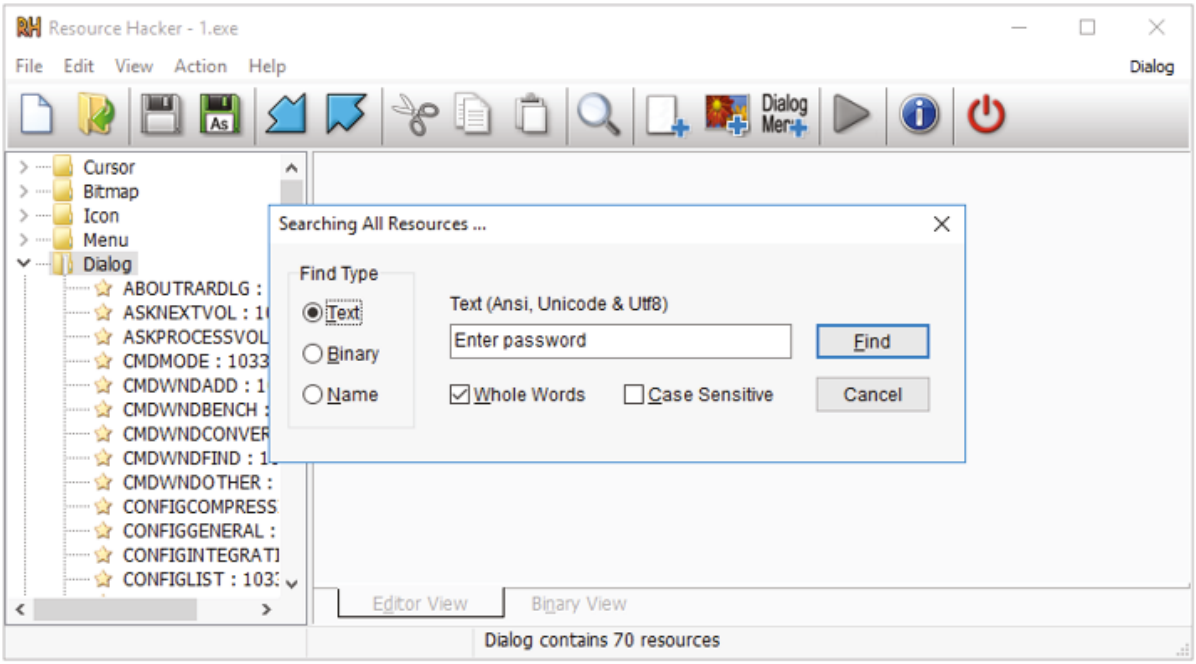


Рис. 10.14. Поиск строки в Resource Hacker

Resource Hacker определяет это окно как диалог `GETPASSWORD2`, как показано на рис. 10.15. Эти сведения помогают направить процесс обратной разработки программы.

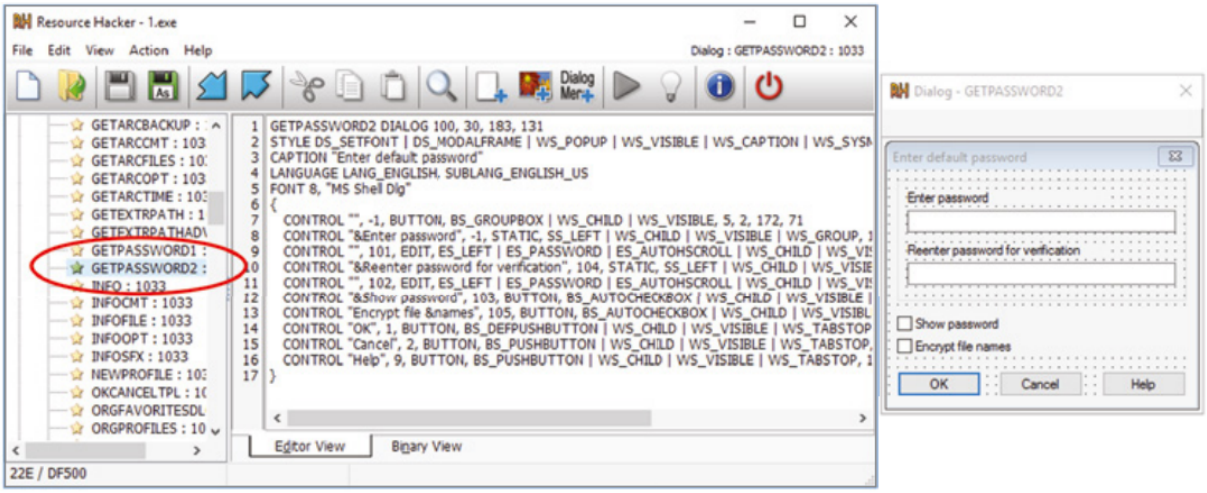


Рис. 10.15. Определение диалогового окна в Resource Hacker

Мини-лабораторная работа: Windows Calculator

Чтобы попробовать Resource Hacker на практике, измените оформление Microsoft Calculator. Как показано на рис. 10.16, окно программы озаглавлено **Calculator**. Попробуйте заменить это значение другим.

Для начала откройте исполняемый файл `calc.exe` в Resource Hacker. Затем найдите слово `Calculator`, как показано на рис. 10.17.

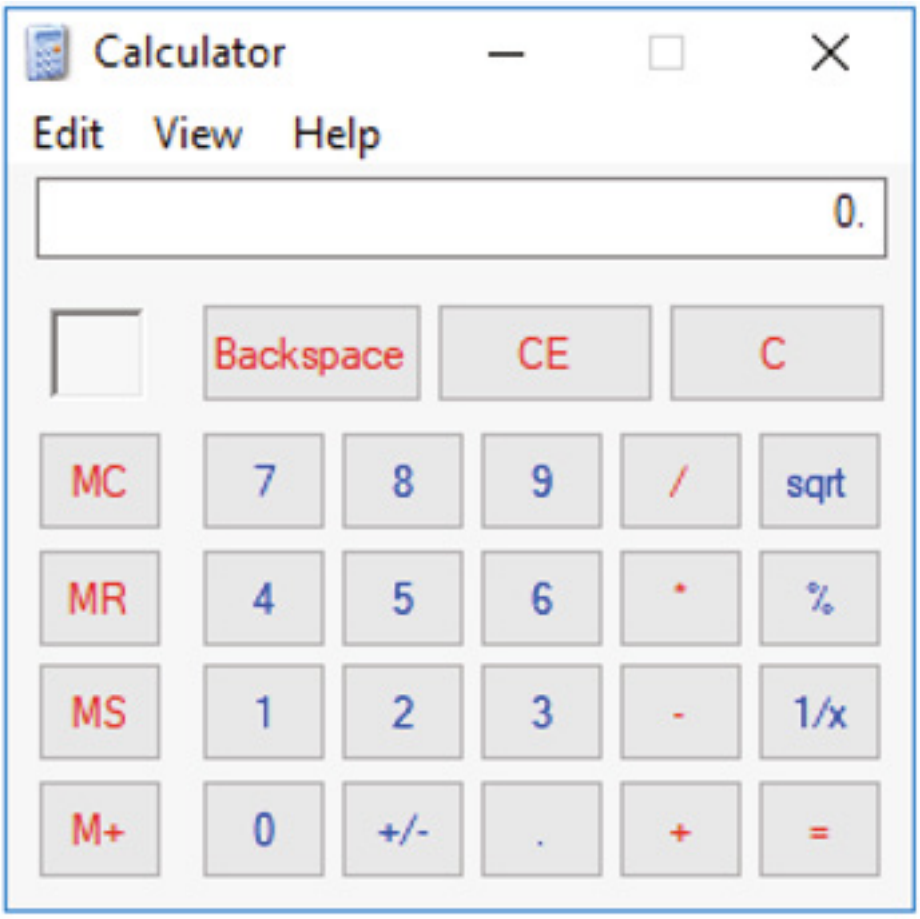


Рис. 10.16. Microsoft Calculator

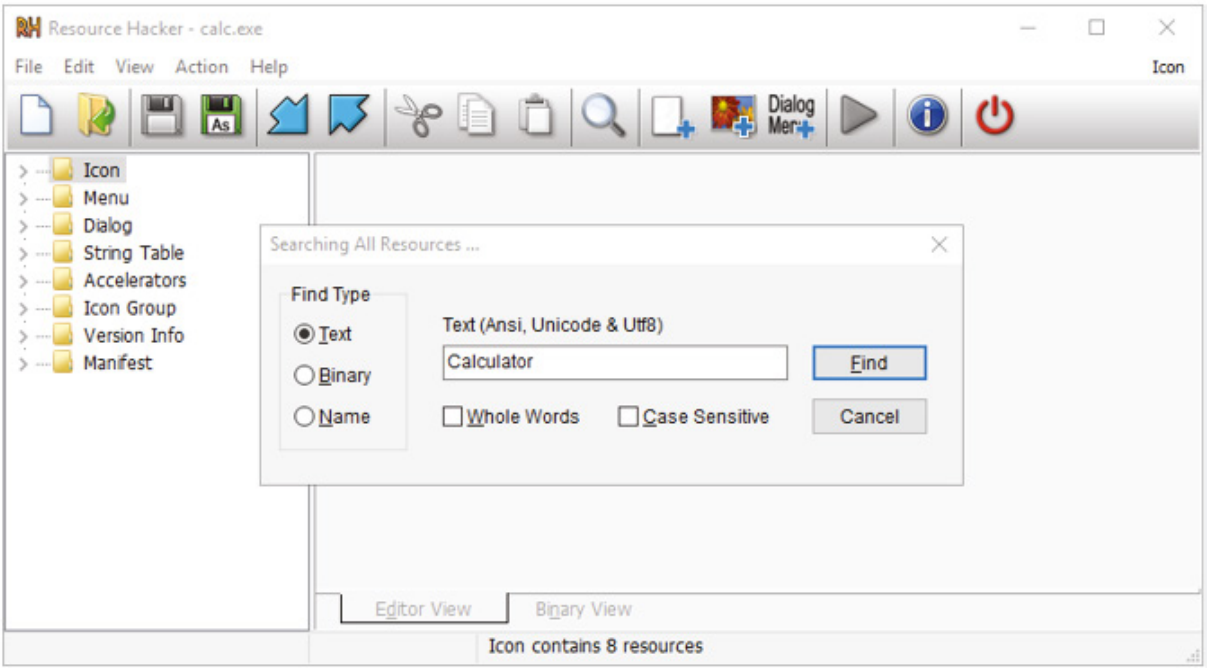


Рис. 10.17. Поиск Calculator в ResHack

Главное окно Calculator может оказаться не первым результатом. Продолжайте поиск, пока не найдёте код, определяющий диалоговое окно Calculator, как показано на рис. 10.18.

На рис. 10.18 строка `CAPTION` определяет заголовок окна приложения. Измените эту строку, чтобы оформить приложение под собственной маркой.

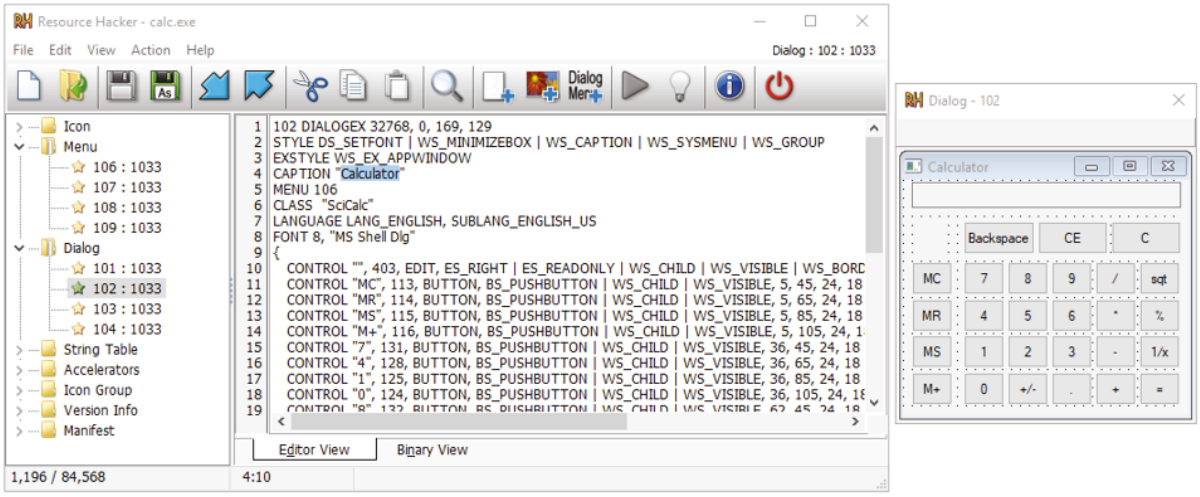


Рис. 10.18. Окно Calculator в Resource Hacker

После изменения `CAPTION` нажмите кнопку с зелёной стрелкой, показанную на рис. 10.19. Это скомпилирует изменённое приложение Calculator.

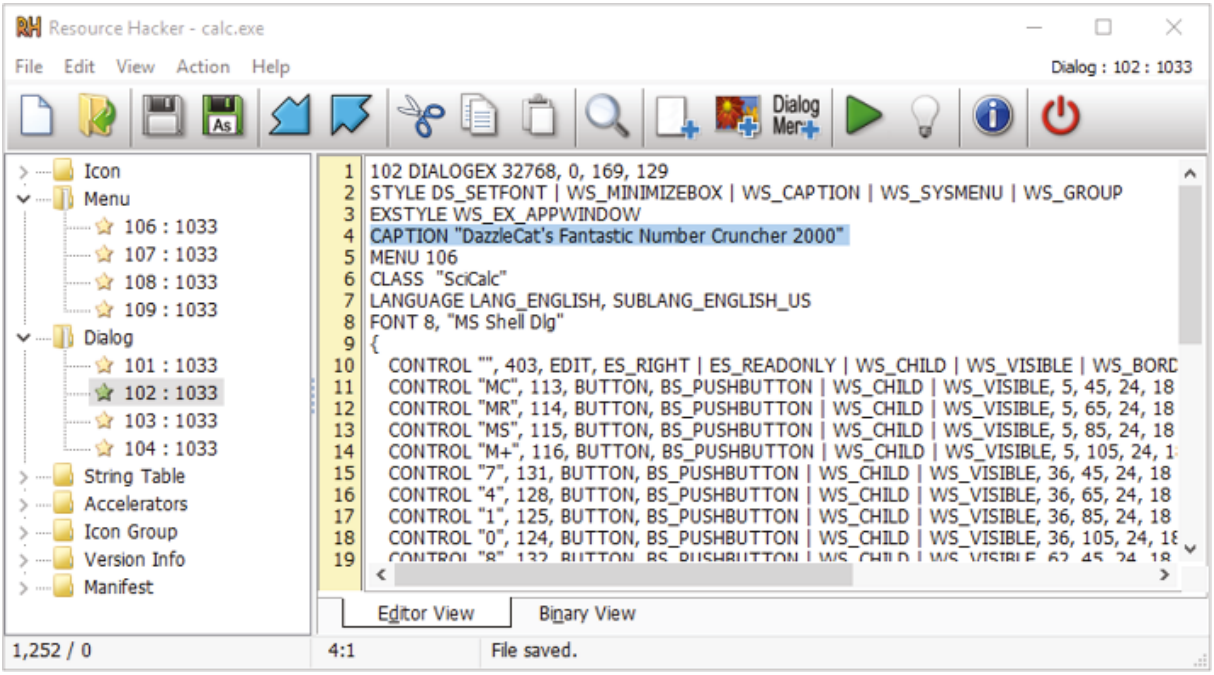


Рис. 10.19. Компиляция изменённого приложения

После компиляции в области предварительного просмотра должна появиться обновлённая версия окна с изменённым заголовком, как на рис. 10.20.

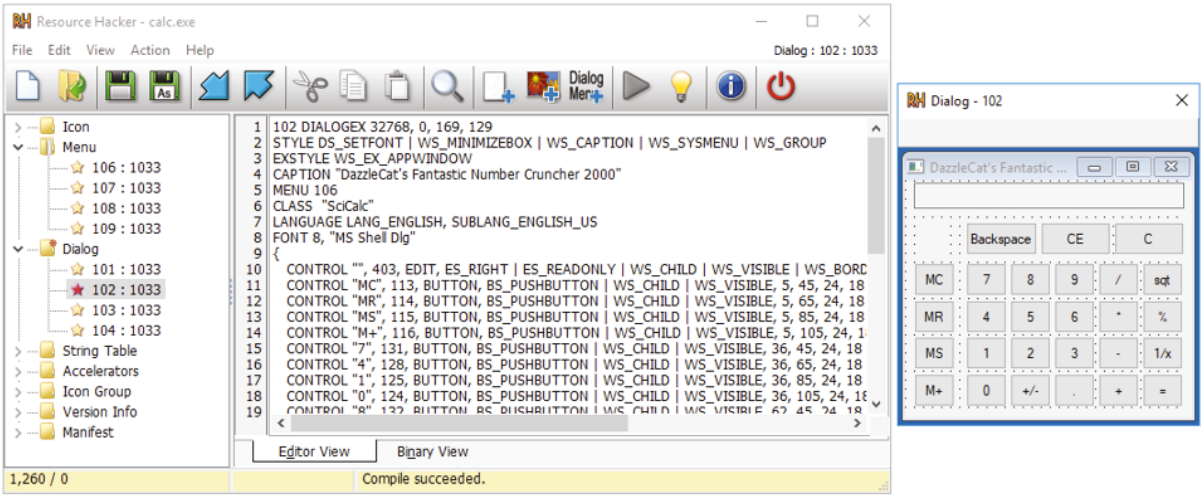


Рис. 10.20. Изменённое окно в Resource Hacker

На снимках новый заголовок задан строкой `CAPTION "DazzleCat's Fantastic Number Cruncher 2000"`.

Компиляция приложения не сохраняет изменённую версию автоматически. Для сохранения выберите **File > Save**, как показано на рис. 10.21.

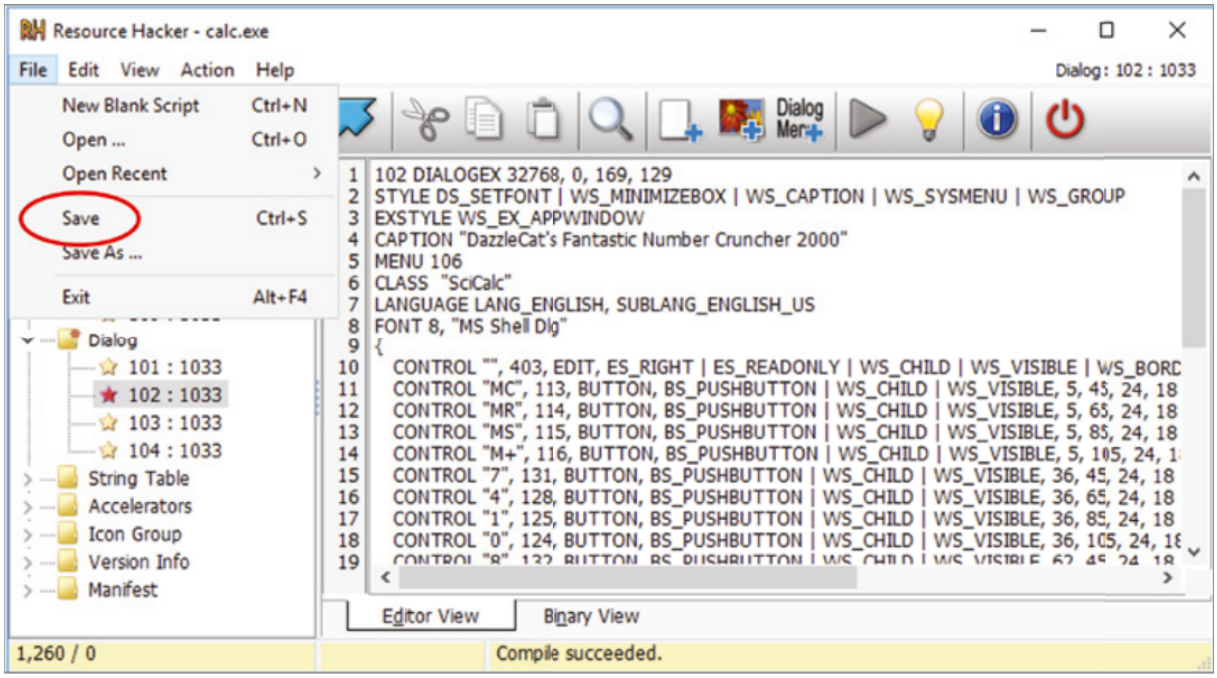


Рис. 10.21. Сохранение изменённого приложения в ResHack

Теперь вы успешно изменили оформление Windows Calculator. Для более сложной задачи попробуйте следующее:

- с помощью Resource Hacker измените размер окна, чтобы в нём помещалось новое имя;
- измените доступные кнопки;
- измените фон калькулятора;
- откройте и отредактируйте другие программы в виртуальной машине.

Исправление двоичного файла

Исправление двоичного файла подразумевает изменение скомпилированного двоичного кода, влияющего на его выполнение. В зависимости от ситуации иногда проще всего исправить приложение, чтобы обойти его защиту.

Исправление или генерация ключей

В некоторых случаях усовершенствованные проверки целостности или обфускация могут затруднить исправление. Например:

• исправить на диске зашифрованную или упакованную программу неосуществимо;
• обход динамических проверок целостности, например постоянно проверяемых контрольных сумм, может оказаться слишком громоздким;
• распространение исправленного исполняемого файла может быть нежелательным по организационным причинам.

В таких ситуациях можно вернуться к генераторам ключей. В остальных случаях, если это возможно, исправить программу, удалив проверку ключа или любую другую нежелательную логику, часто проще.

Где вносить исправление

Исправление можно выполнять в двух местах: в памяти или на диске.

Исправление в памяти изменяет находящийся в памяти машинный код. Это полезно при обратной разработке, поскольку до успеха может потребоваться испытать десятки (или сотни... или больше... страшно подумать) вариантов. Исправление в памяти влияет только на текущее выполнение приложения. При каждом перезапуске все такие изменения теряются.

Исправление на диске изменяет машинный код в скомпилированном двоичном файле. Это полезно, когда рабочий вариант уже найден, и влияет на все будущие запуски приложения. Изменения становятся постоянными и присутствуют при каждом запуске.

NOP

Вспомните инструкцию `nop`. Это однобайтовая инструкция (`0x90`), которая ничего не делает. При исправлении приложений критически важно не перемещать код. Изменение размера или простое удаление кода приведёт к аварийному завершению приложения. Чтобы удалить участки кода, сохранив прежний размер, заполните их инструкциями `nop`.

Тем, кому интересно, почему нельзя просто удалить код: причин несколько, но важнее всего то, что часть кода x86 использует относительные, а часть - абсолютные ссылки. Сначала рассмотрим относительный случай. Такой код преобразуется в команды вроде «перейти на 40 байт вперёд от текущего места». Если удалить код между переходом и его целью в 40 байтах, переход будет нарушен. Он продолжит вести на 40 байт вперёд, но теперь может попасть в середину кода операции или пропустить критически важные инструкции, что приведёт к сбою. Если удалённый код находится вне этого 40-байтового промежутка и переход на 40 байт по-прежнему попадает в прежнее место, воздействия не будет.

Теперь рассмотрим абсолютные ссылки. Они выглядят примерно так: «использовать значение данных по адресу `0x1234567`». Если удалить код в любом месте двоичного файла перед этим

адресом, всё последующее сдвинется. Тогда при всякой попытке абсолютной ссылки получить значение или выполнить абсолютный переход все адреса окажутся неправильными, даже если из двоичного файла был удалён всего 1 байт.

Следовательно, добавление или удаление байтов влияет на относительные ссылки только в том случае, если происходит между местом ссылки и её целью. Однако все абсолютные ссылки разрушаются при сдвиге приложения хотя бы на 1 байт. Поэтому при исправлении критически важно сохранять размер, если, конечно, ваша цель не состоит в том, чтобы всё аварийно завершилось, - тогда ломайте на здоровье!

Возвращаясь к nop: если нужно удалить фрагмент кода, например заставить программу пропустить проверку ключа, не удаляйте код, а замените его инструкциями nop. Это сохраняет байтовое выравнивание приложения, но при достижении нежелательного кода ничего не происходит.

Другие отладчики

Для обратной разработки посредством динамического анализа в Windows существует несколько популярных вариантов:

• OllyDbg;
• Immunity;
• x64dbg;
• WinDbg.

Выбор зависит от ситуации и предпочтений пользователя. Все они обладают похожими возможностями, а навыки работы в одном обычно переносятся на остальные. По ходу книги вы познакомитесь с несколькими программами; цель - дать представление о многих инструментах, чтобы вы почувствовали, когда полезен каждый из них.

OllyDbg

OllyDbg - чрезвычайно популярный и мощный отладчик. Хотя большинство отладчиков сосредоточено на отладке, Olly обладает расширенными возможностями, включая:

• расширяемость, подключаемые модули и сценарии;
• систему трассировки выполнения;
• функции исправления кода;
• автоматические описания параметров большинства функций Windows;
• акцент на анализе двоичного кода, а не на отладке исходного кода;
• малый размер и переносимость.

Благодаря этим возможностям OllyDbg прекрасно подходит для:

• написания эксплойтов;
• анализа вредоносных программ;
• обратной разработки.

Но при всей мощности и популярности OllyDbg имеет ограничения. Одно из них - работа только с 32-разрядными исполняемыми файлами, которые, следует признать, являются вымирающим видом, но пока не исчезли.

Другое ограничение состоит в том, что к интерфейсу OllyDbg часто приходится привыкать: сначала он не кажется надёжным или интуитивно понятным. Тем не менее определённо стоит продолжить

работу с ним, поскольку это мощный инструмент динамического анализа.

Immunity

Immunity является ответвлением OllyDby, то есть обладает многими теми же возможностями. Он также добавляет множество функций, сделавших его популярным у разработчиков эксплойтов, например поддержку сценариев Python.

> **Примечание переводчика.** В исходнике напечатано `OllyDby`; вероятно, имеется в виду `OllyDbg`. Опечатка сохранена.

Однако, подобно OllyDbg, Immunity может отлаживать только 32-разрядные исполняемые файлы. Кроме того, он наследует неинтуитивный интерфейс OllyDbg.

x86dbg

`x86dbg` - замена OllyDbg с поддержкой как 32-разрядных (`x86dbg`), так и 64-разрядных (`x64dbg`) приложений. Благодаря более широкой поддержке его часто выбирают для обратной разработки или отладки 64-разрядных приложений.

WinDbg

WinDbg - универсально применимый отладчик с хорошей поддержкой и превосходной работой с отладочными символами, которая, впрочем, менее полезна при обратной разработке. Однако он ориентирован на отладку и лишён некоторых возможностей специализированных средств обратной разработки.

Отладка с помощью Immunity

Из-за ограничений времени и объёма книги рассмотреть все эти отладчики невозможно. Immunity выбран благодаря популярности в обратной разработке и создании эксплойтов. Однако важно помнить, что у всех этих отладчиков похожие возможности, и навыки, полученные в одном, часто переносятся на другие.

На рис. 10.22 показан интерфейс Immunity в Windows. Четыре окна, начиная с верхнего левого и далее по часовой стрелке, показывают дизассемблированный код программы, регистры, стек и память.

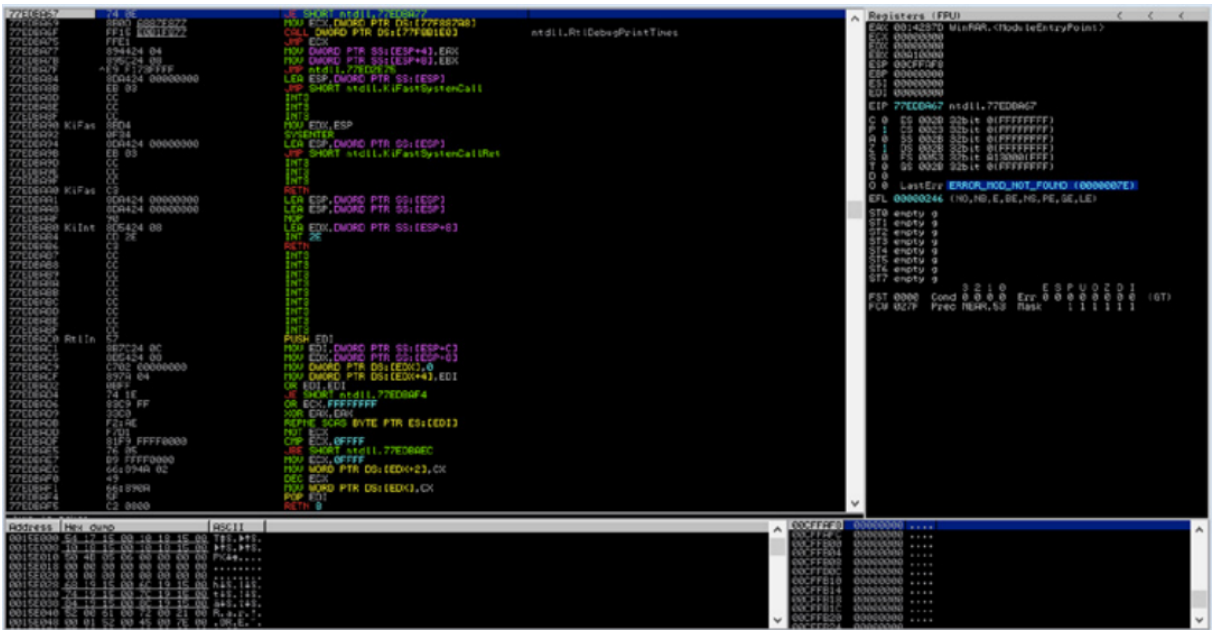


Рис. 10.22. Окно отладчика Immunity

Immunity: ассемблерный код

На рис. 10.23 показан дизассемблированный код программы в Immunity. Обратите внимание: отображаются адрес памяти, машинный код и ассемблерный код x86.

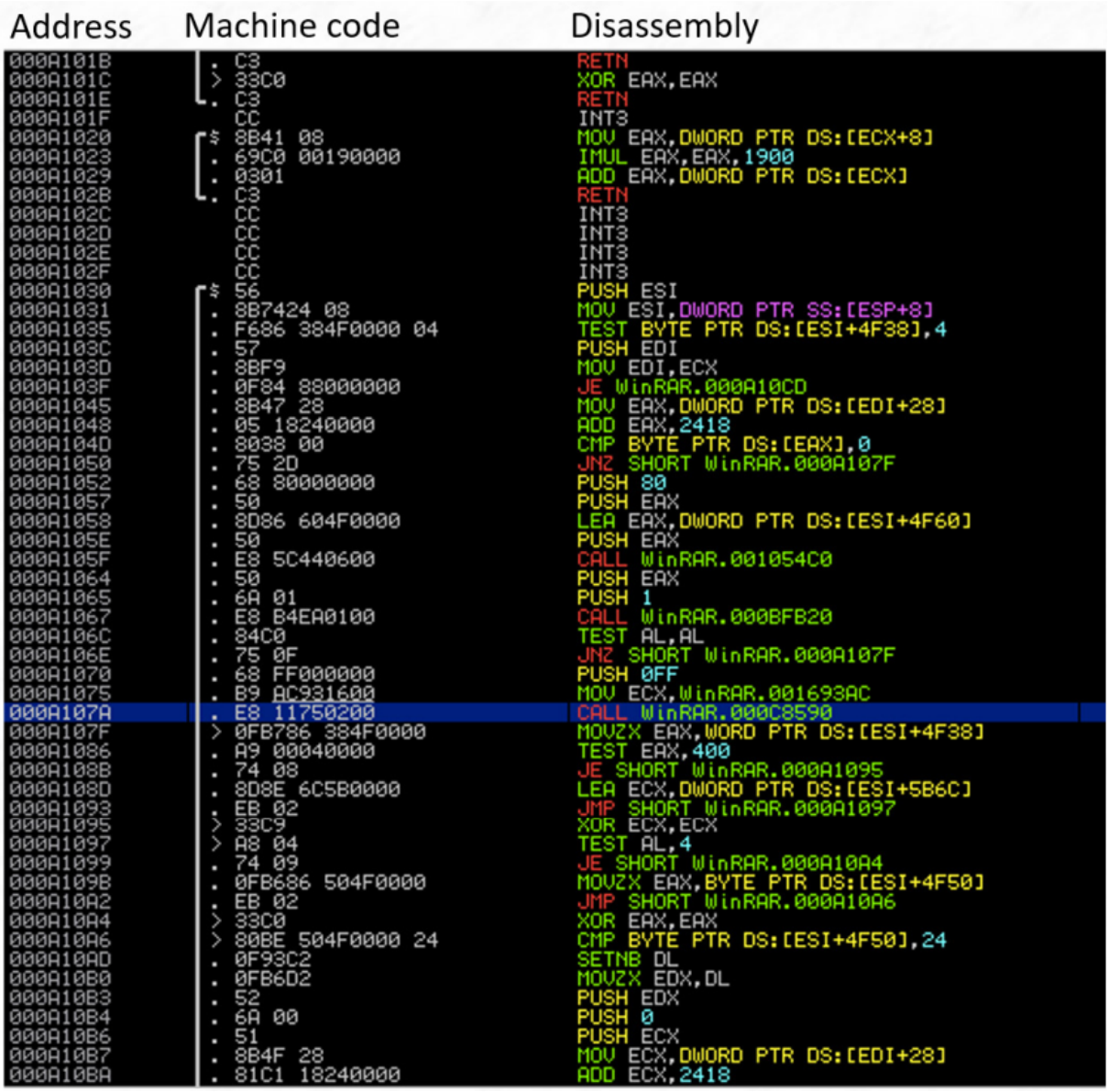


Рис. 10.23. Ассемблерный код в отладчике Immunity

Подписи столбцов на снимке: **Address** - адрес, **Machine code** - машинный код, **Disassembly** - дизассемблированный код.

Чтобы выбрать строку кода, щёлкните её. Для выбранной строки Immunity предоставляет различные клавиатурные сочетания:

• `;`: добавить комментарий к выбранной строке. Это важнейшая часть обратной разработки, помогающая отслеживать проделанную работу.
• `Ctrl+A`: автоматически проанализировать программу. Immunity довольно хорошо добавляет комментарии и предполагает параметры функций.
• `Enter`: перейти к выбранной функции. Например, если видна инструкция `call 0x1234` и требуется узнать, что делает функция по адресу `0x1234`.
• `-`: вернуться в предыдущее место. Например, после анализа функции `0x1234` вернуться туда, откуда вы пришли.
• `+`: перейти в следующее место после нажатия `-`. Например, если вы вернулись в вызывающую функцию с помощью `-`, но хотите снова перейти в функцию `0x1234`.
• `Ctrl+R`: найти перекрёстные ссылки на выбранную строку. Например, если в окне дампа памяти выбрана строка и нужно узнать, кто её использует; либо если в дизассемблированном коде

выбрано начало функции и требуется узнать, кто её вызывает.
• **Двойной щелчок по адресу:** установить в этом месте точку останова.

Immunity: модули

В Immunity список исполняемых модулей открывается нажатием клавиши `e`. Он показывает весь доступный для отладки код, включая динамически загруженные библиотеки, как на рис. 10.24. После открытия списка можно дважды щёлкнуть модуль, чтобы перейти к его коду.

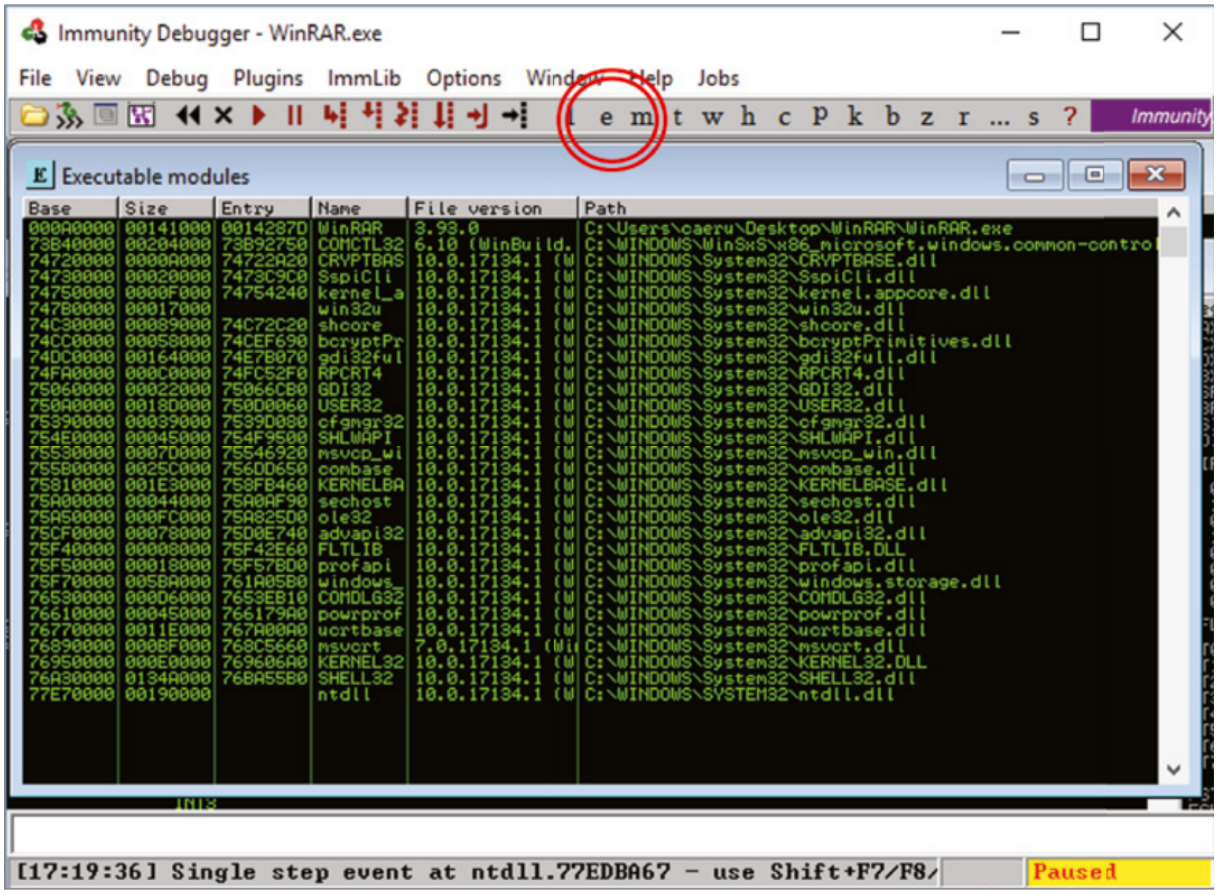


Рис. 10.24. Исполняемые модули в отладчике Immunity

После запуска Immunity проверьте регистр `eip`, чтобы понять, какой модуль сейчас просматривается. Почти всегда начинать отладку следует с главного исполняемого файла, а не с совместно используемой библиотеки вроде `ntdll`. Переключиться на главный файл можно в окне модулей.

Immunity: строки

Часто полезно определить, какой код использует определённую строку исполняемого файла. Чтобы найти все используемые программой строки, щёлкните правой кнопкой мыши и выберите **Search For > All Referenced Text Strings**, как показано на рис. 10.25.

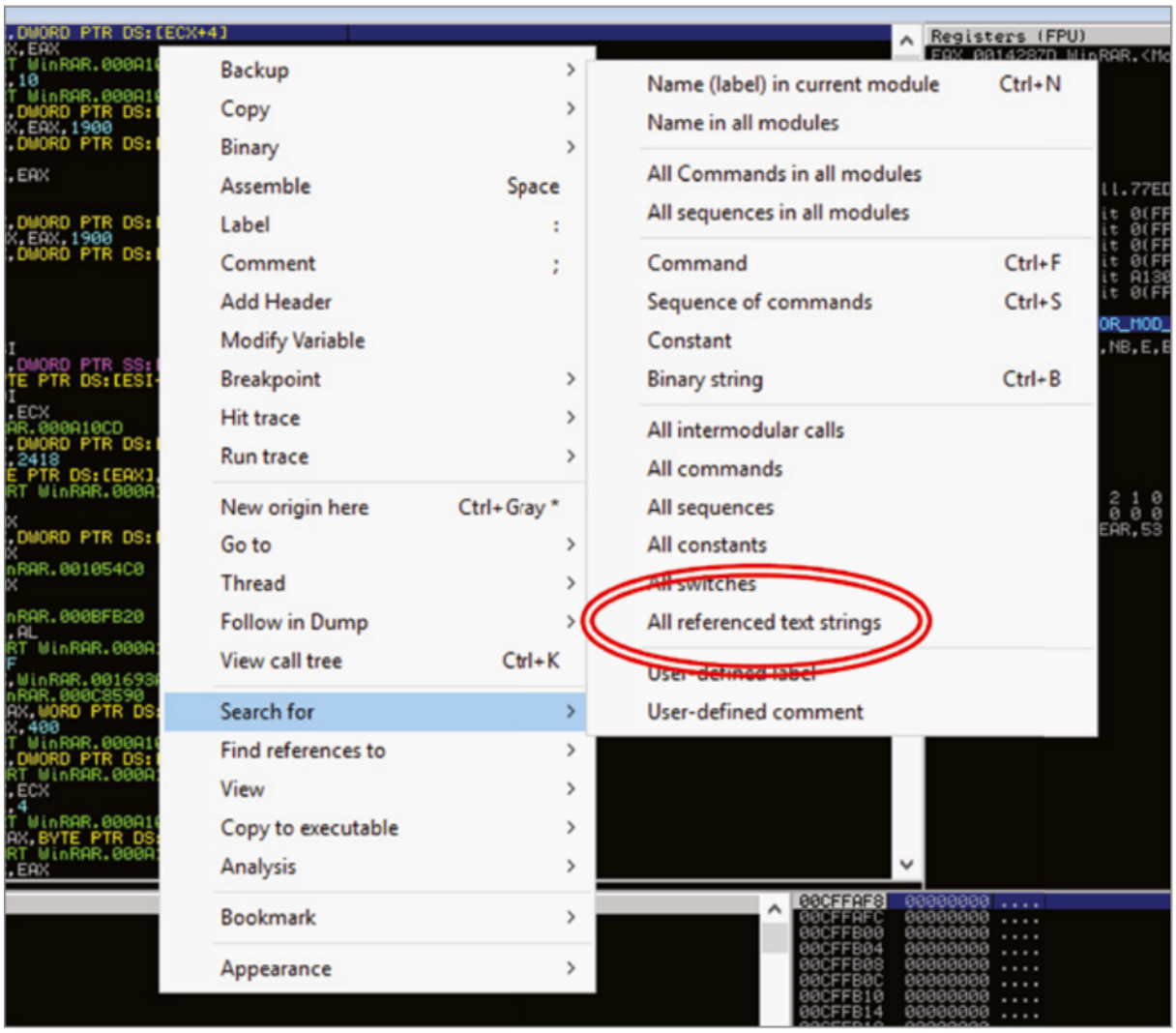


Рис. 10.25. Строки в отладчике Immunity

В окне строк щёлкните правой кнопкой мыши и выберите **Search For Text**, чтобы найти определённую строку, как показано на рис. 10.26. Затем снова щёлкните правой кнопкой и выберите **Search For Next**, чтобы найти следующую ссылку на эту строку. Двойной щелчок по адресу строки переводит к месту её использования в дизассемблированном коде.

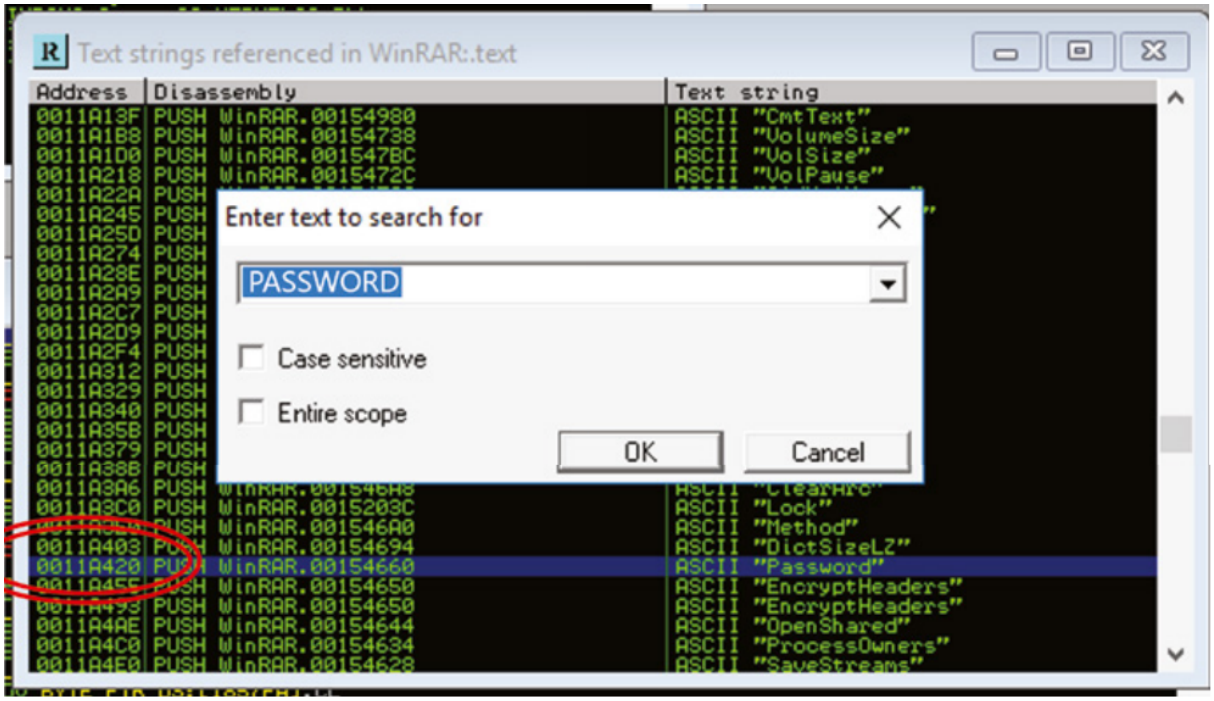


Рис. 10.26. Ссылки на строки в отладчике Immunity

На снимке выполняется поиск текста `PASSWORD`; отмечена соответствующая строка `Password`.

Immunity: запуск программы

Нажмите стрелку воспроизведения, чтобы запустить исполняемый файл под управлением отладчика, как показано на рис. 10.27. Выполнение можно остановить, нажав крест слева от стрелки, или приостановить кнопкой паузы справа от неё. Перезапустить выполнение можно кнопкой с двумя направленными влево стрелками.

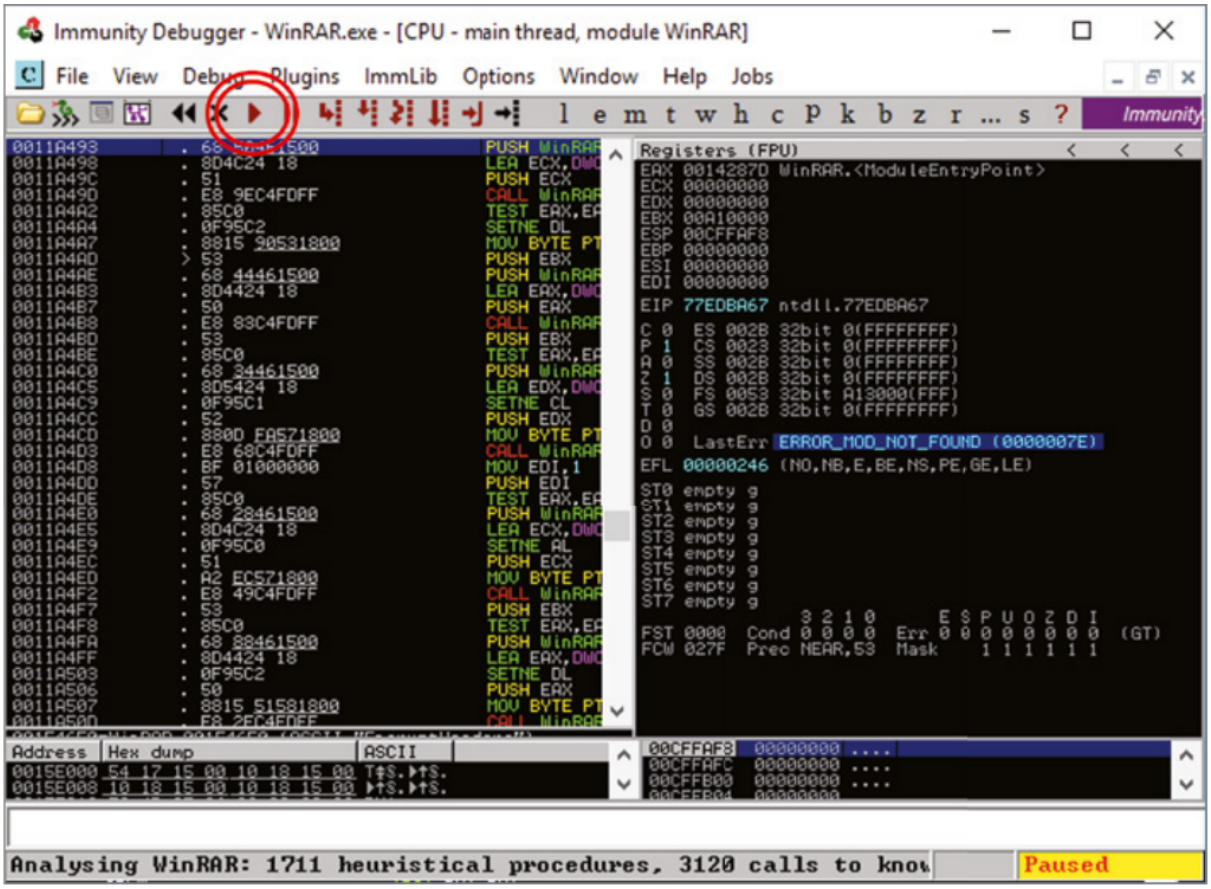


Рис. 10.27. Запуск исполняемого файла в отладчике Immunity

После остановки выполнения точкой останова или кнопкой паузы можно нажать **Step Into**, чтобы продвинуть программу на одну инструкцию, как показано на рис. 10.28. Если же остановка произошла на вызове функции, работа которой уже известна или не представляет интереса, нажмите **Step Over**, как показано на рис. 10.29, чтобы продолжить отладку после возврата из функции.

> **Примечание переводчика.** В исходнике перед `Over` ошибочно напечатано слово `overstep`; в переводе использовано фактическое имя команды `Step Over`.

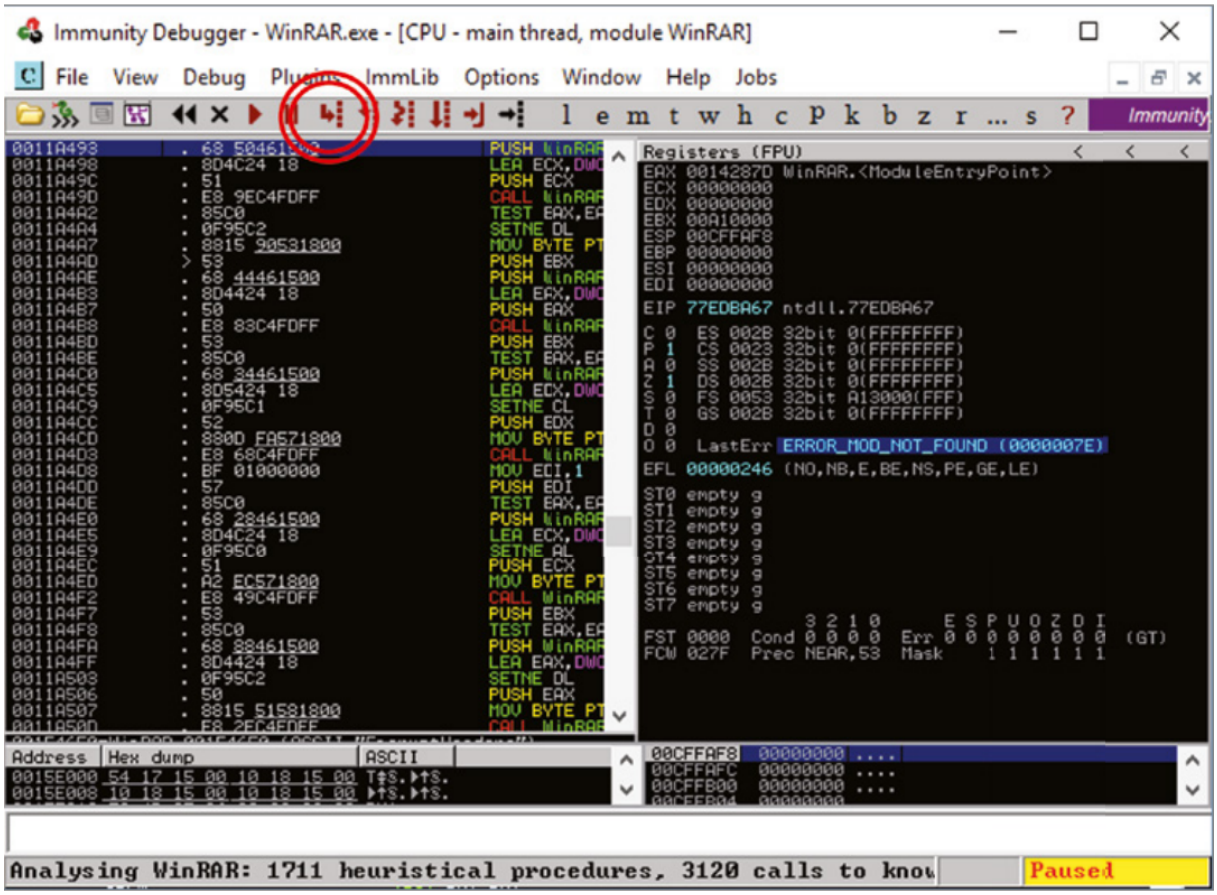


Рис. 10.28. Выполнение по одной инструкции в отладчике Immunity

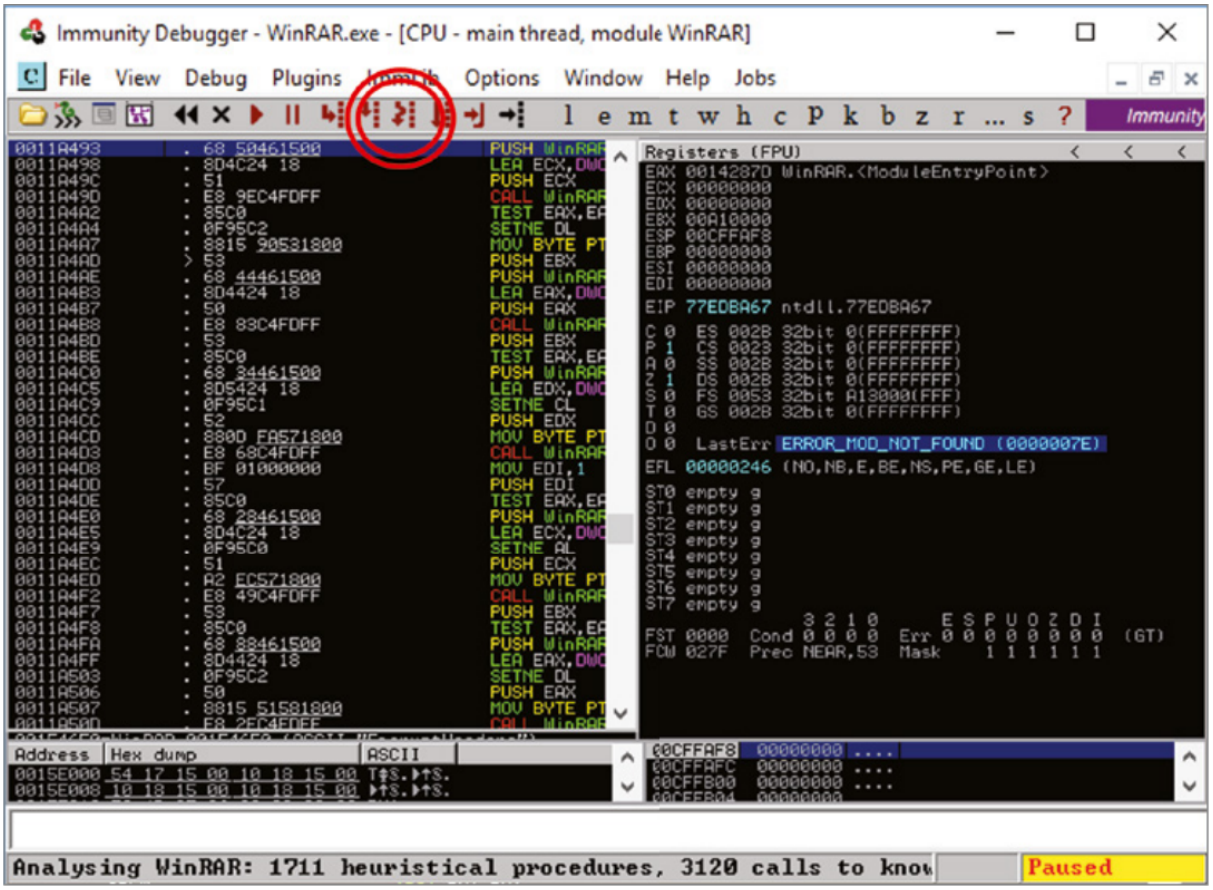


Рис. 10.29. Выполнение инструкций с обходом вызова в отладчике Immunity

Immunity: исключения

Многие приложения создают исключения в ходе обычного выполнения. Например, блок `try {} except {}` создаст исключение, если внутри `try` что-либо пойдёт не так. Средства динамического анализа вроде Immunity, действуя как отладчик, обычно первыми перехватывают исключение, чтобы выяснить, хотите ли вы с ним что-либо сделать.

Но при обратной разработке обычно не следует вмешиваться в нормальное выполнение. Вместо этого нужно позволить приложению обработать исключение обычным способом. Поэтому почти всегда исключение следует передать из отладчика приложению.

Как показано на рис. 10.30, исключения выводятся внизу окна Immunity, хотя каждый отладчик немного отличается. В Immunity нажмите `Shift+F9`, чтобы передать исключение и продолжить выполнение.

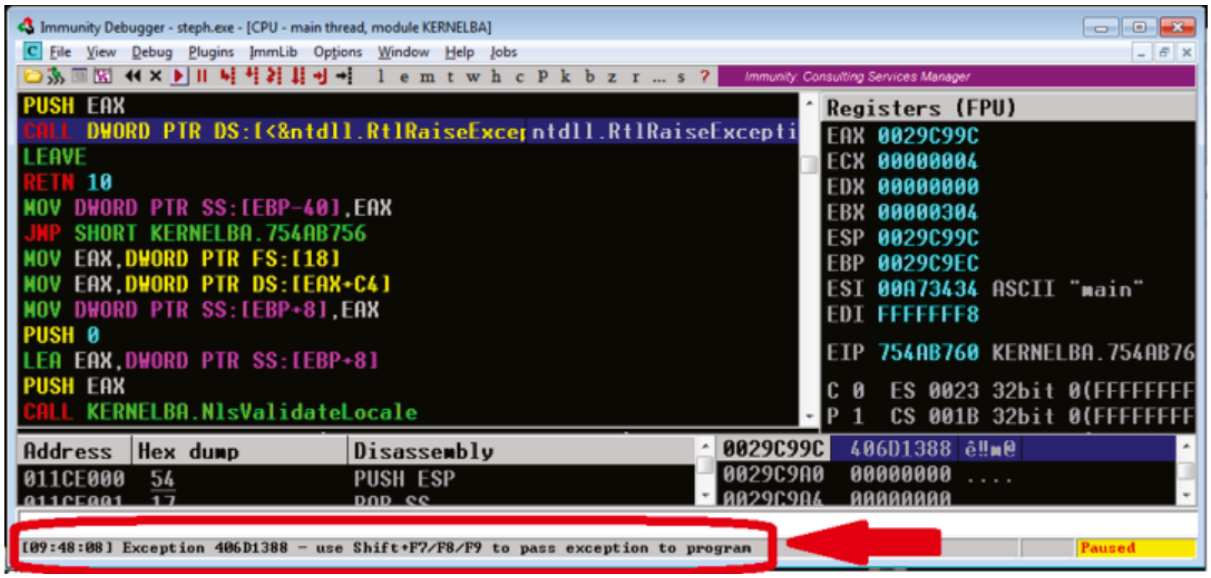


Рис. 10.30. Исключения в отладчике Immunity

Immunity: переписывание программы

Immunity предоставляет множество функций для создания исправлений, изменяющих поведение программы. При взломе это включает редактирование программы для удаления проверок ключей, назойливых экранов и так далее.

В первых взломах вы будете удалять код посредством заполнения инструкциями nop. Для этого инструкции программы заменяются инструкциями nop.

В Immunity сначала выберите инструкции, которые хотите удалить. Затем щёлкните правой кнопкой и выберите **Binary > Fill With NOPs**, как показано на рис. 10.31.

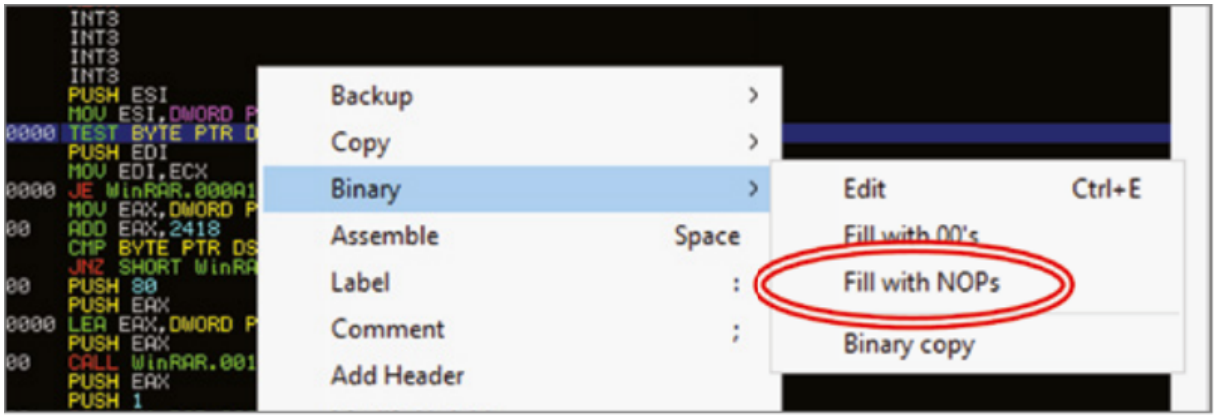


Рис. 10.31. Заполнение кода инструкциями NOP в отладчике Immunity

Выбранные инструкции будут заменены последовательностью nop, как показано на рис. 10.32.

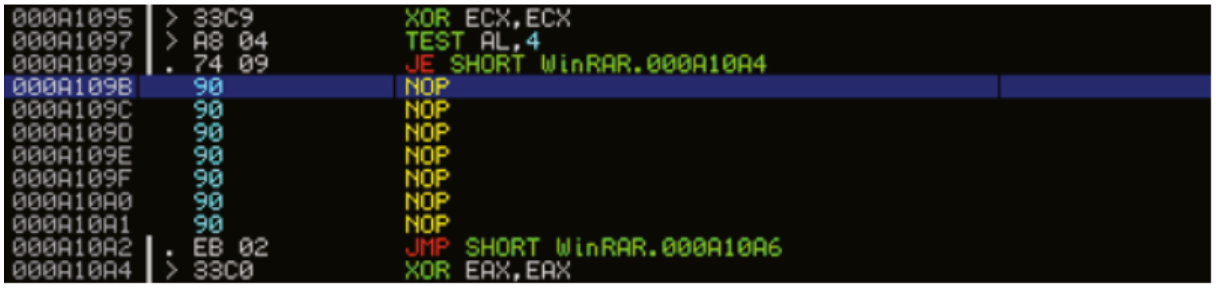


Рис. 10.32. Код, заполненный инструкциями nop в отладчике Immunity

После изменения программы проверьте исправление, снова запустив программу. Если исправлен правильный участок кода, назойливый экран, проверка ключа или другая цель должны исчезнуть.

Если же исправление вызывает сбой или не удаляет цель, изменения легко отменить и попробовать снова. Нажмите кнопку исправлений, чтобы открыть соответствующее окно. Затем щёлкните своё исправление правой кнопкой и выберите **Restore Original Code**, как показано на рис. 10.33, чтобы отменить изменение и повторить попытку.

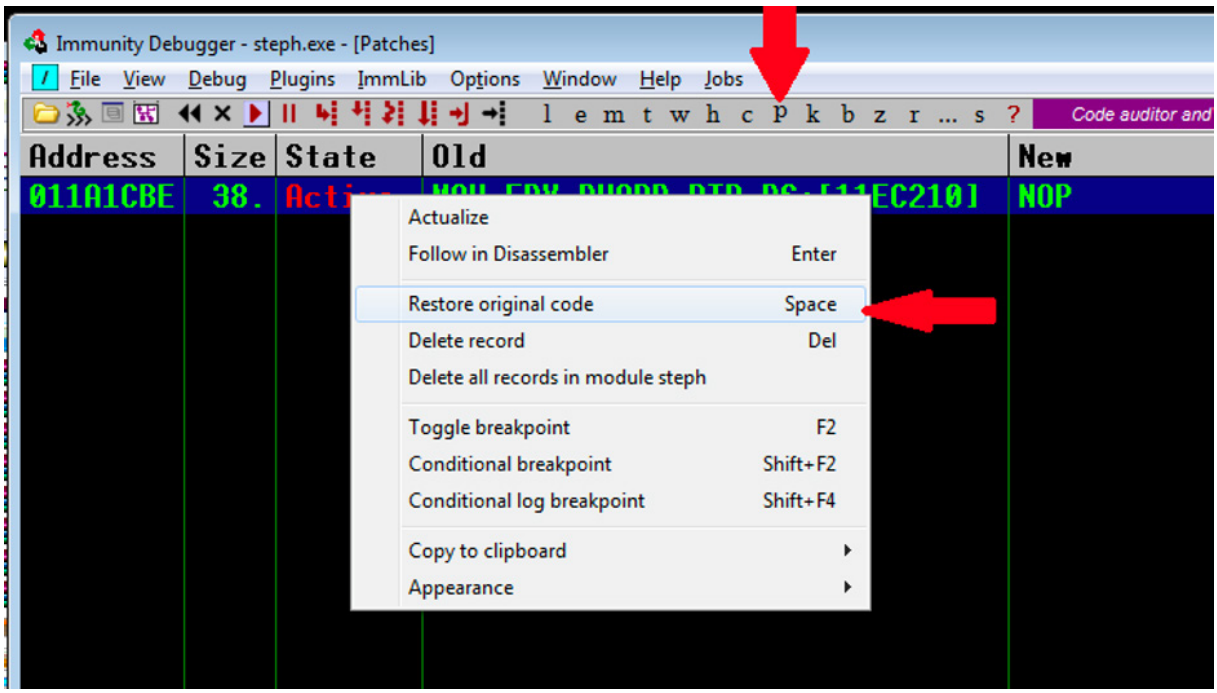


Рис. 10.33. Отмена изменённого кода в отладчике Immunity

Найдя рабочее исправление, сохраните изменения в исполняемом файле, чтобы сделать их постоянными. Как показано на рис. 10.34, щёлкните правой кнопкой и выберите **Copy To Executable > All Modifications**. В окне подтверждения выберите **Copy All**.

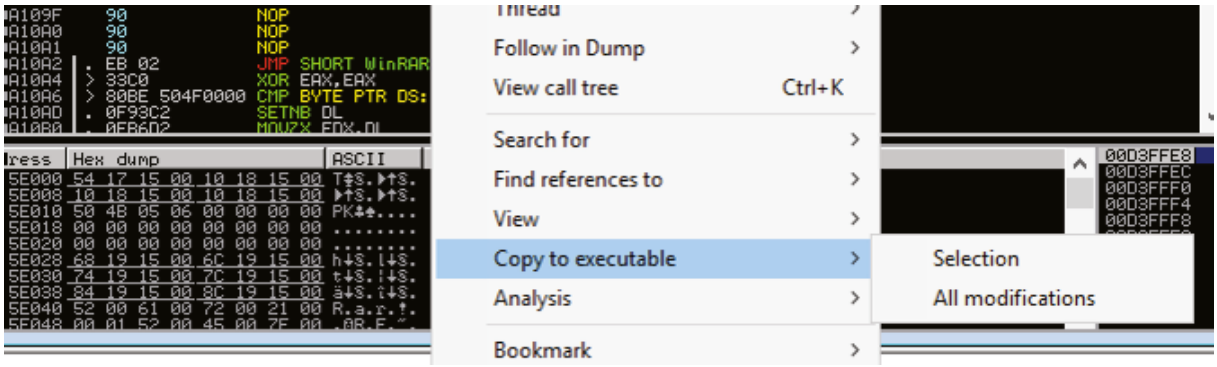


Рис. 10.34. Сохранение изменённого файла в отладчике Immunity

Появится окно изменённого исполняемого файла с внесёнными правками. Закройте его и выберите **Yes**, чтобы сохранить файл. Дайте ему новое имя, например `cracked.exe`.

Если вы уверены в изменении, `cracked.exe` можно запустить непосредственно. Если требуется продолжить отладку с новыми изменениями, необходимо заново загрузить `cracked.exe` в Immunity.

Лабораторная работа: взлом с помощью Immunity

Эта лабораторная работа даёт практический опыт взлома программ с помощью отладчика. Лабораторные работы и все связанные инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой работы найдите **Lab Cracking with Immunity** и следуйте предоставленным инструкциям.

Навыки

В лабораторной работе практикуются обратная разработка, исправление и обход программных защит с помощью Immunity и Resource Hacker. Проверяются, в частности, следующие навыки:

- обратная разработка x86;
- исправление двоичного кода;
- статический и динамический анализ.

Основные выводы

Программное обеспечение можно легко изменить, чтобы добавить, заменить или удалить функциональность. Те же методы позволяют обходить любую защиту, от тривиальной до передовой, если вы понимаете работу программы.

Итоги

Средства проверки ключей предназначены для защиты от распространения и использования нелицензионных и взломанных копий программ, но идеальной защиты не существует. Такие инструменты, как Procmon, Resource Hacker и отладчики, позволяют понять эти средства защиты и преодолеть их с помощью генераторов ключей или исправлений.

Глава 11. Исправление и расширенные инструменты

В предыдущей главе были представлены взлом и исправление программного обеспечения. В этой главе исправление рассматривается подробнее, а также описываются некоторые более совершенные инструменты, которые можно применять для обратной разработки и взлома.

Исправление в 010 Editor

Нередко бывает полезно просматривать и редактировать шестнадцатеричное представление файла. Если вы когда-либо пытались открыть двоичный файл в текстовом редакторе, то видели множество странных символов и пустого пространства. Причина в том, что текстовый редактор пытается интерпретировать всё содержимое файла как ASCII, хотя оно таковым не является. Поэтому нужен редактор, который отображает данные в шестнадцатеричном виде, а не как ASCII. Существует много шестнадцатеричных редакторов, способных это делать. Один из наших любимых - 010 Editor. Ссылки находятся в разделе инструментов нашего репозитория GitHub:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Откройте любой файл - исполняемый файл, файл данных, изображение, музыку и так далее, - чтобы увидеть его шестнадцатеричное представление. На рис. 11.1 показан пример исполняемого файла в 010 Editor.

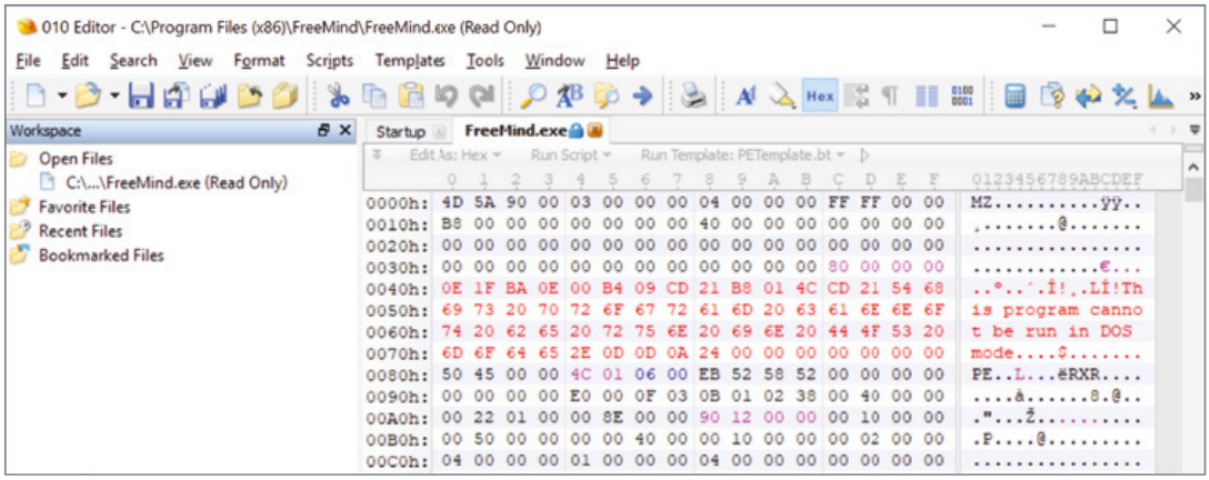


Рис. 11.1. Просмотр файла в 010 Editor

На рис. 11.2 показана панель **Inspector**. На ней представлены различные возможные интерпретации данных, на которые указывает курсор.

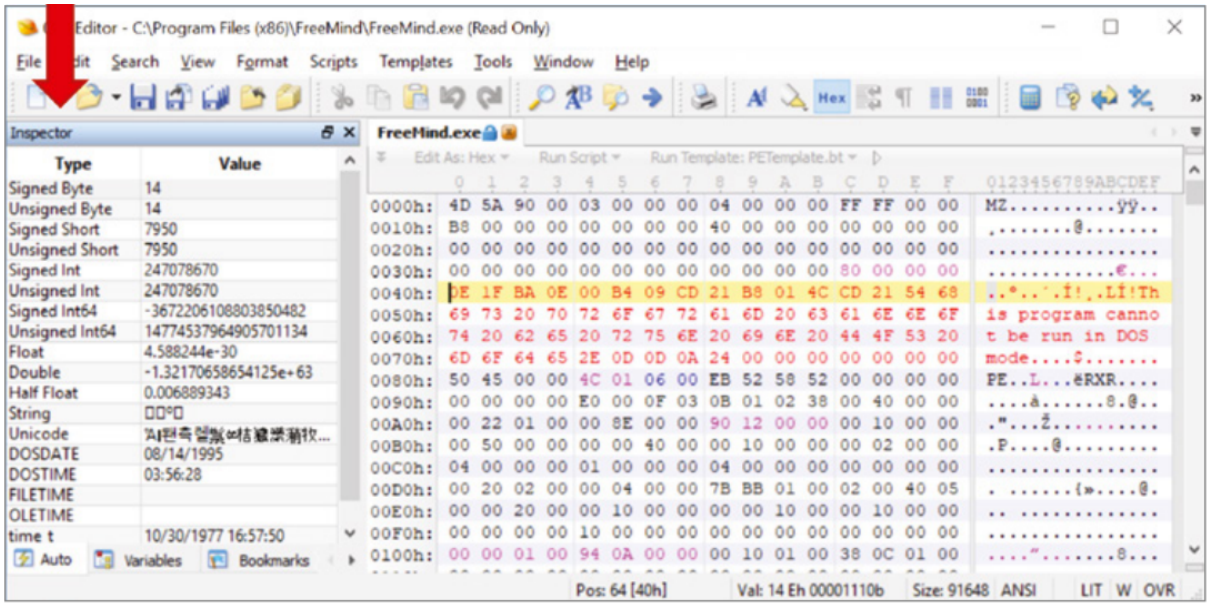


Рис. 11.2. Панель Inspector в 010 Editor

Если вы знаете, что именно ищете, можно выполнить поиск, как показано на рис. 11.3. Поддерживается поиск данных множества типов, в том числе:

- текст;
- шестнадцатеричные байты;
- строка ASCII;
- строка Unicode;
- строка EBCDIC;
- байт со знаком или без знака;
- короткое целое со знаком или без знака;
- целое со знаком или без знака;
- 64-разрядное целое со знаком или без знака;
- число с плавающей точкой;
- число двойной точности;
- имя переменной;
- значение переменной.

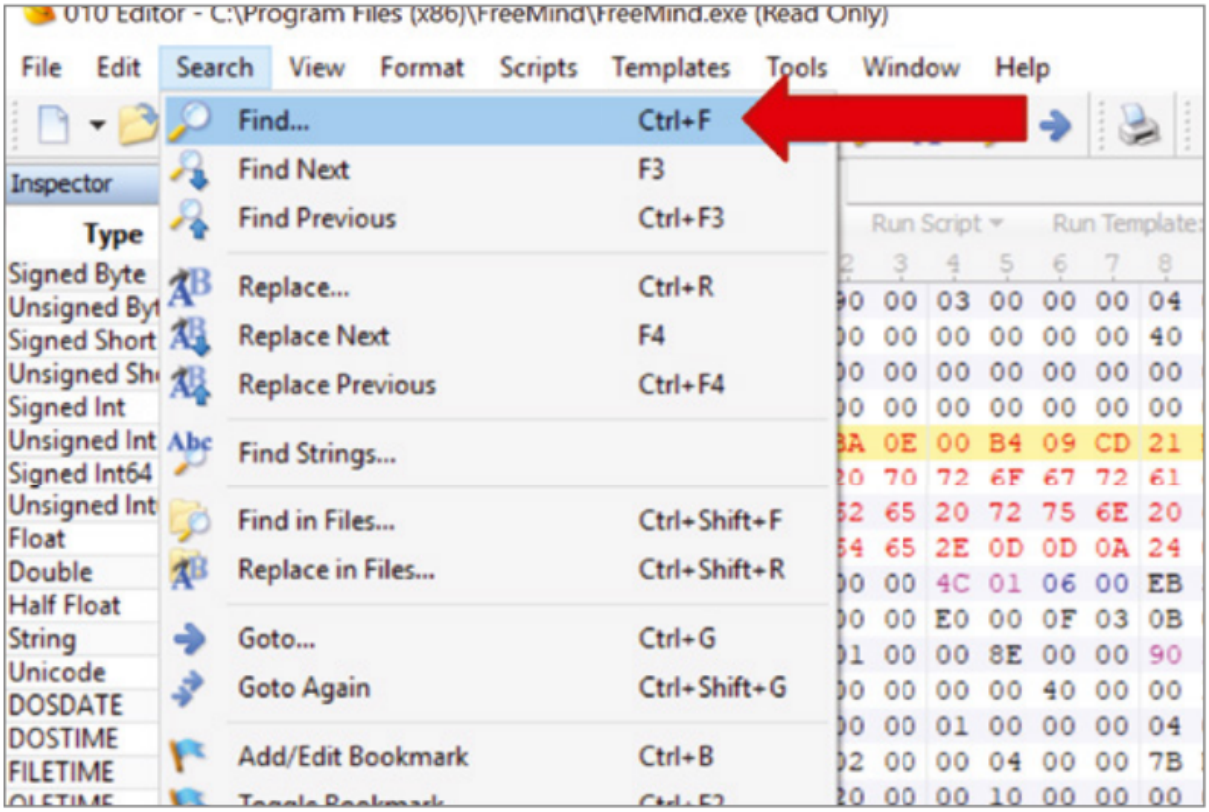


Рис. 11.3. Поиск в 010 Editor

Если вы знаете, куда именно нужно перейти, можно перейти к определённому адресу, как показано на рис. 11.4. Местоположение назначения можно задать как байт, номер строки, сектор или короткое целое.

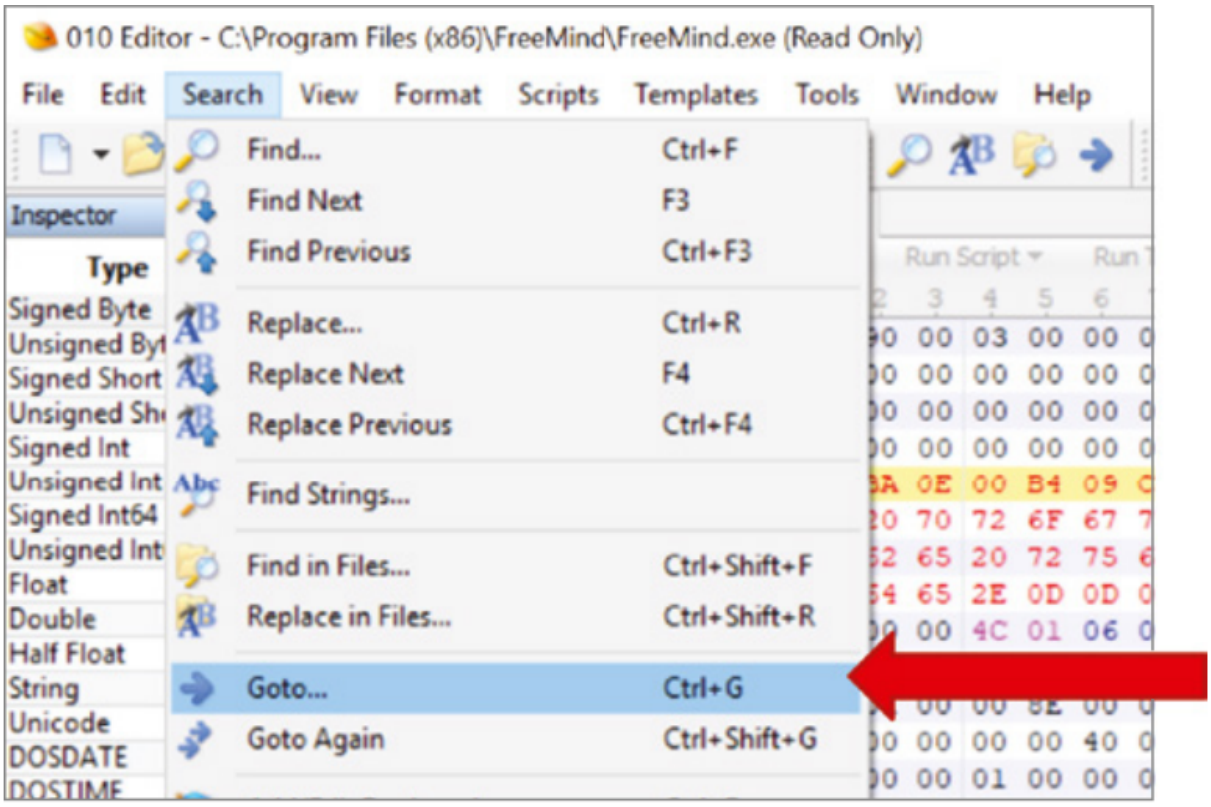


Рис. 11.4. Переход к адресу в 010 Editor

В 010 Editor можно непосредственно изменять шестнадцатеричные данные. Просто установите курсор и начните вводить новые значения поверх существующих.

При этом 010 Editor учитывает, насколько важно сохранять размер файла. Когда вы вводите значения, редактор перезаписывает существующие значения в данном месте. Он не вставляет новые данные, поскольку это увеличило бы размер файла.

Исправление с помощью CodeFusion

После того как исследователь выяснил, как взломать программу, следующим шагом часто становится создание утилиты-исправителя или взломщика. Она позволит другим людям взломать ту же программу.

CodeFusion - популярный генератор исправлений. Он создаёт отдельный исполняемый файл, который можно использовать для взлома конкретного приложения. Ссылки находятся в разделе инструментов нашего репозитория GitHub, приведённого выше.

Чтобы начать создание исправителя, запустите CodeFusion и настройте сведения, которые будут отображаться при его запуске. Эти сведения показаны на рис. 11.5 и включают заголовок окна, название программы, комментарии, значок и прочее. Здесь можно указать любые нужные значения.

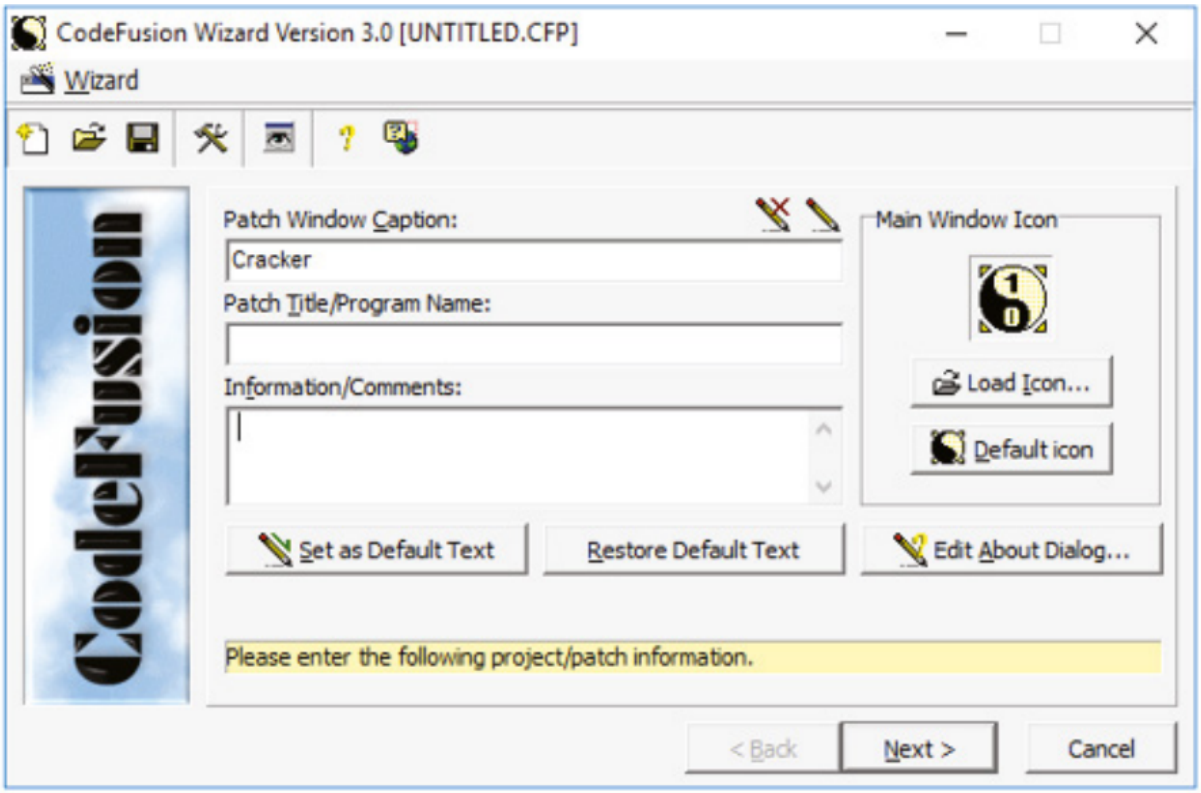


Рис. 11.5. Начальный экран CodeFusion

На следующем экране добавьте файлы, которые требуется исправить, как показано на рис. 11.6. Это исполняемый файл, который вы хотите взломать.

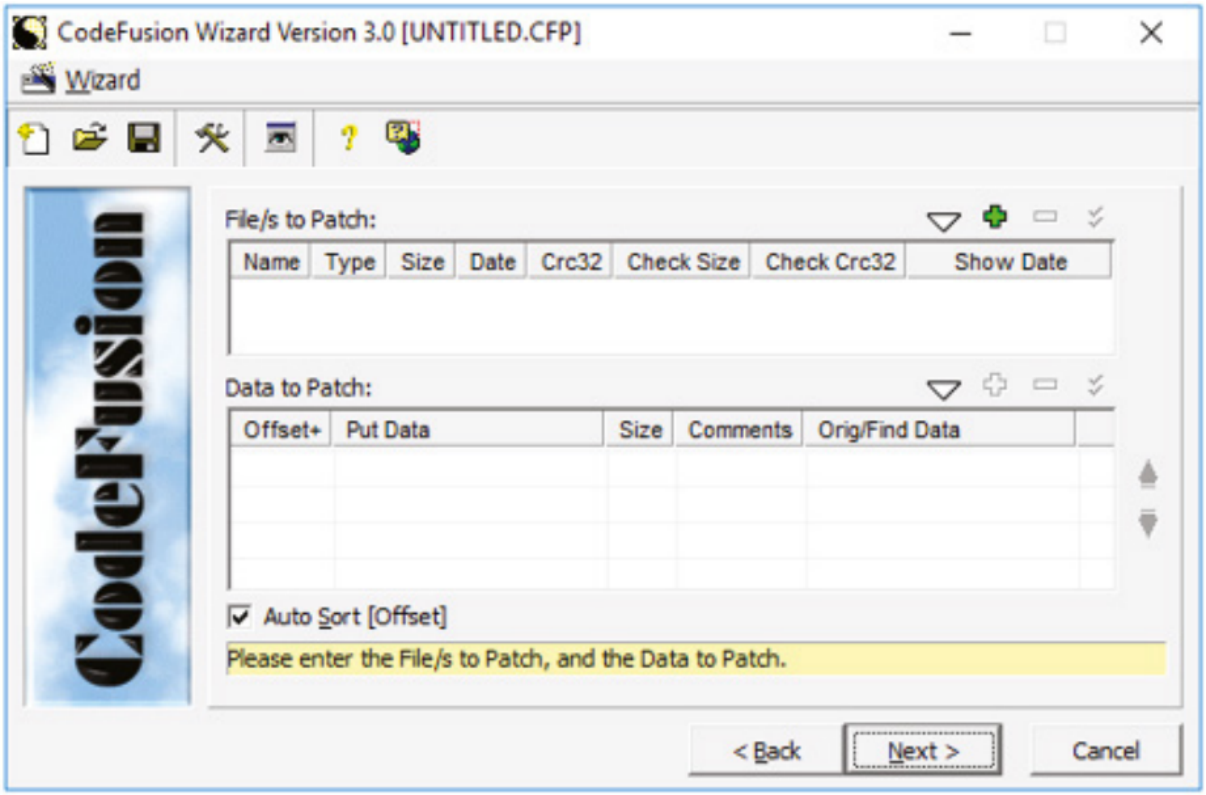


Рис. 11.6. Загрузка файла в CodeFusion

Затем добавьте сведения об исправлении, щёлкнув значок **+**, показанный на рис. 11.7. Обычно это сведения, полученные с помощью Immunity, Cheat Engine, IDA и подобных средств. Как правило, в них входят смещение, по которому требуется внести исправление, и заменяющие байты. Часто для исправления используются байты `0x90`, то есть инструкции `nop`. На следующей странице щёлкните **Make Win32 Executable**, чтобы создать EXE-файл, который будет исправлять целевое приложение.

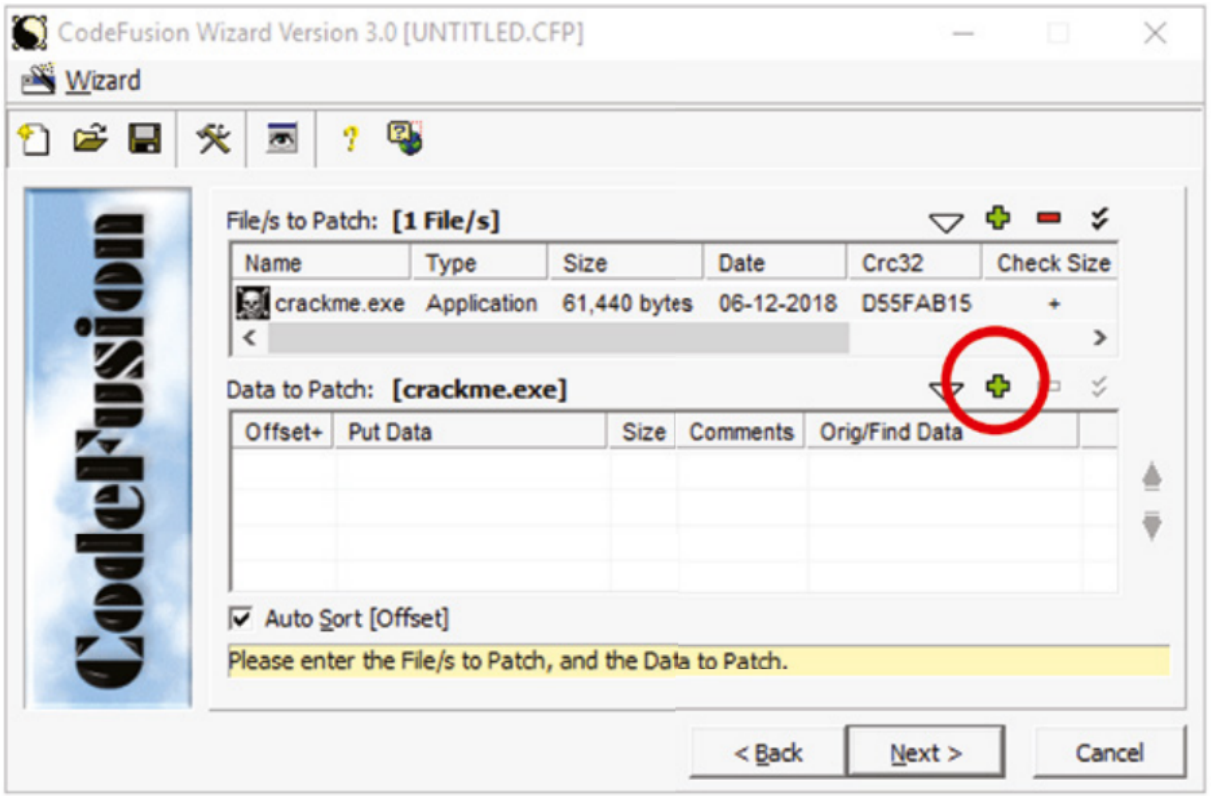


Рис. 11.7. Добавление сведений об исправлении в CodeFusion

CodeFusion добавит новый исполняемый файл рядом с целевым приложением. Как показано на рис. 11.8, запустите этот исполняемый файл, выберите цель и нажмите **Start**, чтобы применить исправление и взломать приложение.

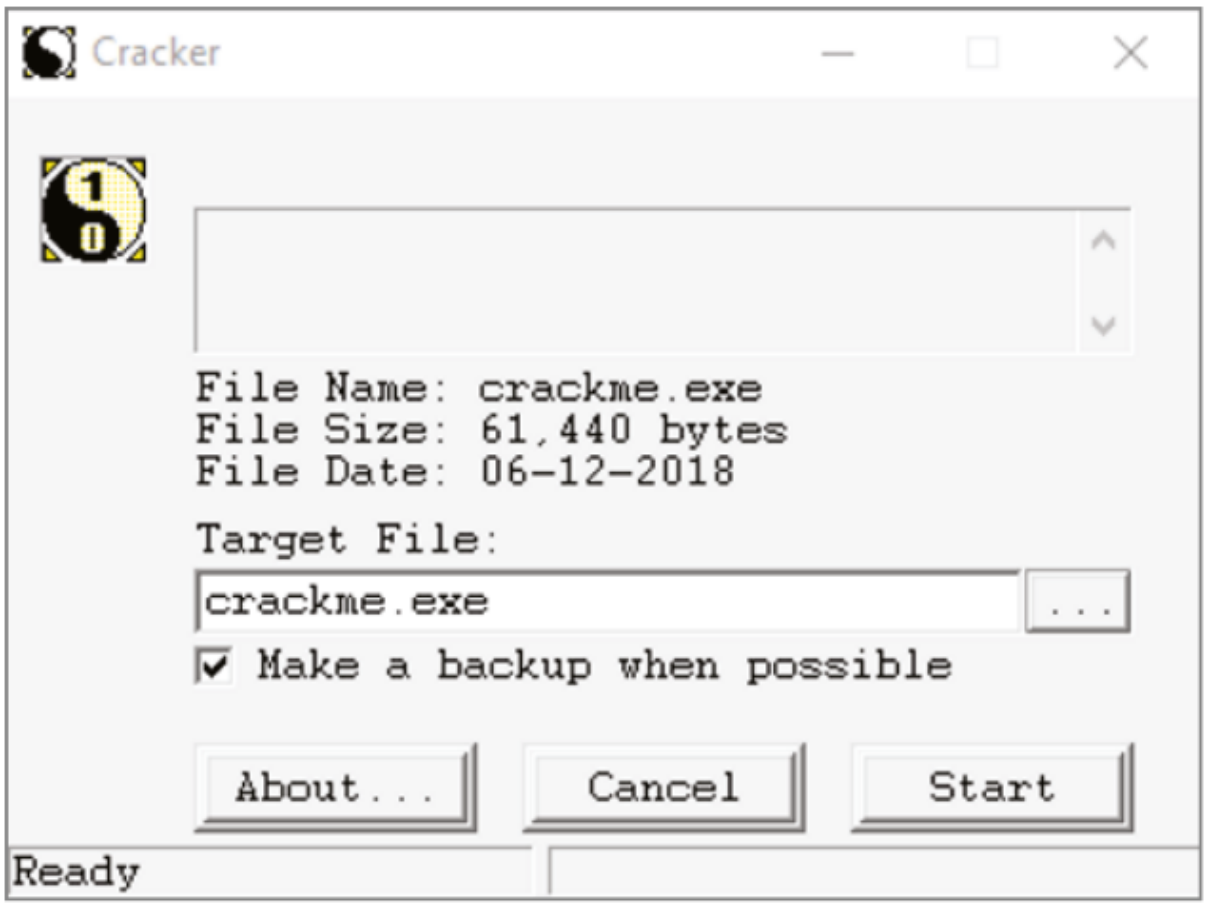


Рис. 11.8. Запуск исполняемого файла исправителя CodeFusion

Именно такой исполняемый файл для взлома часто распространяет группа взломщиков. Он намного меньше и удобнее для переноса, чем полное взломанное приложение. Пользователю достаточно установить настоящее приложение, загрузить небольшой исправитель и запустить его, после чего тот внесёт изменения в подлинный исполняемый файл.

Cheat Engine

Cheat Engine - популярный и мощный сканер памяти, шестнадцатеричный редактор и отладчик с открытым исходным кодом. Хотя этот инструмент главным образом применяется для мошенничества в компьютерных играх, он также часто бывает полезен для быстрого динамического анализа при взломе программ. Ссылки находятся в разделе инструментов нашего репозитория GitHub, приведённого выше.

Cheat Engine позволяет искать введённые пользователем значения, используя широкий набор параметров. Они дают возможность находить нужные данные в памяти компьютера и отбирать их.

Открытие процесса в Cheat Engine

В отличие от других инструментов, обратная разработка с помощью Cheat Engine начинается не с открытия исполняемого файла. Вместо этого вы выбираете работающий процесс, который требуется изменить.

Сначала запустите программу, которую хотите взломать. Затем запустите Cheat Engine и щёлкните **Select A Process To Open**, как показано на рис. 11.9. Появится окно **Process List**, в котором можно выбрать взламываемый процесс и нажать **Open**.

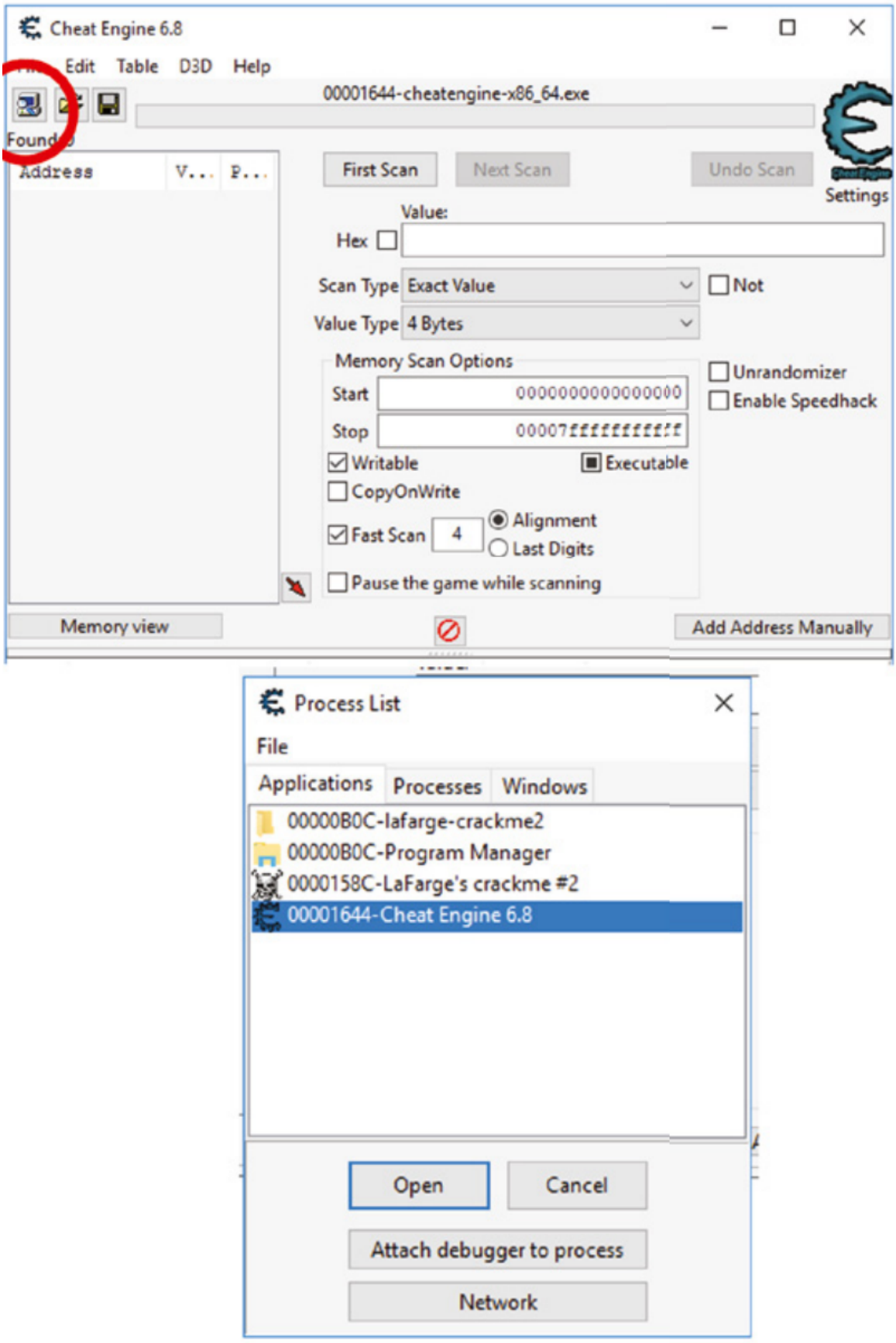


Рис. 11.9. Открытие процесса в Cheat Engine

Просмотр памяти в Cheat Engine

Работа Cheat Engine в значительной степени основана на сканировании памяти.

Главное окно Cheat Engine прежде всего предназначено для сканирования памяти. Однако пока сосредоточимся на более простой функции - просмотре памяти. Чтобы просмотреть память процесса, щёлкните **Memory View**, как показано на рис. 11.10.

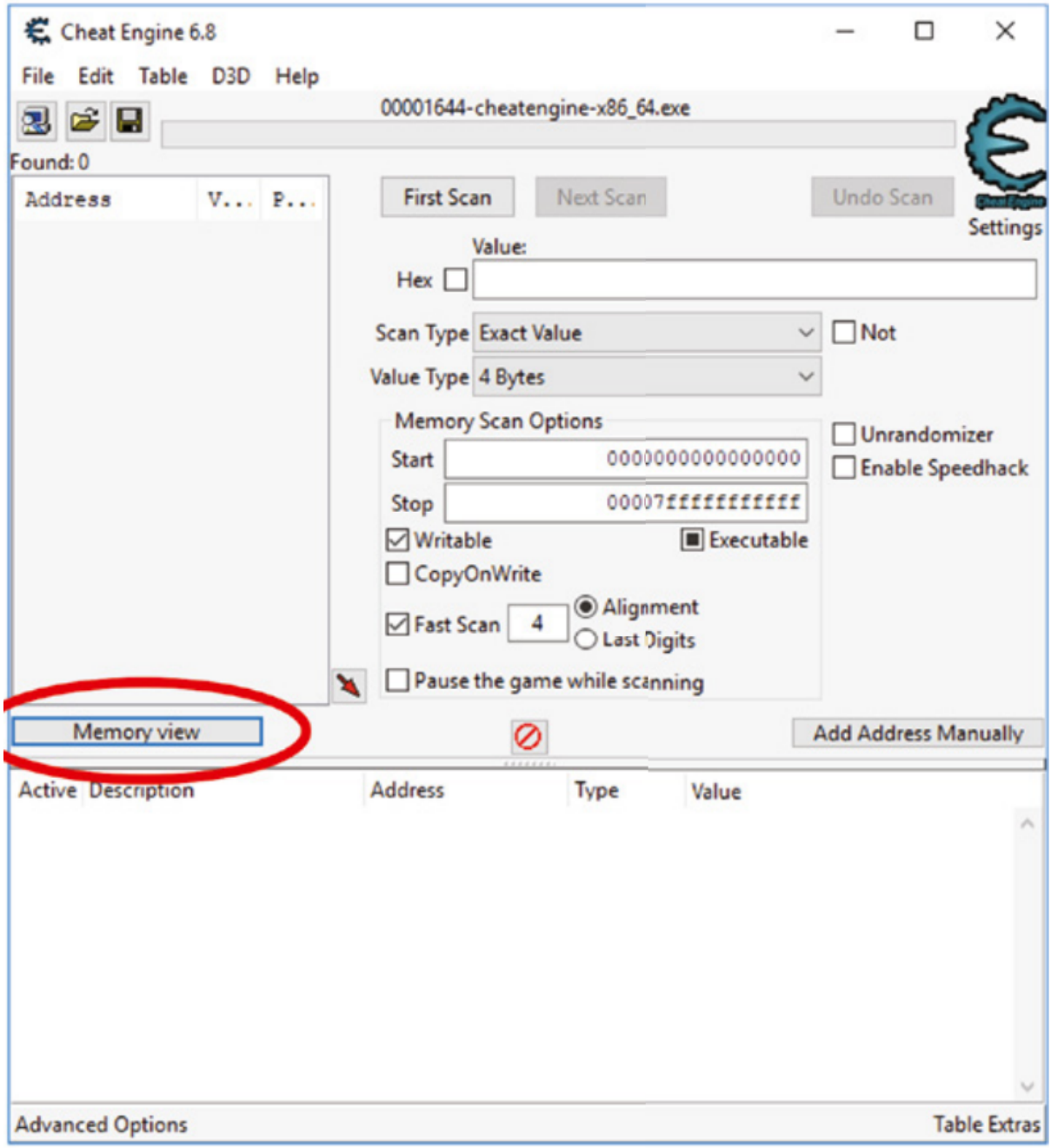


Рис. 11.10. Просмотр памяти в Cheat Engine

Средство просмотра памяти позволяет легко и эффективно просматривать, сканировать и изменять память процесса. Как показано на рис. 11.11, в верхней части окна находится дизассемблированный код, посередине - справка по инструкции, а внизу - шестнадцатеричный дамп.

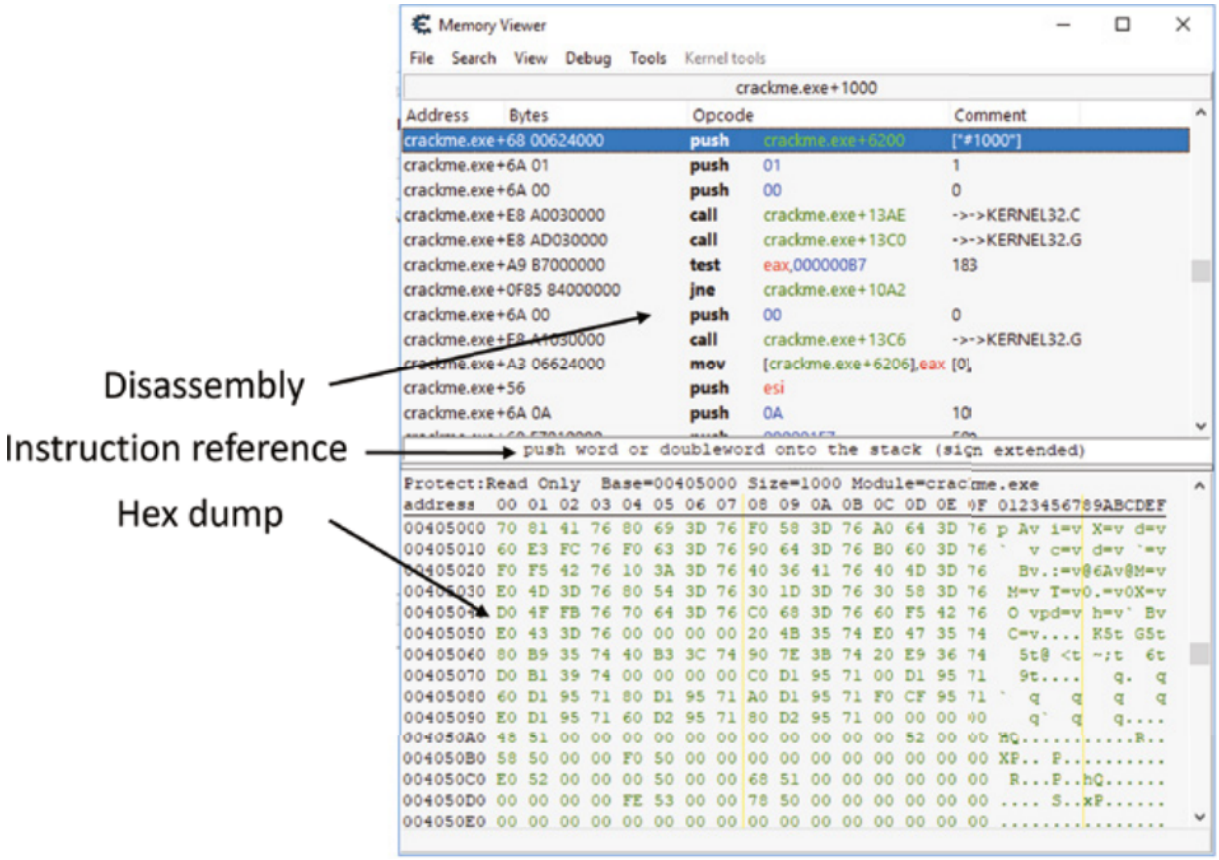


Рис. 11.11. Панель Memory Viewer в Cheat Engine

Ссылки на строки в Cheat Engine

Как уже говорилось, изучение строк в исполняемом файле может дать бесценные подсказки о его функциях. Чтобы просмотреть строки в Cheat Engine, выберите **View > Referenced Strings** и получите список всех строк, используемых программой.

На рис. 11.12 показано открывшееся окно. В нём можно щёлкнуть строку, чтобы увидеть перекрёстные ссылки на неё. Дважды щёлкните адрес перекрёстной ссылки, чтобы перейти к месту использования этой строки в дизассемблированном коде.

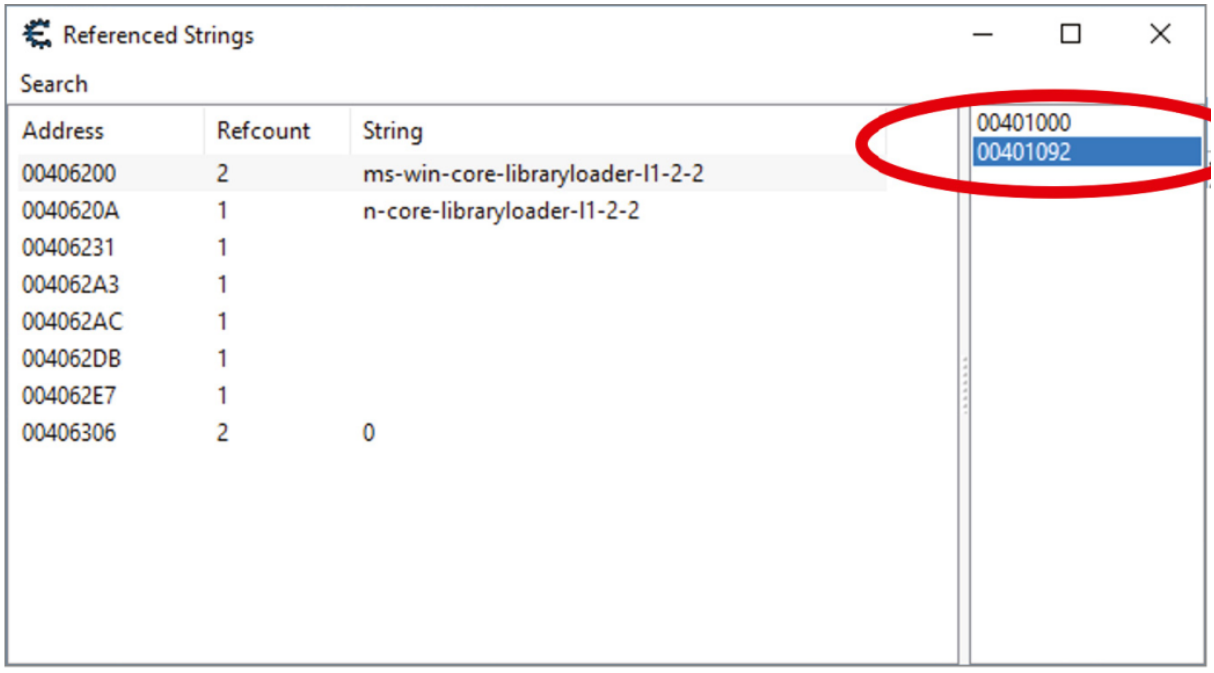


Рис. 11.12. Ссылки на строки в Cheat Engine

Переписывание программ в Cheat Engine

Напомним, что замена фрагмента кода инструкциями nop - самый безопасный и простой способ удалить его, не затрагивая остальную программу. Cheat Engine упрощает эту операцию. Чтобы обойти инструкцию, например последний условный переход при проверке ключа, щёлкните её правой кнопкой и выберите **Replace With Code That Does Nothing**, как показано на рис. 11.13.

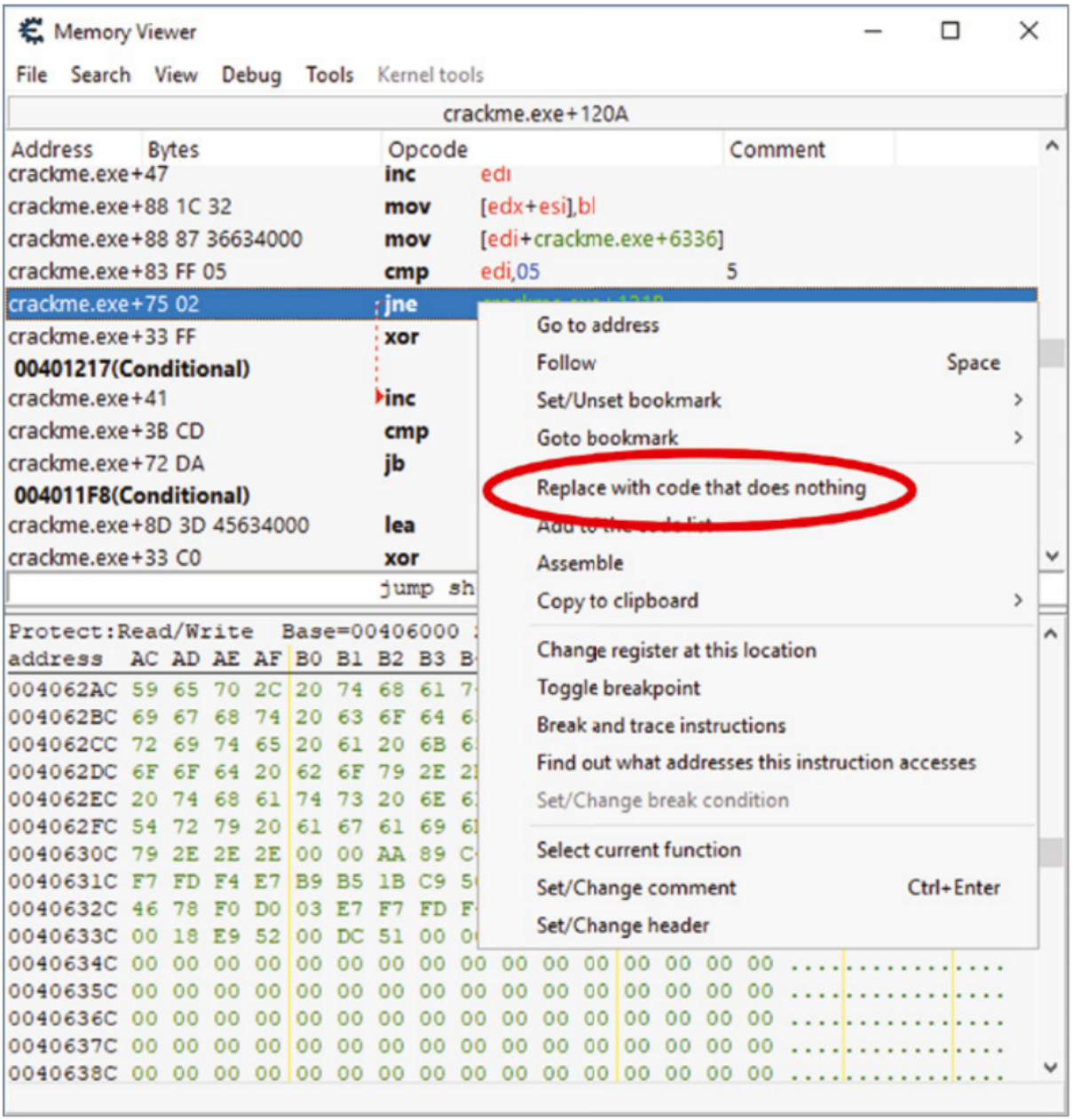


Рис. 11.13. Замена инструкций на nop в Cheat Engine

Cheat Engine чрезвычайно интерактивен. Внесённое изменение можно немедленно испытать в работающей программе. Если оно не сработало или вы хотите его отменить, щёлкните изменённый код правой кнопкой и выберите **Restore With Original Code**, как показано на рис. 11.14.

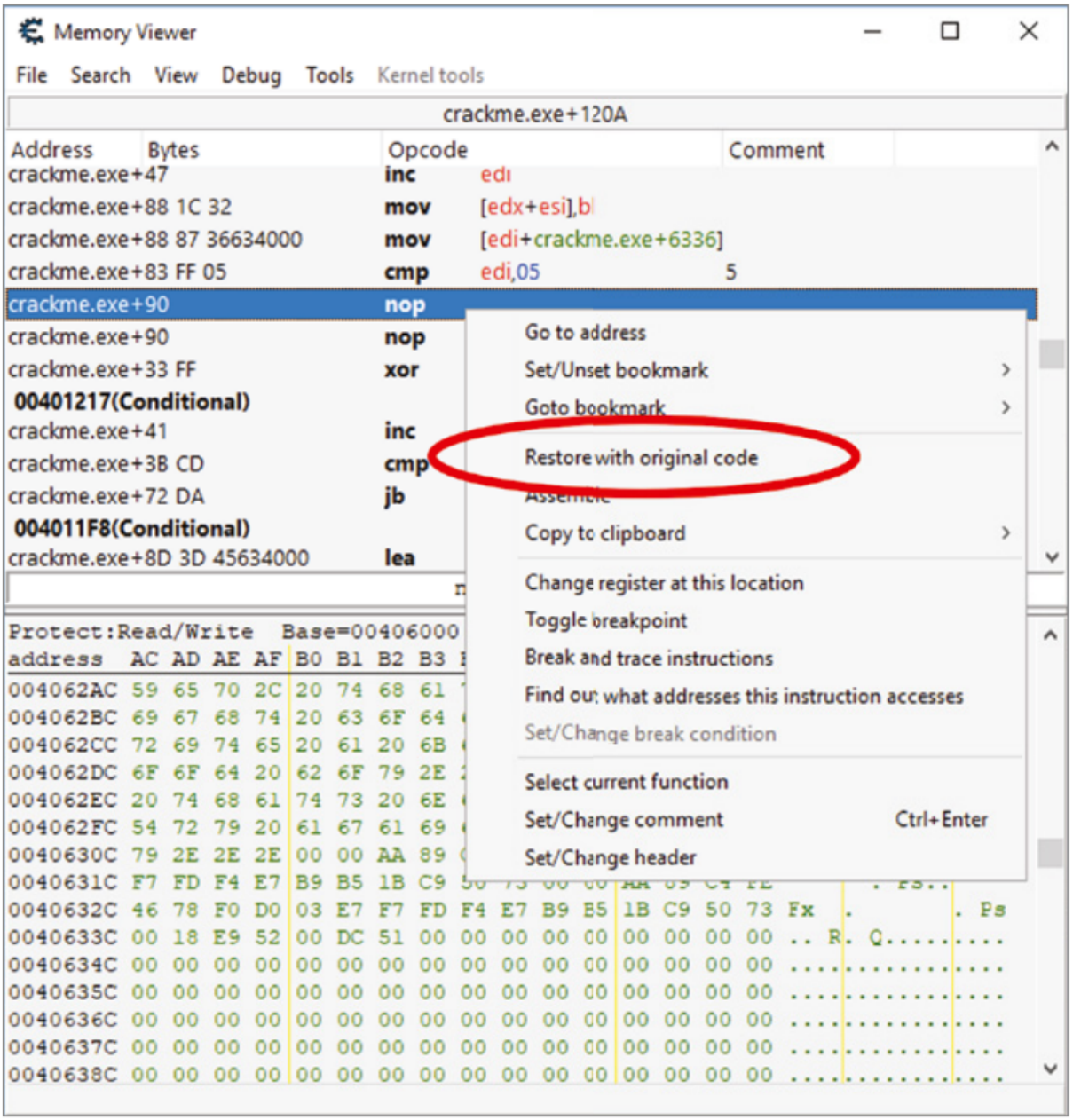


Рис. 11.14. Отмена изменений в Cheat Engine

Копирование байтов в Cheat Engine

Найдя работающее исправление, следует перенести его из работающего процесса в исполняемый файл. Как показано на рис. 11.15, щёлкните место исправления правой кнопкой и выберите **Copy To Clipboard > Bytes Only**, чтобы скопировать эти байты для использования в других инструментах.

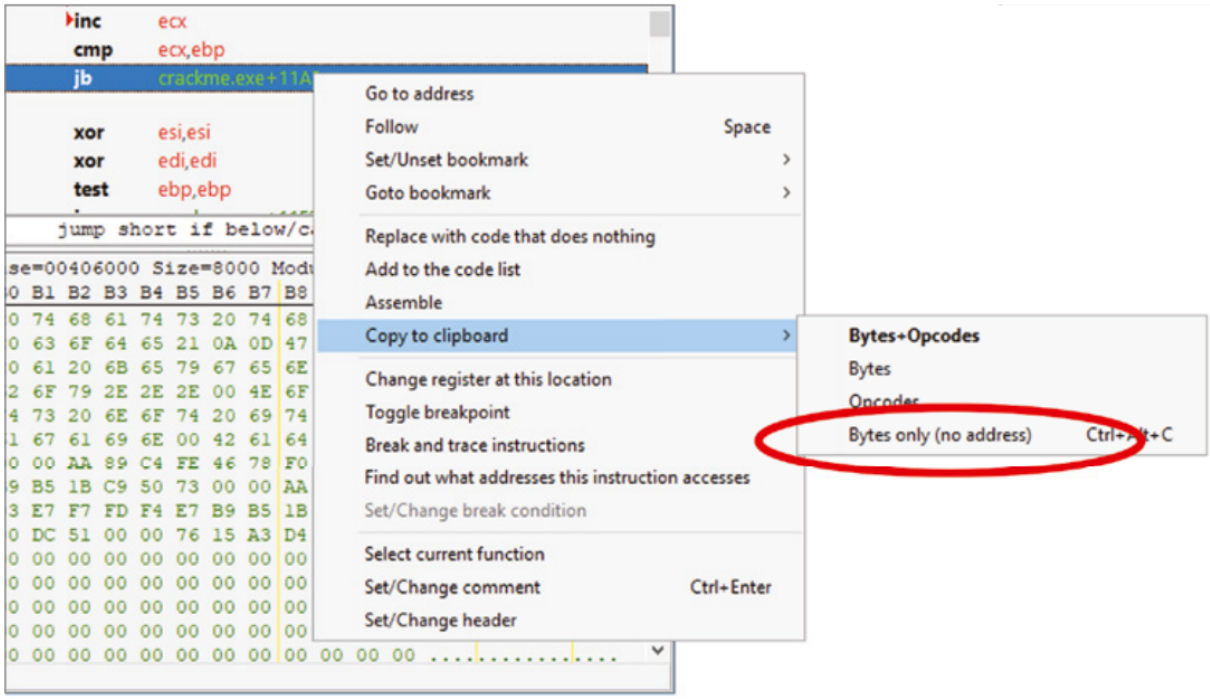


Рис. 11.15. Копирование байтов в Cheat Engine

Получение адресов в Cheat Engine

Чтобы создать исправление, необходимо знать, где именно в файле находятся изменяемые данные. Cheat Engine ориентирован на анализ во время выполнения, поэтому ему неизвестно, где эти данные расположены в файле.

Чтобы найти адрес, воспользуйтесь 010 Editor и выполните поиск заменяемого машинного кода. Найденный адрес является смещением в файле, которое следует указать в CodeFusion или другом исправителе.

Лабораторная работа: взлом LaFarge

В этой лабораторной работе отрабатывается применение рассмотренных инструментов для исправления программ. Лабораторные работы и все связанные инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой работы найдите **Lab LaFarge** и следуйте предоставленным инструкциям.

Навыки

В этой лабораторной работе CodeFusion и Cheat Engine используются для отработки следующих навыков:

• обратная разработка x86;
• исправление кода и создание исправителей.

Основные выводы

Для обратной разработки и взлома существует множество инструментов; выбор подходящего зависит от решаемой задачи и личных предпочтений.

Программы для взлома, или crackme, обычно безопасны, всегда законны и невероятно увлекательны как средство практики навыков взлома.

Знакомство с IDA

Если вы когда-либо искали в Google инструменты для обратной разработки, то наверняка встречали IDA. Это Cadillac среди средств обратной разработки.

IDA, также известный как Interactive Disassembler, позволяет визуально представлять дизассемблированный двоичный код. Он распространяется по условно-бесплатной модели: ограниченный набор функций доступен бесплатно, а некоторые более мощные возможности или поддержка менее распространённых архитектур требуют платной лицензии.

На рис. 11.16 показана загрузка нового файла в IDA. IDA автоматически распознаёт многие распространённые форматы файлов, но если формат определён неверно, можно выбрать универсальный тип **Binary File**. В IDA также имеется раскрывающееся меню **Processor Type** для выбора другой архитектуры.

Одна из важнейших сильных сторон IDA - графическое представление, на котором наглядно показаны ассемблерный код x86 и потоки управления исполняемого файла. На рис. 11.17 показан этот режим и некоторые из его наиболее полезных компонентов: карта памяти исполняемого файла, список функций, представление логических блоков и окно графа.

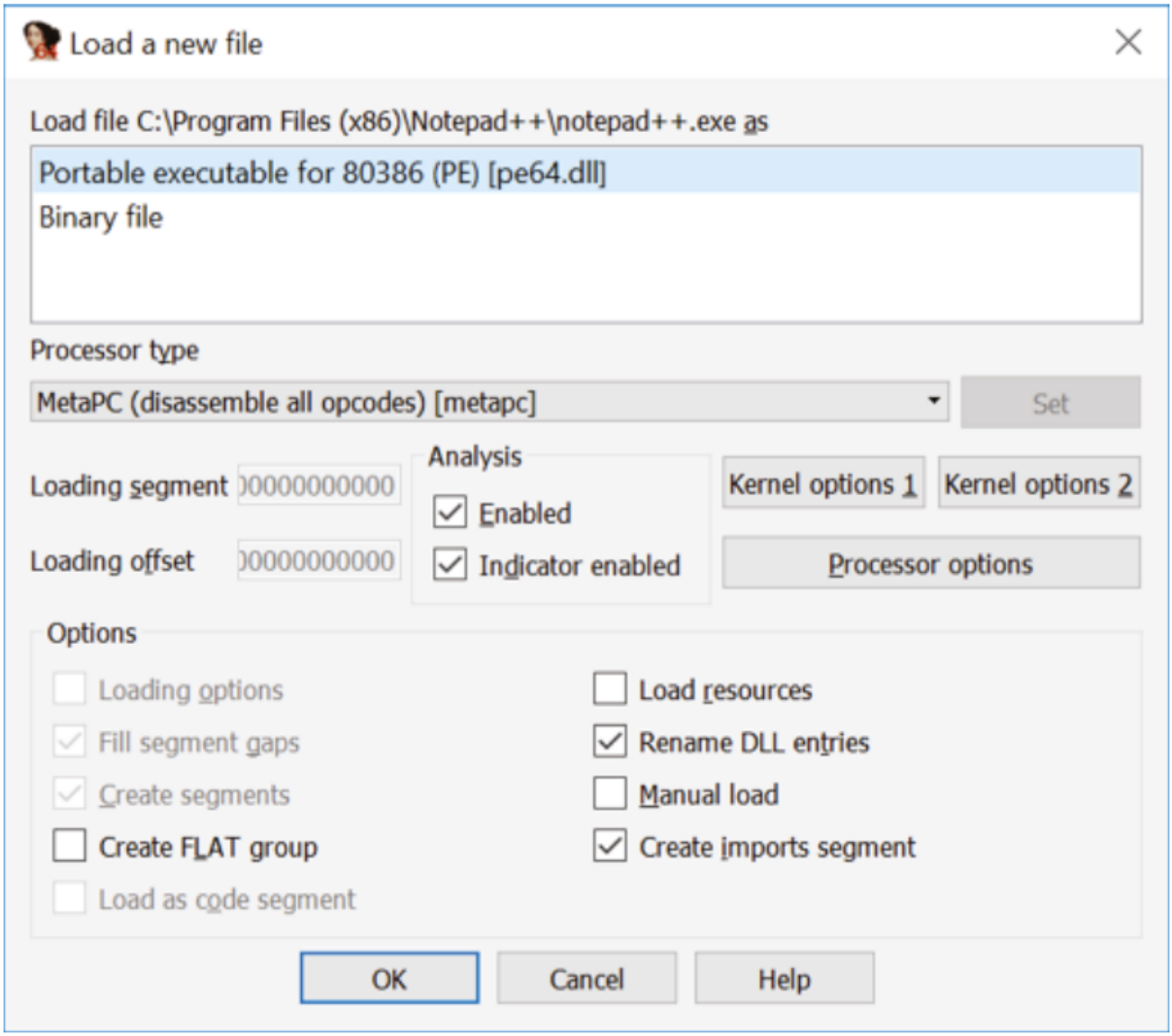


Рис. 11.16. Загрузка файла в IDA

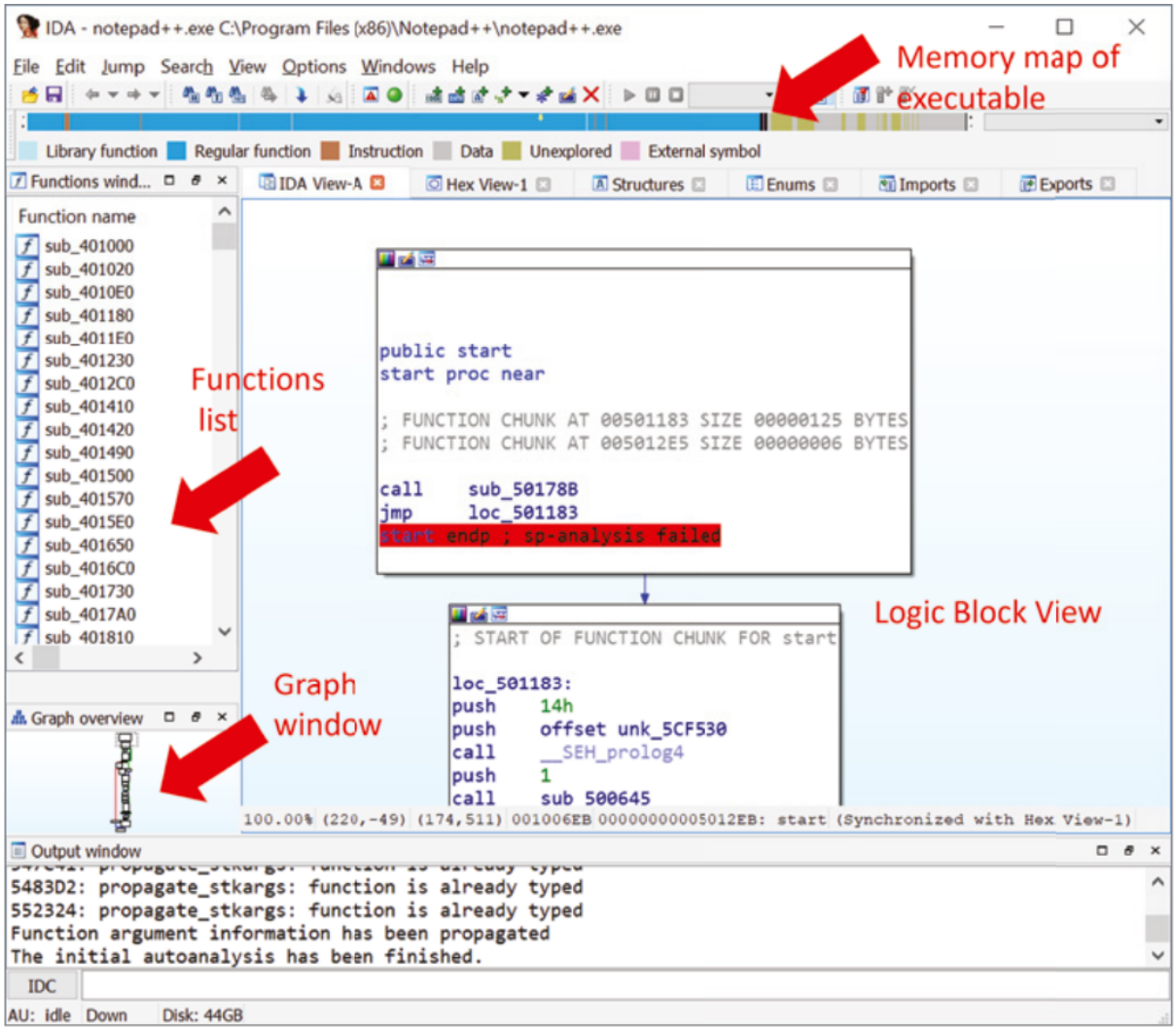


Рис. 11.17. Графическое представление IDA

Строки в IDA

Как всегда, строки служат хорошей отправной точкой при анализе нового исполняемого файла. Однако по умолчанию IDA их не показывает. На рис. 11.18 показано, как открыть представление строк, выбрав **View > Open Subviews > Strings**.

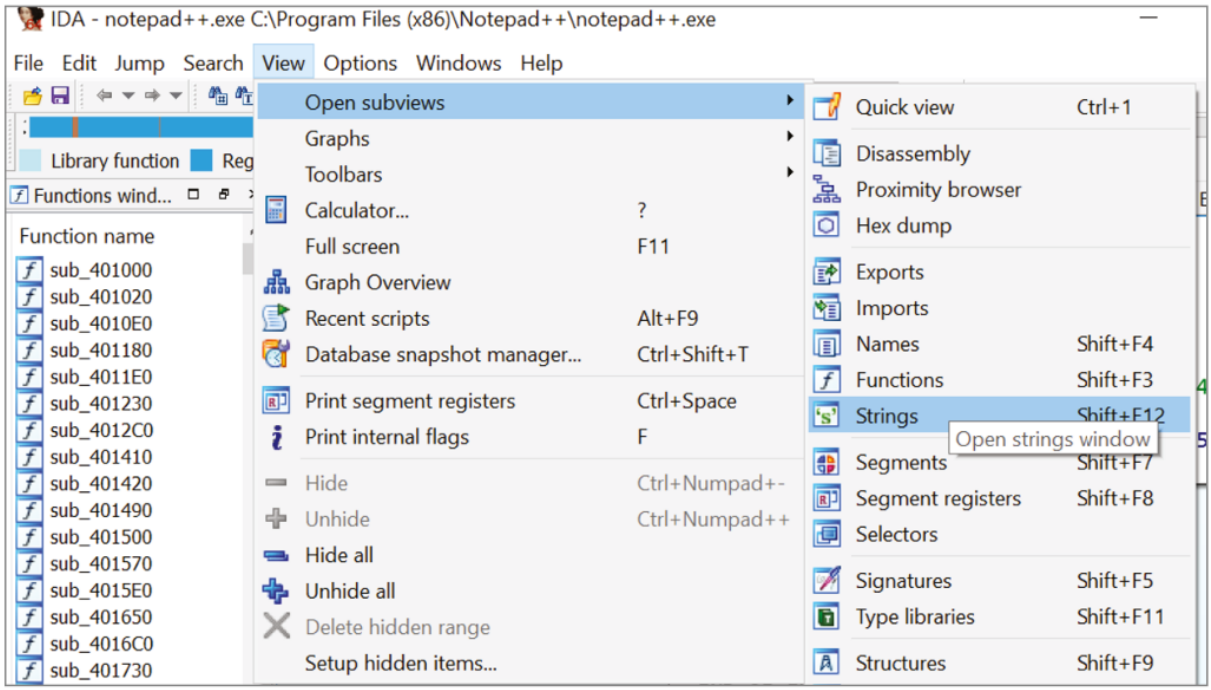


Рис. 11.18. Открытие представления строк в IDA

На рис. 11.19 показан полный список строк в IDA. Для каждой строки IDA отображает собственно текст, адрес и предполагаемую длину.

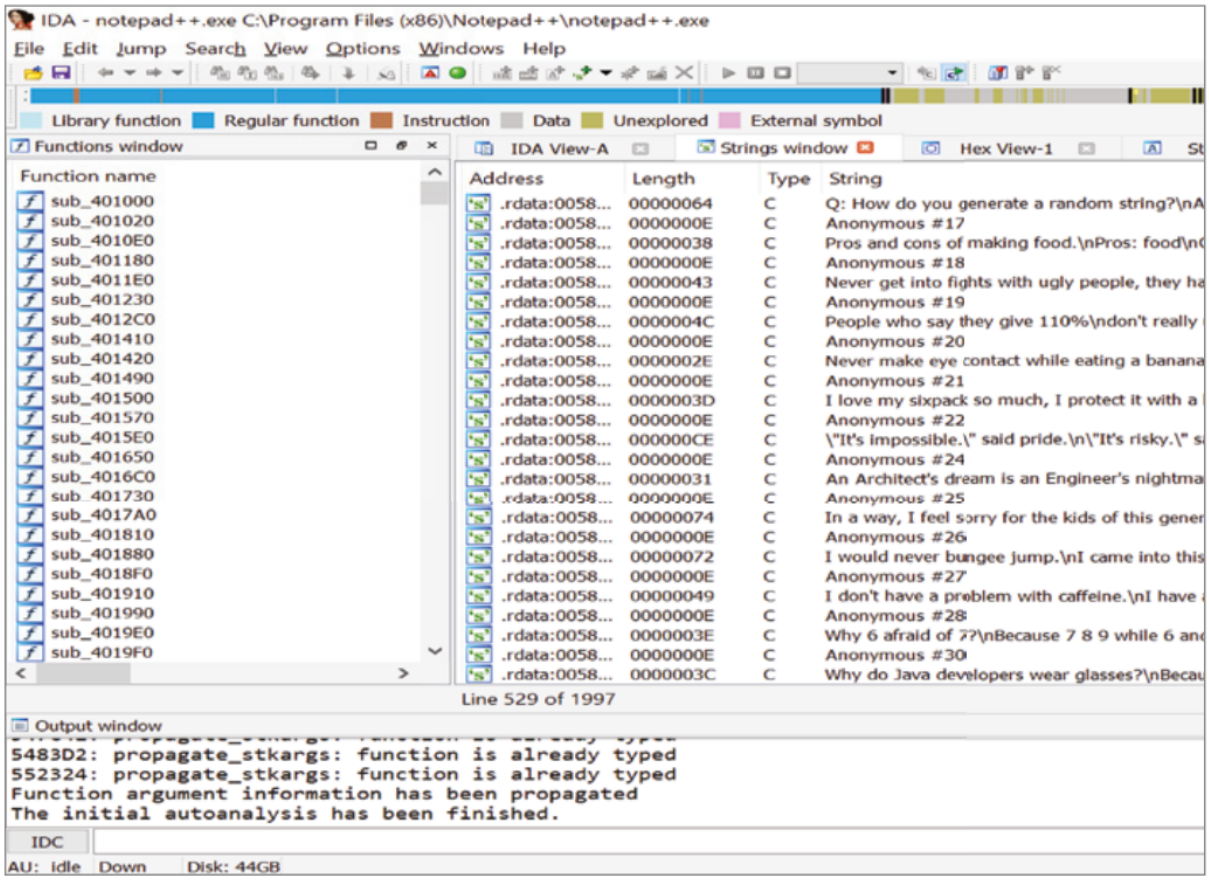


Рис. 11.19. Представление строк в IDA

Щёлкните строку, чтобы выделить её. Затем нажмите **X** либо щёлкните правой кнопкой и выберите **Jump To Xref To Operand**. Откроется окно со всеми местами программы, где используется эта

строка, как показано на рис. 11.20.

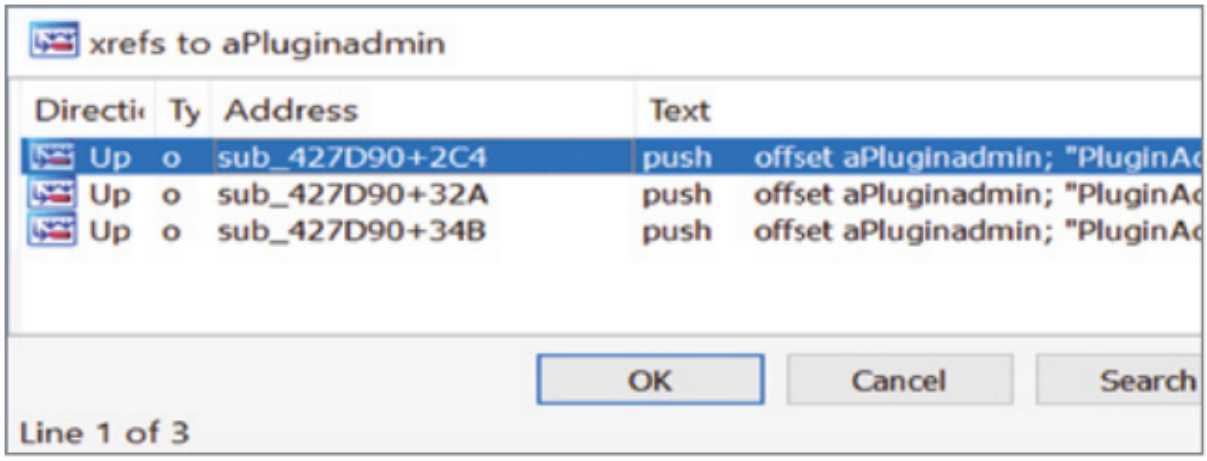


Рис. 11.20. Перекрёстные ссылки на строку в IDA

Переход по одной из перекрёстных ссылок приведёт к дизассемблированному коду, где используется строка. Как показано на рис. 11.21, IDA понимает принцип работы ссылок на строки. Обнаружив такую ссылку, IDA показывает строку в виде комментария.

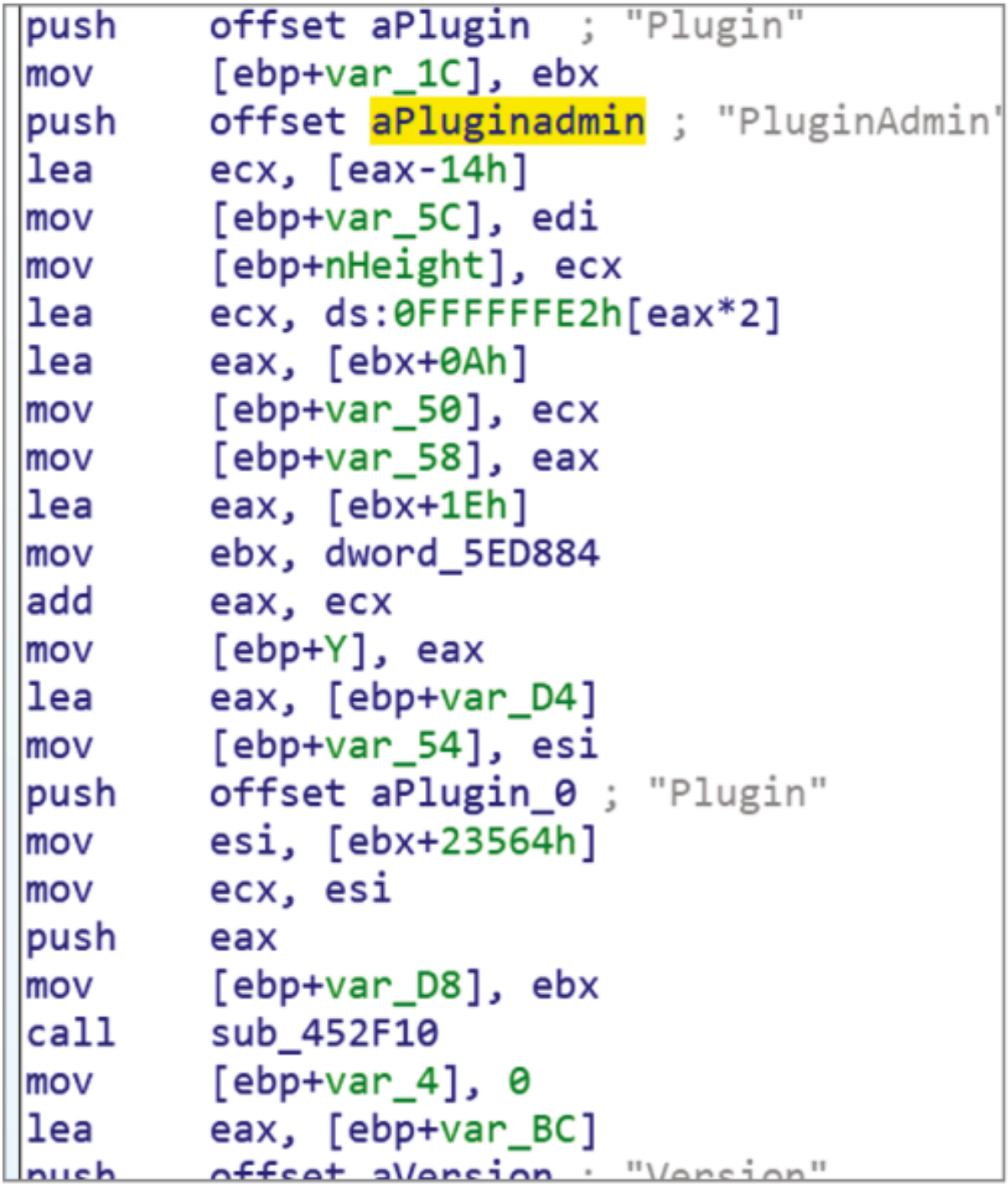


Рис. 11.21. Строки в представлении кода IDA

Базовые блоки в IDA

Графическое представление IDA показывает код в виде базовых блоков. Базовый блок - непрерывная последовательность инструкций, не прерываемая инструкцией ветвления или ссылкой ветвления.

Рассмотрим следующую простую программу в псевдокоде. На рис. 11.22 показано, как выглядит эта программа после дизассемблирования в IDA.

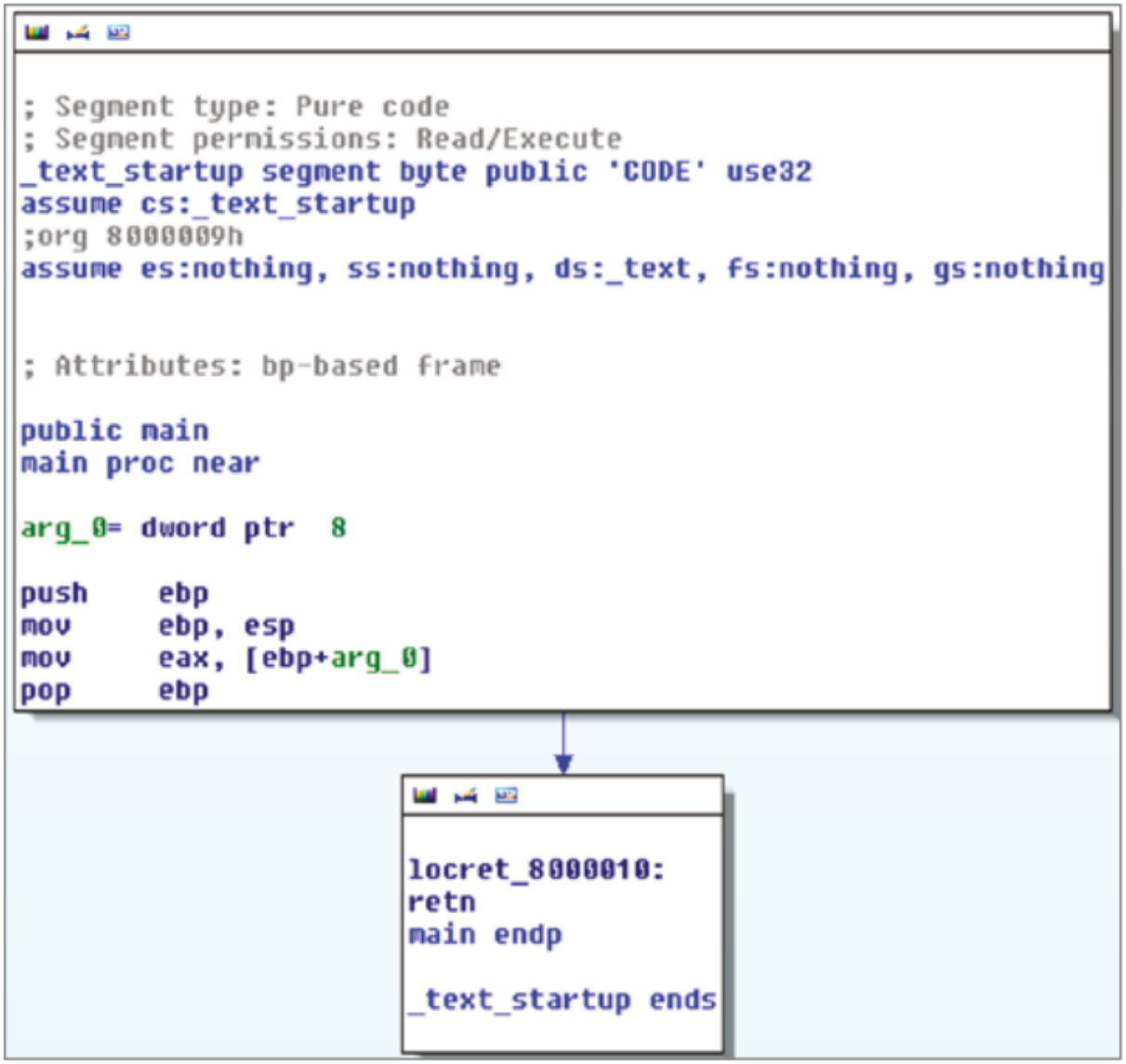


Рис. 11.22. Базовые блоки в IDA

```
int main(int argc, char* argv[])
{
    return argc;
}
```

Функции и переменные в IDA

IDA понимает множество соглашений о вызовах, включая `cdecl`. Она распознаёт `cdecl` и знает, что первый аргумент всегда начинается по адресу `ebp+8`. Чтобы упростить чтение, IDA переименовывает это смещение в `arg_0`. Аналогичным образом переименовываются все входные переменные, получающие имена `arg_X`, как показано на рис. 11.23.

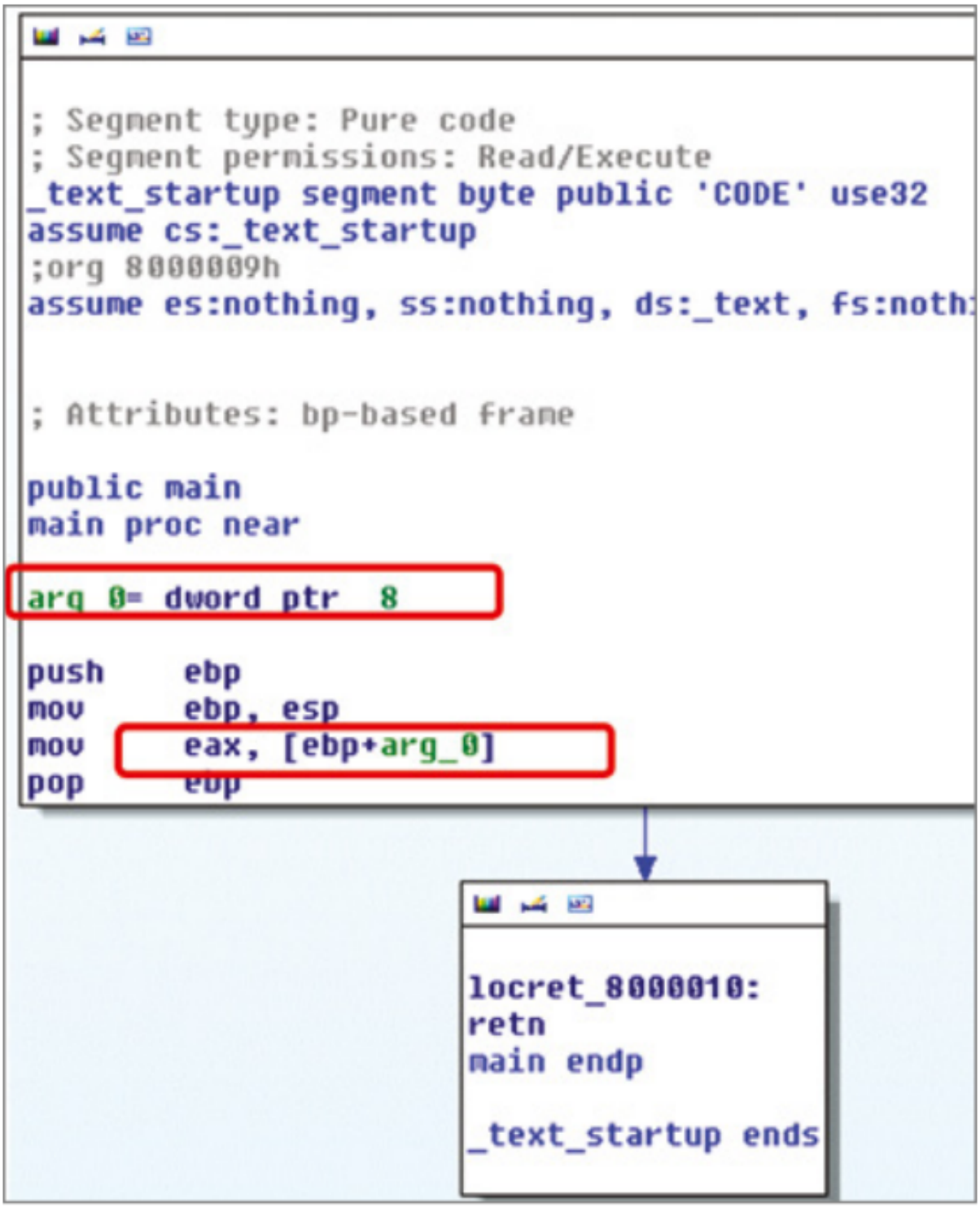


Рис. 11.23. Аргументы функции в IDA

Это понимание распространяется и на обработку локальных переменных в стеке. Например, как показано на рис. 11.24, IDA переименовывает локальные переменные в `var_X`.

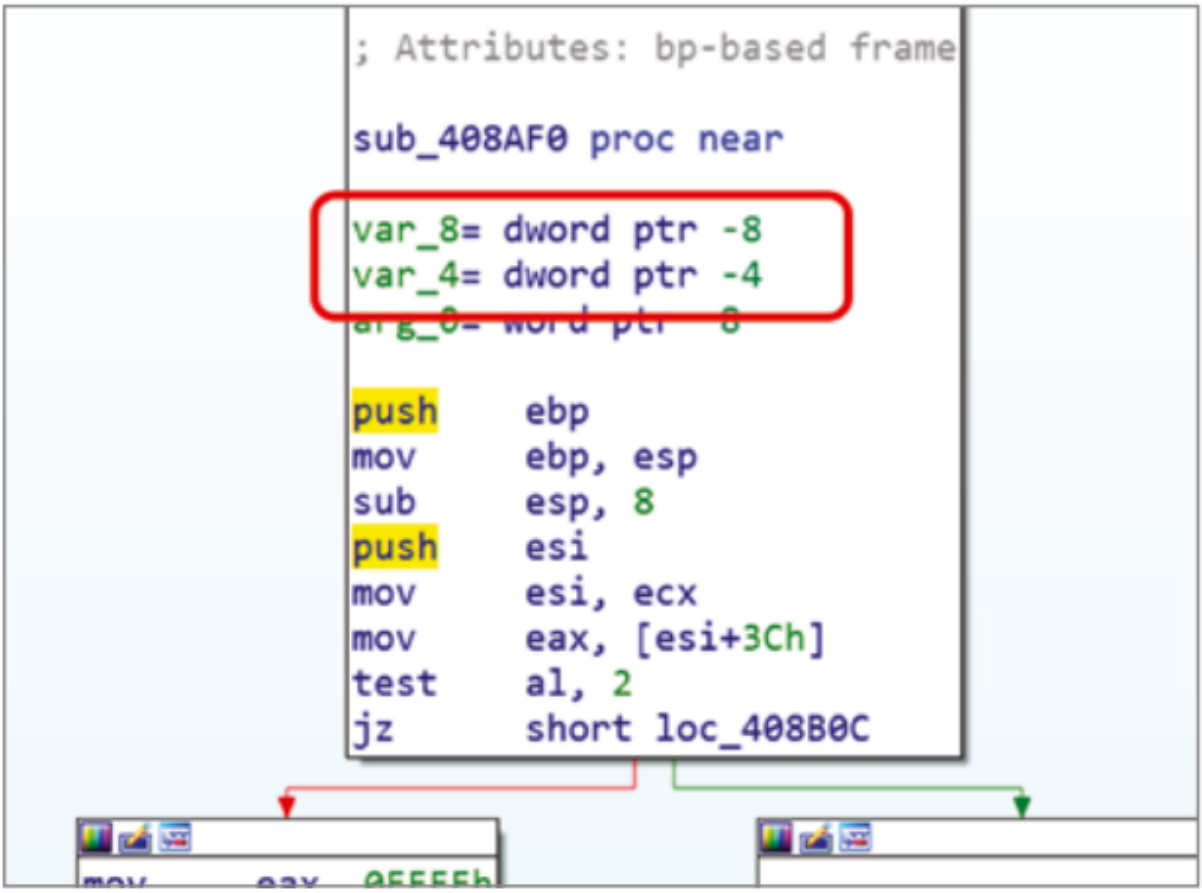


Рис. 11.24. Локальные переменные в IDA

Знание того, как IDA обозначает аргументы и переменные, может значительно помочь при анализе функций. Например, по функции на рис. 11.25 можно очень быстро определить, что у неё есть одна локальная и шесть входных переменных, поскольку нам известны правила именования IDA.

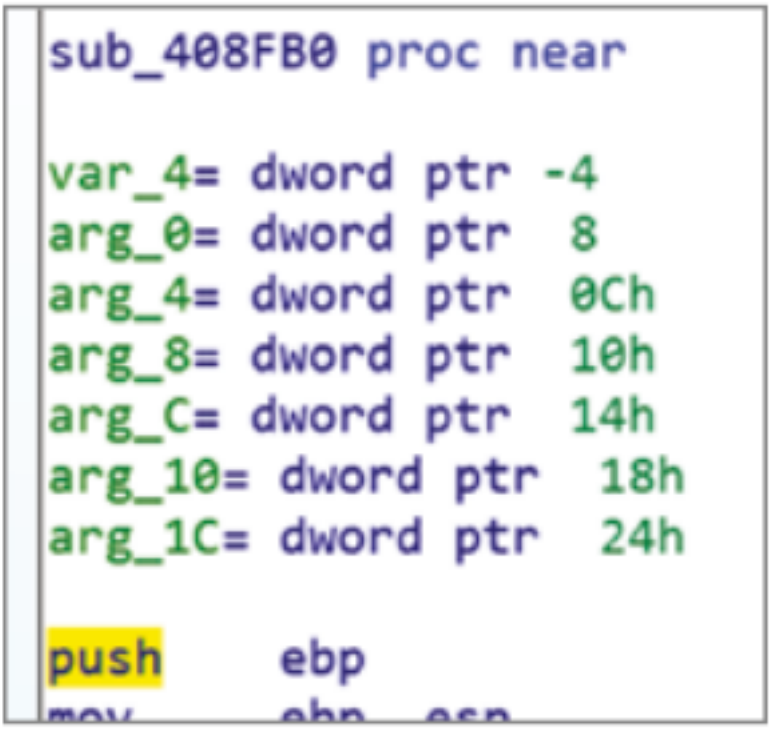


Рис. 11.25. Локальные переменные и аргументы функции в IDA

Часто IDA не располагает сведениями о назначении или контексте использования переменных, поэтому присваивает им последовательные имена. Узнав назначение аргумента, переменной или функции, вы можете переименовать объект, нажав **N** либо щёлкнув его метку правой кнопкой и выбрав **Rename**.

Комментарии в IDA

При обратной разработке приложения крайне важно отслеживать, что уже удалось выяснить и сделать. В IDA нажатие **;** открывает поле для ввода комментария, как показано на рис. 11.26.

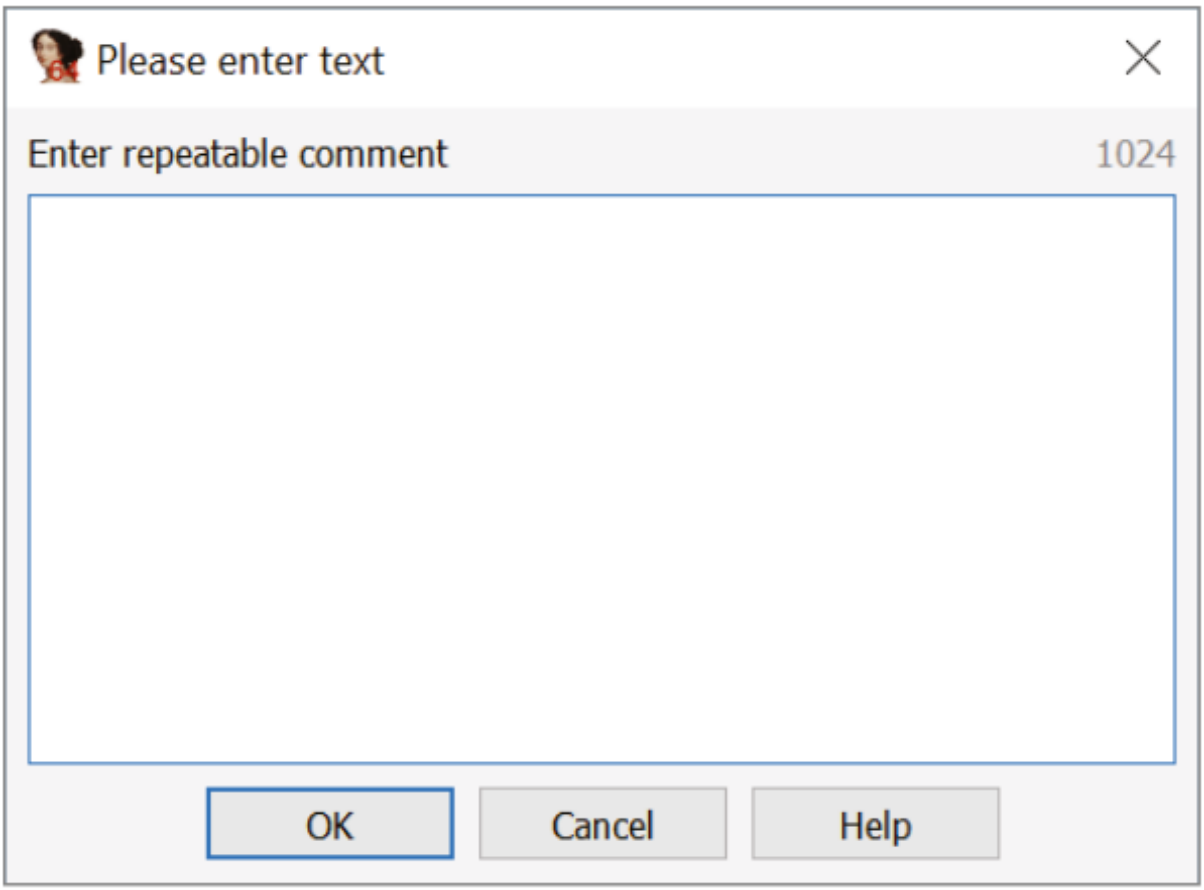


Рис. 11.26. Окно комментария IDA

Полезный приём - добавлять во все комментарии такой идентификатор, как _x. Тогда по нему можно найти все комментарии.

Чтобы начать поиск комментариев, выберите **Search > Text**, как показано на рис. 11.27. Затем выполните поиск _x, включив параметр **Find All Occurrences**, чтобы найти все добавленные в программу комментарии.

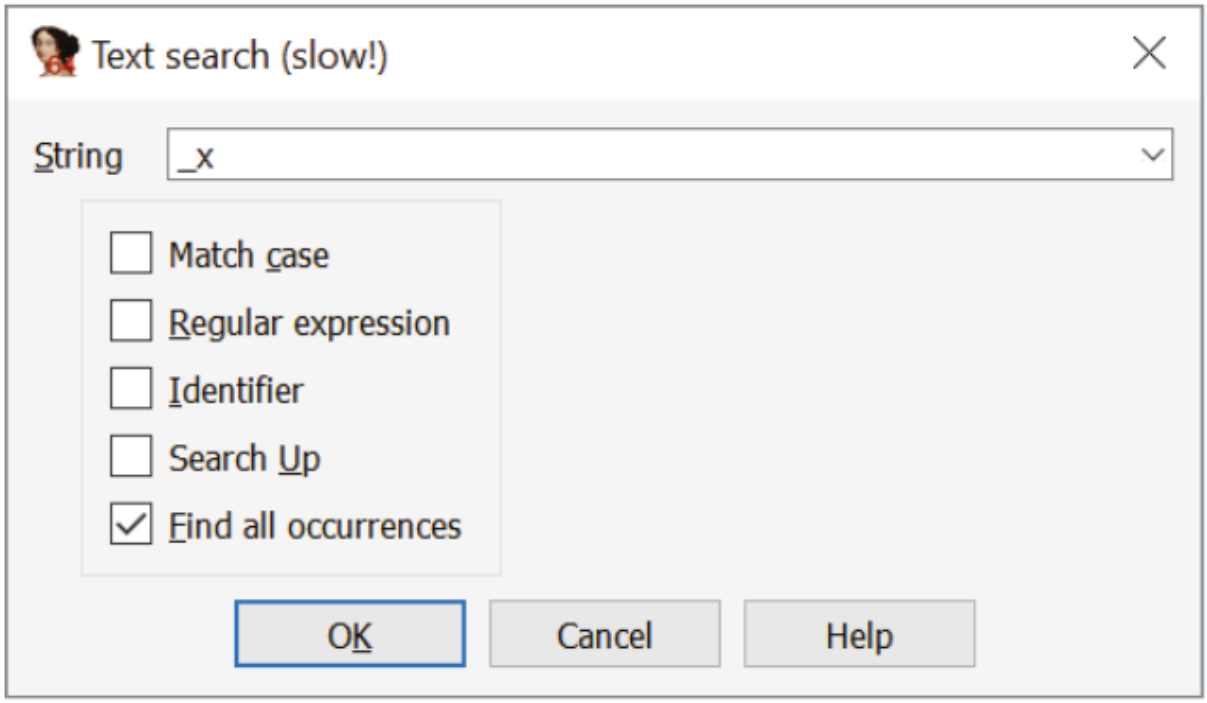


Рис. 11.27. Поиск комментариев в IDA

Единообразное оформление и поиск комментариев позволяют легко находить уже исследованные участки кода. Например, как показано на рис. 11.28, можно быстро обнаружить места, помеченные `TODO` для последующего анализа.

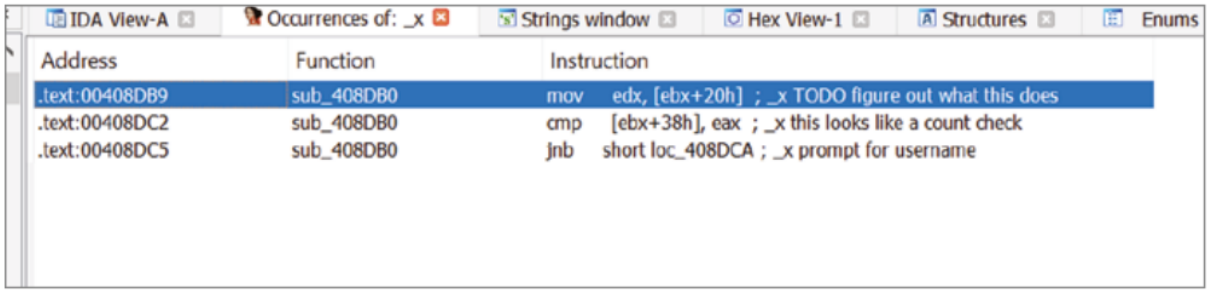


Рис. 11.28. Результаты поиска в IDA

Пути в IDA

IDA показывает три вида путей между базовыми блоками:

- **красный:** путь, выбираемый, если условный переход не выполняется;
- **зелёный:** путь, выбираемый, если условный переход выполняется;
- **синий:** гарантированный путь без условий.

В качестве примера рассмотрим следующий фрагмент кода с простым оператором `if`:

```
int main(int argc, char* argv[])
{
    if (argc > 1)
        return 0;

    return argc;
}
```

На рис. 11.29 показано, как этот код выглядит в IDA. После условного блока пути расходятся. В этой книге цвета не отображаются, но левый путь, который в IDA имеет красный цвет, показывает происходящее, если переход не выполняется. Правый путь, имеющий в IDA зелёный цвет, выбирается, если условное выражение даёт ложный результат.

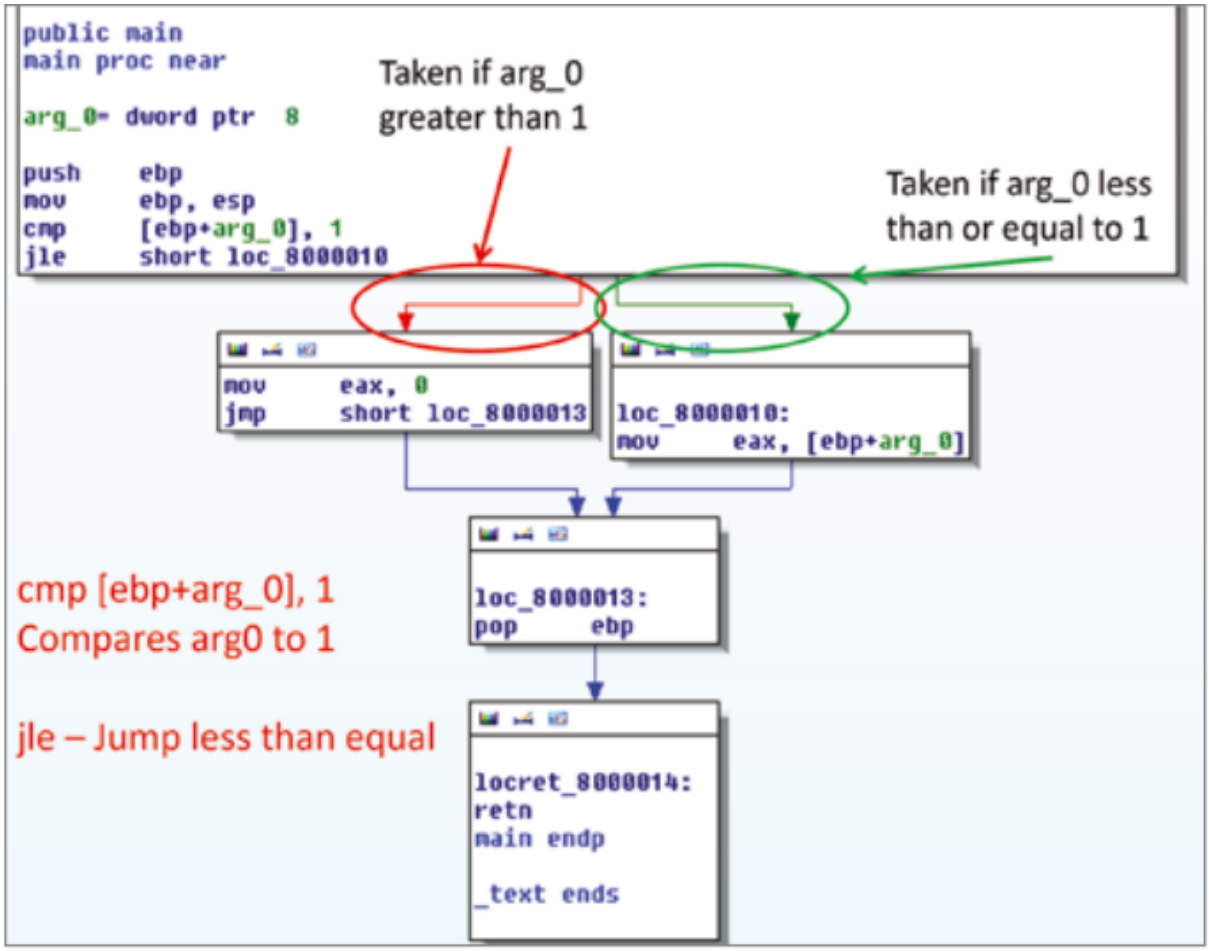


Рис. 11.29. Пути кода в IDA

Ниже ещё несколько стрелок обозначают переходы между базовыми блоками. Поскольку ни один из них не связан с условием, в IDA все они будут синими.

Исправление в IDA

IDA - ещё один инструмент, с помощью которого можно исправлять исполняемые файлы. В качестве примера рассмотрим следующий код:

```
printf("please enter the password\n");
scanf("%s", user_entered_password);
if (strcmp(user_entered_password, correct_password) == 0)
{
    printf("SUCCESS\n");
}
else
{
    printf("Failure\n");
}
```

Этот код реализует простую систему аутентификации. Пользователю предлагается ввести пароль, после чего ответ проверяется. Если ответ правильный, программа выводит SUCCESS; в противном

случае она выводит `Failure`. Хотя пример упрощён, следует помнить, что такой поток с проверкой пароля и выбором одного пути при неверном результате и другого при верном встречается очень часто. В IDA приложение можно исправить так, чтобы нейтрализовать эту проверку пароля.

По умолчанию IDA не показывает машинный код в графическом представлении. Если вы не вносите исправления, он не особенно нужен. Но для исправления его потребуется увидеть. Чтобы показать машинный код, выберите **Options > General** и откройте окно, показанное на рис. 11.30. Затем задайте число байтов кода операции, отображаемых в графическом представлении. Большинство кодов операций не превышает 8 байт, поэтому рекомендуется указать значение 8.

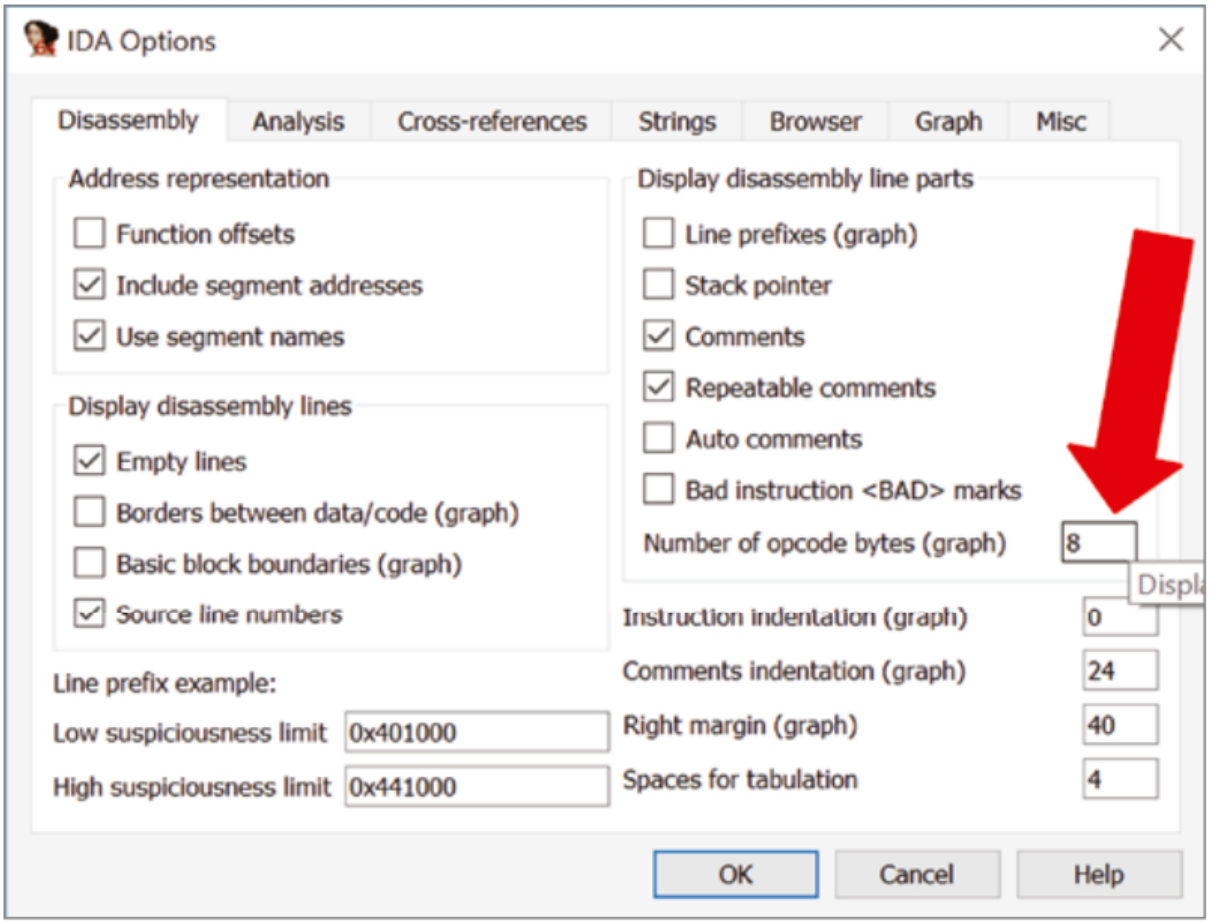


Рис. 11.30. Отображение байтов кода операции в IDA

На рис. 11.31 показана логика проверки пароля приложения в IDA. Как видно, левый, красный путь выбирается, если пароли совпадают, а правый, зелёный - если они не совпадают.

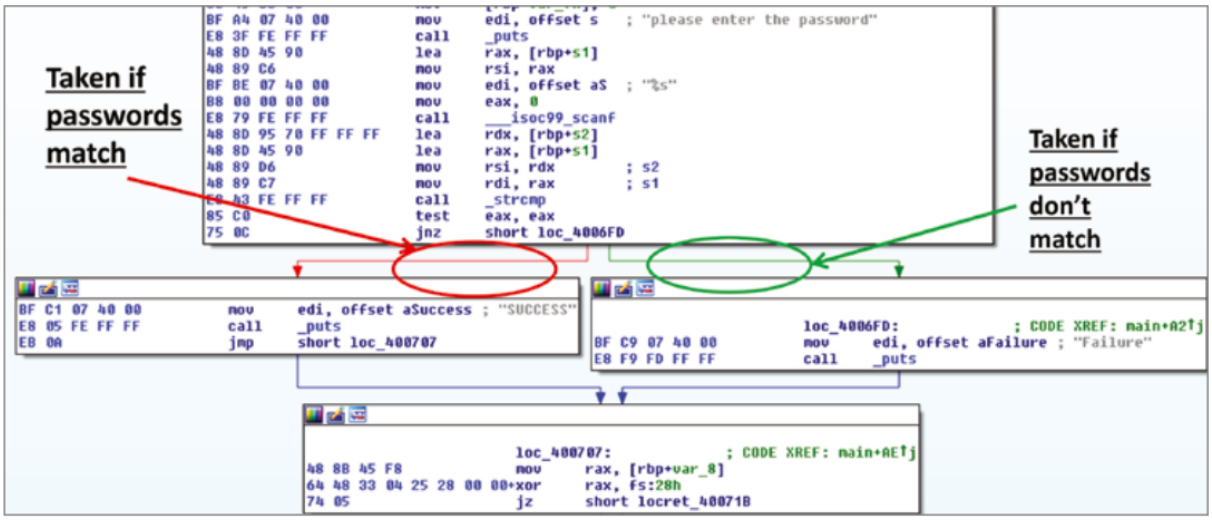


Рис. 11.31. Код проверки пароля в IDA

Инструкция, определяющая выбор пути, - `jnz`. Напомним, что `jnz` означает "jump not zero", то есть "переход, если не ноль".

Эту проверку пароля можно нейтрализовать несколькими способами. Один из вариантов - выяснить, что именно должно быть "не нулём". Для этого требуется определить два сравниваемых значения, чтобы затем, возможно, создать действительный ключ или взломщик.

Более простой вариант - использовать знания x86 и исправить приложение. В текущем виде приложение проверяет условие и выполняет `jnz` (`0x75`), если пароль неверен. Но что произойдёт, если поступить точно наоборот? Замена `jnz` на `jz` (`0x74`) инвертирует логику, и приложение начнёт принимать только неверные пароли. После инверсии неверный пароль приведёт к успеху, а верный - к неудаче.

Чтобы изменить инструкцию, выделите её и выберите **Edit > Patch Program > Change Byte**. Затем в окне **Patch Bytes**, показанном на рис. 11.32, измените первое значение с `74` на `75`.

> **Примечание переводчика.** В исходном тексте эти два указания противоречат друг другу: сначала предлагается заменить `jnz` (`0x75`) на `jz` (`0x74`), но следующая инструкция и рисунки 11.32-11.33 показывают замену `0x74` на `0x75`, то есть `jz` на `jnz`. Оба утверждения сохранены дословно по смыслу.

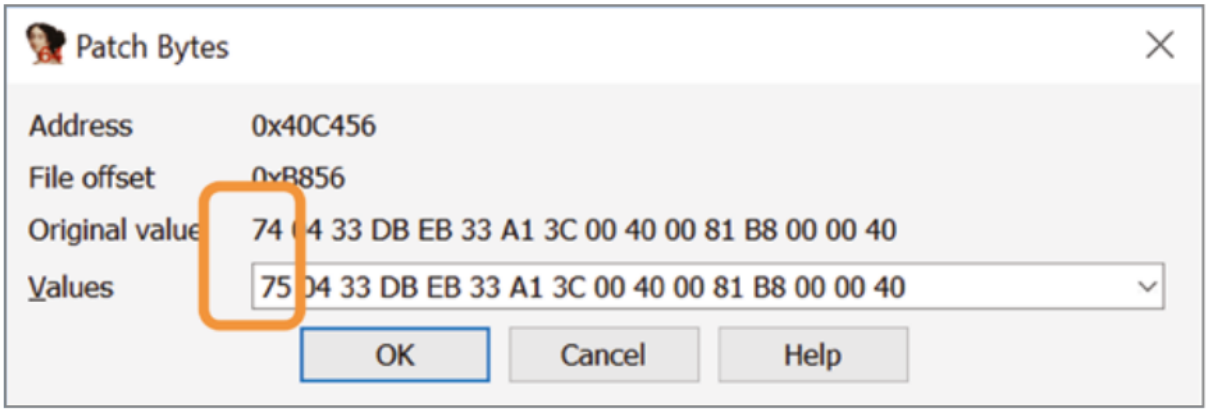


Рис. 11.32. Окно Patch Bytes в IDA

На рис. 11.33 показано, как будет выглядеть приложение после применения исправления. Единственный изменённый бит будет выделен в IDA, а смысл двух путей после перехода поменяется местами. Теперь приложение будет работать с любым паролем, кроме правильного.

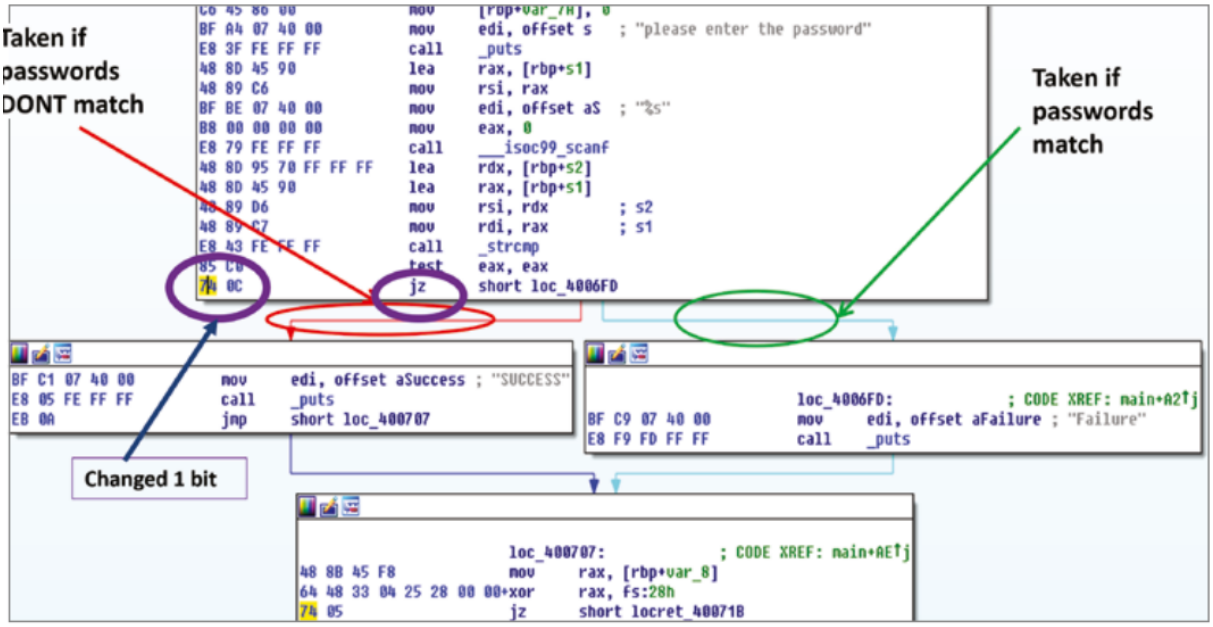


Рис. 11.33. Логика проверки пароля в IDA после исправления

Лабораторная работа: логические потоки в IDA

Эта лабораторная работа знакомит с применением IDA для обратной разработки. Её файлы находятся на рабочем столе виртуальной машины Windows в папке `ida_logic`. Внутри имеется несколько двоичных файлов. Определите, какой из них содержит:

- оператор `if`;
- составной оператор `if`, например `if (cond1 && cond2)`;
- цикл `while`;
- цикл `for`;
- цикл `do while`.

Навыки

В этой лабораторной работе отрабатывается применение IDA для обратной разработки графов потока управления. Цель - научиться быстро распознавать высокоуровневые конструкции языка по характерным схемам потока управления.

Основные выводы

Анализ потока управления программы помогает быстро понять происходящее внутри кода. Умение сразу распознавать такие потоки может значительно повысить эффективность обратной разработки.

Ghidra

Ghidra - инструмент статического анализа, выпущенный АНБ в 2019 году. Он во многом похож на IDA, но, в отличие от IDA, является бесплатным и имеет открытый исходный код. Во многих ситуациях Ghidra служит полноценной заменой IDA.

IDA имеет гораздо более долгую репутацию в этой области, однако Ghidra тоже чрезвычайно мощна и во многих случаях обладает теми же возможностями. В данном примере демонстрируется IDA из-за её долгой истории в сфере обратной разработки, но всё показанное можно выполнить и в Ghidra. Инструменты достаточно похожи, поэтому навыки работы с одним из них часто переносятся на другой. Попробуйте Ghidra в последующих лабораторных работах с открытыми заданиями и во время самостоятельной практики.

Лабораторная работа: взлом с помощью IDA

В этой лабораторной работе в IDA рассматривается более сложное приложение. Лабораторные работы и все связанные инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой работы найдите **Lab Cracking with IDA** и следуйте предоставленным инструкциям.

Навыки

В этой лабораторной работе отрабатывается применение IDA для взлома крупных реальных приложений. Цель - научиться быстро находить интересующие участки и определять приоритет нескольких возможных подходов к взлому.

Основные выводы

Реальные программы слишком велики для единообразного детального анализа. Предварительный отбор критически важен для нахождения интересующих участков.

Обычно у взломщика есть несколько возможностей. Выбор направления исследования может сэкономить или, наоборот, отнять значительное время.

Итоги

В этой главе были рассмотрены некоторые наиболее широко используемые инструменты обратной разработки и взлома. Потратьте время на знакомство с ними. В долгосрочной перспективе это окупится.

Глава 12. Защита

Как защищаться от взлома? Для начала крайне важно правильно спроектировать проверку ключа и не повторять ошибок StarCraft и Half-Life. После этого можно реализовать дополнительные средства защиты.

Однако важно помнить, что невзламываемого программного обеспечения не существует. Задача защитника - замедлить злоумышленников на критически важных участках программы и настолько их разочаровать, чтобы они переключились на другую цель.

Как и во многих вопросах кибербезопасности, главное - не оказаться самой лёгкой добычей. "Когда плывёшь в кишащей акулами воде, не обязательно быть самым быстрым... достаточно быть быстрее человека рядом".

Обфускация

Обфускация - практика сокрытия предполагаемого смысла кода путём намеренного придания логике двусмысленности и неясности. Она может быть полезна для замедления обратной разработки и тем самым:

• замедлить взлом;
• замедлить несанкционированное изменение программы;
• защитить интеллектуальную собственность.

При качественном выполнении обфускация может сделать код практически нечитаемым. Например, следующий код на C, доступный по адресу www.ioccc.org/1988/phillipps.c, после компиляции и запуска выводит полный текст песни "The Twelve Days of Christmas". Он стал одним из победителей IOCCC - конкурса по ручной обфускации кода. От одного взгляда на него у меня начинает болеть голова, и я даже не могу представить, сколько времени потребовалось бы на обратную разработку, прежде чем удалось бы понять его назначение.

```
#include <stdio.h>
main(t,_,a)
char
*
a;
{
       return!
 0<t?
t<3?
  main(-79,-13,a+
 main(-87,1-_,
main(-86, 0, a+1 )
 +a)):
 1,
t<_?
main(t+1, _, a )
:3,
 main ( -94, -27+t, a )
&&t == 2 ?_
<13 ?
 main ( 2, _+1, "%s %d %d\n" )
 :9:16:
t<0?
 t<-72?
main( _, t,
"@n'+,#'/*{}w+/w#cdnr/+,{}r/*de}+,/*{*+,/w{%+,/w#q#n+,/#{l,+,
       /n{n+,/+#n+,/#;\
#q#n+,/+k#;*+,/'r :'d*'3,}{w+K w'K:'+}e#';dq#'l q#'+d'K#!
```

```
      /+k#;\
q#'r}eKK#}w'r}eKK{nl]'/#;#q#n'){}#}w'){){nl]'/+#n';d}rw'
      i;# ){nl]!/n{n#'; \
r{#w'r nc{nl]'/#{l,+'K {rw' iK{;[{nl]'/w#q#\
\
n'wk nw' iwk{KK{nl]!/w{%'l##w#' i; :{nl]'/*{q#'ld;r'}{nlwb!/*de}'c ;;\
{nl'-{}rw]'/+,}##'*}#nc,',#nw]'/+kd'+e}+;\
#'rdq#w! nr'/ ') }+}{rl#'{n' ')# }'+}##(!!/")
:
 t<-50?
 _==*a ?
 putchar(31[a]):
  main(-65,_,a+1)
:
 main((*a == '/') + t, _, a + 1 )
:
 0<t?
 main ( 2, 2 , "%s")
:*a=='/'||
 main(0,
  main(-61,*a, "!ek;dc i@bK'(q)-[w]*%n+r3#l,{}:\nuwloca-O;m .vpbks,
       fxntdCeghiry")
 ,a+1);}
```

Понятие обфускации проникло и в массовую культуру. Следующие реплики звучат в одной из сцен фильма о Джеймсе Бонде "Skyfall", когда Q пытается получить доступ к ноутбуку Сильвы:

• "Там алгоритмы, шифрование и асимметричные схемы!"
• "Похоже на обфусцированный код, скрывающий истинное назначение. Безопасность через неясность!"

Обфускацию можно применять вручную или автоматически на разных этапах жизненного цикла программы, в том числе к следующим представлениям:

• исходный код;
• байт-код;
• объектный код;
• двоичный исполняемый код.

Оценка обфускации

При оценке вариантов обфускации следует учитывать несколько факторов:

• **сила:** насколько сильно обфускация изменяет программу;
• **устойчивость:** насколько хорошо обфусцированный код выдерживает атаки инструментов обратной разработки;
• **незаметность:** насколько хорошо обфусцированный код сливается с остальной программой;
• **стоимость:** потери производительности обфусцированного приложения.

Как правило, эти факторы противодействуют друг другу. Например, чем сильнее обфускация, тем менее незаметной она обычно становится.

На практике ограничивающим фактором часто оказывается производительность. Однако почти все методы обфускации допускают некоторую степень масштабирования или настройки в зависимости от требований.

Автоматическая обфускация

Обфускацию можно выполнять вручную. Однако почти всегда лучше применять специальные инструменты. К распространённым методам относятся:

• искажение имён;
• шифрование строк;
• обфускация потока управления;
• уплощение потока управления;
• непрозрачные предикаты;
• подмена инструкций.

Искажение имён

Искажение имён заключается в обфускации названий функций и переменных. Это можно сделать несколькими способами:

• заменить имя бессмыслицей: `get_key -> aVJ230AM`;
• заменить вводящим в заблуждение именем: `get_key -> draw_screen`;
• заменить неинформативным именем: `get_key -> a`.

После такой обработки назначение функций и переменных перестаёт быть очевидным. Рассмотрим следующий пример:

```
public static void SelectionSort <T> (T[] data, int size)
       where T: IComparable
{
       for (int num1 = size - 1; num1 >= 1; num1--)
       {
              T local1 = data[0];
              int num2 = 0;
              for (int num3 = 1; num3 <= num1; num3++)
              {
                     if (data[num3].CompareTo(local1) > 0)
                     {
                            local1 = data[num3];
                            num2 = num3;
                     }
              }
              T local2 = data[num2];
              data[num2] = data[num1];
              data[num1] = local2;
       }
}
```

После искажения имён код может выглядеть примерно так:

```
public static void a <a> (a[] A_0, int A_1) where a:IComparable
{
       int num1 = A_1 - 1;
Label_004D:
       if (num1 < 1)
       {
              return;
       }
       a local1 A_0[0];
       int num2 = 0;
       int num3 = 1;
       while(true)
       {
              if (num3 <= num1)
              {
                     if (A_0[num3].CompareTo(local1) > 0)
                     {
                            local1 = A_0[num3];
                            num2 = num3;
```

```
                }
            }
            else
            {
                a local2 = A_0[num2];
                A_0[num2] = A_0[num1];
                A_0[num1] = local2;
                num1--;
                goto Label_004D;
            }
            num3++;
        }
    }
```

В исходном варианте даже без имени функции сравнительно легко определить, что код реализует алгоритм сортировки. После искажения имён сделать это значительно труднее.

Шифрование строк

Другой метод обфускации состоит в том, что обфускатор шифрует строки при сборке исполняемого файла. Затем встроенная в код функция расшифровывает отдельные строки по мере необходимости во время выполнения. В результате такие средства, как представление строк IDA, становятся бесполезными.

Шифрование строк может радикально повлиять на читаемость кода. Рассмотрим следующий код:

```
public a() {
       this.a = "Hi, my name is Paul."
}

public static void a() {
       a a1 = new a();
       Console.WriteLine("Enter password: ");
       string text1 = Console.ReadLine();
       if (!text1.Equals(a1.a))
       {
             Console.WriteLine("Incorrect password.");
       }
       else
       {
             Console.WriteLine("Correct password.");
       }
       Console.ReadLine();
}
```

После шифрования строк этот код может выглядеть так:

```
pubic a() {
       int num1 = 5;
       this.a =
a("\ue6ad\u9eb1\u94b3\uc1b7\u9ab9\ud2bb\uadbf\ua7c1\ue4c
      3\uafc5\ubbc7\ueac9\u9ccb\uafcd\ua5cf\ubed1\ufad3", num1;
}

public static void a()
{
       int num1 = 13;
       a a1 = new a();
       Console.WriteLine(a("\uf3b5\ud6b7\uceb9\uccbd\ue0bf\ub2c1\ua5c3\u
             b5c5\ubbc7\ubdc9\ua3cb\ubccd\ub4cf\ue8d1\uf4d3, num1));
       string text1 = Console.ReadLine();
       if (1text1.Equals(a1.a)) {
              Console.WriteLine(a(\uffb5\ud8b7\ud3bb\uccbd\ub2bf\ua7c1
                    \ua7c3\ub2c5\ue8c7\ubac9\uadcb\ubdcd\ua3cf\ua5d1\ubb
                    d3\ua4d5\ubcd7\uf4d9", num1));
       }
       else
```

```
        {
            Console.WriteLine(a("\uf5b5\ud7b7\uc8b9\ucebb\ua3bf\ub6c1
                \ue4c3\ub6c5\ua9c7\ub9c9\ubfcb\ub9cd\ubfcf\ua0d1\ub0
                d3\uf8d5", num1));
        }
        Console.ReadLine();
}
```

> **Примечание переводчика.** Опечатки и синтаксические несоответствия в обфусцированном примере, включая `pubic` и повреждённые кавычки, присутствуют в исходном тексте и сохранены.

В исходном варианте по строкам легко определить, что перед нами код аутентификации, который часто особенно интересует злоумышленников. Без строк понять логику гораздо сложнее. Помните, что одна из главных трудностей при взломе приложения - найти нужный код. В двоичном файле с сотнями тысяч строк кода к проверке ключа могут относиться всего пять, а инструменты поиска строк позволяют быстро сузить область анализа до этих пяти строк. Лишить специалиста по обратной разработке строк весьма болезненно для него.

Уплощение потока управления

При этом методе обфускации поток управления каждой функции "уплощается". Процесс включает следующие шаги:

1. Функция сворачивается в оператор `switch`, находящийся внутри бесконечного цикла.
2. Каждому базовому блоку исходного потока присваивается номер состояния.
3. Оператор `switch` выбирает базовые блоки и передаёт им управление в правильном порядке.

На рис. 12.1 показано, как уплощение преобразует приложение в IDA. Логика остаётся прежней, но анализировать поток управления становится намного труднее.

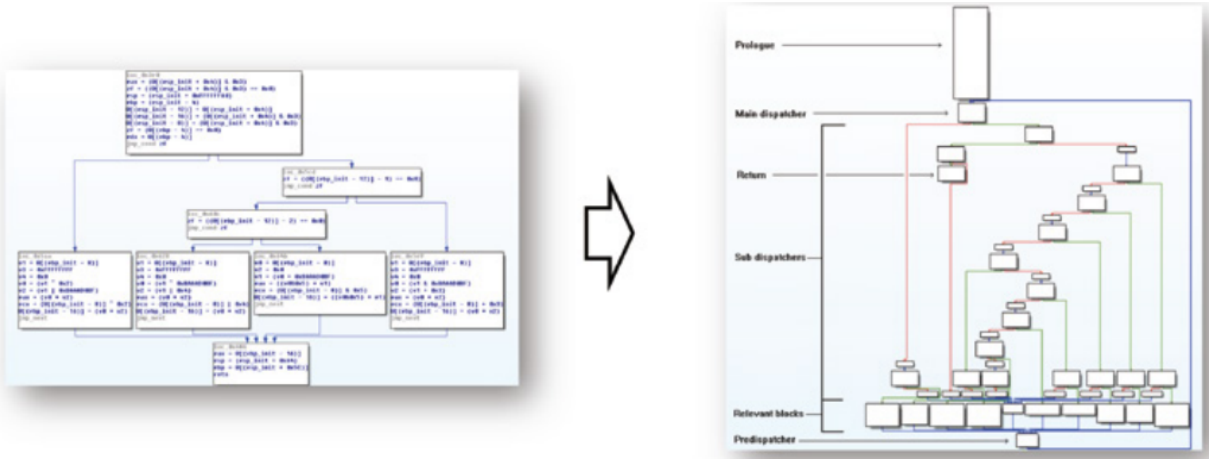


Рис. 12.1. Уплощение потока управления в IDA

Непрозрачные предикаты

Непрозрачные предикаты добавляют мусорный код вперемешку с настоящим. Мусорный код никогда не выполняется, а настоящий выполняется всегда. Однако такой приём хорошо отвлекает специалиста по обратной разработке бесполезным кодом, вынуждая его тратить часы на анализ фрагментов, которые в сущности не имеют значения. Пример в IDA показан на рис. 12.2.

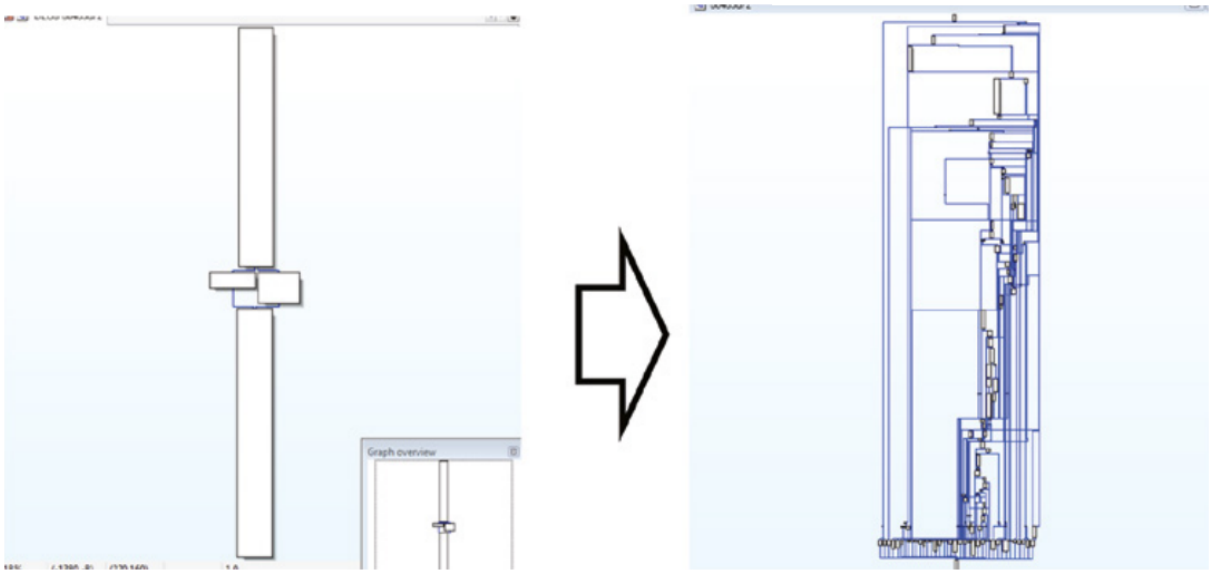


Рис. 12.2. Непрозрачные предикаты в IDA

Путь определяется оператором `if`, который всегда даёт один и тот же результат. Однако на выявление этого факта, то есть "непрозрачного предиката", может потребоваться время, что замедляет анализ.

Рассмотрим следующее выражение:

```
if ( (a<<1)%2 ) { b = a * b + a; } else { a = a + b; }
```

Где здесь находится мусорный код?

Подмена инструкций

Подмена инструкций заключается в замене легко распознаваемых инструкций сложными последовательностями, выполняющими то же действие. Например, рассмотрим следующий код:

```
sub edx, 0x192A6C72
neg ecx
sub edx, ecx
add edx, 0x192A6C72
```

Какой была исходная операция?

Обфускаторы

Обфускаторы обычно предоставляют разработчику "регуляторы", позволяющие менять степень обфускации. Причина в том, что усиление обфускации не всегда приносит пользу. Как правило, оно снижает скорость выполнения и увеличивает размер файла. Кроме того, резкое усиление обфускации не приводит к столь же существенному росту сложности обратной разработки. Для равновесия между удобством и безопасностью необходимо найти золотую середину.

Если это удаётся, обфускация может стать ценным инструментом, особенно для кода, который иначе тривиально декомпилируется, например для рассмотренных ранее языков с JIT-компиляцией, таких как .NET. Однако важно также убедиться, что используемый инструмент не предоставляет легкодоступный деобфускатор.

Для обфускации общего назначения хорошей отправной точкой может быть OLLVM. У него есть несколько преимуществ: он работает с промежуточным представлением LLVM, поддерживает все интерфейсные части LLVM, включая gcc и clang, а также множество исходных языков: C, C++, C#, Lisp, Fortran, Haskell, Python, Ruby и другие.

Использовать OLLVM в рабочем коде не рекомендуется. Однако он может послужить хорошей основой для собственных обфускаторов либо просто для изучения и экспериментов с обфускацией.

Помимо OLLVM существует множество инструментов и приёмов для конкретных языков. Например, для C# имеется Dotfuscator, а для Java - Proguard.

Для программ JavaScript можно применять YUICompressor и UglifyJS. В целом минимизаторы уже как побочный эффект обеспечивают некоторую разумную степень обфускации.

Код Python можно скомпилировать в байт-код, удалив часть имён переменных и комментариев. Затем байт-код можно обфусцировать и распространять вместе с собственным интерпретатором. Среди обфускаторов Python упоминаются Tigress, BitBoost и Opy, но они менее популярны, чем перечисленные ранее средства.

Преодоление обфускаторов

Обфускаторы предназначены для защиты от обратной разработки, повышая её сложность и продолжительность. Однако обфускация несовершенна, и, как уже неоднократно отмечалось, достаточно мотивированные взломщики в конечном счёте способны её преодолеть.

Ускорить анализ обфусцированного двоичного файла специалисту по обратной разработке помогают следующие методы:

• выполнение трасс для различения настоящего и ложного кода;
• символический анализ для упрощения сложной логики;
• написание собственных сценариев для удаления обфускации.

Лабораторная работа: обфускация

В этой лабораторной работе исследуются методы обфускации. Лабораторные работы и все связанные инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой работы найдите **Lab Obfuscation** и следуйте предоставленным инструкциям.

Навыки

В лабораторной работе с помощью `objdump` отрабатывается обход методов обфускации. Цель - понять влияние распространённых способов защиты кода.

Основные выводы

Методы обфускации замедляют, но не предотвращают взлом. Однако помните, что иногда одного замедления достаточно. У опытных специалистов по обратной разработке часто имеются инструменты для автоматического преодоления распространённых методов обфускации.

Противодействие отладке

Отладка часто является самым быстрым способом обратной разработки исполняемого файла. Противодействие отладке представляет собой набор методов, предназначенных для того, чтобы лишить исследователя возможности динамически анализировать приложение с помощью отладчика. В этой области существует много методов, но большинство из них направлено на обнаружение присутствующего отладчика. К распространённым проверкам относятся:

• проверки памяти;
• проверки процессора;
• проверки времени;
• проверки исключений;
• проверки окружения.

Как и большинство средств безопасности, противодействие отладке требует компромиссов в удобстве. Два самых болезненных побочных эффекта - увеличение размера кода и снижение производительности. Поэтому такие возможности часто добавляют избирательно, оставляя их для участков, которые, скорее всего, будут атакованы: средств проверки ключей, конфиденциальных IP-адресов и так далее.

Но, как всегда в безопасности, имеются плюсы и минусы. Если окружить чувствительный код множеством проверок против отладки, вы одновременно нарисуете на нём мишень и точно покажете злоумышленнику, где находится самое интересное. Возможно, отлаживать такой код он не сможет, но теперь будет знать, на чём сосредоточить статический анализ. Это не означает, что защита бесполезна: статический анализ может занимать в сто раз больше времени, чем динамический. Поэтому даже явные указатели на чувствительный код могут быть мощным средством, если вынуждают анализировать его статически.

Главная цель противодействия отладке - определить, подключён ли отладчик, и выполнить определённое действие. Чаще всего применяются следующие действия:

• принудительное отключение отладчика;
• завершение программы;
• выполнение отвлекающего кода для пустой траты времени злоумышленника.

IsDebuggerPresent()

`IsDebuggerPresent` выполняет проверку памяти на наличие отладчика. Функция `IsDebuggerPresent`, объявленная в `Windows.h`, возвращает истину, если программа выполняется под управлением отладчика. Следующий код показывает, как завершить приложение при подключённом отладчике:

```
if (IsDebuggerPresent())
    exit(1);
```

Проверку `IsDebuggerPresent` можно обойти, установив точку останова на инструкции сразу после возврата из функции. Когда она сработает, присвойте `eax` значение `0`, сообщая программе, что отладчик не подключён. Помните, что в `eax` хранится возвращаемое значение. Возвращать `1`

нежелательно, поскольку оно означает обнаружение отладчика, поэтому нужно вернуть `0`.

Это кажется тривиальным, но задача состоит лишь в усложнении анализа. Если код содержит сотню таких проверок, желающему выполнить отладку злоумышленнику придётся найти каждую, всякий раз вручную ставить точку останова и менять возвращаемое значение либо создавать собственные сценарии, автоматически выполняющие замену. Раздражает ли это злоумышленника? Ещё как.

Отладочные регистры

Для проверки наличия отладчика приложение может также использовать отладочные регистры процессора. Напомним, что в разделе об отладке рассматривались программные и аппаратные точки останова. Аппаратная точка останова устанавливается с помощью аппаратных регистров процессора.

Такие точки останова используют отладочные регистры, в x86 это `DR0`, `DR1`, `DR2`, `DR3`, `DR6` и `DR7`, вместо изменений памяти. Следовательно, отладку можно обнаружить, исследовав эти регистры.

Следующий пример проверяет, установлено ли значение в каком-либо отладочном регистре, что указывает на аппаратную точку останова:

```
if (GetThreadContext(hThread, &ctx))
    if ((ctx.Dr0 != 0x00) || ... || (ctx.Dr7 != 0x00))
        exit(1);
```

Вызов `GetThreadContext()` критически важен для этого метода противодействия отладке. Чтобы его обойти, установите точку останова после вызова и измените структуру контекста, присвоив наблюдаемым значениям всех отладочных регистров `0x0`. Можно ли обойти защиту? Да. Раздражает ли необходимость постоянно вносить такие изменения злоумышленника? Ещё как. Раздражённый злоумышленник означает успех защитника.

Напомним также, что в исходнике утверждается: IDA версии 6.3 и новее поддерживает аппаратные точки останова, которые не используют отладочные регистры, а вместо этого опираются на разрешения страниц. Иными словами, такая проверка против отладки не обнаружит эту аппаратную точку останова.

RDTSC

RDTSC - сокращённое название инструкции x86 **Read Timestamp Counter**, читающей счётчик временных меток. У этого счётчика много интересных применений, одно из которых - проверка времени с целью обнаружения отладчика.

При обычном выполнении приложения без отладчика процессор работает очень быстро, а с подключённым отладчиком - нет. Даже если не выполнять пошаговую отладку, а просто позволить коду работать, он исполняется на порядки медленнее, чем непосредственно процессором. При пошаговом выполнении замедление ещё сильнее. С помощью RDTSC приложение может получить временные метки до и после блока кода и измерить продолжительность его выполнения. Большая разница с высокой вероятностью означает, что код встретил точку останова или выполнялся вручную по шагам под управлением отладчика.

Следующий псевдокод показывает, как RDTSC может использоваться для обнаружения отладчика:

```
a = __rdtsc();
keycheck();
b = __rdtsc();
if (b -a > 0x10000)
```

```
    exit(1);
```

Чтобы обойти такую проверку, можно остановиться на втором вызове RDTSC. Затем измените либо a, приблизив его к b, либо b, приблизив его к a. Иначе говоря, сделайте разницу между значениями очень маленькой, чтобы программа решила, что выполнение прошло как обычно. Обход возможен? Да. Раздражает необходимость вносить исправление при каждой отладке? Да.

Недопустимый вызов CloseHandle()

Вызов `CloseHandle` с недопустимым дескриптором служит примером проверки отладчика по исключению. Функция Windows `CloseHandle`, вызванная с недопустимым дескриптором под управлением отладчика, генерирует исключение, а без отладчика - нет. Приложение может использовать это свойство и вызвать `CloseHandle` для недопустимого дескриптора, чтобы обнаружить отладчик.

Следующий код демонстрирует такое применение `CloseHandle`:

```
HANDLE hInvalid = (HANDLE)0xDEADBEEF;
__try { CloseHandle(hInvalid); }
__except (EXCEPTION_EXECUTE_HANDLER) { exit(1); }
```

Чтобы обойти проверку, установите точку останова на `CloseHandle`. Когда она сработает, измените аргумент на `INVALID_HANDLE_VALUE`.

Сканирование каталогов

Сканирование каталогов является проверкой окружения на наличие отладчика. Оно заключается в поиске в файловой системе установленных распространённых отладчиков и инструментов взлома. Обнаружив их, приложение может завершить работу.

Однако такой поиск неизбирателен: найденные инструменты могут вообще не участвовать в отладке приложения. В результате страдают законные пользователи этих средств.

Чтобы обойти проверку, установите точку останова на обходе каталогов. Затем скройте каталоги инструментов, чтобы приложение не видело и не просматривало их.

Наступательное противодействие отладке

Методы противодействия отладке не обязаны ограничиваться пассивным обнаружением отладчиков. Существует множество подходов "активной защиты", в том числе:

• **NtUserBlockInput:** блокирование ввода с клавиатуры в подключённый отладчик;
• **NtUserFindWindowEx:** получение дескриптора окна отладчика;
• **атаки на конкретные отладчики:** например, версии IDA до 7.0 аварийно завершаются примерно после 10 000 инструкций без ветвления.

Имеется и множество других вариантов. Для наступательного противодействия отладке сначала нужно распознать присутствие отладчика, а затем выполнить определённое наступательное действие. В том числе доступны подключаемые модули с открытым исходным кодом, некоторые из которых используются в следующей лабораторной работе.

При защитном противодействии отладке важно помнить, что изобретать всё заново не требуется. Существуют готовые решения, включая бесплатные проверки против отладчиков Windows с открытым исходным кодом.

Преодоление противодействия отладке

Как и другие средства защиты программ, код противодействия отладке можно обойти, хотя при правильной реализации это весьма болезненно. Первый шаг - найти и подвергнуть обратной разработке проверку против отладки. Часто для этого двигаются назад от места, где программа обнаружила использование отладчика.

После выявления такого кода имеется несколько вариантов его преодоления:

• удалить проверку с помощью инструкций `nop`;
• установить на проверке точку останова и изменить память или регистры, чтобы скрыть отладчик;
• использовать встроенные подключаемые модули или сценарии отладчика.

Как правило, незаметнее всего скрыть отладчик непосредственно в момент проверки. Например, если приложение использует `IsDebuggerPresent`, измените возвращаемое этой функцией значение, а не вмешивайтесь в оператор `if` или код завершения, использующий результат.

Лабораторная работа: противодействие отладке

В этой лабораторной работе отрабатывается обход методов противодействия отладке. Лабораторные работы и все связанные инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой работы найдите **Lab Anti-Debugging** и следуйте предоставленным инструкциям.

Навыки

В лабораторной работе x64dbg используется для обхода методов противодействия отладке. Цель - понять влияние распространённых защитных приёмов программирования.

Основные выводы

Ещё раз: зачастую достаточно просто замедлить специалиста по обратной разработке, и защита не обязана быть идеальной. Однако у опытных исследователей имеются инструменты для преодоления распространённых защитных методов.

Итоги

Разработчики хотят защитить себя и свой код от специалистов по обратной разработке и взломщиков. В этой главе были рассмотрены некоторые распространённые способы достижения этой цели, включая обфускацию и средства противодействия отладке.

Глава 13. Расширенные методы защиты

В предыдущей главе были представлены базовые способы защиты приложения от обратной разработки и взлома. В этой главе рассматриваются более совершенные методы, которые труднее преодолеть: защита от изменений, упаковка, виртуализация и применение крипторов.

Защита от изменений

Один из мощных рассмотренных нами методов взлома - исправление кода, применяемое как для долговременного изменения программы, так и в помощь обратной разработке. Защита от изменений представляет собой набор методов, затрудняющих злоумышленнику модификацию программного обеспечения. Распространённые подходы включают:

• хеширование;
• цифровую подпись;
• водяные знаки;
• программные сторожевые проверки.

Все описанные далее методы можно преодолеть, но, и это невозможно переоценить, наличие способа обхода не означает, что защиту не стоит применять. Каждый метод создаёт один из уровней эшелонированной защиты. Даже если описание обхода умещается в нескольких предложениях, на практике это вовсе не означает простоту его выполнения.

Хеширование

Приложение может использовать хеш-функции для защиты от изменений, выполнив следующие действия:

1. Вычислить хеш программного обеспечения.
2. Встроить хеш в программу.
3. Заставить программу перед выполнением проверять собственный хеш.
4. Любое изменение программы будет изменять хеш.

Защита основана на том, что изменения приложения приводят к неудаче проверки хеша. Чтобы её преодолеть, злоумышленнику потребуется внести изменения, заново вычислить хеш и заменить проверяемое значение либо полностью удалить проверку хеша.

Цифровые подписи

Цифровые подписи способны обеспечить строгую защиту целостности и подлинности данных. Они используют криптографию с открытым ключом, в которой создаётся пара открытого и закрытого ключей. Для защиты от изменений выполните следующие действия:

1. Подпишите программу закрытым ключом, создав подпись.
2. Встройте подпись в программу.
3. Заставьте программу перед выполнением проверять подпись с помощью открытого ключа.
4. Любое изменение программы сделает подпись недействительной.

Одно из главных преимуществ цифровых подписей состоит в том, что без знания закрытого ключа создать действительную подпись практически невозможно. Чтобы преодолеть такую защиту, злоумышленнику пришлось бы полностью удалить проверку подписи либо завладеть закрытым ключом и с его помощью создать новую действительную подпись.

Водяной знак

При реализации водяных знаков каждый покупатель программного обеспечения получает уникальную версию исполняемого файла, в которой изменены:

• порядок инструкций;
• имена функций;
• порядок параметров;
• способы подмены инструкций;
• и так далее.

Конкретный набор изменений служит "водяным знаком" данного экземпляра, позволяя связать его с владельцем и обнаруживать модификации. Кроме того, любые изменения программы повреждают водяной знак и становятся заметными.

Чтобы преодолеть эту защиту, злоумышленнику потребуется определить участки с водяными знаками, а затем заменить их другим знаком, скрывающим источник изменённой программы.

Сторожевые проверки

При использовании сторожевых проверок код внутри программы контролирует изменения чувствительных участков. Например, он может специально проверять критически важный переход и убеждаться, что тот по-прежнему ведёт по предусмотренному адресу. Обычно контролируются средства проверки ключей, инструкции переходов, другие сторожевые проверки и тому подобное.

Любые изменения этих участков обнаруживаются сторожевыми проверками. После этого они изменяют поведение программы: завершают её, выбирают другой путь, отменяют модификации и так далее.

Защита зависит от присутствия сторожевого кода и его способности требуемым образом изменять программу. Чтобы преодолеть этот метод, злоумышленнику придётся удалить код программных сторожевых проверок.

Упаковка

Упаковка - широкий термин, обозначающий методы, обычно применяемые к исполняемым файлам для сжатия и обфускации их содержимого. К распространённым приёмам упаковки относятся:

• сжатие или шифрование разделов данных;
• перемешивание разделов кода;
• сжатие или шифрование разделов кода;
• противодействие обратной разработке.

Одно из главных преимуществ упаковки состоит в усложнении обратной разработки. Например, упаковщик может включать возможности, противодействующие многим распространённым угрозам:

• **противодействие отладке:** упаковщики могут скрывать использование `IsDebuggerPresent`, усложняя обнаружение проверки;
• **противодействие виртуализации:** упаковщики могут определять выполнение приложения на такой платформе виртуализации, как VMware, и скрывать код обнаружения;
• **противодействие созданию дампа:** упаковщики могут стирать заголовки в памяти, затрудняя создание дампа;
• **противодействие изменениям:** его можно реализовать с помощью контрольных сумм, включая распространённые скользящие суммы, CRC32, MD5 и SHA-1, а также Tiger, Whirlpool, MD4 и Adler.

Упаковщики могут применять шифрование для сокрытия кода. Нередко используются простые алгоритмы: побитовые операции `XOR`, `ROL` и другие, LCG, RC4 и TEA. Однако могут применяться и более совершенные алгоритмы: DES, AES, Blowfish, Trivium, IDEA, ElGamal и так далее.

Если разделы кода и данных приложения зашифрованы упаковщиком, после загрузки файла в дизассемблер или шестнадцатеричный редактор будет виден лишь небольшой фрагмент кода и множество бессмысленных данных. Доступный маленький фрагмент - распаковщик. Для выполнения программы он должен распаковать себя в память во время работы, но статический анализ остальной код увидеть не сможет.

Упаковщики могут также использовать мутаторы, то есть обфускацию, изменяющую код при сохранении того же набора инструкций и архитектуры. Среди применяемых мутаций упоминаются изменение компоновки, олигоморфизм и другие методы обфускации, рассмотренные в главе 12 "Защита".

Принцип работы упаковщиков

Упаковщик, представляющий собой отдельный инструмент, упаковывает исполняемый файл: сжимает, обфусцирует и так далее. Затем он добавляет распаковщик в начало исполняемого файла. При запуске первым выполняется распаковщик, который распаковывает исходный код и данные в память, причём только в память.

На рис. 13.1 показано, как упакованный исполняемый файл выглядит в IDA. IDA видит начальный переход к распаковщику, но остальной код выглядит как данные.

```
.text:0805C050
.text:0805C050                  public start
.text:0805C050 start            proc near
.text:0805C050                  jmp     start_0
.text:0805C050 start            endp
.text:0805C050
.text:0805C050 ; ---------------------------------------------------------------------------
.text:0805C055                  db 0B3h, 58h, 12h
.text:0805C058                  dd 7A2EF958h, 73DE6856h, 297708BCh, 41272C37h, 492E3AE1h
.text:0805C058                  dd 0BB91C39Fh, 17072049h, 0B6572788h, 26CD82A6h, 0B6B2FBB8h
.text:0805C058                  dd 0D7D87B3h, 0EEA8772Ch, 0A5E71B1Bh, 0E5E1B170h, 1AA084B1h
.text:0805C058                  dd 0F67F9497h, 27E22E54h, 0D2EF54E5h, 5FC8B0FCh, 107274EDh
.text:0805C058                  dd 33DF407Ch, 3321B16Eh, 9054A231h, 25097B7Ah, 0C959BE98h
.text:0805C058                  dd 0FA156FA4h, 3A6C5D35h, 482937C8h, 4BD4AEBh, 51CE4A8h
.text:0805C058                  dd 0AFC3DC5h, 28340E28h, 0F8732F8h, 0F67A534h, 8C997059h
.text:0805C058                  dd 7C092F51h, 0F9C12512h, 41D3B406h, 413DA6AEh, 48A47C67h
.text:0805C058                  dd 9DCCFC00h, 472B12C6h, 4C19D2A2h, 472E7E3Eh, 509B602h
```

Рис. 13.1. Упакованный код в IDA

Насколько сильна эта защита?

В следующих разделах мы рассмотрим несколько методов защиты и зададим вопрос, насколько они сильны. Оценки предназначены для очень общего распределения областей, на которые каждая защита влияет сильнее всего. Наша книга преимущественно посвящена наступательной стороне,

но мы сочли важным кратко рассмотреть некоторые средства защиты.

Эффективность каждого средства против взлома будет оцениваться с помощью так называемой триады CIA: **confidentiality**, **integrity**, **availability**, то есть конфиденциальность, целостность и доступность. Для незнакомых с этим понятием поясним: это распространённый способ рассматривать средства безопасности, поскольку не каждое средство охватывает все три части триады, и важно понимать, в какой из них оно полезно.

Целостность означает подлинность объекта: соответствует ли он исходному замыслу или был изменён? Конфиденциальность означает доступность объекта только уполномоченным сторонам. Доступность показывает, насколько объект способен выполнять предназначенную функцию. Вместе эти свойства обычно называют триадой CIA.

Оценка упаковщиков по триаде CIA:

• **конфиденциальность:** да; за исключением распаковывающего фрагмента, остальной код находится в нечитаемом формате;
• **целостность:** да; изменения двоичного файла повредят упакованные разделы и, вероятно, приведут к отказу приложения;
• **доступность:** упаковщики могут снижать производительность и тем самым влиять на доступность, но при осторожной настройке это влияние можно свести к минимуму.

Преодоление упаковки

Как преодолеть упаковщик? Отлаживайте программу и наблюдайте, как она расшифровывается в памяти. После распаковки в памяти её можно анализировать, но все исправления будут применимы только к распакованному двоичному коду. Сохранить их обратно в упакованный файл нельзя.

Возникает естественная мысль: если программа уже распакована в памяти, нельзя ли создать дамп и получить новый распакованный двоичный файл? Технически это возможно, но сделать это трудно. Приложения содержат много кода запуска, а правильное размещение в памяти, настройка стека и прочие необходимые действия не возникают автоматически при сохранении дампа памяти и простом присвоении ему расширения EXE.

Другой вариант - попытаться распаковать программу. Для некоторых распространённых упаковщиков имеются инструменты, отменяющие применённую защиту. В качестве примеров можно привести UPX, MEW и ASPack.

Однако отдельного распаковщика может не существовать, а код распаковки может находиться только в упакованном исполняемом файле. Но и это не означает тупик. Для такой задачи создано множество отличных подключаемых модулей и инструментов, например OllyDumpEx и ImpRec, предназначенных для восстановления таблицы импорта. Это сложный, но выполнимый процесс, не входящий в тематику книги. Заинтересованный читатель найдёт в Интернете хорошие статьи о восстановлении импорта.

PEiD

При первом знакомстве с файлом бывает трудно определить, каким преобразованиям он подвергался. Если каким-либо образом заранее известно, что файл был упакован определённым инструментом, можно сразу идти по соответствующему пути. Но взлом обычно не сопровождается удобным руководством с перечнем применённых средств защиты.

PEiD обнаруживает большинство распространённых упаковщиков, крипторов и компиляторов переносимых исполняемых файлов, например приложений. Он способен распознавать сигнатуры более 470 различных инструментов обфускации. Более современный инструмент в этой области - Detect It Easy.

Как уже упоминалось, для многих защитных средств, включая упаковщики и крипторы, существуют также распаковщики и дешифраторы. Определение использованного инструмента может сократить время анализа на порядок, позволив удалить множество уровней защиты приложения.

На рис. 13.2 показан пример использования PEiD. Сначала выберите проверяемый файл. Затем PEiD отобразит сведения о его упаковке, шифровании и компиляции.

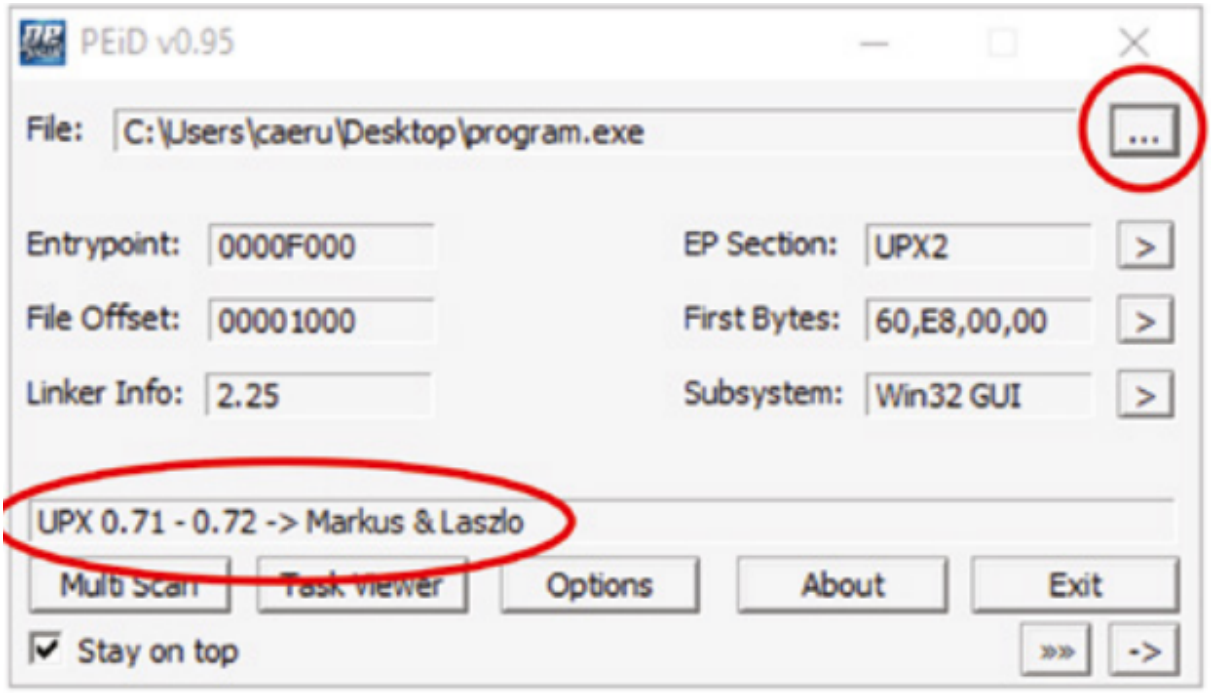


Рис. 13.2. Определение упаковщика с помощью PEiD

Лабораторная работа: обнаружение и распаковка

В этой лабораторной работе исследуется обнаружение и преодоление распространённого упаковщика. Лабораторные работы и все связанные инструкции находятся в соответствующей папке репозитория:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Для этой работы найдите **Lab Detecting and Unpacking** и следуйте предоставленным инструкциям.

Навыки

Упаковщики являются распространённой защитой от обратной разработки. В этой лабораторной работе IDA, Cheat Engine и PEiD используются для отработки следующих навыков:

• обнаружение присутствия упаковщиков;
• распаковка программ существующими инструментами;

• распаковка программ с помощью расширенной отладки.

Основные выводы

Для многих упаковщиков доступны готовые распаковщики, поэтому не следует изобретать их заново. Если распаковщика нет, исходную программу после распаковки всё равно можно вручную восстановить из памяти.

Виртуализация

Виртуализация представляет собой разновидность обфускации и упаковки. Она переводит программу в собственный машинный язык и создаёт виртуальную среду или виртуальную машину, VM, для его интерпретации. Виртуальная машина встраивается в приложение и запускается вместе с ним.

В данном случае речь идёт не о типичных крупных виртуальных машинах, например Windows или Linux, работающих под управлением гипервизора. Здесь виртуализация может означать всего лишь дополнительный уровень абстракции или интерпретации между основной архитектурой x86 и кодом.

Рассмотрим простую программу "hello world":

```
#include <stdio.h>
int main(void)
{
    printf("hello, world!\n");
    return 0;
}
```

Затем программу можно скомпилировать в произвольный машинный язык. Например, так она выглядит в Brain$#@!:

```
++++++++[>++++[>++>+++>+++>+<<<<-]>+>+>->>+[<]<-]>>.>---
        .+++++++..+++.>>.<-.<.+++.------.--------.>>+.>++.#
```

После этого приложение упаковывается вместе с интерпретатором, написанным для целевой архитектуры, то есть x86.

```
#include <stdio.h>

char data[30000];
char program[30000];
int ip=0; /* instruction pointer */
int dp=0; /* data pointer */

char read_byte(void) { return getchar(); }
void write_byte(char b) { putchar(b); }

int main(void) {
    int i=0; char b;

    do {
      b=read_byte();
      program[i]=b;
      i++;
    } while (b!='#');
    while (1) {
      b=program[ip];
      if (b==0) {
        break;
```

```
      } else if (b=='>') {
        dp++;
      } else if (b=='<') {
        dp--;
      } else if (b=='+') {
        data[dp]++;
      } else if (b=='-') {
        data[dp]--;
      } else if (b=='.') {
        write_byte(data[dp]);
      } else if (b==',') {
        data[dp]=read_byte();
      } else if (b=='[') {
        if (!data[dp]) {
          int c=1;
          do {
            ip++;
            if (program[ip]=='[') { c++; }
            else if (program[ip]==']') { c--; }
          } while (c);
        }
      } else if (b==']') {
        if (data[dp]) {
          int c=1;
          do {
            ip--;
            if (program[ip]=='[') { c--; }
            else if (program[ip]==']') { c++; }
          } while (c);
        }
      } else {
        /* do nothing */
      }
      ip++;
    }
    return 0;
}
```

> **Примечание переводчика.** В исходной вёрстке часть одинарных и двойных кавычек в листингах заменена типографскими. Здесь они нормализованы до ASCII, чтобы сохранить синтаксис C; логика примеров не изменена.

Так появляется дополнительный уровень абстракции, который должен преодолеть взломщик или специалист по обратной разработке. Сначала необходимо подвергнуть обратной разработке промежуточный язык виртуальной машины. Знакомый с Java читатель знает, что Java выполняется внутри виртуальной машины JVM. Она была создана ради переносимости, а не безопасности, но всё равно добавляет уровень сложности. Существуют и другие языки, выполняемые внутри виртуальной машины, а также можно создать собственную, как в приведённом примере.

Принцип работы виртуализации кода

В отличие от упрощённого примера, хороший виртуализатор на лету создаёт уникальный произвольный машинный язык, а не использует статический или известный. Это затрудняет разработку средств девиртуализации.

Логика программы переводится в собственный набор инструкций. В результате инструменты обратной разработки нельзя применить непосредственно, поскольку они не способны восстановить и проанализировать логику программы. Затем виртуальная машина компилируется для собственной архитектуры, то есть x86.

Для обратной разработки приложения требуется выполнить обе задачи:

• подвергнуть обратной разработке VM и расшифровать собственный набор инструкций;
• подвергнуть обратной разработке логику приложения в новом наборе инструкций.

Процесс сложен и утомителен из-за ограниченного доступа к отладке. Нельзя непосредственно отлаживать логику целевой программы, можно отлаживать лишь виртуальную машину. Среди инструментов, помогающих в этой работе, исходник называет Themida и VMProtect.

Многоуровневая виртуализация

Средства виртуализации можно накладывать друг на друга следующим образом:

• виртуальная машина VM0 реализует собственный набор инструкций IS0;
• IS0 запускает виртуальную машину VM1, реализующую собственный набор инструкций IS1;
• IS1 запускает исходное приложение.

Пример многоуровневой виртуализации:

1. Скомпилировать исходный код C в собственный язык, например DazzleZ.
2. Написать интерпретатор DazzleZ на другом собственном языке, например CatCat.
3. Написать интерпретатор CatCat на x86.
4. Запустить программу на обычной платформе x86.
5. Для обратной разработки потребуется последовательно преодолеть все уровни виртуализации.

Проблемы виртуализации

Виртуализация может эффективно замедлять обратную разработку и взлом. Однако у неё есть недостатки:

• **обнаружение антивирусом:** вредоносные программы часто используют виртуализацию для сокрытия, поэтому многие антивирусы автоматически помечают применяющие её приложения;
• **раздувание файла:** приложение должно содержать встроенную виртуальную машину, что увеличивает размер файла;
• **замедление выполнения:** виртуализированное приложение должно выполнять как VM, так и виртуализированный код, что снижает скорость работы.

Наложение нескольких виртуальных машин экспоненциально усугубляет проблемы размера и скорости.

Насколько сильна эта защита?

Оценка виртуализации по триаде CIA:

• **конфиденциальность:** да; исходный код скрыт за уровнями виртуализации;
• **целостность:** да; изменение любого уровня, вероятно, вызовет каскад отказов и затруднит исправление;
• **доступность:** каждый добавленный уровень влияет на производительность; слишком большое число уровней может резко снизить скорость и доступность кода, выборки данных и других операций.

Преодоление виртуализации

Виртуализация может быть эффективной защитой, потому что её преодоление требует много времени и усилий. Обычно применяется следующий процесс:

• **обратная разработка схемы диспетчеризации кода:** виртуальные машины обычно используют знакомый процессорный цикл выборки, декодирования и выполнения, что позволяет понять способ диспетчеризации кода;
• **снижение сложности:** сопоставление с шаблонами, символический анализ и подобные методы удаляют ненужную сложность;
• **девиртуализация программы:** предпринимается попытка восстановить представление исходного кода, однако для сложных VM это не всегда простое обратное преобразование, а восстановить оригинал может оказаться невозможно, что вынуждает анализировать виртуализированный код;
• **обратная разработка восстановленного кода:** к восстановленному коду применяются традиционные инструменты; если работоспособную программу восстановить нельзя, возможно, придётся полагаться на статический анализ.

Виртуализацию можно преодолеть, подвергнув обратной разработке виртуальную машину, а затем преобразовав приложение обратно в машинный код x86 для анализа. Среди помогающих в этом инструментов исходник называет Themida, VMProtect и Tigress.

Крипторы и дешифраторы

Крипторы шифруют разделы кода приложения, применяя подмножество методов из рассмотренного ранее раздела об упаковщиках, часто для уклонения от обнаружения вредоносного ПО. Многие антивредоносные средства анализируют программу перед запуском и блокируют её по вызовам подозрительных функций операционной системы. Шифруя раздел кода, вредоносная программа лишает защитное средство возможности исследовать содержимое приложения до выполнения.

Как правило, зашифрованная программа должна расшифровать себя перед выполнением. Обычно это означает, что ключ расшифрования где-либо находится внутри неё. Следовательно, с помощью обратной разработки можно найти ключ и расшифровать программу.

Однако существуют исключения. Например, программа с привязкой к узлу может формировать ключ на основе конкретной системы, где она установлена. Другой вариант: вредоносная программа может связаться с сервером и получить ключ расшифрования непосредственно во время работы.

Полезна ли эта защита?

Преимущества крипторов:

• **конфиденциальность:** да; шифрование всегда добавляет уровень конфиденциальности, и расшифрование произойдёт только при правильных условиях;
• **целостность:** частично; большинство алгоритмов шифрования добавляют некоторую защиту целостности, поскольку изменение зашифрованных данных приводит к повреждению, а не к предсказуемому изменению конечного кода;
• **доступность:** отсутствует; шифрование на неё не влияет.

Преодоление крипторов

Для большинства шифраторов существуют сопутствующие дешифраторы, автоматически восстанавливающие исходную программу. Часто дешифратор представляет собой тот же шифратор с другим входным флагом.

Если вы исследуете зашифрованное приложение, расшифрование вернёт исходный двоичный файл. Анализировать его будет намного проще, поэтому перед началом обратной разработки проверьте наличие готового дешифратора. Среди распространённых крипторов следует проверить Yoda's Cryptor, Morphine и PGMP.

Итоги

Среди вариантов защиты нет универсального решения. У большинства методов противодействия обратной разработке имеются недостатки.

Обфускация снижает производительность и усложняет законную отладку. Однако при разумной интенсивности она хорошо замедляет обратную разработку, особенно в случае легко декомпилируемых языков.

Противодействие отладке сравнительно слабо увеличивает время обратной разработки, поскольку многие отладчики имеют подключаемые модули для обхода всех распространённых приёмов, и одновременно усложняет законную отладку. Однако его может быть достаточно для остановки начинающих взломщиков.

Упаковщики ещё выше поднимают порог сложности обратной разработки и исправления. Но следует с осторожностью относиться к готовым коммерческим упаковщикам, COTS, для которых обычно имеются общедоступные распаковщики.

Крипторы и дешифраторы значительно усложняют обратную разработку, что делает их полезными для защиты программ. Однако при неосторожном применении они могут вызвать срабатывание распространённых антивирусов, поскольку при вредоносном использовании позволяют также защищать вредоносное ПО.

Решая, применять ли средства против обратной разработки и когда именно, следует взвесить их силу, устойчивость, незаметность и стоимость. Учитывайте противника и его цели:

• конкурирующая компания, похищающая интеллектуальную собственность;
• случайный взломщик, ищущий лёгкую добычу;
• профессиональный взломщик, атакующий крупные и ценные цели.

Также подумайте, что именно требуется защищать:

• проверку ключа или всю программу; большинство средств можно применять к отдельным функциям;
• учтите, что добавление защиты может привлечь внимание к цели.

Часто можно услышать ошибочное утверждение: "Всё можно взломать и подвергнуть обратной разработке, если постараться достаточно сильно, поэтому не стоит тратить силы на защиту, обфускацию, шифрование и тому подобное". Это грубое непонимание задачи защиты. Нередко взломщик прекращает взламывать, анализировать или ломать продукт просто потому, что занятие перестало приносить удовольствие.

Если удалось достаточно замедлить специалистов по обратной разработке, задача выполнена. Часто лучшим доступным вариантом становится надёжная архитектура с умеренными настройками готового коммерческого обфускатора. Каждый подход следует оценивать с учётом потребностей проекта.

Универсального решения нет. Не позволяйте идеальному стать врагом хорошего. После выбора разумного подхода обфускаторы, средства противодействия отладке, упаковщики и другие защитные средства можно встроить в DevOps.

Глава 14. Обнаружение и предотвращение

Разработчики приложений используют различные механизмы для обнаружения обратной разработки и взлома и защиты от них. Однако одни методы эффективнее других. В этой главе рассматриваются некоторые наиболее распространённые приёмы, их относительные сильные и слабые стороны и способы их преодоления.

CRC

Циклический избыточный код, CRC, представляет собой математическое вычисление над байтами защищаемых данных. Результат сохраняется как CRC и часто добавляется к данным, например: `данные данные данные данные данные данные CRC`. Для проверки данных вычислите CRC заново и сравните результаты.

У алгоритмов CRC есть преимущества:

• высокая скорость и компактность;
• простота аппаратного ускорения;
• быстрое вычисление и сравнение;
• множество доступных вариантов, включая IEEE 802.3, CRC-32 и другие.

В целом CRC отлично обнаруживает случайные ошибки или изменения, например ошибки передачи.

Однако это плохая защита от преднамеренных ошибок и изменений. Противник может легко заново вычислить и обновить CRC. Например, простейший CRC может складывать все байты и сохранять результат. Если данные в какой-либо части файла повредятся, новая сумма не совпадёт, что позволит принять меры. Если повреждение произойдёт в части файла, содержащей CRC, вычисленная сумма не совпадёт с испорченным CRC, что также позволит принять меры. Такой подход отлично обнаруживает, например, случайное изменение бита во время загрузки.

Но из-за тривиальности повторного вычисления CRC злоумышленник может просто внести изменения и обновить CRC, включив в него новые значения.

Насколько сильна эта защита?

Сравнение CRC с триадой CIA даёт неутешительные результаты:

• **конфиденциальность:** отсутствует;
• **целостность:** очень слабая, поскольку злоумышленнику слишком легко вычислить новый CRC и записать его в файл;
• **доступность:** отсутствует.

Защита легко преодолевается созданием нового действительного CRC. Другой вариант - просто удалить исправлением проверку CRC. CRC эффективен для обнаружения случайного повреждения, но бесполезен против преднамеренного.

Подписание кода

Многие организации перед выпуском подписывают свой код цифровой подписью. Это делают потому, что подписание кода даёт два основных преимущества:

• **подлинность:** цифровую подпись можно создать только с помощью правильного закрытого ключа, поэтому она доказывает, что программа поступила от заявленного автора;
• **целостность:** изменение данных с цифровой подписью делает подпись недействительной, поэтому подписание кода подтверждает, что после выпуска программа не изменялась.

Подписание кода защищает от широкого спектра возможных атак. Однако с точки зрения взломщика важнее всего то, что оно может помешать исправлению, если программа проверяет подпись перед выполнением.

Как подписать код

Подписание кода работает на основе криптографии с открытым ключом, или асимметричной криптографии. Эти алгоритмы используют пару открытого и закрытого ключей. Сначала для подписи кода необходимо создать такую пару.

Цифровая подпись проверяется с помощью открытого ключа, но нужен способ доказать, что конкретный открытый ключ принадлежит именно вам. Здесь появляется инфраструктура открытых ключей, PKI. Используя созданный открытый ключ, вы подаёте заявку на сертификат в центр сертификации подписей кода, CA. Центр проверяет личность и выдаёт цифровой сертификат, содержащий открытый ключ и подтверждающий владение им.

С этим сертификатом можно создавать цифровые подписи. Для этого вычисляется хеш исполняемого файла и шифруется закрытым ключом. При распространении исполняемого файла полученная подпись и цифровой сертификат поставляются вместе с ним.

Процесс можно выполнить вручную, но многие средства сборки сделают это за вас. Сертификат всё равно потребуется купить и загрузить в средство сборки, после чего ему можно поручить подписать приложение. Если это первое знакомство читателя с PKI, следует понимать, что здесь намеренно затронуты лишь самые основы: данной теме посвящены целые книги.

Как проверить подписанное приложение

Подпись кода по существу является зашифрованным хешем исполняемого файла. Убедившись с помощью связанного сертификата, что открытый ключ действителен, можно расшифровать хеш исполняемого файла. Затем независимо вычисляется хеш приложения с помощью той же хеш-функции, которую использовал разработчик. Если два хеша совпадают, приложение подлинно и не изменено. Если они различаются, приложение поддельно или подверглось изменению.

Большинство операционных систем проверяет подписи кода автоматически. Операционная система также выдаёт предупреждение, если открытый ключ, которым была создана подпись, не проверен, как показано на рис. 14.1. Однако большинство пользователей всё равно нажмёт **Run**.

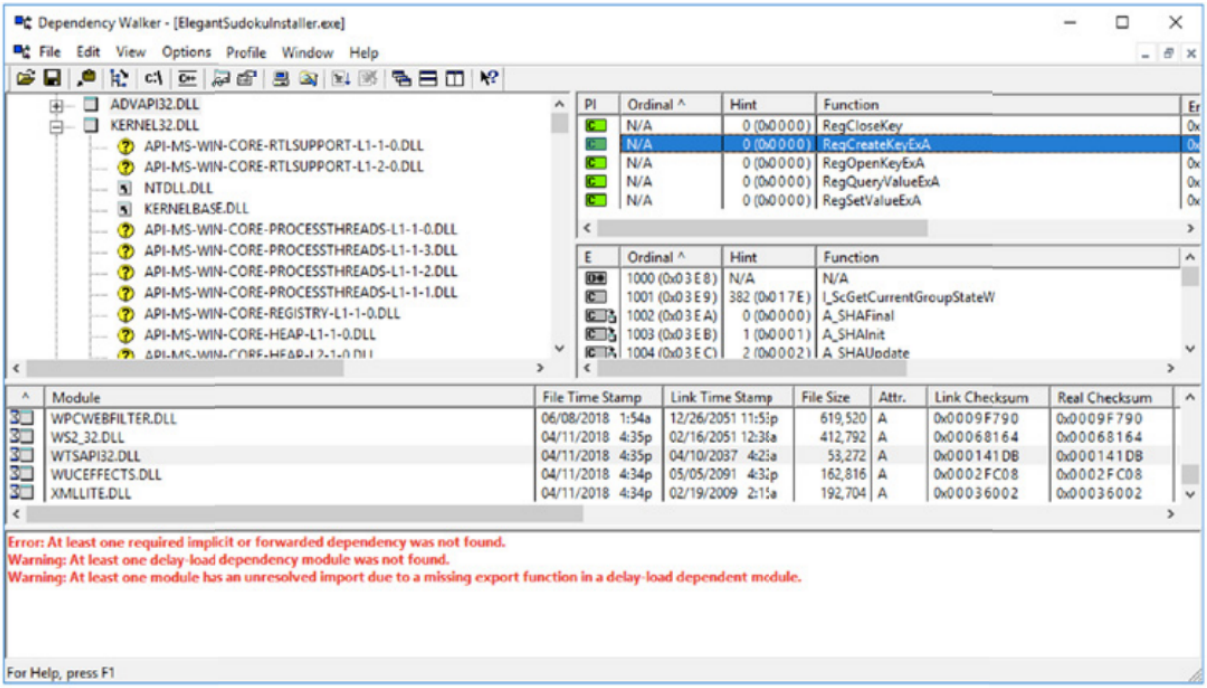


Рис. 14.1. Предупреждение Windows о непроверенной программе

> **Примечание переводчика.** Исходная подпись называет изображение предупреждением Windows о непроверенной программе, хотя на самом снимке показано окно Dependency Walker с сообщениями о неразрешённых зависимостях и импортах. Подпись сохранена по источнику.

Эффективно ли подписание кода?

Останавливает ли подписание кода все атаки с исправлением? Нет.

Причина в том, что должен существовать фрагмент неподписанного кода, проверяющий подпись. Он выполняет несколько действий:

1. Вычисляет хеш кода.
2. Проверяет, соответствует ли он ожидаемому.

• **2a.** Если ответ верен, запускает код.
• **2b.** Если ответ неверен, не запускает код.

Код проверки подписи нельзя включить в подпись, потому что он должен содержать или получать значение хеша для сравнения. Невозможно предсказать это значение, не вычислив хеш приложения. Если вычислить хеш приложения, включающего данное значение, а затем поместить хеш в приложение, изменённое приложение получит уже другой хеш.

Поскольку код проверки подписи нельзя подписать, появляется другое место, которое можно исправить для обхода подписания. Тем не менее подписание кода, несомненно, является одним из лучших способов защиты целостности программы как от случайных, так и от преднамеренных изменений.

Подписание кода и CRC

CRC обычно применяется для обнаружения битовых ошибок в данных, передаваемых по сети. Однако он защищает только от случайных изменений, а не от преднамеренных. Противник легко вычислит CRC заново.

Подписание кода настолько надёжно, насколько надёжно защищён закрытый ключ. Без закрытого ключа злоумышленник не сможет создать новый действительный подписанный хеш.

Насколько сильна эта защита?

По сравнению с CRC подписание кода гораздо лучше выглядит в триаде CIA:

• **конфиденциальность:** отсутствует;
• **целостность:** да, превосходная;
• **доступность:** отсутствует.

Более сложный способ преодолеть подписание кода - похитить закрытые ключи подписи и использовать их для цифрового подписания изменённой версии приложения.

RASP

Самозащита приложения во время выполнения, RASP, встраивает защиту в работающее приложение. Она перехватывает системные вызовы и проверяет, поступают ли они из ожидаемого источника. Кроме того, RASP перехватывает операции с данными и проверяет, выполняются ли они уполномоченными источниками.

RASP является реагирующей защитой. Её можно настроить на немедленную "остановку" атак. Например, RASP может:

• отбросить или удалить вызов, сочтённый вредоносным, например подозрительный SQL-вызов к приложению;
• завершить сеанс пользователя;
• остановить выполнение.

Перехват функций

Один из методов RASP - перехват функций. Первые несколько байтов кода функции перезаписываются переходом к коду RASP.

Код RASP содержит проверки законности вызова. Они могут включать:

• проверку параметров и контекста вызова;
• проверку отсутствия изменений кода, например сравнение хеша функции с заведомо правильным хешем.

В конце код RASP выполняет перезаписанные инструкции и затем возвращается переходом к исходной функции.

Риски RASP

Обнаружив атаку, RASP может остановить выполнение. Однако в зависимости от назначения программы это может быть неприемлемо. В больницах, на производстве, в критической инфраструктуре, автомобилях и подобных средах внезапная остановка приложения способна создать серьёзную угрозу здоровью и безопасности.

У RASP есть недостатки даже при отсутствии атаки:

• **скорость:** из-за перехвата функций RASP оказывает заметное влияние на быстродействие;
• **размер:** перехват функций и таблицы поиска помогают обеспечивать безопасность, но одновременно раздувают двоичные файлы.

Насколько сильна эта защита?

Сравнение RASP с критериями CIA даёт смешанные результаты:

• **конфиденциальность:** нет;
• **целостность:** да; для разделов, защищаемых RASP, проверка контекста во время выполнения является очень мощным средством;
• **доступность:** нет, и влияние может быть даже отрицательным.

При правильной настройке RASP может быть трудно преодолеть. Если включено подписание кода, приложение нельзя легко исправить; если включено противодействие отладке, его нельзя легко подвергнуть обратной разработке.

Однако RASP открывает потенциальное направление атаки на доступность. Если определить входные данные, воспринимаемые системой как "атака", можно заставить её самостоятельно завершить работу.

Списки разрешений

Список разрешений, также называемый белым списком, предоставляет среде выполнения перечень "хороших" объектов. Например, компьютер может разрешать запуск только приложений из такого списка.

Существует множество коммерческих продуктов, предоставляющих списки разрешений. Например, в операционную систему Windows встроены политики ограничения программного обеспечения.

С точки зрения взлома списки разрешений могут помешать использованию инструментов обратной разработки. Такие программы, как Procmon, отладчики, Cheat Engine, Resource Hacker, Dependency Walker и другие средства обратной разработки и взлома, скорее всего, не будут включены в список.

Правильно составить список разрешений трудно. Не всегда просто определить все библиотеки, необходимые приложению. При создании белого списка требуется тщательное тестирование, чтобы включить в него все необходимые приложения и библиотеки.

Принцип работы списков разрешений

Существует два основных подхода. Список может основываться на имени процесса или на его хеше. Проверка применяется только при первом запуске приложения.

Обход списков по имени

Список разрешений хранит имена процессов или приложений, которым позволено выполняться. Для обхода дайте вредоносному приложению разрешённое имя. Например, вы определили, что имя `solitare.exe` находится в списке, и называете своё вредоносное приложение `solitare.exe`.

> **Примечание переводчика.** Написание `solitare.exe`, вместо обычного `solitaire.exe`, сохранено по исходному тексту.

Обход списков по имени и хешу

Если список использует одновременно имена и хеши приложений, обойти его простым переименованием нельзя. Хеш вредоносного приложения не совпадёт с хешем настоящего.

Однако такую защиту можно преодолеть внедрением в процесс. После запуска разрешённого приложения можно внедрить вредоносные библиотеки, если удастся добиться выполнения кода. Это легко сказать, но получить выполнение кода зачастую вовсе не просто. Перед нами один из случаев, когда обход звучит просто, потому что описывается одним предложением, но на практике обязательное условие выполнения кода в разрешённом процессе может стать для взломщика непреодолимым препятствием.

Если трудно достижимое выполнение кода в разрешённом приложении всё же получено, существует множество способов загрузиться в процесс. В Windows можно использовать `LoadLibrary()` или `SetWindowsHookEx()`. В Linux можно применять `ptrace()`, `PTRACE_POKEDATA` или коды операций системного вызова `uselib()`.

Хеш приложения проверяется до его запуска. Изменения, внесённые после запуска, списком разрешений обнаружены не будут.

Пример: Metasploit

Metasploit - популярный инструмент взлома. Его главная задача - эксплуатировать уязвимость приложения и внедрить Meterpreter, предоставляющий злоумышленнику удалённый доступ к заражённому компьютеру. Ссылки находятся в разделе **Tools** репозитория книги.

При работе Metasploit новые приложения не запускаются: Meterpreter внедряется во взломанный процесс. Из него он может "переместиться" в любое другое работающее приложение.

Насколько сильна эта защита?

Списки разрешений обеспечивают ограниченную защиту:

• **конфиденциальность:** нет;
• **целостность:** да, если проверяются и имя, и хеш, хотя проверка целостности обычно выполняется только при запуске приложения;
• **доступность:** нет.

Список разрешений можно преодолеть несколькими способами. Вредоносная программа может выдать себя за законное приложение, обойдя список по имени, либо использовать внедрение кода против списка, основанного одновременно на именах и хешах.

Списки запретов

Список запретов, также называемый чёрным списком, является полной противоположностью списка разрешений. Вместо перечисления всего разрешённого он содержит всё запрещённое. Список запретов может основываться на именах, ключах или хешах.

Такие списки легко создавать, но трудно поддерживать. Например, представьте список, содержащий вредоносный файл `virus1.exe`. Что произойдёт на следующей неделе, когда появится `virus2.exe`?

В контексте взлома в список можно включать заведомо плохие, например взломанные, ключи. В зависимости от алгоритма создания ключей иногда можно запретить целые их подмножества.

Программа может также отказаться работать при обнаружении определённых других приложений. Например, она может не запускаться, если установлен отладчик. Многие антивирусы применяют такой подход для обнаружения и блокирования известного вредоносного ПО. Они содержат список "сигнатур" известных плохих приложений и помечают как вредоносное всё, что совпадает с сигнатурой.

Насколько сильна эта защита?

Список запретов защищает слабее списка разрешений:

• **конфиденциальность:** отсутствует;
• **целостность:** частичная, если используются хеши или ключи;
• **доступность:** отсутствует.

Способ обхода зависит от данных, по которым список распознаёт вредоносные приложения. Если используются имена, измените имя. Если хранятся хеши известных плохих программ, выполните мутацию: внесите небольшое изменение в код или данные приложения, чтобы изменить хеш.

Удалённая аутентификация

В случае большинства стратегий против обратной разработки и взлома злоумышленник располагает всеми компонентами, необходимыми для преодоления защиты. Имея достаточно времени, он может подвергнуть приложение обратной разработке и исправлению.

Удалённая аутентификация требует, чтобы приложение для работы получило нечто с удалённого узла. Например, оно может загрузить с сервера ключ для расшифрования критически важного кода.

Большинство злоумышленников исследует систему "автономно". Они не хотят, чтобы она обращалась к вашим серверам, поскольку не желают раскрывать IP-адрес и сам факт запуска программы. При попытке взлома программное обеспечение, его код запуска и проверки, скорее всего, будут запускаться многократно. Законный пользователь обычно запускает приложение не более нескольких раз в день. Такое различие легко заметить на сервере удалённой аутентификации: пользователь, проходящий проверку сто раз в день, вероятно, занимается чем-то подозрительным.

Архитектура, при которой приложение не может работать без сведений с сервера, помогает предотвращать обратную разработку. Злоумышленнику придётся либо анализировать его "в сети",

либо отказаться.

Пример удалённой аутентификации

Один из вариантов реализации - зашифровать всё приложение, кроме загрузчиков. Загрузчик отправляет серверу сведения о системе, хеш программы и ключ активации.

Сервер проверяет ожидаемый хеш и ключ активации. Если они действительны, он с помощью алгоритма создаёт ключ расшифрования и отправляет его приложению. После этого загрузчик расшифровывает приложение, позволяя ему работать.

Злоумышленник не сможет "имитировать" удалённый сервер и алгоритм без доступа к серверному коду. Исследовать программу можно будет только после сетевой активации. Код расшифрования приложения может находиться исключительно в памяти, чтобы при каждом запуске требовалось взаимодействие с сервером.

Главная сложность такого подхода состоит в нетривиальности реализации криптографии и корпоративных систем управления ключами. Ошибка может позволить взломщику обойти код проверки и создавать собственные ключи расшифрования.

Как говорилось об упакованных приложениях, после распаковки в памяти можно создать дамп для последующего статического анализа. Однако такой дамп трудно, а иногда невозможно, превратить в расшифрованное работоспособное приложение. Дамп не будет полезен для исправления или проверки изменений, но никогда не следует недооценивать ценность статического анализа.

Насколько сильна эта защита?

Удалённая аутентификация получает смешанную оценку по триаде CIA:

• **конфиденциальность:** частичная; в конечном счёте приложение расшифровывается в памяти, но двоичный файл в состоянии покоя остаётся закрытым;
• **целостность:** да; перед отправкой ответа сервер должен выполнять определённую проверку целостности;
• **доступность:** возможно отрицательное влияние.

Одна из возможных атак состоит в создании поддельного сервера. Сначала активируйте приложение в сети и перехватите весь обмен данными между приложением и сервером.

Затем запустите поддельный сервер с подходящими ответами. Код приложения будет расшифрован, после чего его можно сохранить на диске.

Для этого требуется один сетевой запуск приложения, позволяющий получить расшифрованный код. После него можно создать полный расшифрованный двоичный файл, и дальнейшая сетевая аутентификация станет ненужной. Однако подход сработает не всегда, если приложение добросовестно требует от сервера определённые сертификаты или если запрос и ответ сервера непостоянны, например меняются в зависимости от времени или даты.

Лабораторная работа: ProcMon

Эта лабораторная работа показывает, что программу можно взломать несколькими способами. Вернитесь на страницу книги в GitHub:

https://github.com/DazzleCatDuo/X86-SOFTWARE-REVERSE-ENGINEERING-CRACKING-AND-COUNTER-MEASURES

Найдите лабораторную работу **ProcMon**.

Навыки

В этой работе ProcMon и IDA используются для изучения альтернативных способов взлома. Проверяются следующие основные навыки:

• динамический анализ поведения программы;
• выявление косвенных подходов к обходу защиты программного обеспечения.

Основные выводы

Наблюдать за действиями процесса снаружи может быть быстрее и проще, чем изнутри, то есть отладка не всегда является лучшим подходом.

Обычно программу можно взломать многими способами; умение находить лучший приходит с практикой.

Итоги

В этой главе были представлены различные способы защиты от взлома и обратной разработки программ. Одни методы в целом неэффективны, другие способны работать, но имеют недостатки.

Важно помнить, что при достаточных времени и усилиях можно преодолеть почти любую защиту. Цель состоит в том, чтобы замедлить злоумышленника и в идеале настолько его разочаровать, чтобы он отказался от продолжения.

Глава 15. Правовые вопросы

В разделе "Введение" правовые последствия и аспекты обратной разработки и взлома рассматривались на общем уровне. В этой главе подробнее обсуждаются соответствующие законы США, их влияние и толкование.

> **Примечание переводчика.** Опечатка `Introducton` в названии раздела исходника нормализована до `Introduction`, то есть "Введение".

> **ПРЕДУПРЕЖДЕНИЕ.** Мы не юристы, и приведённый материал не является юридической консультацией. Если вам нужна правовая помощь, обратитесь к заслуживающему доверия юристу или в Electronic Frontier Foundation по адресу www.eff.org, обладающую глубокой специализацией в области безопасности.

> **Примечание переводчика.** Правовые сведения переведены по исходному изданию и не актуализировались. Формулировки описывают понимание законодательства США авторами книги на момент её подготовки.

Законы США, влияющие на обратную разработку

Законы об авторском праве, взломе и подобных вопросах различаются в зависимости от юрисдикции. В этом разделе рассматриваются некоторые законы, действующие в Соединённых Штатах. Если вы находитесь в другой стране, изучите местные законы и ограничения.

Закон об авторском праве в цифровую эпоху

Управление цифровыми правами, DRM, предназначено для защиты интеллектуальной собственности. Системы DRM могут отслеживать и контролировать защищённое содержимое после его выхода на рынок.

Закон об авторском праве в цифровую эпоху, DMCA, был принят Конгрессом в 1998 году. Он привёл законодательство США в соответствие с международными соглашениями об авторском праве.

Закон о компьютерном мошенничестве и злоупотреблениях

Закон о компьютерном мошенничестве и злоупотреблениях, CFAA, был принят в 1984 году. Это федеральный закон против взлома, запрещающий несанкционированный доступ к компьютерам и сетям.

Законодатели составили его настолько неудачно, что с тех пор изобретательные прокуроры злоупотребляют им. Однако в последние годы предпринимались попытки защитить исследователей безопасности от преследования на его основании. В статье Wired.com 2014 года о CFAA говорилось следующее:

> Громкий случай ненадлежащего применения закона произошёл в 2008 году, когда трём студентам MIT запретили выступать на хакерской конференции Def Con. Студенты

> обнаружили недостатки электронной билетной системы Massachusetts Bay Transportation Authority, позволявшие любому ездить бесплатно. MBTA запросила и получила временный судебный запрет, не позволивший студентам рассказать об уязвимостях. Выдавая временный запрет на разглашение, судья сослался на CFAA и заявил, что сведения, которые студенты собирались представить, предоставили бы другим средства для взлома системы. Слова судьи подразумевали, что простой разговор о взломе равнозначен самому взлому. Однако решение подверглось общественной критике как неконституционное предварительное ограничение свободы слова. Когда впоследствии MBTA запросила постоянный запрет, другой судья отклонил требование, постановив, в частности, что CFAA не распространяется на речь и потому не имеет отношения к делу.

Источник, приведённый в книге: www.wired.com/2014/11/hacker-lexicon-computer-fraud-abuse-act

Второе громкое и крайне печальное злоупотребление CFAA привело к широко известному самоубийству. Оно произошло после того, как федеральный прокурор США применил CFAA для чрезмерно сурового преследования интернет-активиста Аарона Шварца за действие, которое многие считали незначительным нарушением.

Шварц, участвовавший в разработке стандарта RSS и бывший одним из основателей правозащитной организации Demand Progress, был обвинён после того, как проник в служебное помещение MIT и, предположительно, подключил ноутбук к университетской сети, чтобы загрузить миллионы научных статей, распространявшихся подписной службой JSTOR. Шварца обвинили в неоднократной подмене MAC-адреса компьютера для обхода блокировки, которую MIT установил для использованного им адреса.

Хотя JSTOR не подала жалобу, Министерство юстиции продолжило преследование Шварца. Федеральный прокурор Кармен Ортис настаивала, что "кража остаётся кражей" и власти лишь обеспечивают соблюдение закона. Шварц, отчаявшийся из-за предстоящего суда и перспективы осуждения за тяжкое преступление, покончил с собой в 2013 году.

После трагедии два законодателя предложили давно назревшую поправку, которая помогла бы предотвратить чрезмерно широкое применение закона прокурорами. Поправка, получившая название "закон Аарона", была внесена через несколько месяцев после смерти Шварца членом Палаты представителей Зои Лофгрен, демократом от Калифорнии, и сенатором Роном Уайденом, демократом от Орегона.

Она исключила бы из сферы закона нарушения условий обслуживания и пользовательских соглашений, а также сузила бы определение несанкционированного доступа, чётко различив преступный взлом и незначительное превышение разрешённого доступа. Вместо этого предлагалось определить несанкционированный доступ как "обход одной или нескольких технологических мер, исключающих или не позволяющих неуполномоченным лицам получать или изменять" сведения на защищённом компьютере. Поправка также разъяснила бы, что простая смена пользователем MAC- или IP-адреса для получения доступа к системе не считается таким обходом.

Закон об авторском праве

Согласно Закону об авторском праве 1976 года, авторское право на компьютерную программу возникает по мере написания программистом её исходного кода. Для возникновения защиты программа не обязана быть завершённой или даже работоспособной. Судебная практика в области авторского права рассматривает защиту исходного и объектного кода как равнозначную.

Если вы не являетесь правообладателем, без разрешения обычно незаконно выполнять следующие действия:

• копировать программу или её части, чтобы передать или продать другому лицу;
• предварительно устанавливать программу на жёсткий диск продаваемого компьютера;
• распространять программу через Интернет;
• обходить средства контроля, препятствующие доступу к материалам, защищённым авторским правом.

Однако существует множество исключений и тонкостей. Первое авторское право на программное обеспечение было зарегистрировано в 1964 году. Обоснованием начала защиты программ стало их восприятие как своеобразной "книги с практическими инструкциями". Закон об авторском праве 1976 года прямо назвал программное обеспечение объектом авторского права.

Что именно защищается, когда программа получает охрану авторского права? Авторское право защищает выражение идеи, а не саму идею. Например, если вы придумали игру о киоске с лимонадом, можно защитить авторским правом её конкретную реализацию, но не саму идею такой игры.

Далее исходный текст утверждает, что защита распространяется на объектную, то есть исполняемую, программу, а не на исходный код. Наконец, она охватывает экранные изображения, создаваемые программой во время выполнения.

Исходный код программ обычно сохраняется как коммерческая тайна и не публикуется для широкой аудитории на основании авторского права.

> **Примечание переводчика.** На этой странице исходник сначала говорит о равнозначной защите исходного и объектного кода, а затем утверждает, что защищён объектный код, но не исходный. Оба противоречащих утверждения сохранены.

Важные судебные дела

Помимо законов, для определения законности действий в США важны судебные прецеденты. Среди значимых дел приводятся следующие.

• **Apple Computer v. Franklin Computer.** Установило, что объектные программы могут охраняться авторским правом.

В начале 1980-х корпорация Franklin Computer начала выпускать компьютер Franklin Ace как конкурента Apple II. Franklin ACE был совместим с программами Apple II. Для обеспечения совместимости Franklin Ace непосредственно скопировал из ПЗУ Apple II некоторые функции операционной системы. Apple подала иск к Franklin Computers за нарушение авторского права путём копирования объектного кода и выиграла дело.

• **Sega v. Accolate.** Установило, что дизассемблирование объектного кода для определения технических характеристик является добросовестным использованием.

Производитель видеоигр Accolate хотел выпустить часть своих игр для Sega Genesis. Однако Sega не раскрывала технические характеристики системы, поэтому Accolade дизассемблировала объектный код игры Sega, чтобы понять его работу. Sega подала иск о нарушении авторского права. На этот раз суд вынес решение в пользу Accolade, поскольку её действия представляли собой добросовестное использование программного обеспечения.

> **Примечание переводчика.** В заголовке дела исходник использует написание `Accolate`, но далее в описании переходит к правильному `Accolade`. Это различие сохранено.

Из этих двух дел следовало, что обратная разработка допустима до тех пор, пока не нарушает авторское право. Franklin Computer нарушила его, скопировав часть кода Apple. Accolate авторское

право не нарушила: она не копировала защищённые материалы, а лишь изучала их.

Один из способов остаться в допустимой ситуации, подобной Accolate, и не оказаться в положении Franklin Computers состоит в применении стратегии разработки в "чистой комнате". Для этого создаются две изолированные команды, каждая из которых выполняет свою часть работы. Первая исследует систему или программу конкурента и составляет техническое описание её работы. Вторая использует это описание для разработки новой системы.

Если бы Franklin Computer применила такой подход, часть сотрудников могла бы выяснить принцип работы системы Apple и описать её функции. Затем вторая команда, никогда не видевшая реализацию Apple, могла бы самостоятельно воспроизвести функциональность. Это потенциально изменило бы исход дела. При таком подходе компания, вероятно, избежала бы иска, поскольку не использовала бы код Apple.

Главное - избежать неосознанного копирования кода. Если бы одна и та же команда сначала исследовала Apple II, а затем реализовывала спецификацию, она, вероятно, была бы склонна использовать код, похожий на код системы Apple, поскольку уже видела его.

Добросовестное использование

Иногда воспроизводить защищённое авторским правом произведение без разрешения законно. Обычно суды рассматривают четыре фактора, определяя, подпадает ли действие под исключение добросовестного использования:

• **цель и способ использования:** критика, комментарий, освещение новостей, преподавание или исследование, вероятно, будут допустимы, тогда как коммерческое использование, скорее всего, нет;

Важнейшая характеристика использования состоит в степени преобразования исходного произведения. Если новый автор добавил новое выражение или смысл, работа потенциально может считаться добросовестным использованием;

• **характер произведения:** добросовестное использование охотнее признаётся в отношении документальных, чем художественных произведений;
• **объём копируемого произведения:** краткий фрагмент с большей вероятностью допустим, чем копирование всей книги или главы;
• **влияние на рынок защищённого произведения:** например, копирование материала, снятого с печати, оказывает не такое влияние на рынок, как копирование недавно написанного и изданного произведения.

Согласно Закону об авторском праве, 17 U.S.C. § 107, обратная разработка относится к "добросовестному использованию", когда выполняется "в таких целях, как критика, комментарий, сообщение новостей, преподавание, включая изготовление нескольких копий для занятий, научная работа или исследование". Однако это сопоставляется с "влиянием использования на потенциальный рынок защищённого произведения или его ценность".

Исключение DMCA для исследований

В октябре 2016 года в DMCA было добавлено исключение для добросовестных исследований безопасности. Оно говорит о "доступе к компьютерной программе исключительно в целях добросовестного тестирования, когда такая деятельность проводится в контролируемой среде, предназначенной для предотвращения какого-либо вреда людям или обществу, и не используется и не поддерживается способом, содействующим нарушению авторского права".

Исключение может распространяться и на обратную разработку и взлом. В исходнике говорится, что "исследователи могут обходить цифровые средства контроля доступа, выполнять обратную разработку, получать доступ, копировать и изменять цифровое содержимое, защищённое авторским правом, не опасаясь преследования, в разумных пределах".

Это не освобождающая от ответственности универсальная привилегия и не безусловное разрешение взламывать всё подряд. Исключение отражает развитие отрасли и признание того, что исследования безопасности, проводимые с надлежащими целями, полезны и что закон теперь защищает добросовестных исследователей.

Законность

В вопросах обратной разработки и изменения кода законодательство об авторском праве уделяет большое внимание намерениям и последствиям. Действуя самостоятельно, проконсультируйтесь с юристом... или никому не рассказывайте. Последняя фраза не призывает к скрытности; вспомните, что одним из факторов добросовестного использования является влияние работы на рынок.

Если вы экспериментируете со взломом ради обучения или исследования и результаты остаются у вас, они фактически не влияют на потенциальный рынок произведения. Это важный фактор добросовестного использования. Как только знания используются для создания генератора ключей, опубликованного в Интернете и причиняющего поставщику убытки, действие перестаёт считаться добросовестным использованием. Если же всё остаётся у вас и не влияет на рынок или других людей, вы в значительной степени приблизились к условиям добросовестного использования.

> **ПРЕДУПРЕЖДЕНИЕ.** Повторим: мы не юристы, это не юридическая консультация, а лишь наше толкование и понимание нормативно-правовой среды США, влияющей на обратную разработку.

Итоги

В этой главе были рассмотрены некоторые правовые аспекты обратной разработки и взлома, но мы не юристы. Для получения юридической консультации рекомендуем обратиться в EFF.

Глава 16. Расширенные методы

До этого момента в книге рассматривались основные инструменты и навыки, применяемые для обратной разработки и взлома. Однако эта область продолжает развиваться, и появляются новые методы, делающие работу быстрее и проще. В этой главе на общем уровне описываются некоторые передовые методы и инструменты, находящиеся на острие обратной разработки.

Наша цель состоит в следующем: если к этому моменту вы всё ещё увлечены взломом программ и хотите перейти на следующий уровень, мы хотим показать вам множество направлений, в которые можно углубиться. Надеемся, что в зависимости от ваших интересов следующие разделы укажут подходящий путь дальнейшего изучения.

Отладка с обращением времени

Отладка с обращением времени также называется обратной отладкой. Её основная идея: "Что, если во время отладки можно было бы двигаться назад?"

Представьте, что во время отладки что-либо пошло не так. Возможно, исправление не сработало, вы пропустили проверку против отладки, не понимаете, как попали в текущее состояние, и так далее.

Существует несколько инструментов для отладки с обращением времени:

• Visual Studio Ultimate для .NET;
• rr;
• gdb.

Для начала посмотрите выступление Джорджа Хотца на USENIX Enigma 2016: George Hotz @ Enigma.

Инструментирование двоичного кода

Инструментирование двоичного кода заключается во внедрении кода для наблюдения за процессом или его изменения во время выполнения. Оно полезно для поиска утечек памяти, трассировки средств проверки ключей, противодействия средствам против отладки и других задач.

Инструменты двоичного инструментирования:

• PIN;
• DynamoRIO;
• Frida;
• Valgrind;
• QBDI.

Для знакомства с двоичным инструментированием посмотрите доклад Black Hat USA 2015 Augmenting Static Analysis Using Pintool: Ablation.

Промежуточные представления

Обычно для обратной разработки и взлома приходится изучать и писать инструменты для каждой новой архитектуры. Идея промежуточных представлений состоит в переводе ассемблерного кода всех архитектур в один общий язык. Тогда достаточно изучить и создавать инструменты только для этого языка.

Для работы с промежуточными представлениями можно использовать:

• Binary Ninja;
• REIL;
• VEX;
• BNIL;
• Ghidra PCode;
• микрокод IDA;
• LLVM IR.

Для начала работы с промежуточными представлениями посмотрите доклад Джордана Винса с LevelUp 0x03 Finding Bugs with Binary Ninja.

Декомпиляция

Идея декомпиляции состоит в восстановлении исходного кода посредством сложного автоматизированного анализа ассемблерного кода. Декомпиляцию поддерживают следующие инструменты:

• Hex-Rays в IDA;
• Ghidra;
• Binary Ninja;
• Snowman Decompiler.

Подробнее о декомпиляции рассказывает материал Decompiling a Virus using IDA Pro.

Автоматическое восстановление структур

Автоматическое восстановление структур заключается в автоматическом поиске шаблонов и связей в памяти, позволяющих делать выводы об используемых типах данных. Для этого применяются:

• dynStruct;
• Cheat Engine.

Подробнее об автоматическом восстановлении структур можно узнать из идеи и статьи проекта dynStruct.

Визуализация

Листинги кода и текст бывает трудно осмысливать. Визуализация позволяет глубже понять структуру файла и процесс выполнения.

Полезные средства визуализации предлагают следующие инструменты обратной разработки:

• BinWalk;

• Hopper;
• подключаемые модули IDA;
• Veles;
• ..cantor.dust..;
• Cheat Engine.

Хорошая отправная точка для понимания применения визуализации в обратной разработке - доклад Кристофера Домаса с DerbyCon Dynamic Binary Visualization.

Деобфускация

Обфускация предназначена для замедления обратной разработки в надежде, что взломщик сдастся. Идея деобфускации состоит в применении таких средств, как Tigress Protection, для автоматического удаления обфускации из программ.

Посмотрите материал Lets break modern binary code obfuscation.

Средства доказательства теорем

Средства доказательства теорем используют математику для анализа кода, включая его сокращение, деобфускацию, исследование границ, входных данных и прочего. В обратной разработке применяются:

• Z3;
• STP;
• Boolector;
• Yices.

Пример применения таких средств показан в материале Using z3 to find a password and reverse obfuscated JavaScript.

Обратите внимание и на ежегодное соревнование SMT-COMP!, где представлены интересные тесты множества уникальных решателей.

Символический анализ

Идея символического анализа состоит в поиске входных данных, приводящих к интересующим результатам. Например, какие входные данные вызовут сбой, пройдут проверку ключа, откроют секрет и так далее?

Инструменты символического анализа прослеживают движение пользовательского ввода по программе. На каждом ветвлении они спрашивают средство доказательства теорем, какие входные данные приведут на выбранный путь, а какие - на невыбранный.

Рассмотрим следующий код:

```
if (strlen(username) > 10)
    if (key_1^sum(username)==key_2)
        printf("key passed");
```

Инструмент символического анализа автоматически определит сочетание `username`, `key_1` и `key_2`, которое пройдёт проверки и достигнет инструкции вывода `key passed`.

Инструменты символического анализа:

• Angr;
• Mayhem;
• KLEE;
• Triton;
• S2E.

Пример символического анализа с Angr приведён в докладе Шошитаишвили и Вана с DEF CON 23 Angry Hacking: The next gen of binary analysis.

Итоги

На этом этапе лучший способ совершенствовать навыки обратной разработки и взлома - больше практиковаться. На рабочем столе виртуальной машины Windows в папке `allthethings` находится множество программ crackme для упражнений, распределённых по уровням сложности.

Глава 17. Дополнительные темы

Эта последняя глава книги знакомит с обратной разработкой и взломом программного обеспечения. В основном речь идет о том, как понять работу программы и обойти либо изменить нежелательную функциональность, например проверку ключа.

В этой главе полученные знания применяются к взлому реальных программ. Разрушение стека и шелл-код опираются на понимание того, как работают программа и стек, чтобы выполнить внутри программы вредоносный код.

Разрушение стека

Разрушение стека, также известное как переполнение стекового буфера, относится к наиболее классическим атакам на программное обеспечение. Оно использует тот факт, что в языках без безопасной работы с памятью, таких как C/C++, нет встроенной защиты, которая не позволяла бы приложению обращаться к данным в других областях памяти или перезаписывать их. Например, C/C++ не проверяет автоматически, помещаются ли записываемые в массив данные в границы этого массива. Если вы не знаете C, не беспокойтесь. Если вы знаете любой язык программирования, то сможете следить за изложением.

Поскольку разрушение стека известно уже очень давно, многие компиляторы встраивают в скомпилированный код автоматические защитные средства. Хотя теперь провести такую атаку не так легко, как раньше, всем следует хорошо понимать, как она работает, потому что:

• некоторые ее аспекты по-прежнему работают;
• она лежит в основе других видов атак;
• защита стека есть не в каждом приложении.

Если вы собираете следующие примеры на C с помощью gcc, защиту необходимо отключить флагом `-fno-stack-protector`. В книге полная командная строка для сборки в Linux напечатана так: `gcc myfile.c-fno-stack-protector`.

> **Примечание переводчика.** В команде оригинала отсутствует пробел перед `-fno-stack-protector`. Синтаксически корректная форма: `gcc myfile.c -fno-stack-protector`.

Рассмотрим простую программу на C:

```
void function(int a, int b, int c) {
   char buffer1[5];
   char buffer2[10];
}

void main() {
  function(1,2,3);
}
```

После компиляции приложения и дампа объекта из памяти получается следующий ассемблерный код:

```
function:
      push ebp
      mov esp, ebp
      sub ebp, 20  (*стек показан здесь)
      leave
      ret
main:
```

```
push ebp
mov ebp, esp
push 3
push 2
push 1
call function
add esp 0xc
leave
ret
```

> **Примечание переводчика.** Листинг воспроизведен как в оригинале, включая `mov esp, ebp`, `sub ebp, 20` и комментарий в скобках. Для обычного пролога ожидались бы `mov ebp, esp` и выделение места через `sub esp, ...`.

После выполнения первых трех инструкций функции, включая `sub ebp, 20`, стек будет выглядеть как в следующей таблице; адреса увеличиваются сверху вниз:

Имя	Размер
buffer2	10
buffer1	5
ebp	4
ret	4
a	4
b	4
c	4
ebp	4

Теперь рассмотрим такой код:

```
void function(char *str) {
   char buffer[16];

   strcpy(buffer,str); //Копирует входную строку str в buffer
}

void main() {
  char large_string[256];
  int i;

  for( i = 0; i < 255; i++)
    large_string[i] = 'A';  //создает строку из 255 символов 'A'

  function(large_string);
}
```

> **Примечание переводчика.** Типографские кавычки в символьном и строковом литералах оригинала заменены на допустимые для C ASCII-кавычки.

Функция `main` создает строку из 255 символов A. Затем она передает указатель на этот буфер функции `function`. Та выделяет 16 байт под локальный буфер, но без проверки длины вслепую копирует в него входной буфер с помощью `strcpy`. Поэтому входной буфер из 255 символов A переполняет локальный буфер размером всего 16 байт.

Результатом запуска будет `Segmentation fault (core dumped)`. Ошибка сегментации возникает, когда приложение пытается прочитать, записать или выполнить недопустимый адрес памяти. Разберемся подробнее, что произошло. После трансляции код выглядит так:

```
0804840c <function>:
 804840c:    55                      push   ebp
 804840d:    89 e5                   mov    ebp,esp
 804840f:    83 ec 28                sub    esp,0x28
 8048412:    8b 45 08                mov    eax,DWORD PTR [ebp+0x8]
 8048415:    89 44 24 04             mov    DWORD PTR [esp+0x4],eax
 8048419:    8d 45 e8                lea    eax,[ebp-0x18]  ;[1]
 804841c:    89 04 24                mov    DWORD PTR [esp],eax
 804841f:    e8 cc fe ff ff          call   80482f0 <strcpy@plt>
 8048424:    c9                      leave
 8048425:    c3                      ret
```

Здесь видно, что `ebp-0x18` является адресом начала буфера, отмеченным `[1]`. Из пролога и настройки стека видно, что для стека выделено `0x28` байт. Напомним: `ebp` указывает на нижнюю границу стека, а `esp` - на верхнюю. Следовательно, `ebp = esp+0x28`.

Поэтому после настройки функции начало массива относительно `esp` находится по адресу `esp+0x10`. Это лишь означает, что буфер расположен за `0x10` байт до конца выделенной функции области стека, что логично: `0x10` равно 16 в десятичной системе, и функции выделен 16-байтовый буфер.

Чтобы увидеть разрушение стека в действии, запустите приложение в gdb и установите точку останова непосредственно перед `strcpy`. В точке останова вывод памяти по указателю стека должен быть похож на рис. 17.1.

```
(gdb) x/16x $esp
0xffffd120:     0x00000000      0xf7fdab18      0x00000000      0x00000000
0xffffd130:     0x00000000      0x00000003      0xf63d4e2e      0x000003f3
0xffffd140:     0x00000000      0xf7e25938      0xffffd278      0x08048470
0xffffd150:     0xffffd16c      0x00000000      0x00000026      0xf7e4a95d
```

Рис. 17.1. Кадр стека функции до `strcpy`

На изображении выделенный буфер занимает строку с адресом `0xffffd130`, а через `0x10` байт заканчивается кадр стека функции. За ним находятся сохраненное значение `ebp` предыдущего стека и, наконец, адрес возврата. Сохраненное значение регистра `ebp`, то есть кадра стека предыдущей функции, равно `0xffffd278`, а адрес возврата - `0x08048470`.

После шага через `strcpy` та же область памяти будет выглядеть как на рис. 17.2. Операция `strcpy` перезаписывает буфер, сохраненный регистр `ebp` и адрес возврата значением `0x41`, то есть символом A.

```
(gdb) s
6       }
(gdb) x/16x $esp
0xffffd120:     0xffffd130      0xffffd16c      0x00000000      0x00000000
0xffffd130:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffd140:     0x41414141      0x41414141      0x41414141      0x41414141
0xffffd150:     0x41414141      0x41414141      0x41414141      0x41414141
```

Рис. 17.2. Стек функции после `strcpy`

Когда приложение доходит до `ret`, оно снимает со стека адрес возврата и пытается продолжить выполнение по нему. Однако `0x41414141` является недопустимым адресом, поэтому процессор вызывает ошибку сегментации.

В этом примере приложение аварийно завершается, но возможны и другие последствия. На высоком уровне мы получили возможность управлять адресом возврата и кадром стека предыдущей функции. Манипуляции с кадром стека тоже полезны, но гораздо чаще целью становится адрес возврата, поэтому сосредоточимся на нем. В первом случае он был перезаписан мусором. А что, если подобрать содержимое для перезаписи более осмысленно?

Следующий пример меняет адрес возврата, чтобы управлять выполнением кода. Цель - пропустить инструкцию `x=1`:

```
#include <stdio.h>
void function(int a, int b, int c) {
//сделать что-нибудь, чтобы после возврата пропустить x=1
}
void main() {
     int x;
     x = 0;
     function(1,2,3);
     x = 1;
     printf("%d\n",x);
 }
```

Функция `main` создает локальную переменную `x` и присваивает ей начальное значение 0. Затем она вызывает `function` с фиксированными значениями. Пока внутри `function` кода нет. Нужно понять, какой код позволит переписать адрес возврата.

После возврата из `function` функция `main` меняет `x` на 1 и печатает ее. Можно ли с помощью знаний о cdecl и стеке не выполнить `x=1`, а вывести `x=0`? Да. Содержимое `function` должно заставить программу пропустить инструкцию `x=1` в `main`.

Стек внутри `function` выглядит так:

Имя	**Адрес**
ebp	ebp
адрес возврата	ebp+4
a	ebp+8
b	ebp+12
c	ebp+16

Это обычная настройка стека cdecl. Поскольку речь идет о переполнениях, добавим в `function` буфер. Для работы с отдельными значениями в нем добавим указатель. Можно было бы писать `buffer[z]`, но указатель нагляднее показывает смещения памяти.

```
#include <stdio.h>
void function(int a, int b, int c) {
char buffer[16];
int *r;
r = 0x99;  //нужно, чтобы r не была удалена при оптимизации
buffer[0] = 0x88; //нужно, чтобы buffer не был удален при оптимизации
}
void main() {
     int x;
     x = 0;
     function(1,2,3);
     x = 1;
     printf("%d\n",x);
 }
```

После трансляции получаем:

```
0804840c <function>:
 804840c:    55                      push   ebp
 804840d:    89 e5                   mov    ebp,esp
 804840f:    83 ec 20                sub    esp,0x20
 8048412:    c7 45 fc 99 00 00 00    mov    DWORD PTR [ebp-0x4],0x99
 8048419:    c6 45 ec 88             mov    BYTE PTR [ebp-0x14],0x88
 804841d:    c9                      leave
 804841e:    c3                      ret
```

В стеке появились указатель и буфер:

Имя	**Адрес**
`buffer`	`ebp-0x14`
`r`	`ebp-4`
`ebp`	`ebp`
адрес возврата	`ebp+4`
`a`	`ebp+8`
`b`	`ebp+12`
`c`	`ebp+16`

В этом кадре адрес возврата находится по адресу `buffer+0x18`. Следующий шаг - заставить указатель из `function` указывать на этот адрес памяти.

В C оператор & означает "адрес", поэтому следующий код направляет `ret` на адрес `buffer+0x18`. По схеме стека видно, что там хранится сохраненный адрес возврата. Пока он не изменен, но указатель на него уже получен. Осталось определить новое значение, при котором будет пропущено x=1.

```
#include <stdio.h>
void function(int a, int b, int c) {
     char buffer[16];
     int *ret;

     //теперь у нас есть значение возврата; что с ним делать?
     ret = (unsigned int)&buffer+0x18;
     buffer[0] = 0x88; //нужно, чтобы buffer не был удален при оптимизации
}
void main() {
     int x;
     x = 0;
     function(1,2,3);
     x = 1;
     printf("%d\n",x);
 }
```

Чтобы понять, как изменить адрес возврата, посмотрим на код `main`:

```
0804841f <main>:
 804841f:    55                      push   ebp
 8048420:    89 e5                   mov    ebp,esp
 8048422:    83 e4 f0                and    esp,0xfffffff0
 8048425:    83 ec 20                sub    esp,0x20
 8048428:    c7 44 24 1c 00 00 00    mov    DWORD PTR [esp+0x1c],0x0
 8048430:    c7 44 24 08 03 00 00    mov    DWORD PTR [esp+0x8],0x3
 8048438:    c7 44 24 04 02 00 00    mov    DWORD PTR [esp+0x4],0x2
```

```
8048440:    c7 04 24 01 00 00 00     mov    DWORD PTR [esp],0x1
8048447:    e8 c0 ff ff ff           call   804840c <function>
804844c:    c7 44 24 1c 01 00 00     mov    DWORD PTR [esp+0x1c],0x1;x=1
8048454:    8b 44 24 1c              mov    eax,DWORD PTR [esp+0x1c]
8048458:    89 44 24 04              mov    DWORD PTR [esp+0x4],eax
804845c:    c7 04 24 08 85 04 08     mov    DWORD PTR [esp],0x8048508
8048463:    e8 88 fe ff ff           call   80482f0 <printf@plt>
8048468:    c9                       leave
8048469:    c3                       ret
```

Обычно адресом возврата из функции был бы `0x804844C`, а по этому адресу как раз находится нежелательная инструкция x=1. Следующая инструкция начинается с `0x8048454`.

Есть два способа изменить адрес возврата. Можно записать через указатель жестко заданное значение `0x8048454`. Однако это виртуальный адрес, выбранный компилятором при сборке. Он будет одинаковым при каждом запуске, но после повторной компиляции может измениться. Такой подход слишком жесткий.

Лучше учесть, что инструкция x=1 имеет длину 8 байт. Это останется неизменным, поэтому надежнее прибавить 8 байт к текущему адресу возврата.

> **Примечание.** В выводе ассемблерного кода gdb нередко обрезает шестнадцатеричное представление, поэтому в строке x=1 можно насчитать лишь 7 байт. Причина только в обрезке. Всегда вычисляйте длину по адресам.

Для пропуска x=1 увеличим адрес возврата на 8 байт:

```
#include <stdio.h>
void function(int a, int b, int c) {
char buffer[16];
int *ret;

ret = (unsigned int)buffer+0x18; //получить значение возврата
*ret +=0x8; //увеличить значение возврата на 8
buffer[0] = 0x88; //нужно, чтобы buffer не был удален при оптимизации
}
void main() {
     int x;
     x = 0;
     function(1,2,3);
     x = 1;
     printf("%d\n",x);
 }
```

> **Примечание переводчика.** В предыдущем листинге оригинала записано `&buffer`, а в итоговом - `buffer`. Это расхождение сохранено.

Запуск этого кода с флагом `-fno-stack-protector` должен вывести 0. Значит, адрес возврата успешно изменен и программа пропустила x=1. Победа!

Шелл-код

Возможность менять адреса возврата дает управление выполнением кода. Одно из распространенных применений - "открыть оболочку" и получить возможность выполнять более мощные команды.

Для этого нужно выполнить внутри приложения собственный произвольный код. Шелл-код помещают в переполняемый буфер, а адрес возврата меняют так, чтобы он указывал на начало этого кода. Само слово shellcode буквально означает код, запускающий командную оболочку. В зависимости от доступного места шелл-код может находиться до адреса возврата или после него.

Цель - поместить его в какой-либо буфер и направить на него адрес возврата.

Ниже показан очень простой шелл-код. Он использует системный вызов Linux `execve` для запуска `/bin/sh`, распространенной командной оболочки. Через `execve` программа просит ядро Linux выполнить действие, в данном случае запустить оболочку.

```
#include <stdio.h>

void main() {
   char *name[2];

   name[0] = "/bin/sh";
   name[1] = NULL;
   execve(name[0], name, NULL);
      exit(0);
}
```

Этот простой код транслируется так:

```
0804843c <main>:
 804843c:    55                       push   ebp
 804843d:    89 e5                    mov    ebp,esp
 804843f:    83 e4 f0                 and    esp,0xfffffff0
 8048442:    83 ec 20                 sub    esp,0x20
8048445:    c7 44 24 18 18 85 04     mov    DWORD PTR [esp+0x18],
0x8048518
 804844c:    08
 804844d:    c7 44 24 1c 00 00 00     mov    DWORD PTR [esp+0x1c],0x0
 8048454:    00
 8048455:    8b 44 24 18              mov    eax,DWORD PTR [esp+0x18]
 8048459:    c7 44 24 08 00 00 00     mov    DWORD PTR [esp+0x8],0x0
 8048460:    00
 8048461:    8d 54 24 18              lea    edx,[esp+0x18]
 8048465:    89 54 24 04              mov    DWORD PTR [esp+0x4],edx
 8048469:    89 04 24                 mov    DWORD PTR [esp],eax
 804846c:    e8 cf fe ff ff           call   8048340 <execve@plt>
 8048471:    c7 04 24 00 00 00 00     mov    DWORD PTR [esp],0x0
 8048478:    e8 a3 fe ff ff           call   8048320 <exit@plt>
```

Код опирается на стандартные функции C `execve` и `exit`, которые перемещаются в памяти, поэтому предсказать их адреса и встроить их в код трудно. Если взять эти машинные коды как есть, поместить их в буфер и направить туда адрес возврата, то на `call execve` скорее всего возникнет ошибка сегментации. Адрес `0x8048340` соответствует тому месту, куда `execve` загрузилась в данном приложении, и не является универсальным. Нужно было бы знать адрес `execve` в целевом приложении, если она там вообще есть. Следовательно, оболочку придется запускать без библиотек C.

Если дизассемблировать `execve` и `exit`, можно увидеть базовую реализацию:

```
mov   eax, 0xb
mov   ebx, string_addr
lea   ecx, string_addr
lea   edx, null_string
int   0x80   ;системный вызов exec
mov   eax, 0x1
mov   ebx, 0x0
int   0x80   ;системный вызов exit
":/bin/sh"\0
```

> **Примечание переводчика.** В оригинале перед `/bin/sh` напечатано двоеточие, а сама строка заключена в типографские кавычки. Двоеточие сохранено, кавычки нормализованы в ASCII.

Таким образом, вызовы библиотеки C сводятся к системным вызовам `int 0x80`, рассмотренным ранее. Но возникает другая трудность: значения `string_addr` и `null_string` неизвестны, потому что нельзя предсказать их размещение в памяти. Собранный шелл-код поместил их в свое

локальное адресное пространство, где `0x8048518` является адресом `/bin/sh`, однако после переноса кода в целевой буфер эти адреса окажутся неверными.

Нужен относительный, а не жестко заданный способ получить адрес. Можно воспользоваться адресами возврата при вызовах функций. Если поместить вызов функции непосредственно перед строкой, то внутри этой функции адрес строки окажется на вершине стека, поскольку строка начинается по адресу возврата.

Сначала добавим в имеющийся шелл-код несколько заполнителей:

```
jmp    ??
pop   esi
mov   [esi+0x8],esi
mov   [esi+0x7],0x0
mov   [esi+0xc],0x0
mov   eax, 0xb
mov   ebx, esi
lea   ecx, [esi+0x8]
lea   edx, [0xc+esi]
int    0x80
mov   eax, 0x1
mov   ebx, 0
int    0x80
call   ??
.string \"/bin/sh\"
```

К исходному шелл-коду добавлены две инструкции в начале и две в конце. Теперь нужно определить адрес строки, находящейся в конце блока. Начальная инструкция `jmp` должна перейти к новому `call` внизу.

Этот `call`, в свою очередь, должен вызвать новую строку `pop esi`. При использовании `call`, а не перехода, адрес возврата, то есть адрес следующей инструкции, помещается в стек. Обычная настройка кадра cdecl здесь не нужна: знания об x86 намеренно используются не по назначению.

После вызова `pop esi` на вершине стека находится адрес возврата, которым в данном случае является адрес строки оболочки. Его можно снять со стека в `esi` и использовать в предыдущем шелл-коде.

Вместо заполнителей для перехода и вызова нужны точные смещения. Для этого подсчитаем длины скомпилированных инструкций:

```
jmp    0x26                    # 2 bytes
pop   esi                      # 1 byte
mov   [esi+0x8],esi            # 3 bytes
mov   [esi+0x7],0x0            # 4 bytes
mov   [esi+0xc],0x0            # 7 bytes
mov   eax, 0xb                 # 5 bytes
mov   ebx, esi                 # 2 bytes
lea   ecx, [esi+0x8]           # 3 bytes
lea   edx, [0xc+esi]           # 3 bytes
int    0x80                    # 2 bytes
mov   eax, 0x1                 # 5 bytes
mov   ebx, 0                   # 5 bytes
int    0x80                    # 2 bytes
call   -0x2b                   # 5 bytes
.string \"/bin/sh\"
```

Измененный код решает проблему поиска строки в памяти: все адреса стали относительными, и для этого используются базовые механизмы x86. Последняя задача - запустить код, поместив его двоичное представление в стек через переполнение буфера.

Разрушение стека и защита стека

Как уже говорилось, многие современные компиляторы по умолчанию встраивают защиту от простейших атак на стек. Например, gcc и g++ версий после 4.1 имеют встроенную защиту стека. Чтобы практиковаться в разрушении стека, исполняемые файлы нужно собирать с `-fno-stack-protector`. Посмотрим, как выглядит защита. Следующая программа собрана с включенной защитой стека:

```
0804845c <function>:
 804845c:    55                      push   ebp
 804845d:    89 e5                   mov    ebp,esp
 804845f:    83 ec 48                sub    esp,0x48
 8048462:    8b 45 08                mov    eax,DWORD PTR [ebp+0x8]
 8048465:    89 45 d4                mov    DWORD PTR [ebp-0x2c],eax
 8048468:    65 a1 14 00 00 00       mov    eax,gs:0x14
 804846e:    89 45 f4                mov    DWORD PTR [ebp-0xc],eax
 8048471:    31 c0                   xor    eax,eax
 8048473:    8b 45 d4                mov    eax,DWORD PTR [ebp-0x2c]
 8048476:    89 44 24 04             mov    DWORD PTR [esp+0x4],eax
 804847a:    8d 45 e4                lea    eax,[ebp-0x1c]
 804847d:    89 04 24                mov    DWORD PTR [esp],eax
 8048480:    e8 bb fe ff ff          call   8048340 <strcpy@plt>
 8048485:    8b 45 f4                mov    eax,DWORD PTR [ebp-0xc]
 8048488:    65 33 05 14 00 00 00    xor    eax,DWORD PTR gs:0x14
 804848f:    74 05                   je     8048496 <function+0x3a>
 8048491:    e8 9a fe ff ff          call   8048330 <__stack_chk
_fail@plt>
 8048496:    c9                      leave
 8048497:    c3                      ret
```

В оригинале жирным выделены строки, добавленные компилятором для защиты стека. Компилятор добавил код, который при входе в функцию сохраняет адрес возврата и после `strcpy` проверяет, не изменился ли он. Компилятор знает, что вызовы вроде `strcpy` могут быть опасны; такая проверка не дает `strcpy` перезаписать адрес возврата.

> **Примечание переводчика.** Объяснение предыдущего абзаца воспроизводит формулировку оригинала. Фактически показанная последовательность сохраняет и проверяет стековую канарейку из `gs:0x14`, а при несовпадении вызывает `__stack_chk_fail`.

Защититься от разрушения стека можно несколькими способами: использовать встроенную защиту gcc, языки с безопасной памятью и проверкой границ, а также предотвращение выполнения данных (Data Execution Prevention, DEP). Тем не менее переполнения буфера все еще опасны: не каждый компилятор поддерживает защиту стека или DEP, а защита имеет нюансы и не ставит ограждение вокруг каждого вызова. Она сосредоточена на конкретных опасных операциях вроде `strcpy`, и многие компиляторы достаточно хорошо определяют, какие из них требуют защиты.

Связывание C и x86

Любую программу, которую можно написать на C или другом языке, можно написать и на ассемблере. Более того, перед исполнением процессором код высокого уровня компилируется в ассемблерный. Но иногда C и ассемблер удобно смешивать. Для собственных эксплойтов и взломов это мощное сочетание. Некоторые действия достаточно тонки и требуют контроля на уровне ассемблера, а другие представляют собой обычный код, который быстрее написать на C.

Чтобы вызвать функцию, написанную на другом языке, нужно знать ее расположение в памяти. Линкер может предоставить эту информацию автоматически. Нужно также знать, как передать ей данные, то есть ее соглашение о вызовах. Будем считать, что функции C используют cdecl.

Напомним правила cdecl:

• аргументы передаются через стек и помещаются туда справа налево;
• после возврата вызывающая сторона очищает стек;
• возвращаемое значение функции хранится в `eax`;
• регистры `eax`, `ecx` и `edx` доступны вызываемой функции. Вызывающая сторона должна сохранить их значения, если они нужны, а вызываемая должна сохранить и восстановить значения всех прочих используемых регистров.

Соблюдая правильное соглашение о вызовах, можно вызывать функции C из ассемблерного кода.

Использование функций C в коде x86

Чтобы код x86 мог использовать функции C, ему нужно сообщить, что функция определена в другом месте. Для этого служит директива `extern`. Например, функцию C `x()` вызывают так:

```
extern x
call x
```

Сначала в начале ассемблерного файла указывают `extern function_name`. Эта директива сообщает ассемблеру, что функция будет использована, но ее расположение пока неизвестно. Строка `call function_name` первоначально собирается как `call 0x????????`. Программу нельзя запустить, пока линкер не подставит нужный адрес.

Затем нужную функцию вызывают по правилам cdecl. Например, для функции C `int add(int x, int y)` используется следующий код. Аргументы помещаются справа налево, после вызова стек очищается, а возвращаемое значение берется из `eax`.

```
push [y]
push [x]
call add
add esp, 8
mov [sum], eax
```

Ассемблерный код собирают с помощью nasm, например: `nasm example.asm -o example.o`.

> **Примечание переводчика.** Здесь и далее типографские тире, ошибочно использованные в оригинале внутри параметров командной строки, заменены на ASCII-дефисы.

Пока все будет переведено в машинный код, кроме заполнителей. Без внешних функций программа уже была бы готова к запуску, но теперь нужна помощь линкера. Последний шаг - связать ассемблерный код с функцией C. Если используется gcc и вызываются функции библиотеки C, gcc сделает это автоматически. Например, `gcc example.o -o example` подставит все известные линкеру адреса, превратив `call 0x????????` в `call 0x08048320`.

Рассмотрим пример, который через `printf` выводит `hello world 42`:

```
extern printf
global main

section .text
main:

push 42
push world
push hello
call printf
add esp, 12

mov eax, 1
mov ebx, 0
```

```
int 0x80

section .data
hello: db "hello %s %d", 0xa, 0
world: db "world"
```

Этот код можно собрать командой `nasm -f elf example.asm`, а затем скомпоновать командой `gcc -m32 example.o -o example`.

Возможность вызывать простые функции вроде `printf` из ассемблерного кода очень удобна при проверке идей для взлома или исправления программы.

Использование функций x86 в коде C

Ассемблерные функции можно вызывать и из C. Программа на C должна содержать прототип нужной функции x86. Например, для ассемблерной функции `f` нужен прототип `int f(void);`. Прототип описывает, как выглядело бы определение функции на языке высокого уровня: ее имя, аргументы и возвращаемое значение.

Чтобы использовать функции x86 из C, их нужно экспортировать из ассемблерного кода, чтобы линкер мог их найти. Для этого служит директива `global`:

```
global f
f:
  mov eax, 0xdabbad00
  ret
```

Ассемблерный код собирают nasm, а всю программу компилируют и компонуют gcc. Рассмотрим программу на C:

```
// x.c

#include <stdio.h>

int add(int,int);

int main(void)
{
    int x=add(1,2);
    printf("%d\n",x);
    return 0;
}
```

Она использует функцию `add`, определенную в следующем ассемблерном коде:

```
; y.asm

add:
  push ebp
  mov ebp, esp

  mov eax, [ebp+8]
  add eax, [ebp+12]

  leave
  ret
```

Для сборки и компоновки выполните:

```
nasm -f elf y.asm # создает объектный файл y.o
gcc -m32 -c x.c # создает объектный файл x.o
gcc x.o y.o -o adder # создает исполняемый файл adder
# запустить командой ./adder
```

_start и main()

Ассемблерные программы x86 часто начинаются с метки `_start`, а программы на C - с функции `main()`. В чем различие?

Выполнение программы на C, ассемблере или любом другом языке на самом деле начинается не с `main`. Рассмотрим простейшую функцию C:

```
int main()
{
return 0;
}
```

При компиляции командой `gcc simple.c -o simple` программа переводится в ассемблерный код. Компилятор добавляет функцию `_start`, которая вызывает `main`.

Скомпилированная функция `main` выглядит так:

```
80483b4:     55                   push   ebp
80483b5:     89 e5                mov    ebp,esp
80483b7:     5d                   pop    ebp
80483b8:     c3                   ret
```

Функция `_start` выглядит так:

```
8048300:     31 ed                xor    ebp,ebp
8048302:     5e                   pop    esi
8048303:     89 e1                mov    ecx,esp
8048305:     83 e4 f0             and    esp,0xfffffff0
8048308:     50                   push   eax
8048309:     54                   push   esp
804830a:     52                   push   edx
804830b:     68 30 84 04 08       push   0x8048430
8048310:     68 c0 83 04 08       push   0x80483c0
8048315:     51                   push   ecx
8048316:     56                   push   esi
8048317:     68 b4 83 04 08       push   0x80483b4
804831c:     e8 cf ff ff ff       call   80482f0 <__libc_start_main@plt>
8048321:     f4                   hlt
```

Функция `_start` отвечает за несколько задач:

• инициализирует указатель кадра;
• настраивает стек;
• подготавливает стандартные аргументы, то есть параметры `main()`;
• вызывает `libc_start_main`, которая выполняет проверки безопасности, настраивает подсистему потоков и инициализацию, вызывает `main`, а затем `exit()`.

В чистом ассемблерном коде все пишется вручную. Полная настройка, выполняемая C, не обязательна: можно написать собственную `_start`.

При сочетании ассемблера и C нужен gcc. Обычно gcc предоставляет свою `_start` и ожидает от вас функцию `main()`. Для ассемблерной программы, компонуемой со стандартной библиотекой C, соблюдайте следующие правила:

1. Используйте `main` вместо `_start`: `libc_start_main` сама вызовет `main()`.
2. Настройте только кадр стека, а не весь стек: `_start` уже его настроила.
3. Завершайте код через `ret`, а не `int 0x80`: `ret` вернет управление в `libc_start_main`, та вызовет функцию C `exit`, а уже она выполнит `int 0x80`.
4. Перед `ret` установите возвращаемое значение в `eax`, обычно в 0.

Например, автономная ассемблерная программа определяет собственную `_start`:

```
global _start

section .text
```

```
_start:
    mov  esp, stack
    mov  ebp, esp

    ...

    mov  esp, ebp

    mov  eax, 1
    mov  ebx, 0
    int  0x80

section .data
times 128 db 0
stack equ $-4
```

При компоновке с libc вместо нее следует использовать `main`:

```
global main

section .text
main:
    push ebp
    mov  ebp, esp

    ...

    mov  eax, 0
    leave
    ret
```

Стандартные аргументы

В C аргументы командной строки можно читать с помощью стандартных аргументов. Функцию `main()` часто определяют как `int main(int argc, char **argv)`, что дает доступ к этим аргументам. `argc` содержит количество переданных аргументов, а `argv` является массивом самих аргументов.

> **Примечание переводчика.** В оригинале в этом месте напечатаны выражения `command link` и `stdargs`; по смыслу они переданы как "командная строка" и "стандартные аргументы".

Аргументы командной строки доступны и в ассемблерной функции `main`. Она автоматически вызывается по соглашению cdecl. Напомним:

• аргументы передаются через стек справа налево;
• они находятся по адресам `[ebp+8]`, `[ebp+12]` и далее;
• `argc` является последним аргументом и находится первым в списке аргументов стека, по адресу `[ebp+8]`;
• `argv` помещается в стек первым и находится по адресу `[ebp+12]`.

Например, следующая программа выводит первый аргумент командной строки:

```
extern printf
global main

main:
   push  ebp
   mov   ebp, esp

   mov   eax, [ebp+12]  ; загрузить argv в eax
   push  dword [eax+4]  ; поместить argv[1] в стек
   call  printf         ; вывести argv[1]
   add   esp, 4         ; очистить стек
```

```
    mov   eax, 0
    leave
    ret
```

Смешивание C и ассемблера

В C можно бесшовно переключаться между C и ассемблерным кодом. Это называется встроенным ассемблером, поскольку ассемблерные инструкции встраиваются прямо в исходный код.

Встроенный ассемблер не входит в спецификацию C, но большинство компиляторов поддерживает его как расширение. Синтаксис зависит от компилятора. В gcc применяется синтаксис x86 AT&T.

Базовая форма имеет вид `__asm__ ("assembly code here");`. При компиляции gcc переводит C в ассемблерный код и вставляет туда код из директивы `__asm__`.

Например:

```
int main(void)
{
    // установить регистр управления клавиатурой

    __asm__ ("mov  $0x10010001, %eax");
    __asm__ ("out  %eax, $0x64");

    return 0;
}
```

Расширенная форма встроенного ассемблера позволяет задавать дополнительные ограничения:

• входные переменные: переменные C, которыми нужно манипулировать из ассемблерного кода;
• выходные переменные: значения, созданные встроенным ассемблерным кодом и нужные в C;
• изменяемые регистры: gcc переводит C в ассемблерный код и сам выбирает регистры. Этот список не дает регистрам кода C и ассемблерного кода конфликтовать.

Расширенная форма в оригинале записана так:

```
__asm__(
             "assembly"
             : входные ограничения
             : выходные ограничения
             : список изменяемых регистров
             );
```

> **Примечание переводчика.** Порядок полей сохранен по оригиналу. В синтаксисе расширенного asm GCC после шаблона обычно указываются выходные операнды, затем входные, а затем список изменяемых регистров.

Пример расширенного ассемблера в C:

```
#include <stdio.h>

int main(void)
{
    // получение адреса возврата текущей функции

    int x;

    __asm__("\
            movl 0x4(%%ebp), %%eax  \n\
            movl %%eax, %0          \n\
            "
            :"=r"(x)
            :
            :"%eax"
```

```
            );

    printf("%08x\n", x);

    return 0;
}
```

Встроенный ассемблер широко применяется:

• в ядре операционной системы, например в исходном коде ядра Linux;
• во встраиваемых системах;
• в коде, работающем с аппаратурой;
• в коде, который должен быть очень быстрым;
• он время от времени встречается при работе с C, и вам, возможно, придется применять его самостоятельно.

При использовании встроенного ассемблера в gcc нужен дополнительный флаг. Например, команда `gcc -masm=intel myFile.c` сообщает gcc, что в файл C встроен ассемблерный код в синтаксисе Intel.

Итоги

В этой главе было показано, как применять знания об x86 и стеке для взлома. Разрушив стек и вставив шелл-код, специалист по обратной разработке может заставить программу выполнить код атакующего.